CS 6120

Natural Language Processing

Instructor: Amir Tahmasebi, PhD

Fall 2026



Introduction to Machine Learning

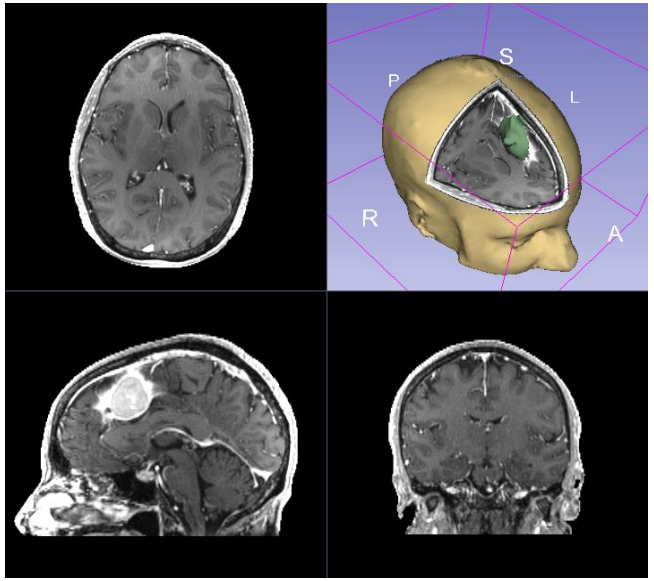
Part II



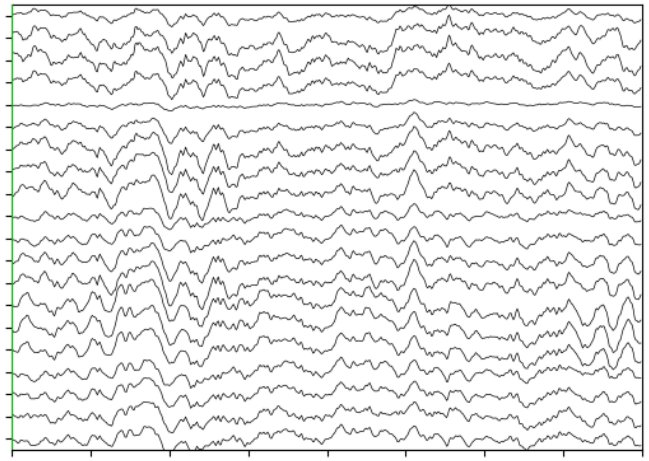
Syllabus

- What is Natural Language Processing?
  - History
  - Applications
- Python Primer
- Text Processing Fundamentals
  - Regular Expressions
  - Tokenization
  - Normalization
  - Libraries and Toolkits
- Parsing
  - Syntactic
  - Semantic
- Word Representation
  - TF-IDF
  - word2vec
  - Glove
- Intro to Machine Learning
  - Linear Regression
  - Logistic Regression (LR)
  - Clustering
  - Neural Networks
  - Recurrent Neural Networks (RNN)
  - Convolutional Neural Networks (CNN)
  - Transformers
- Language Models
- NLP Use Cases
  - Named Entity Recognition
  - Machine Translation
  - QA
- Large Language Models
  - chatGPT

Classical Machine Learning



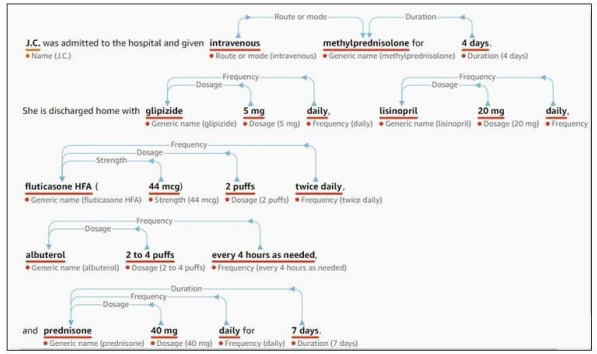
Image



Time-series

J.C. was admitted to the hospital and given intravenous methylprednisolone for 4 days. She is discharged home with glipizide 5 mg daily, lisinopril 20 mg daily, fluticasone HFA (44 mcg) 2 puffs twice daily, albuterol 2 to 4 puffs every 4 hours as needed, and prednisone 40 mg daily for 7 days.

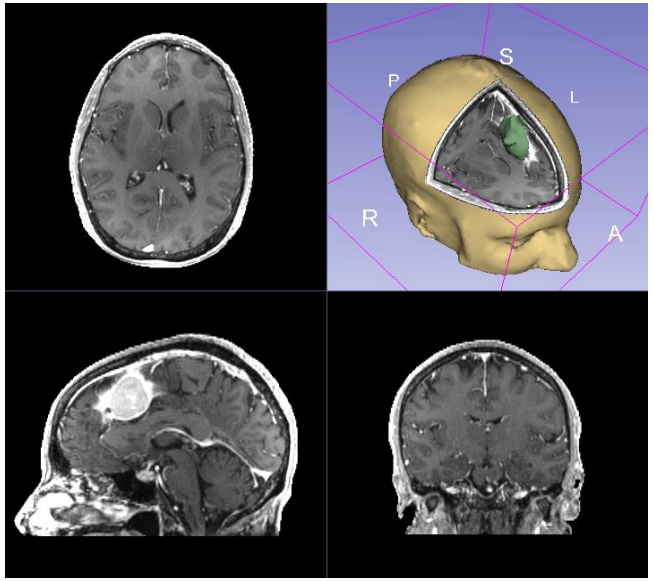
A. Sample clinical text note



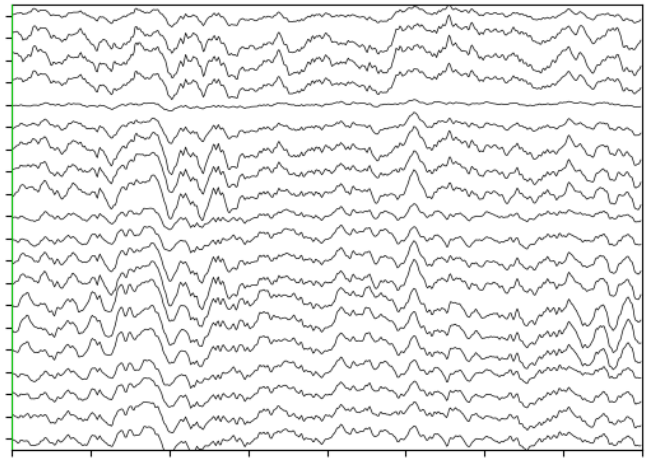
B. ACM Output

Textual

Classical Machine Learning



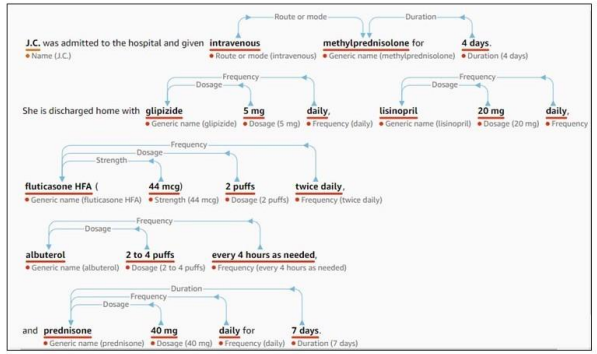
Image



Time-series

J.C. was admitted to the hospital and given intravenous methylprednisolone for 4 days. She is discharged home with glipizide 5 mg daily, lisinopril 20 mg daily, fluticasone HFA (44 mcg) 2 puffs twice daily, albuterol 2 to 4 puffs every 4 hours as needed, and prednisone 40 mg daily for 7 days.

A. Sample clinical text note

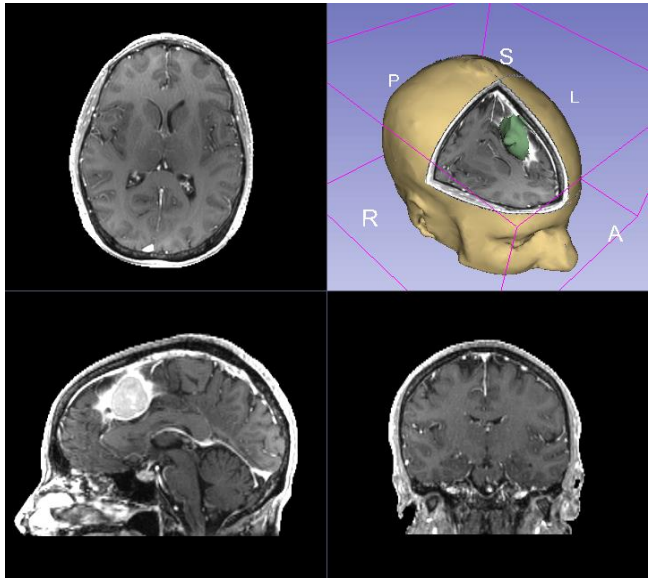


B. ACM Output

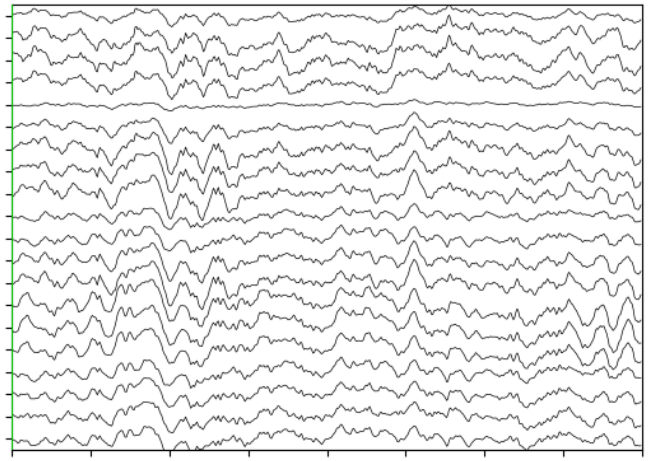
Textual

Feature Extraction/Feature Engineering

Classical Machine Learning



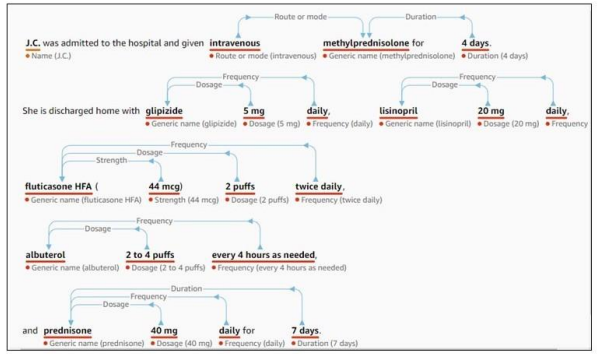
Image



Time-series

J.C. was admitted to the hospital and given intravenous methylprednisolone for 4 days. She is discharged home with glipizide 5 mg daily, lisinopril 20 mg daily, fluticasone HFA (44 mcg) 2 puffs twice daily, albuterol 2 to 4 puffs every 4 hours as needed, and prednisone 40 mg daily for 7 days.

A. Sample clinical text note



B. ACM Output

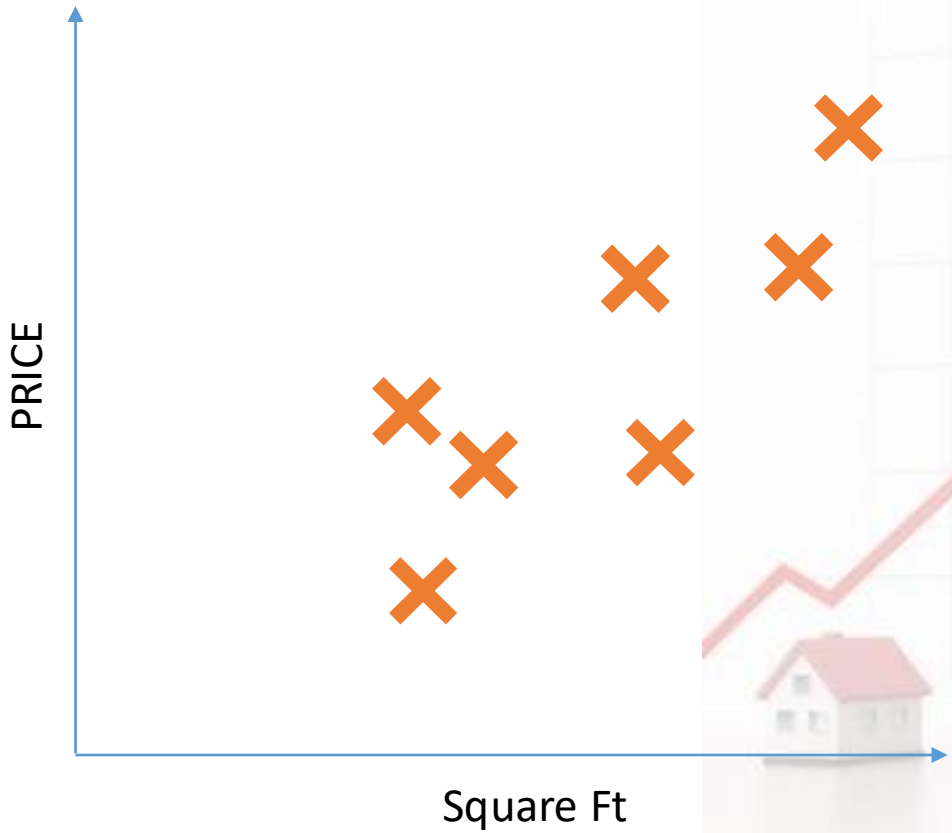
Textual

Feature Extraction/Feature Engineering

Hypothesis/Model Fitting/Training

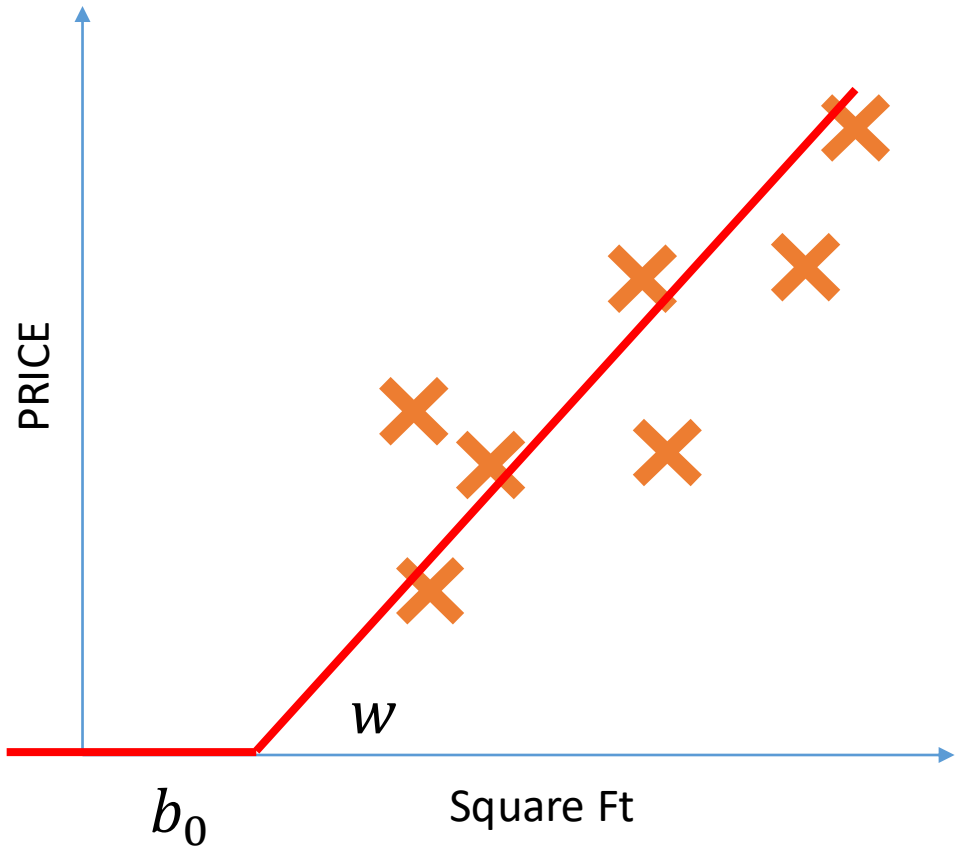
Machine Learning: *Neural Networks*

Prediction



Machine Learning: *Neural Networks*

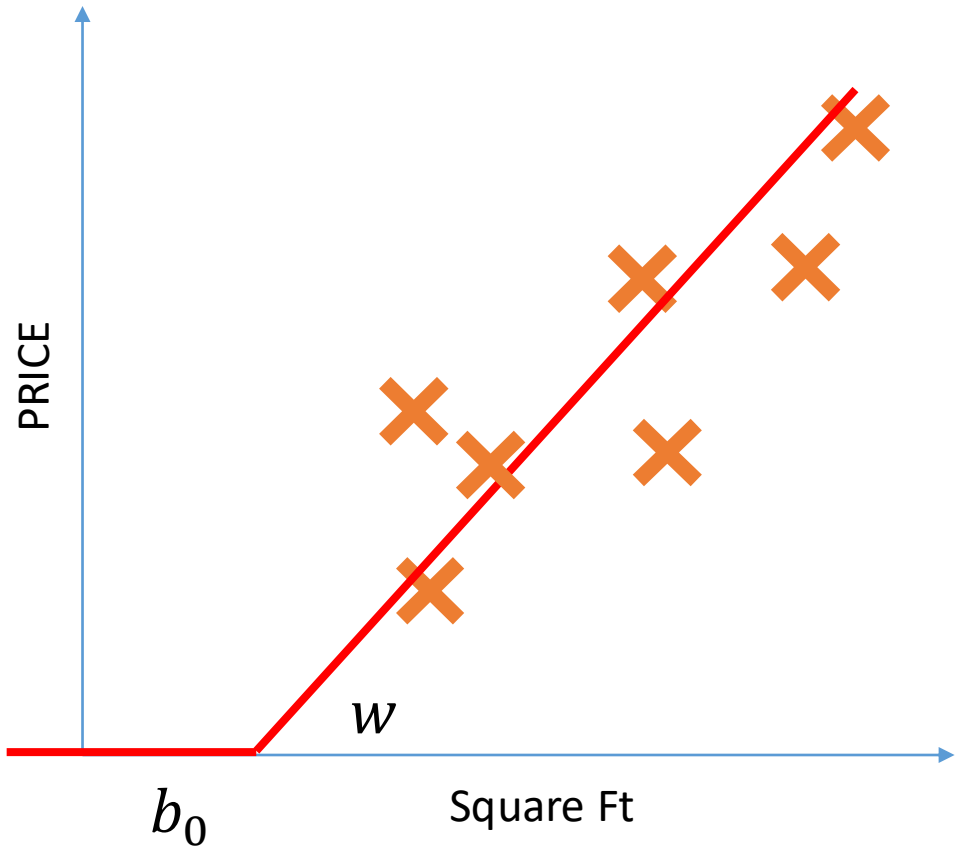
Prediction



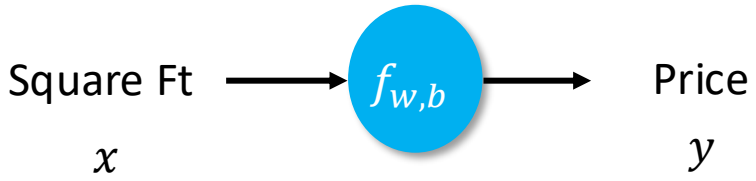
$$y = \begin{cases} wx, & x > b_0 \\ 0 & x \leq b_0 \end{cases}$$

Machine Learning: *Neural Networks*

Prediction

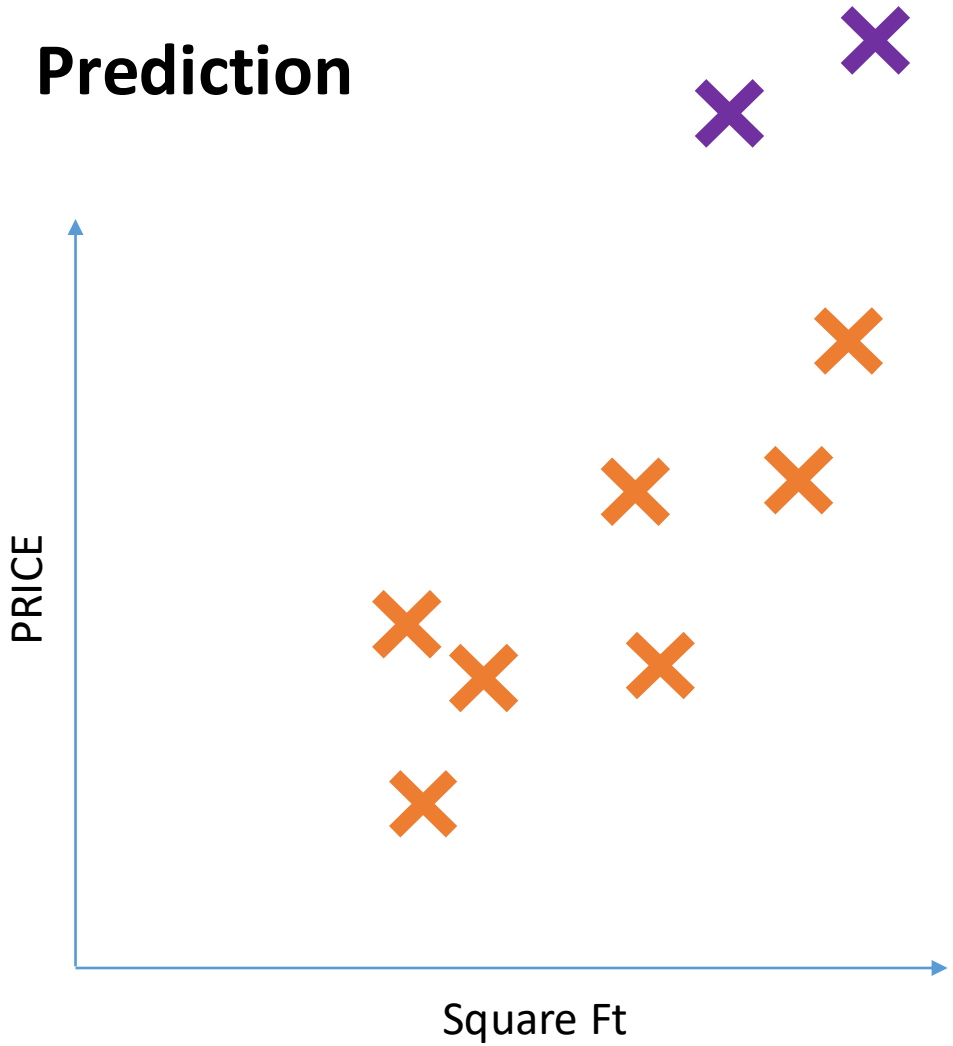


$$y = \begin{cases} wx, & x > b_0 \\ 0 & x \leq b_0 \end{cases}$$



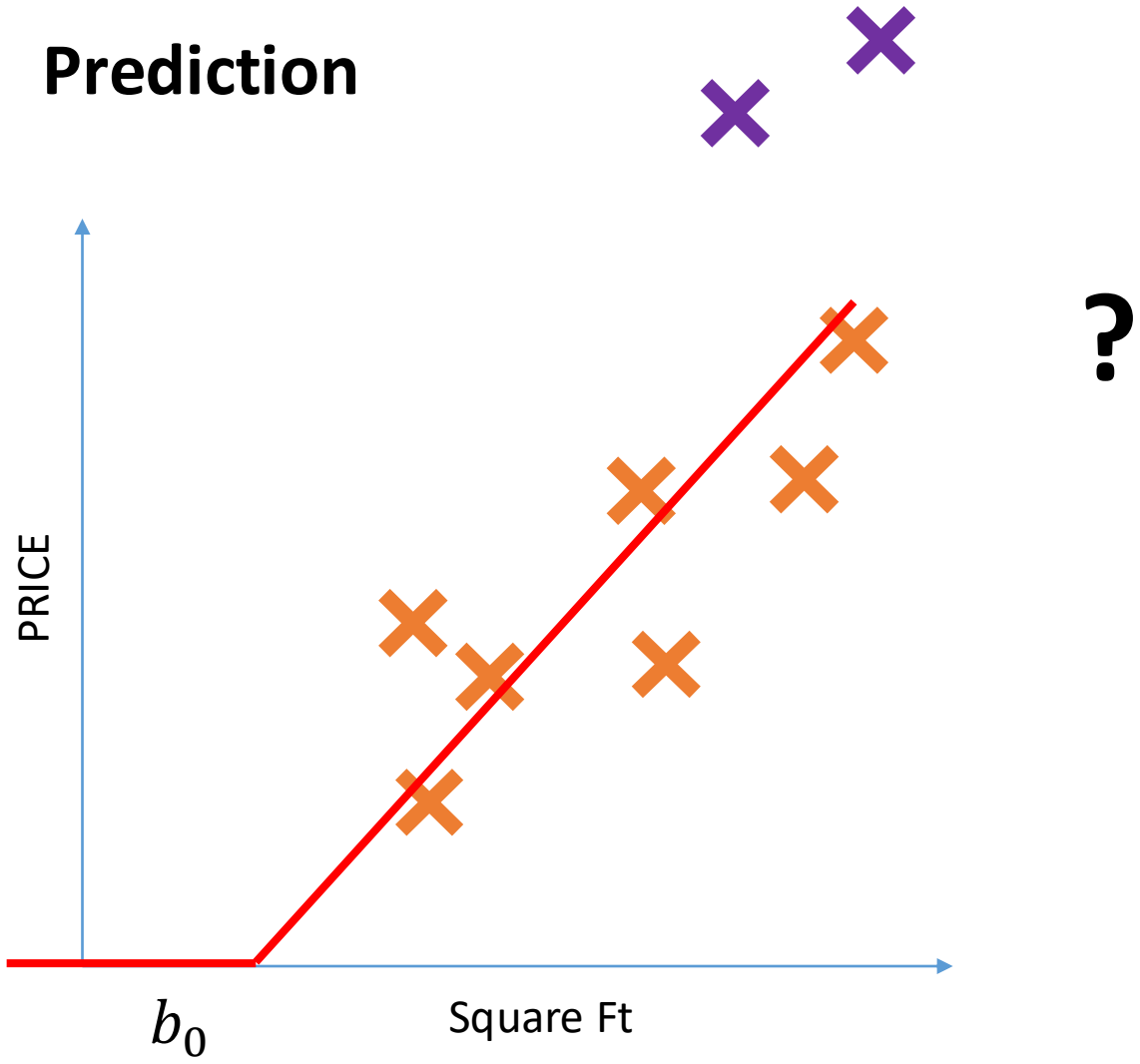
Machine Learning: *Neural Networks*

Prediction



Machine Learning: *Neural Networks*

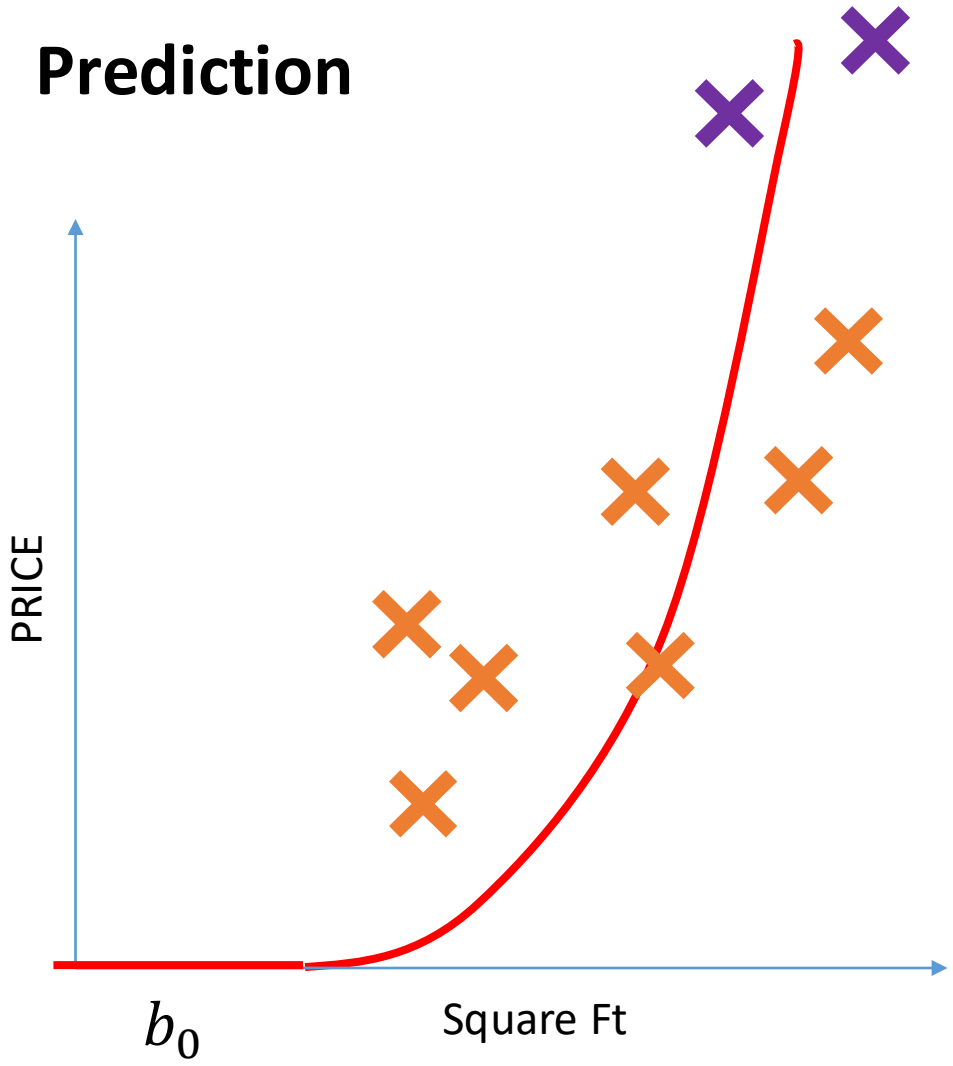
Prediction



?

Machine Learning: *Neural Networks*

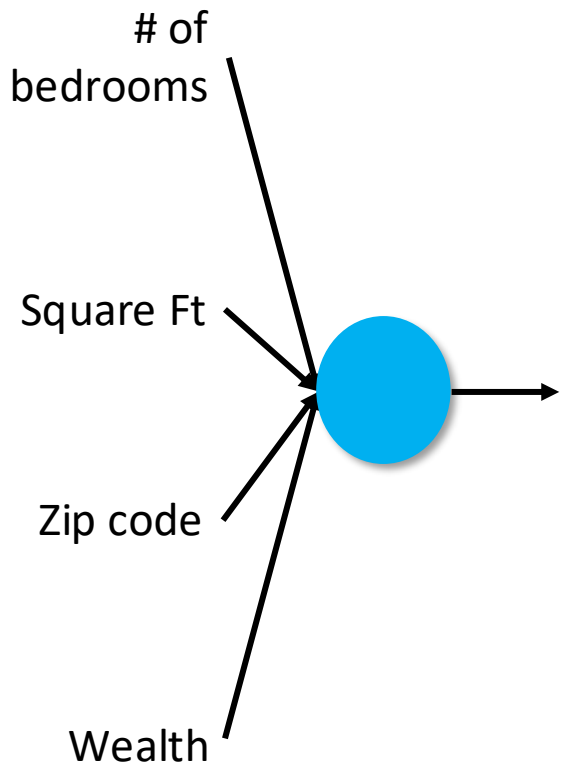
Prediction



$$y = \begin{cases} wx^2, & x > b_0 \\ 0 & x \leq b_0 \end{cases}$$

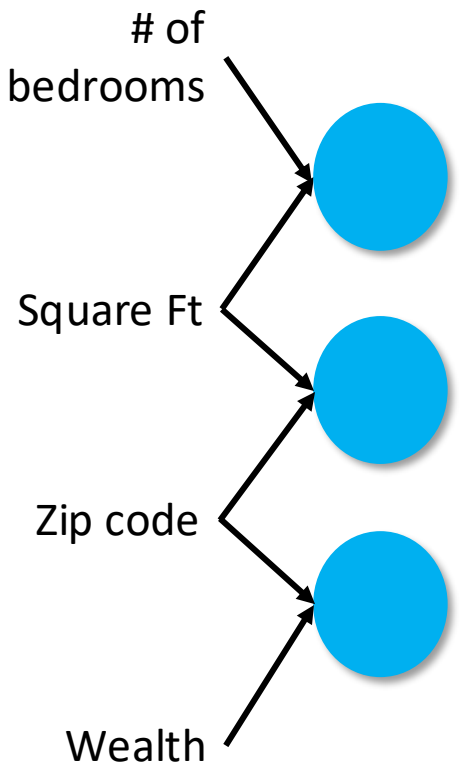
Machine Learning: *Neural Networks*

Prediction



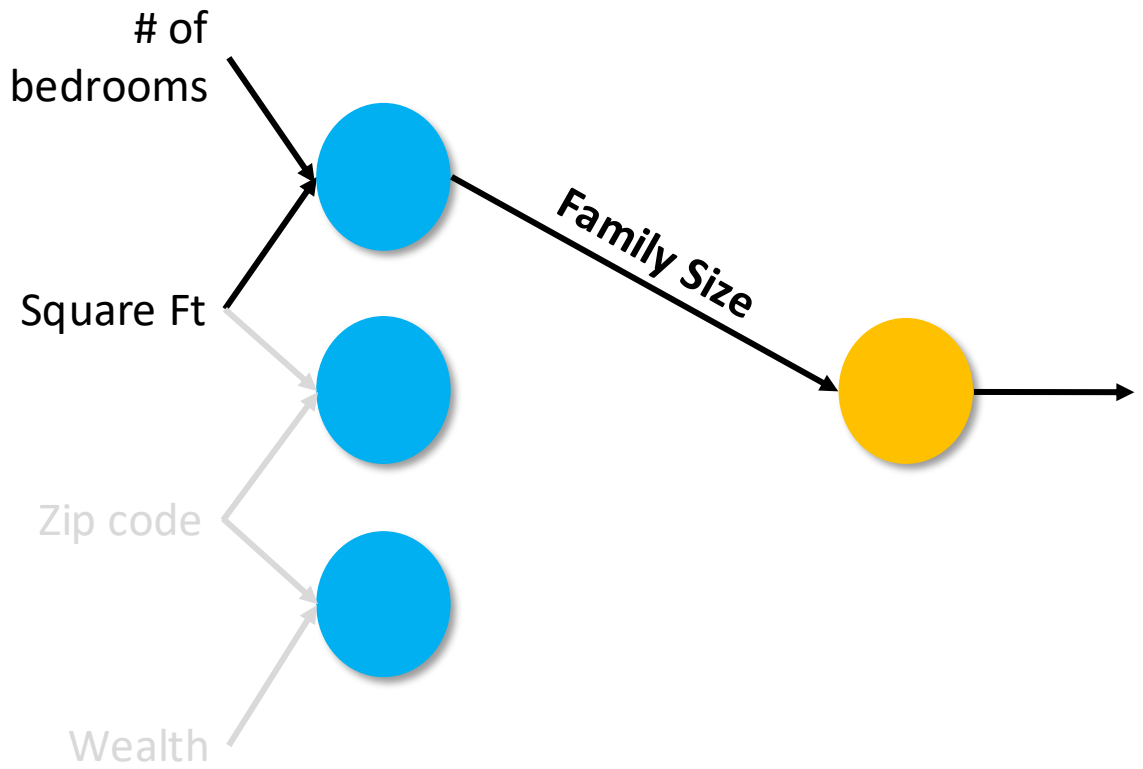
Machine Learning: *Neural Networks*

Prediction



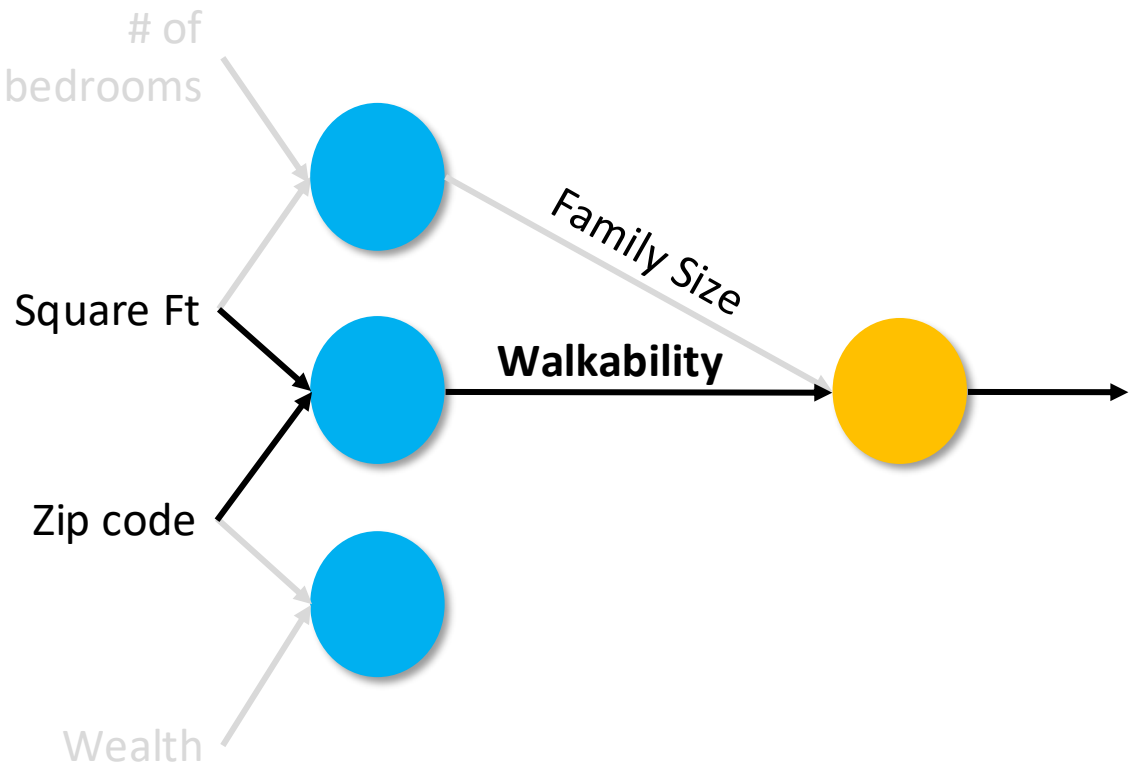
Machine Learning: *Neural Networks*

Prediction



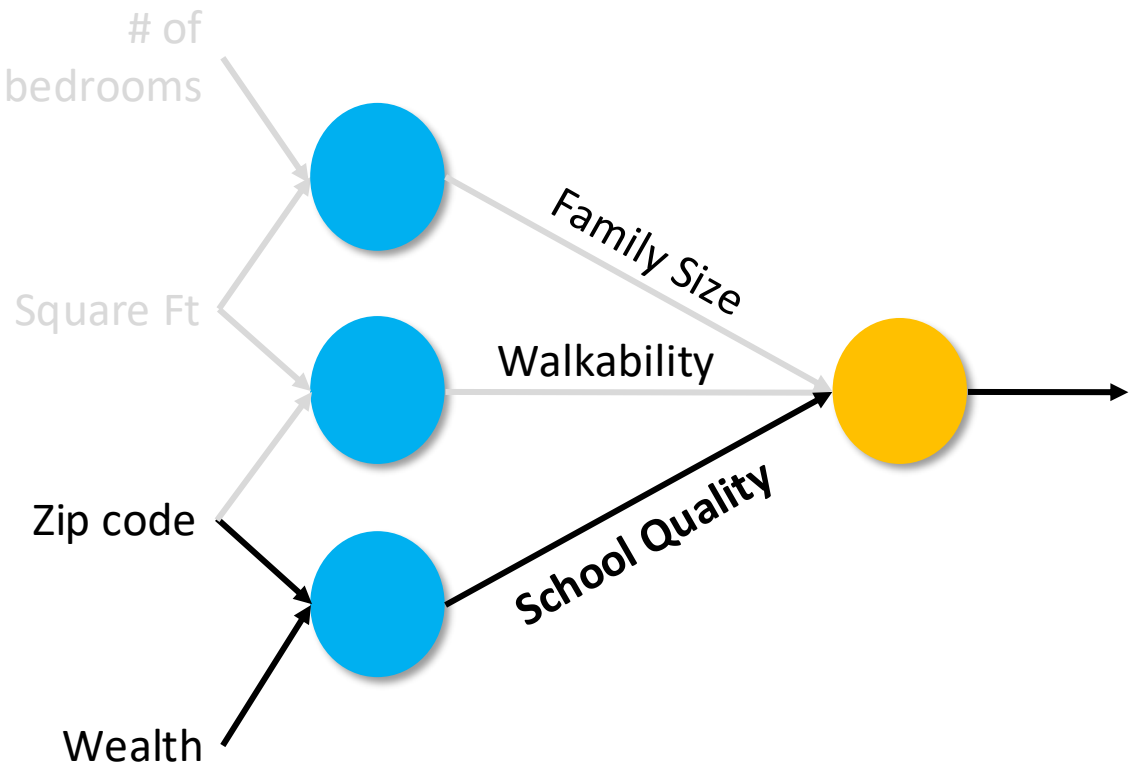
Machine Learning: *Neural Networks*

Prediction



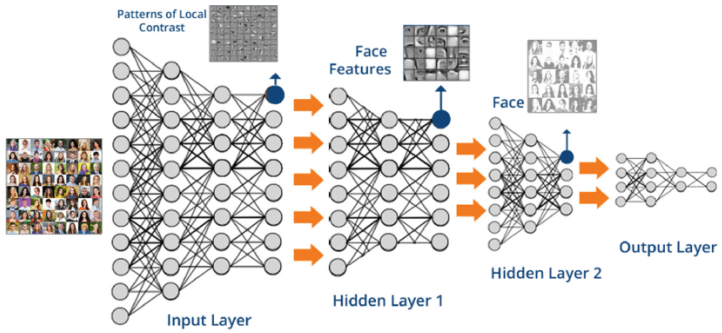
Machine Learning: *Neural Networks*

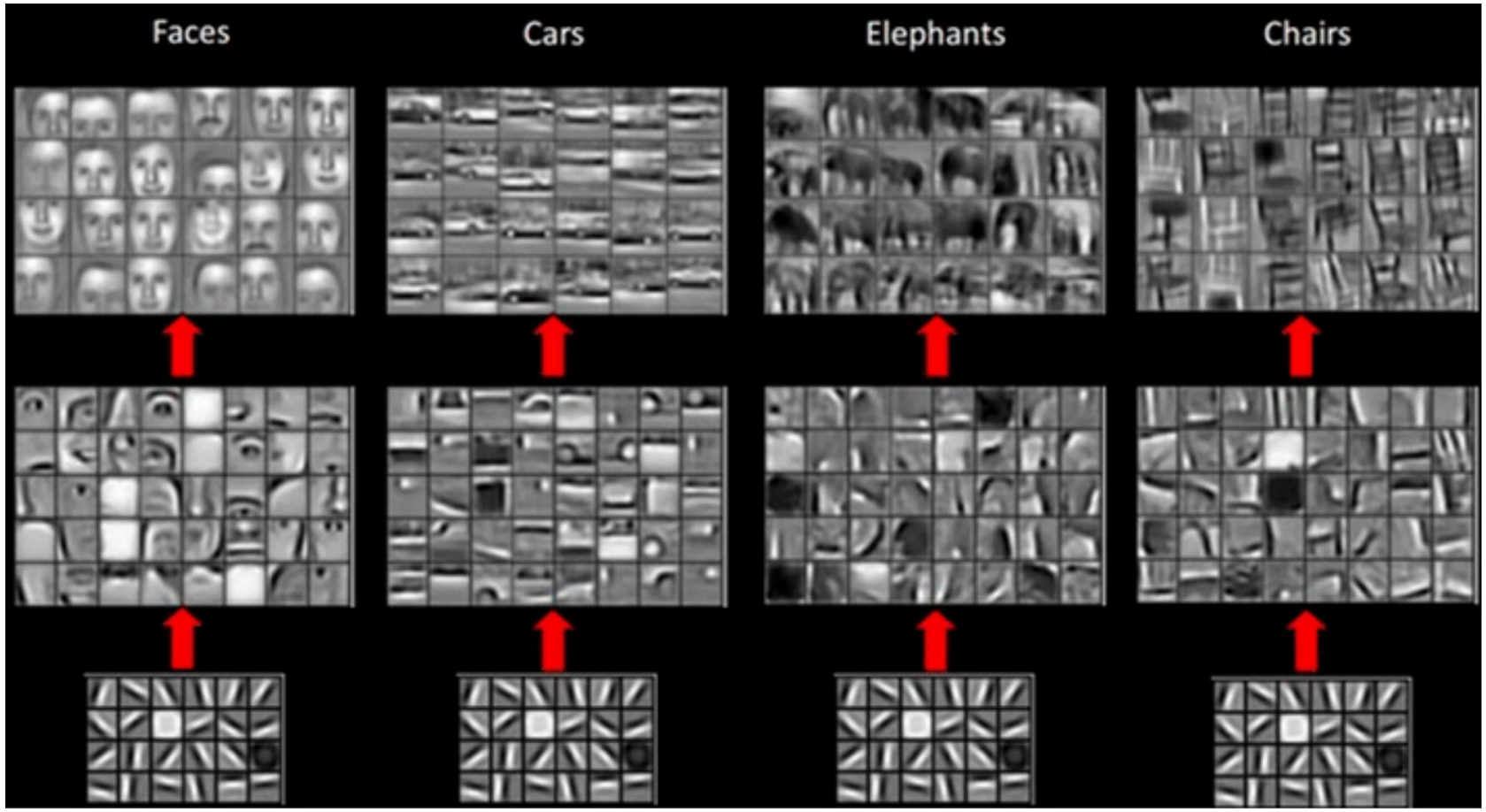
Prediction



Machine Learning: *Neural Networks*

Prediction

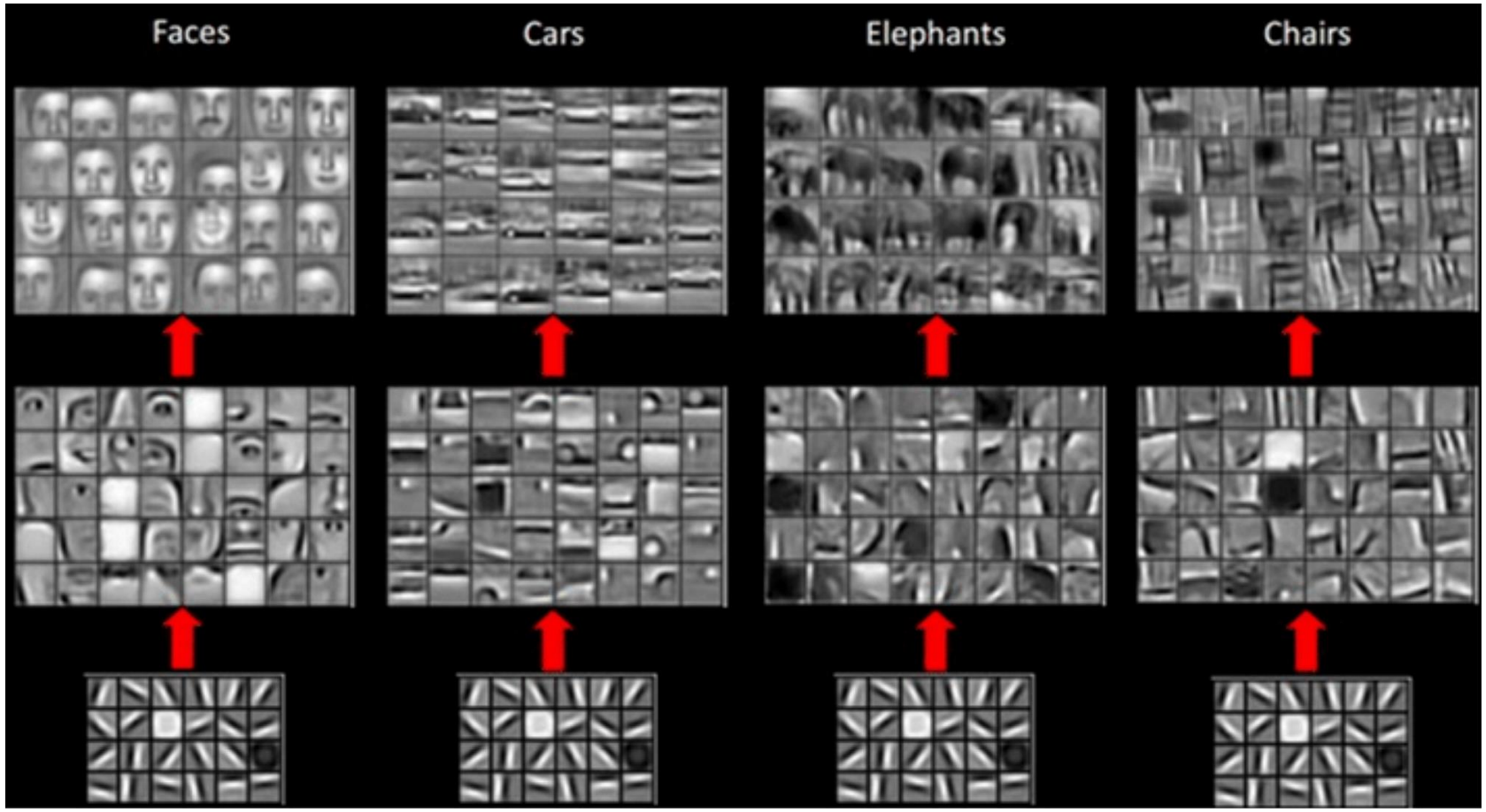




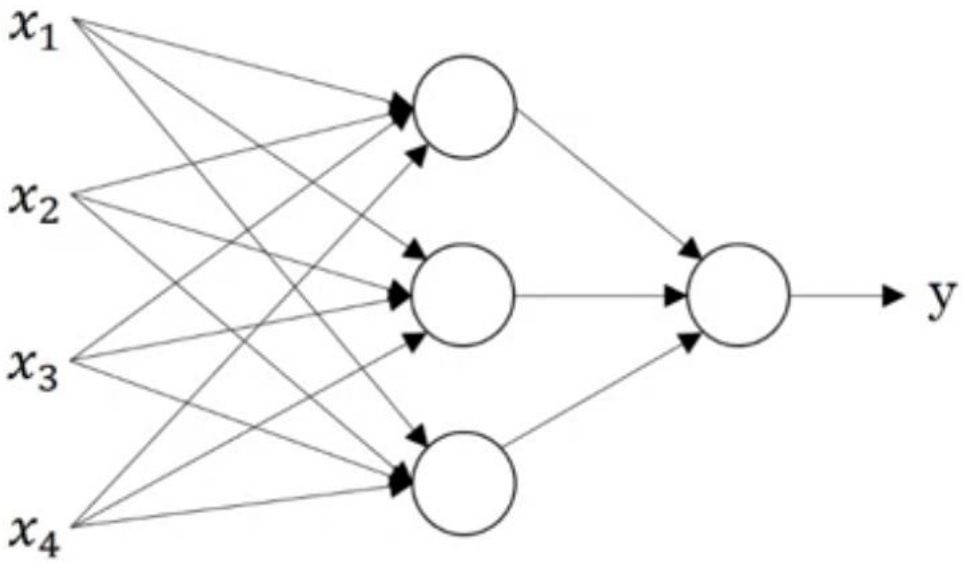
Machine Learning: *Neural Networks*

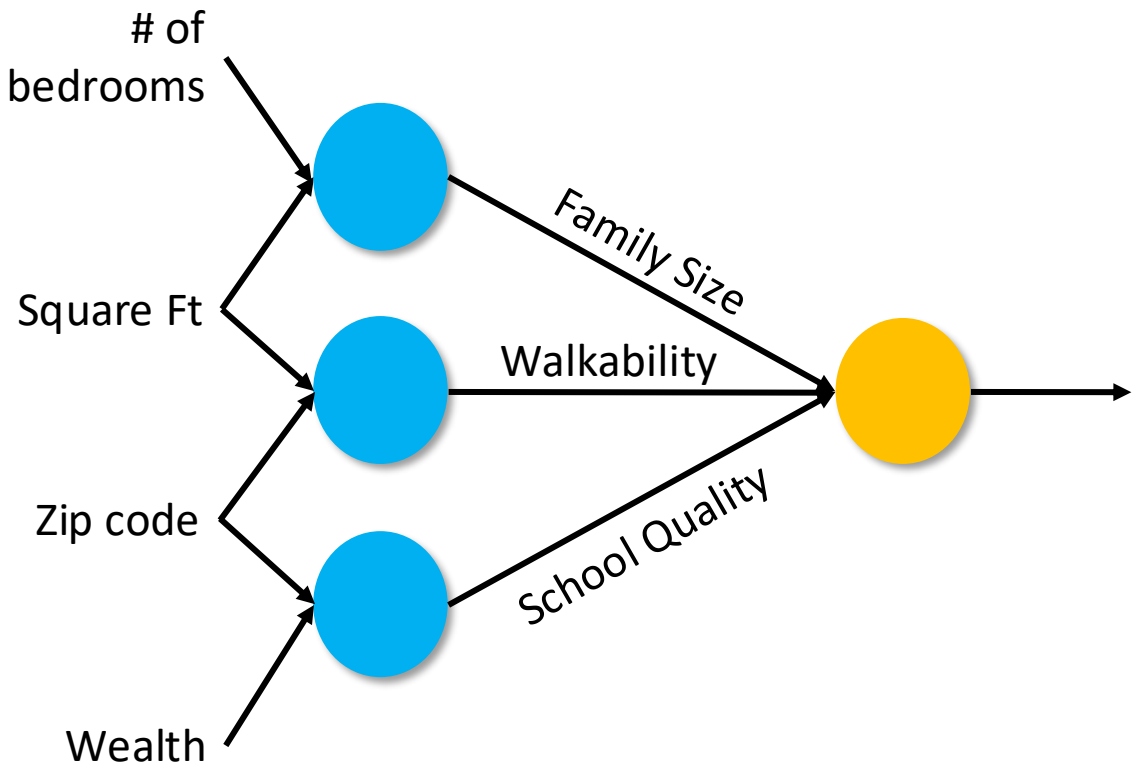
Prediction

DEMO



Machine Learning: *Neural Networks*

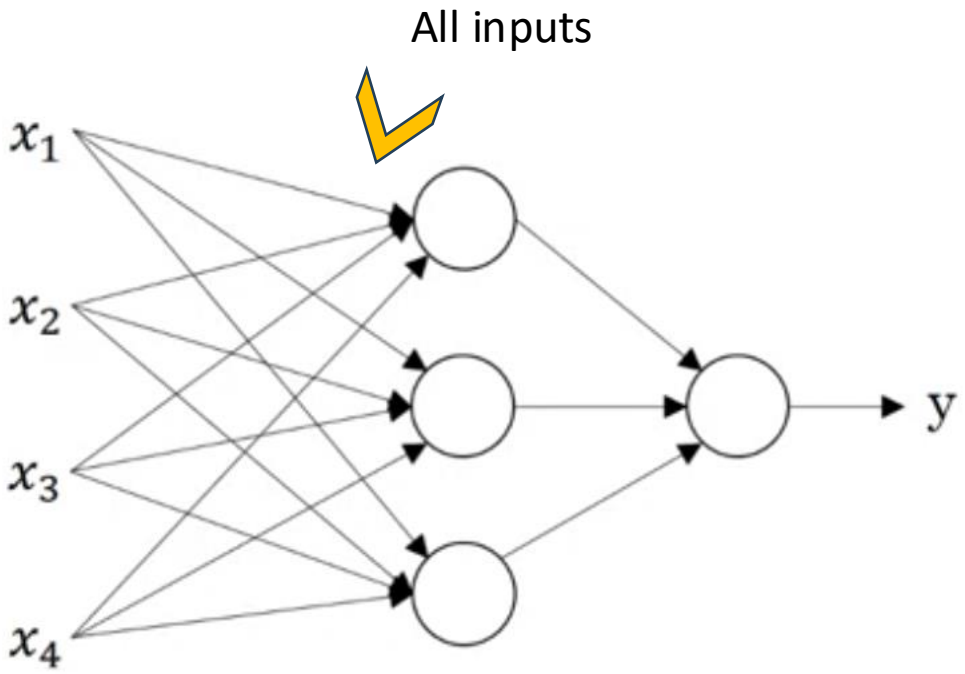




Machine Learning: *Neural Networks*

Feed Forward NN

Instead of we define what inputs need to be mixed together, give all to the nodes and it will figure out on its own based on the training data.

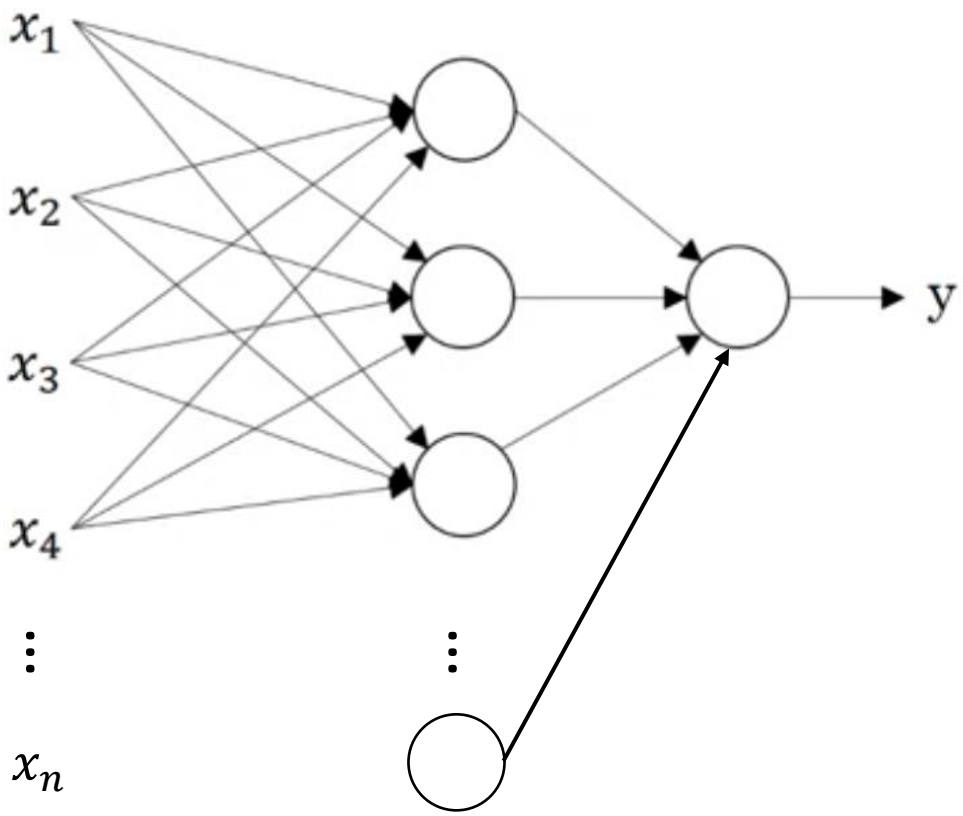


Machine Learning: *Neural Networks*

Feed Forward NN

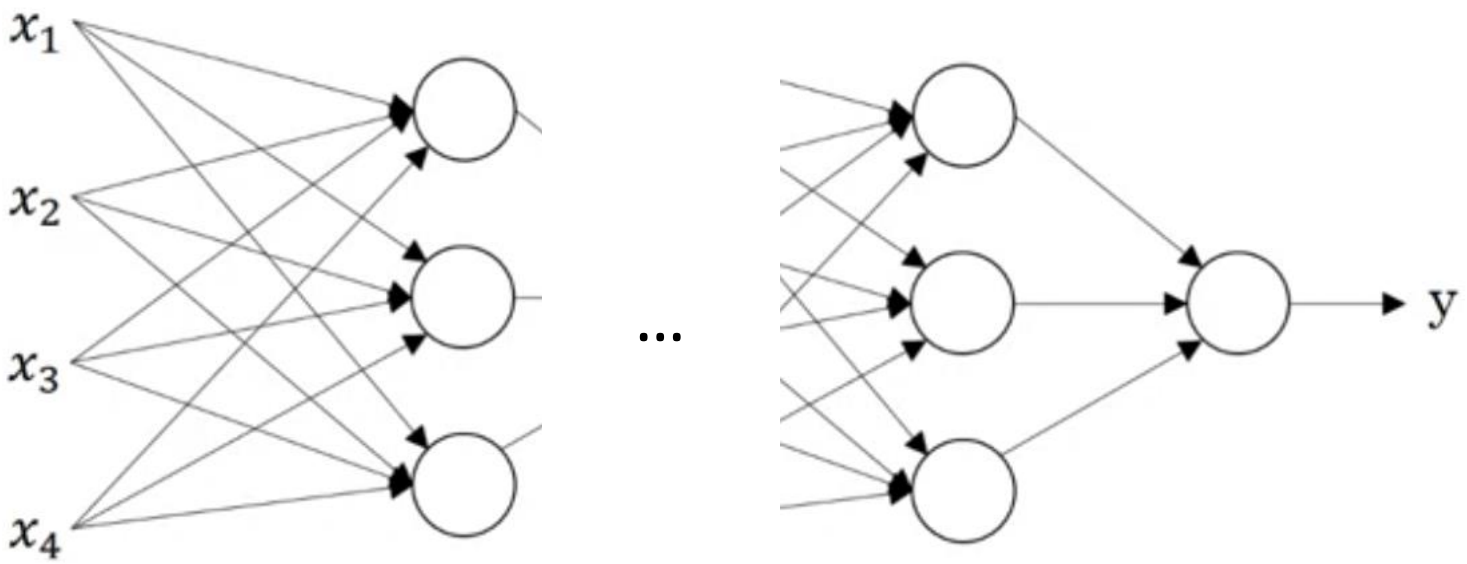
Increase input/feature size





Machine Learning: *Neural Networks*

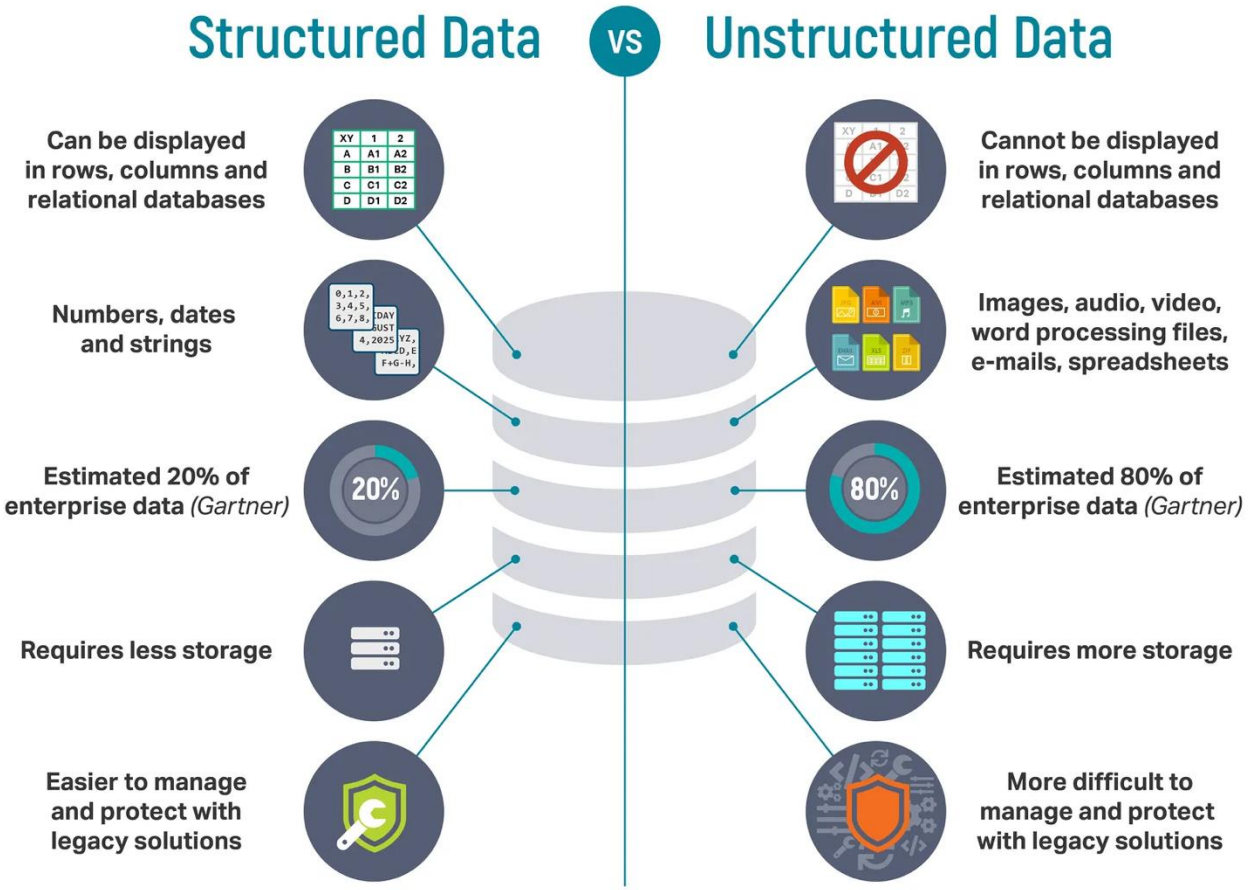
Feed Forward NN



Increase Model Complexity

Machine Learning: *Neural Networks*

Structured vs. Unstructured Data

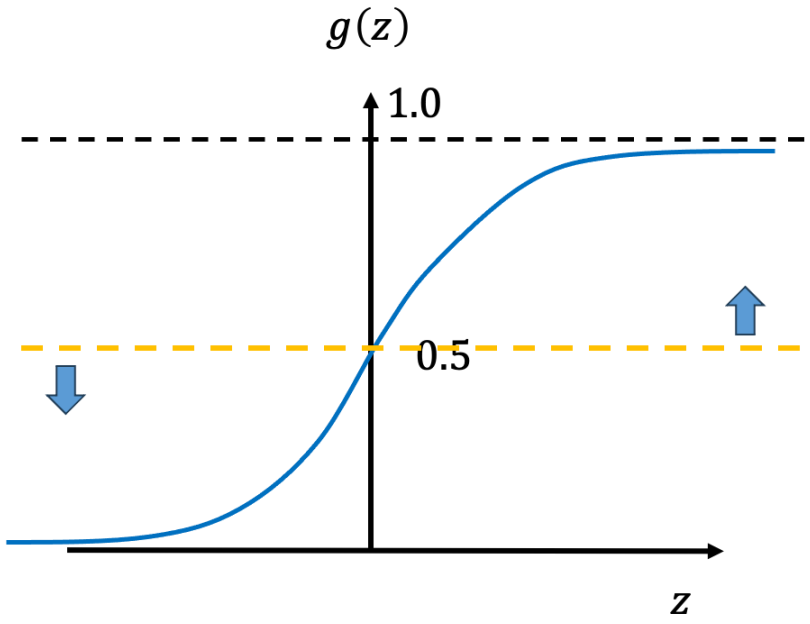


https://lawtomated.com/structured-data-vs-unstructured-data-what-are-they-and-why-care/

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

$$f_{\vec{w},b}(\vec{x}) = P(y = 1|\vec{x}; \vec{w}, b)$$



$$f_{\vec{w},b}(\vec{x}) = g(z)$$

$$z = \vec{w}.\vec{x} + b$$

$$g(z) = \frac{1}{1 + e^{-z}}, 0 < g(z) < 1$$

$$f_{\vec{w},b}(\vec{x}) = g(\underbrace{\vec{w}.\vec{x} + b}_{z}) = \frac{1}{1 + e^{-(\vec{w}.\vec{x}+b)}}$$

Machine Learning: *Neural Networks*

(reminder)

Logistic Regression as a Neural Network

Cost Function

Given training samples: $\{(x^1, y^1), (x^2, y^2), \dots, (x^m, y^m)\}$, want $\hat{y}^{(i)} \approx y^{(i)}$

$$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) = -y^{(i)} \log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right) - \left(1 - y^{(i)}\right) \log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right)$$

$$L\left(\hat{y}^{(i)}, y^{(i)}\right) = -y^{(i)} \log\left(\hat{y}^{(i)}\right) - \left(1 - y^{(i)}\right) \log\left(1 - \hat{y}^{(i)}\right)$$

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} L\left(\hat{y}^{(i)}, y^{(i)}\right) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log\left(\hat{y}^{(i)}\right) + \left(1 - y^{(i)}\right) \log\left(1 - \hat{y}^{(i)}\right)\right]$$

Machine Learning: *Neural Networks*

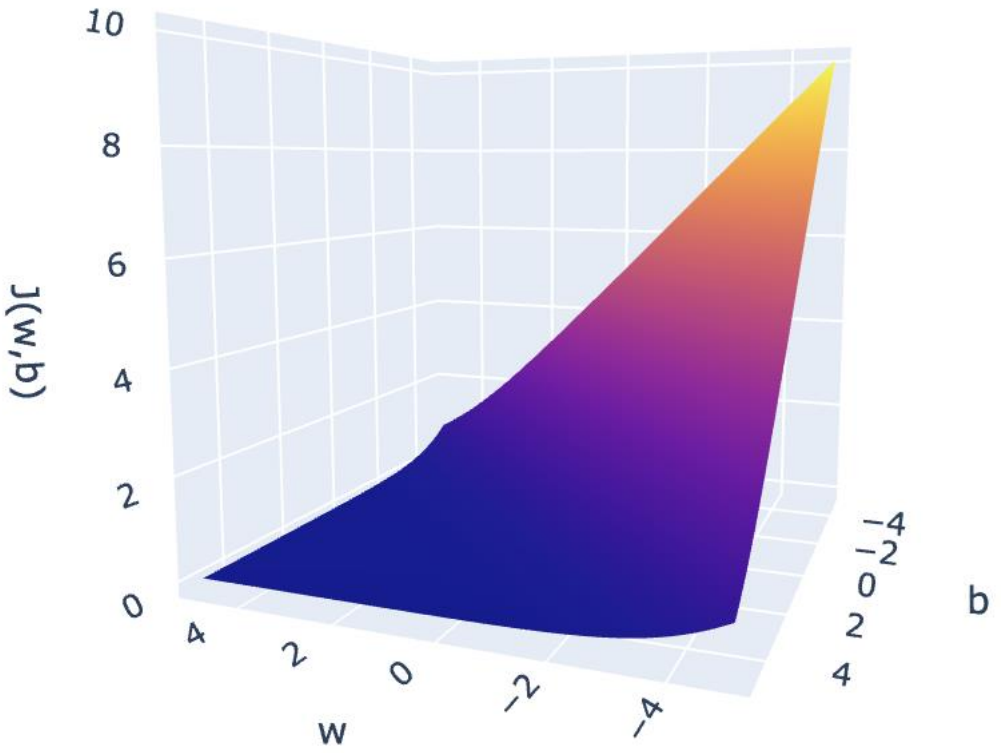
(reminder)

Logistic Regression as a Neural Network

Gradient Descent

Find w, b to minimize $J(\vec{w}, b)$

$$J(\vec{w}, b) = \frac{1}{m}\sum_{i=1}^{m} L\left(\hat{y}^{(i)}, y^{(i)}\right)$$



Machine Learning: *Neural Networks*

(reminder)

Logistic Regression as a Neural Network

Gradient Descent

Repeat Until Convergence:

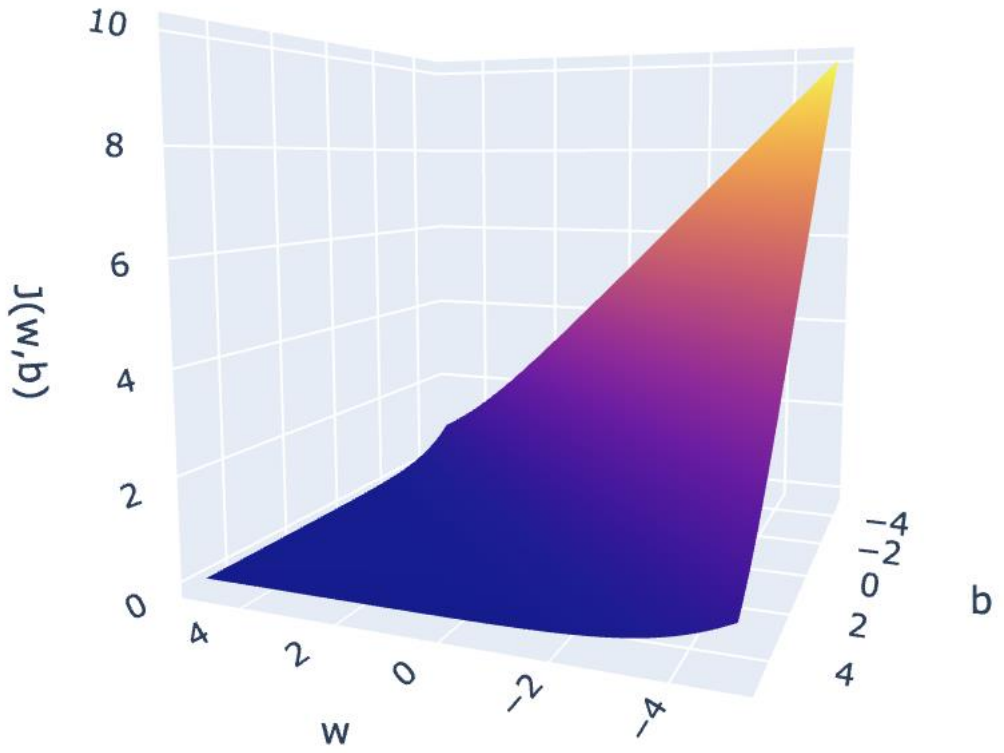
{

$$w_j = w_j - \alpha \frac{\partial J(\vec{w}, b)}{\partial w_j}$$

$$b = b - \alpha \frac{\partial J(\vec{w}, b)}{\partial b}$$

}

Simultaneously update $\vec{w}, b$



Machine Learning: *Neural Networks*

(reminder)

Logistic Regression as a Neural Network

Gradient Descent

Repeat Until Convergence:

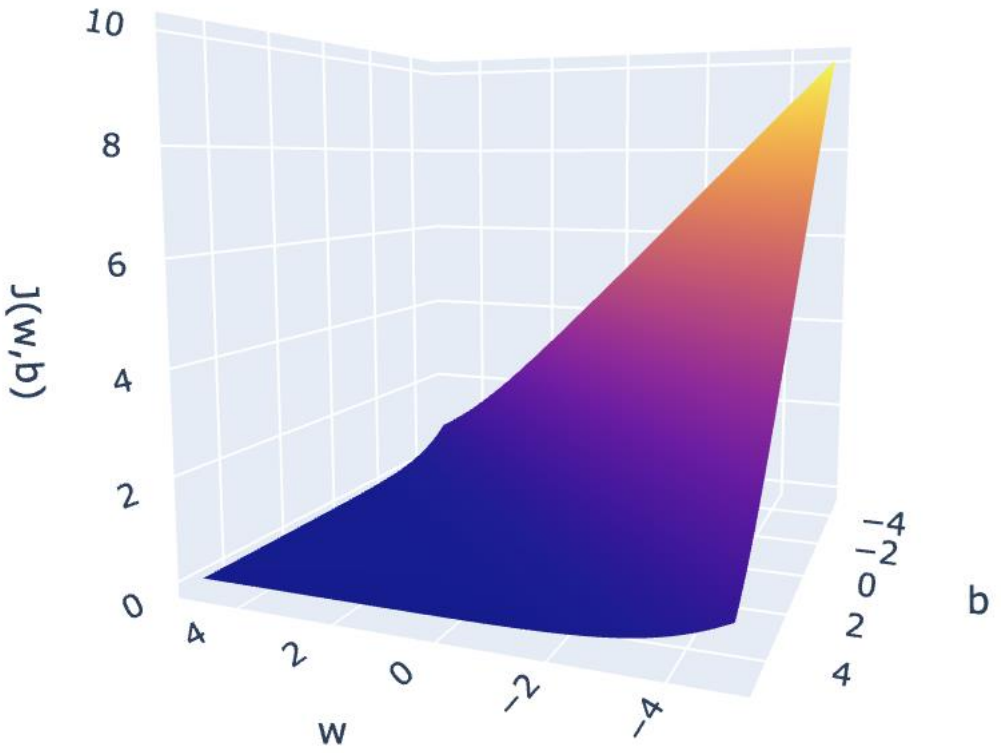
{

$$w_j = w_j - \alpha \frac{\partial J(\vec{w}, b)}{\partial w_j}$$

$$b = b - \alpha \frac{\partial J(\vec{w}, b)}{\partial b}$$

}

Simultaneously update $\vec{w}, b$



Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent

Reminder (chain rule)

$$\frac{\partial J(z(w))}{\partial w} = \frac{\partial J(z(w))}{\partial z} \cdot \frac{\partial z(w)}{\partial w}$$

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent

$$z = w.x + b$$
$$\hat{y} = g(z)$$
$$L(\hat{y}, y) = -y\log(\hat{y}) - (1-y)\log(1-\hat{y})$$

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent

$z = w.x + b$

$\hat{y} = a = g(z)$

$L(a, y) = -y \log(a) - (1 - y) \log(1 - a)$

Machine Learning: *Neural Networks*

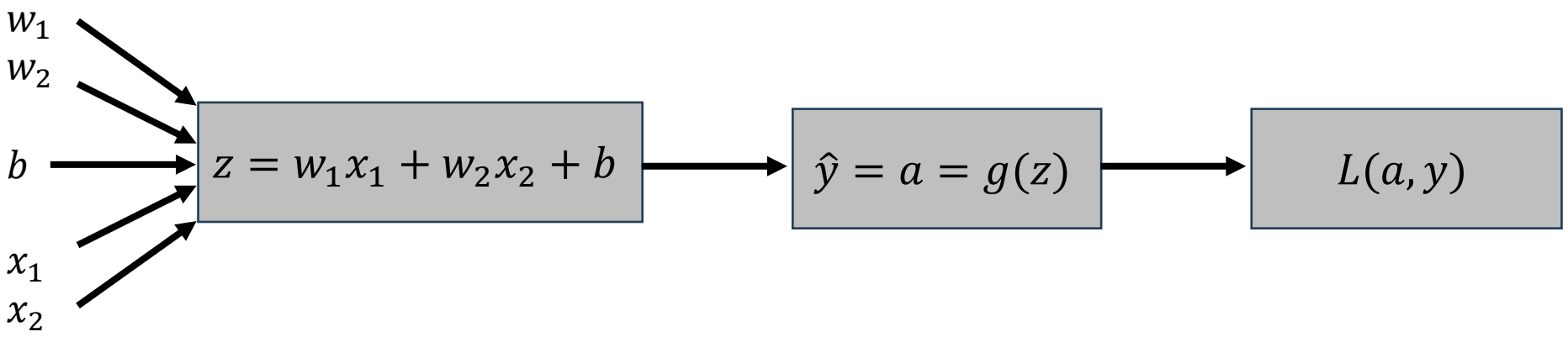
Logistic Regression as a Neural Network

Gradient Descent

$z = w.x + b$

$\hat{y} = a = g(z)$

$L(a, y) = -y \log(a) - (1 - y) \log(1 - a)$



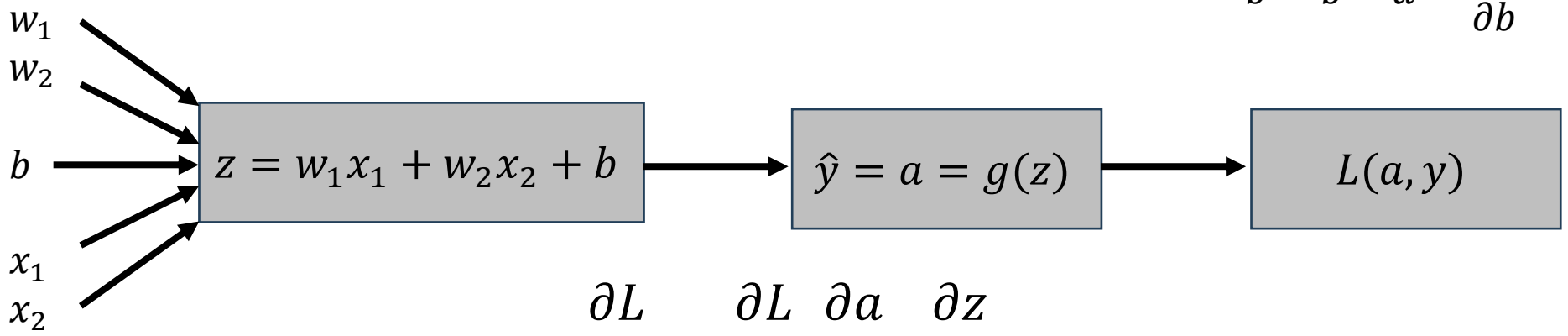
Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent

$$w_j = w_j - \alpha \frac{\partial J(\vec{w}, b)}{\partial w_j}$$

$$b = b - \alpha \frac{\partial J(\vec{w}, b)}{\partial b}$$



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_2}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial b}$$

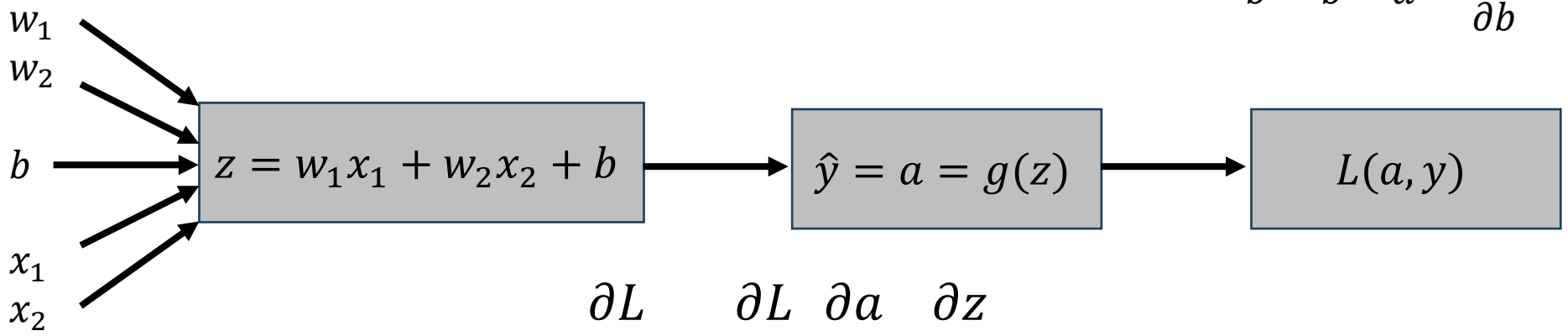
Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent

$$w_j = w_j - \alpha \frac{\partial J(\vec{w}, b)}{\partial w_j}$$

$$b = b - \alpha \frac{\partial J(\vec{w}, b)}{\partial b}$$



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_2}$$

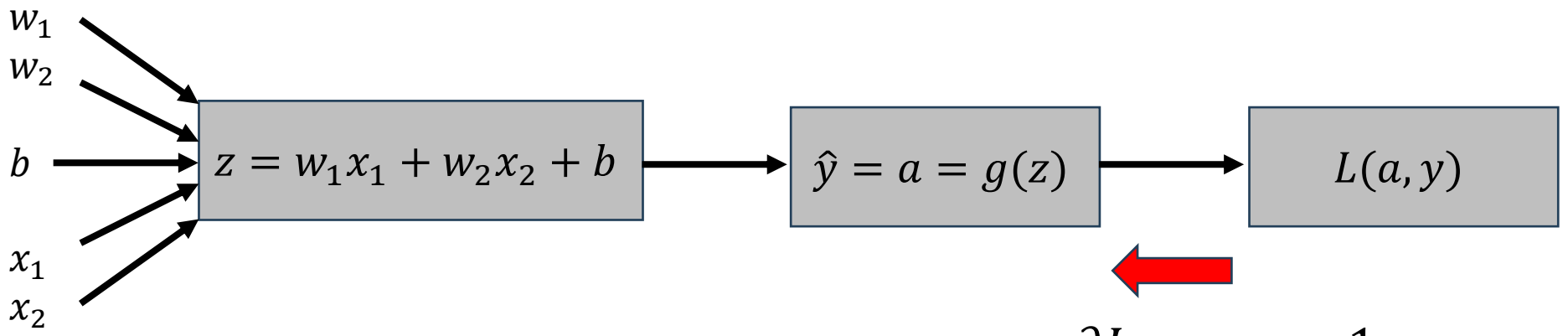
$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial b}$$

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$



$$\frac{\partial L}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a}$$

$$z = w.x + b$$

$$a = g(z) = \frac{1}{1+e^{-z}}$$

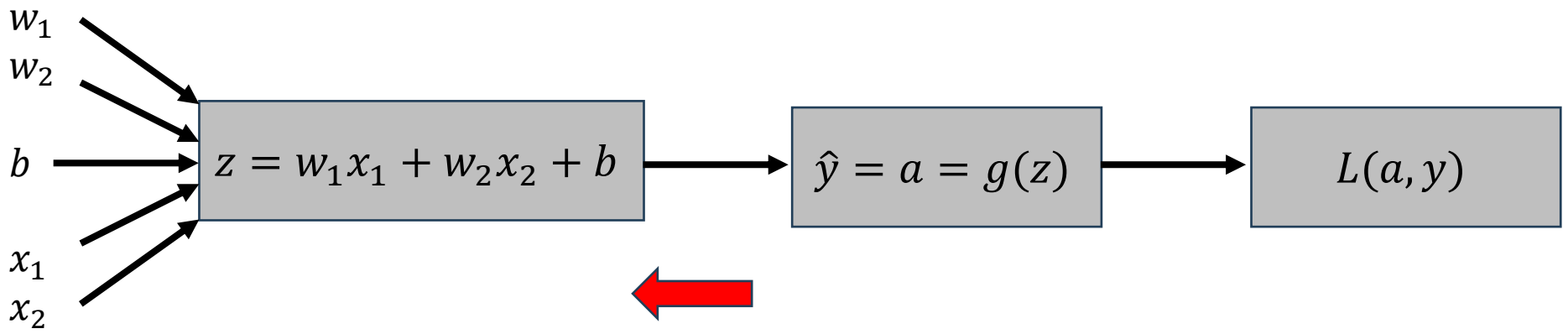
$$L(a, y) = -y \log(a) - (1-y) \log(1-a)$$

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$



$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z}$$
$$= \left(-\frac{y}{a} + \frac{1-y}{1-a}\right) . [a(1-a)]$$
$$= a - y$$

$z = w . x + b$

$a = g(z) = \frac{1}{1 + e^{-z}}$

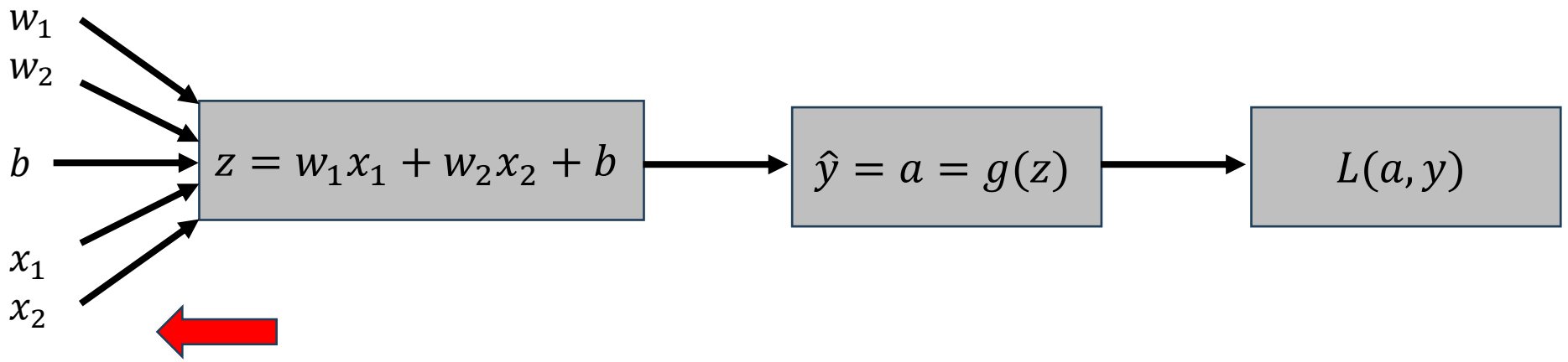
$L(a, y) = -y \log(a) - (1 - y) \log(1 - a)$

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$
$$= x_1 \cdot \frac{\partial L}{\partial z}$$
$$= x_1 \cdot (a - y)$$

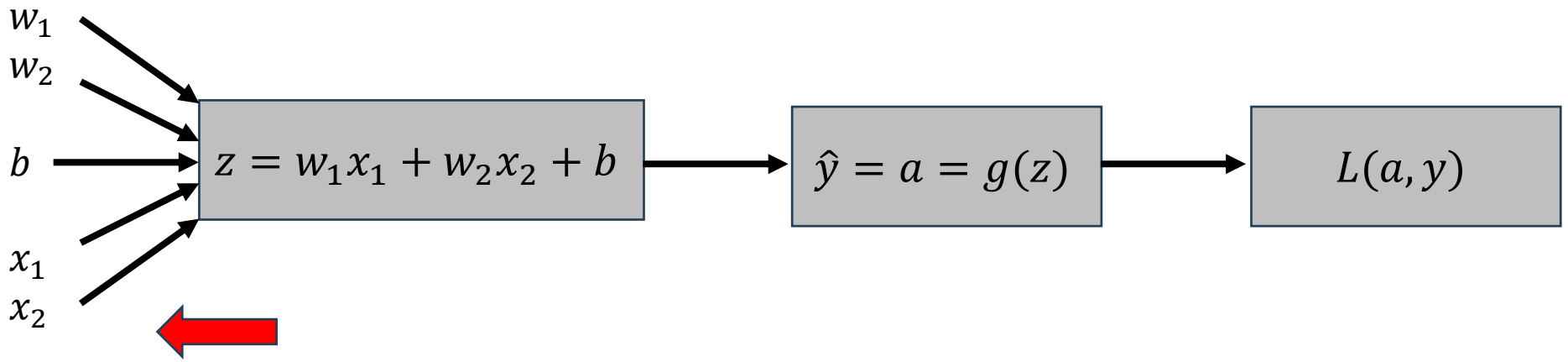
$$z = w . x + b$$
$$a = g(z) = \frac{1}{1 + e^{-z}}$$
$$L(a, y) = -y \log(a) - (1 - y) \log(1 - a)$$

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$

$$= x_1 . \frac{\partial L}{\partial z}$$

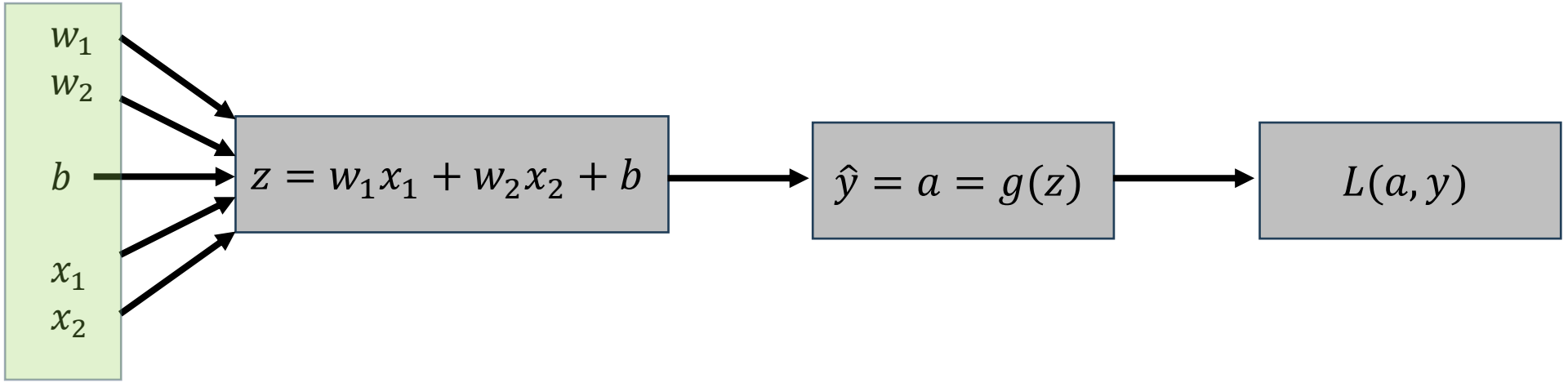
$$= x_1 . (a - y)$$

$$w_1 = w_1 - \alpha \frac{\partial L}{\partial w_1}$$

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

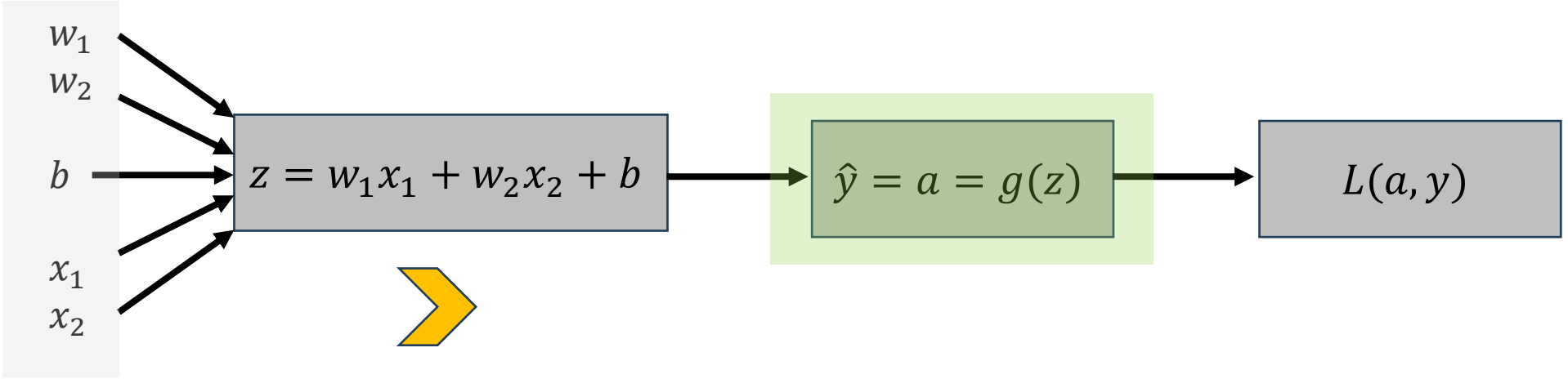
Gradient Descent: *Forward Pass*



Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

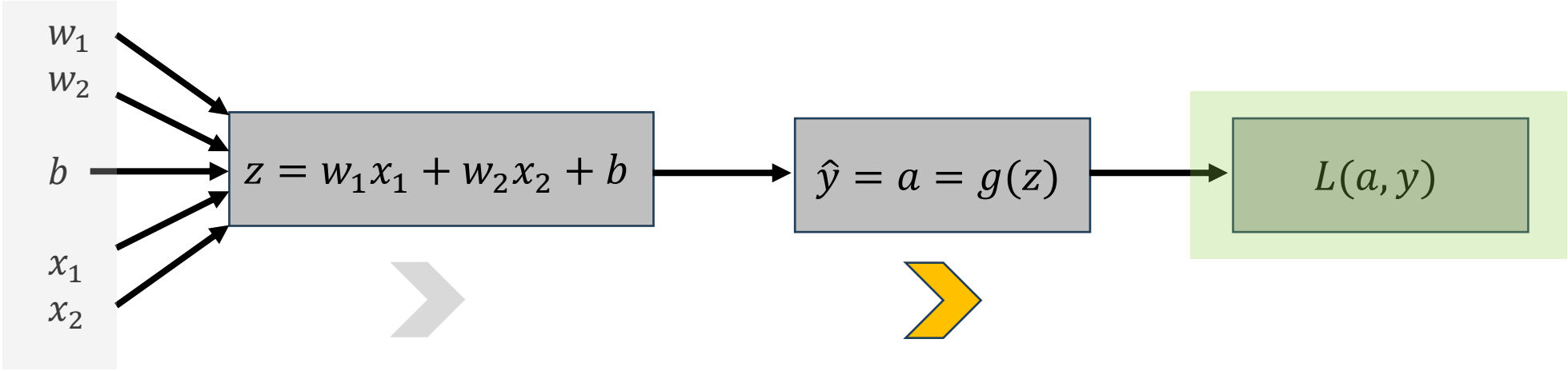
Gradient Descent: ***Forward Pass***



Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

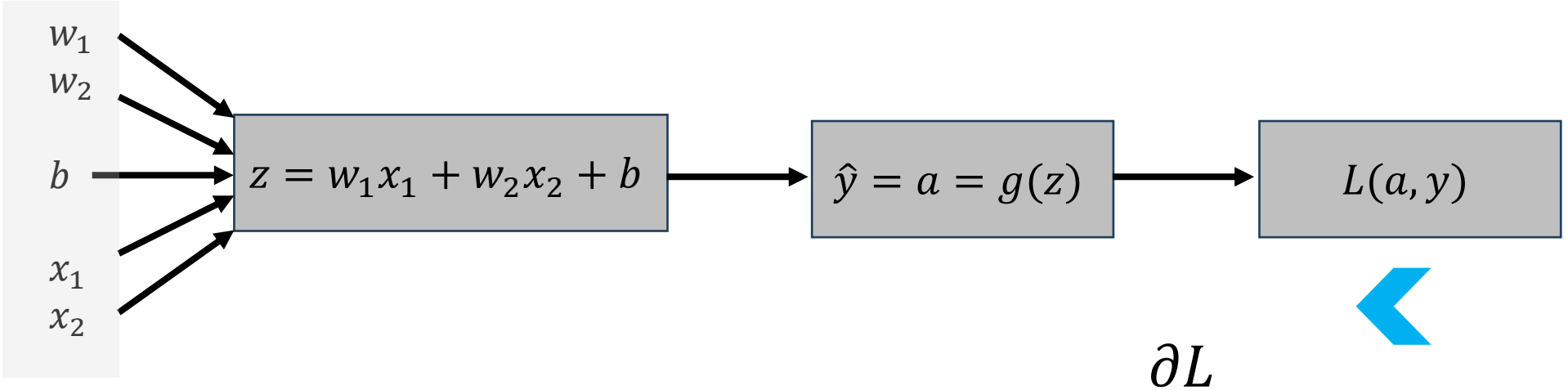
Gradient Descent: *Forward Pass*



Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

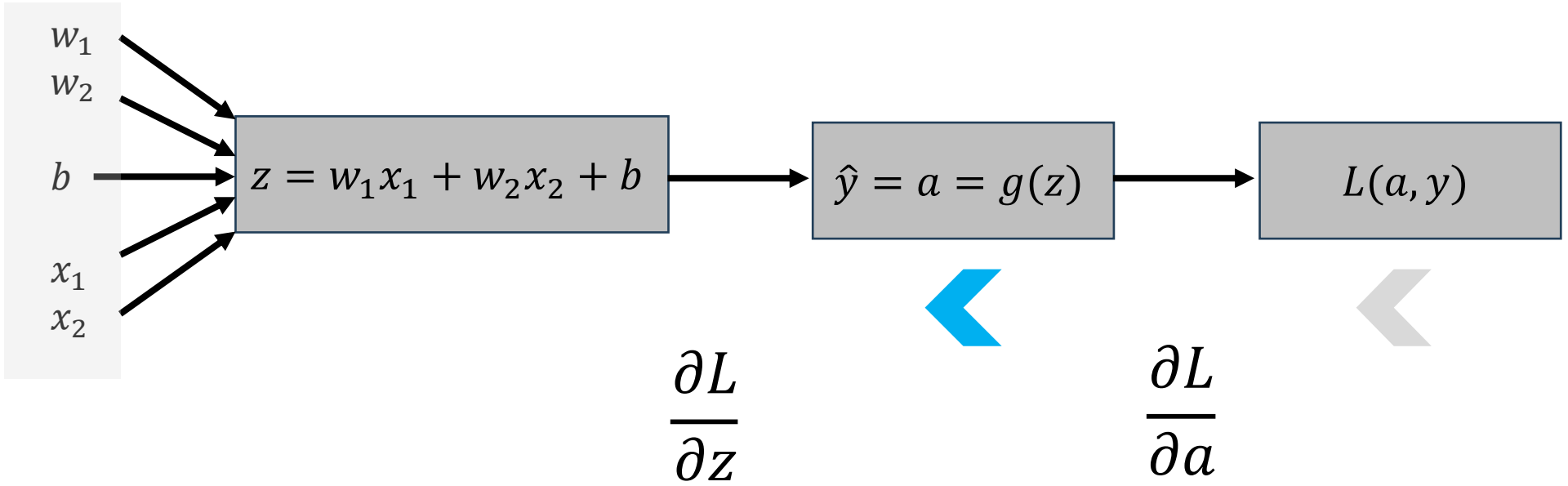
Gradient Descent: Backward Pass



Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

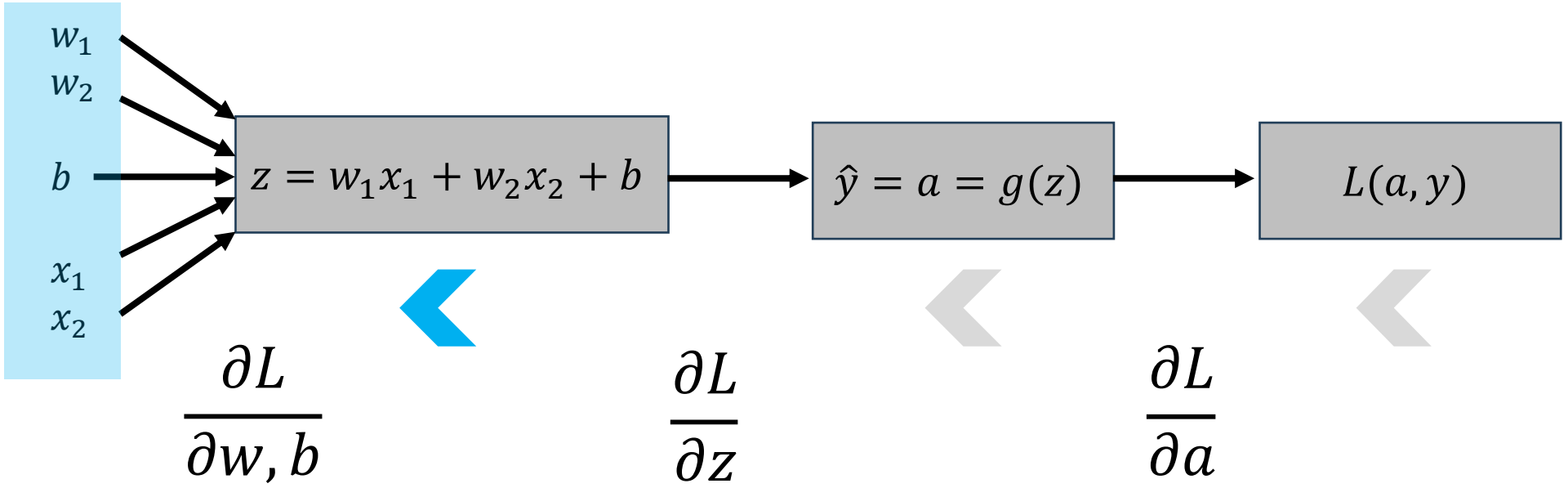
Gradient Descent: *Backward Pass*



Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent: *Backward Pass*



Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent over n features and m training samples

For a given feature j

$for\ i = 1{:}\,m$

$$z^{(i)} = w^T x^{(i)} + b$$

$$a^{(i)} = g\left(z^{(i)}\right)$$

$$J \mathrel{+}= -[y^{(i)} \log\left(a^{(i)}\right) + \left(1 - y^{(i)}\right) \log(1 - a^{(i)})]$$

$$dz^{(i)} = a^{(i)} - y^{(i)}$$

$$dw_j \mathrel{+}= x_j^{(i)} dz^{(i)}$$

$$db \mathrel{+}= dz^{(i)}$$

$$J = J/m$$

$$dw_j = dw_j/m$$

$$db = db/m$$

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent over n features and m training samples

For a given feature j

$for\ i = 1{:}\,m$

$$z^{(i)} = w^T x^{(i)} + b$$
$$a^{(i)} = g\left(z^{(i)}\right)$$
$$J \mathrel{+}= -[y^{(i)} \log\left(a^{(i)}\right) + \left(1 - y^{(i)}\right) \log(1 - a^{(i)})]$$
$$dz^{(i)} = a^{(i)} - y^{(i)}$$
$$dw_j \mathrel{+}= x_j^{(i)} dz^{(i)}$$
$$db \mathrel{+}= dz^{(i)}$$

$$J = J/m$$
$$dw_j = dw_j/m$$
$$db = db/m$$

$$w_j = w_j - \alpha dw_j$$
$$b = b - \alpha db$$

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent over n features and m training samples

Vectorization

$$z = w.x + b$$
$$\hat{y} = a = g(z)$$

$$X = [x^{(1)} \cdots x^{(m)}]$$

$$Z = \left[z^{(1)} \cdots z^{(m)}\right] = w^T X + [b \ldots b]_{1\times m}$$

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent over n features and m training samples

Vectorization

$$z = w.x + b$$
$$\hat{y} = a = g(z)$$

$$X = [x^{(1)} \cdots x^{(m)}]$$

$$Z = [z^{(1)} \cdots z^{(m)}] = w^T X + [b \dots b]_{1\times m}$$

Compute all $z^{(i)}$ at the same time

$$A = [a^{(1)} \cdots a^{(m)}] = g(Z)$$

Compute all $a^{(i)}$ at the same time

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent over n features and m training samples

Vectorization

For a given feature j

$for\ i = 1{:}\ m$

$z^{(i)} = w^T x^{(i)} + b$

$a^{(i)} = g(z^{(i)})$

$J\ += -[y^{(i)} \log(a^{(i)}) + (1 - y^{(i)}) \log(1 - a^{(i)})]$

$dz^{(i)} = a^{(i)} - y^{(i)}$

$dw_j\ += x_j^{(i)} dz^{(i)}$

$db\ += dz^{(i)}$

$J = J/m$

$dw_j = dw_j/m$

$db = db/m$

$$Y = \left[y^{(1)} \cdots y^{(m)}\right]$$

$$A = \left[a^{(1)} \cdots a^{(m)}\right]$$

$$dZ = \left[dz^{(1)} \cdots dz^{(m)}\right]$$

$$dZ = A - Y$$

Machine Learning: *Neural Networks*

Logistic Regression as a Neural Network

Gradient Descent over n features and m training samples

Vectorization

For a given feature j

$for\ i = 1{:}\ m$

$$z^{(i)} = w^T x^{(i)} + b$$
$$a^{(i)} = g(z^{(i)})$$
$$J\ +\!= -[y^{(i)} \log(a^{(i)}) + (1 - y^{(i)}) \log(1 - a^{(i)})]$$
$$dz^{(i)} = a^{(i)} - y^{(i)}$$
$$dw_j\ +\!= x_j^{(i)} dz^{(i)}$$
$$db\ +\!= dz^{(i)}$$

$J = J/m$

$dw_j = dw_j/m$

$db = db/m$

$$Y = \left[y^{(1)} \cdots y^{(m)}\right]$$
$$A = \left[a^{(1)} \cdots a^{(m)}\right]$$

$$dZ = [dz^{(1)} \cdots dz^{(m)}]$$

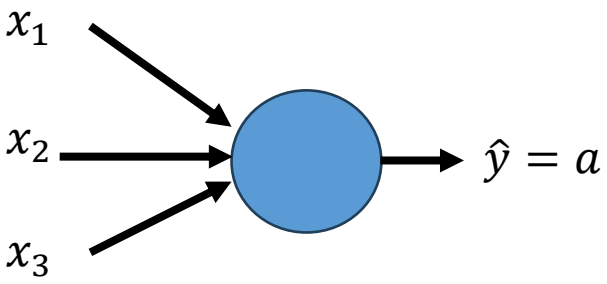
$$dZ = A - Y$$

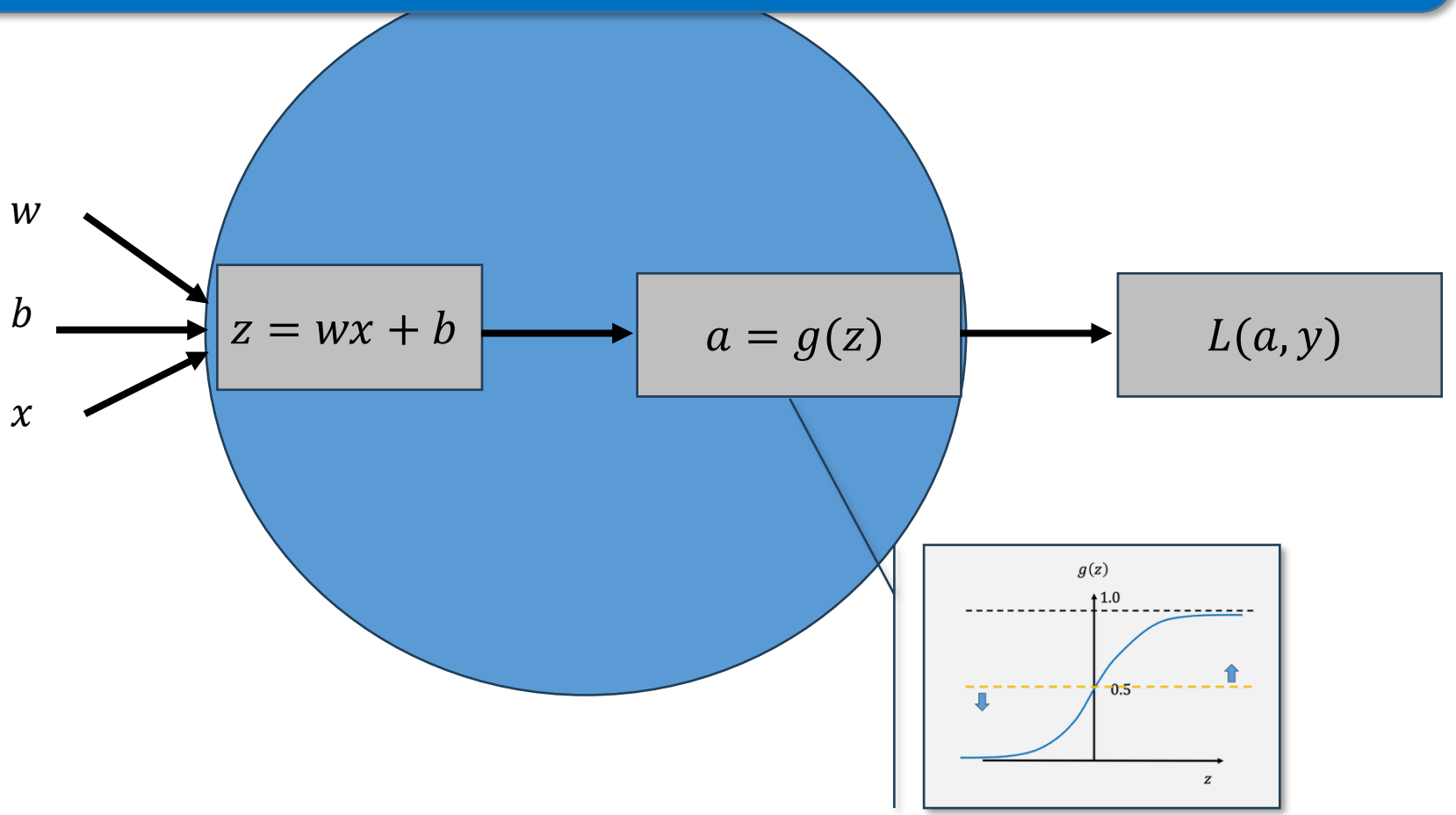


$$dw = \frac{1}{m} X dZ^T$$
$$db = \frac{1}{m} sum(dZ)$$

Machine Learning: *Neural Networks*

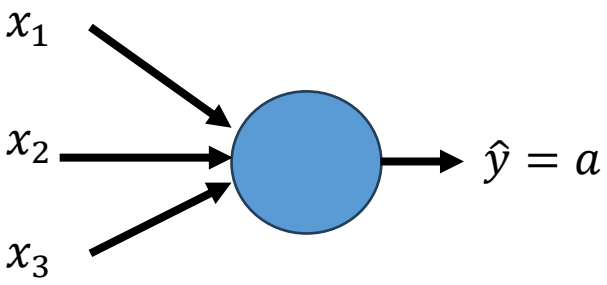
General Formulation

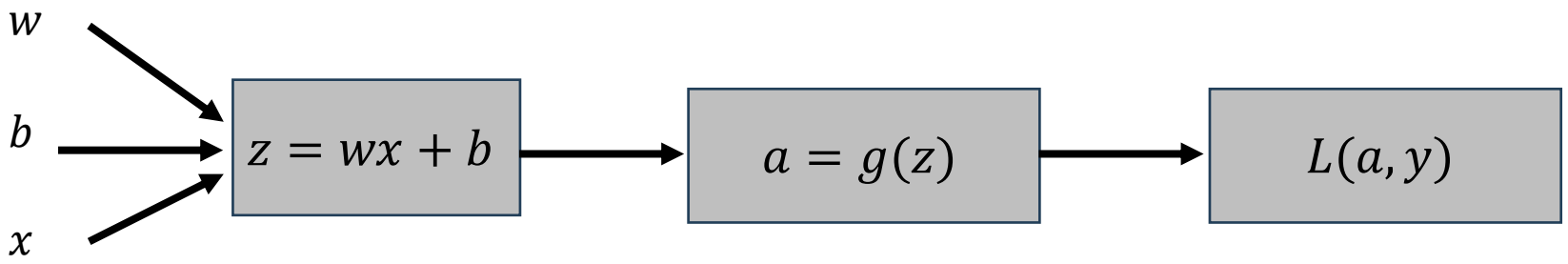


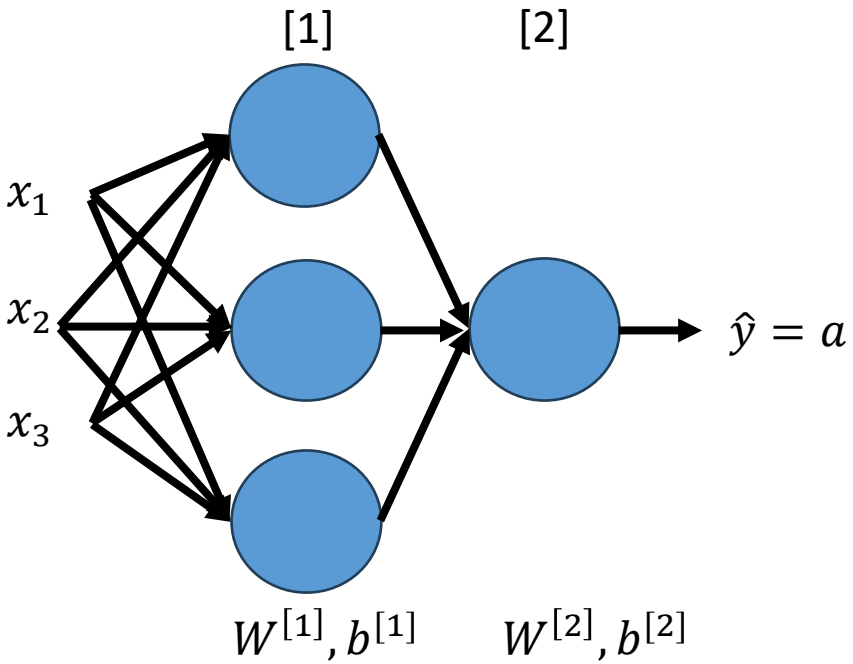


Machine Learning: *Neural Networks*

General Formulation

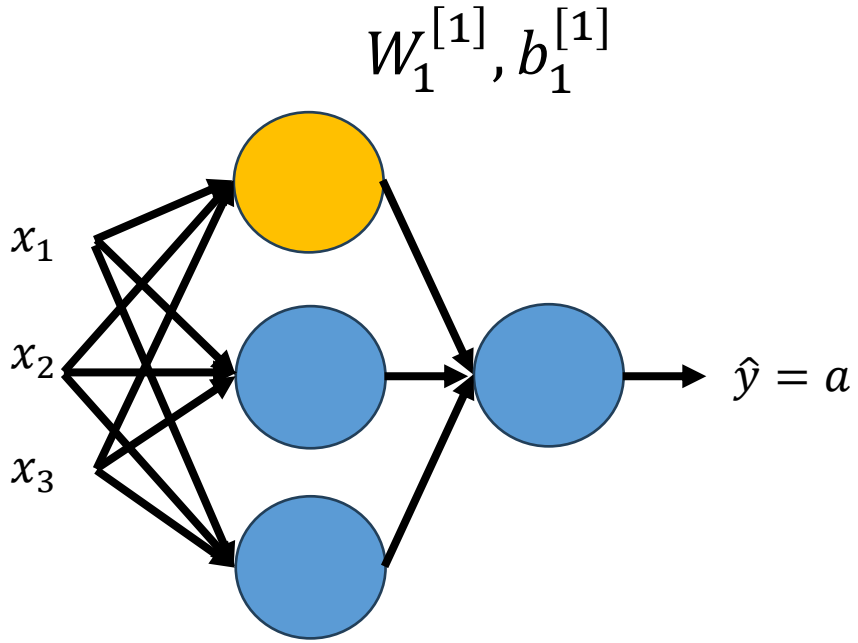






Machine Learning: *Neural Networks*

Notation

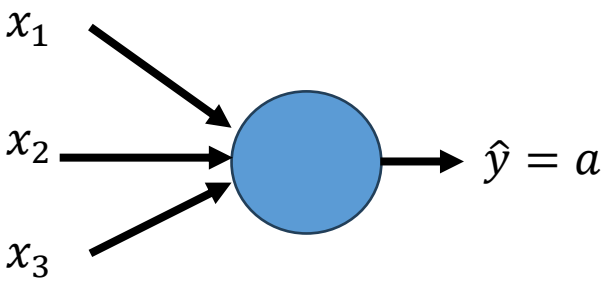


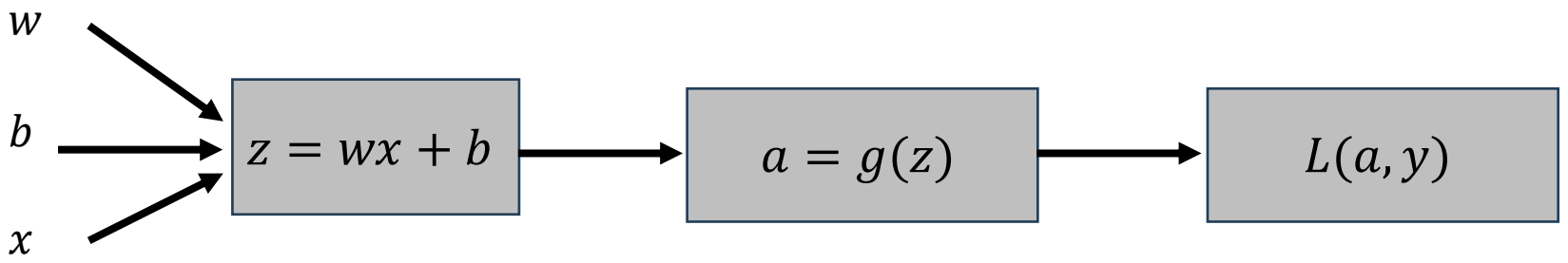
$W_j^{[i]}$ → layer

→ Node within layer i

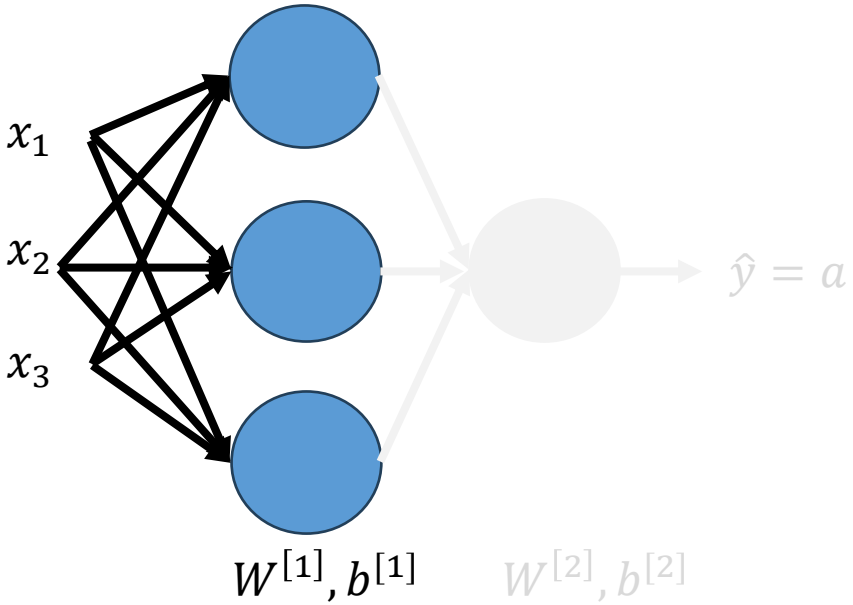
Machine Learning: *Neural Networks*

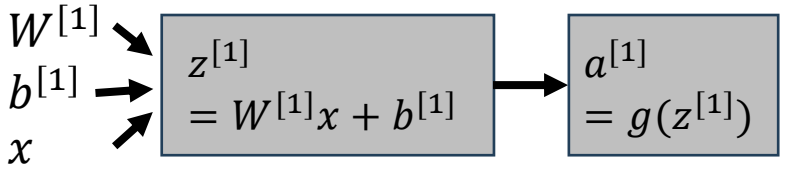
General Formulation





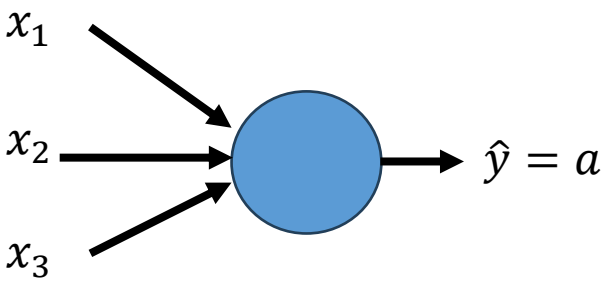
[1] [2]

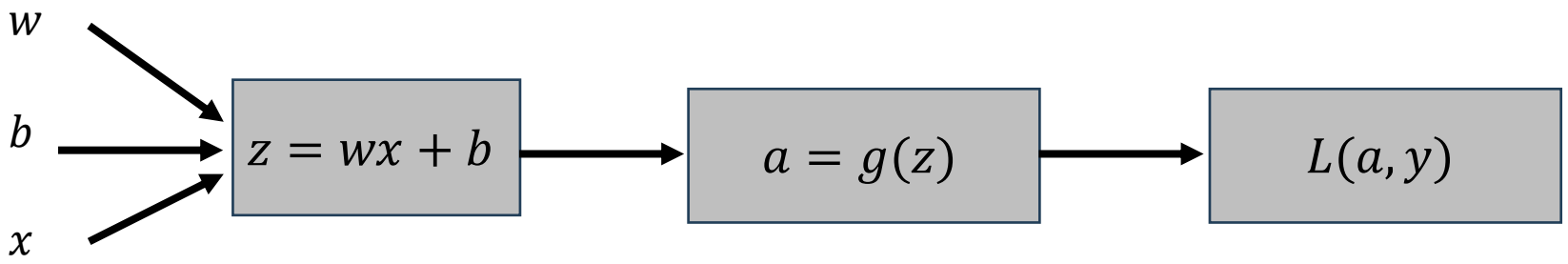




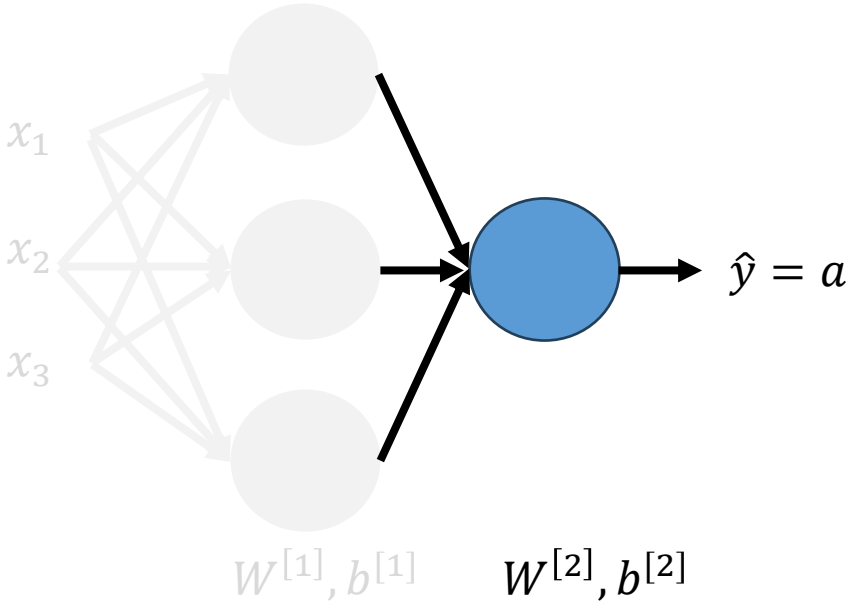
Machine Learning: *Neural Networks*

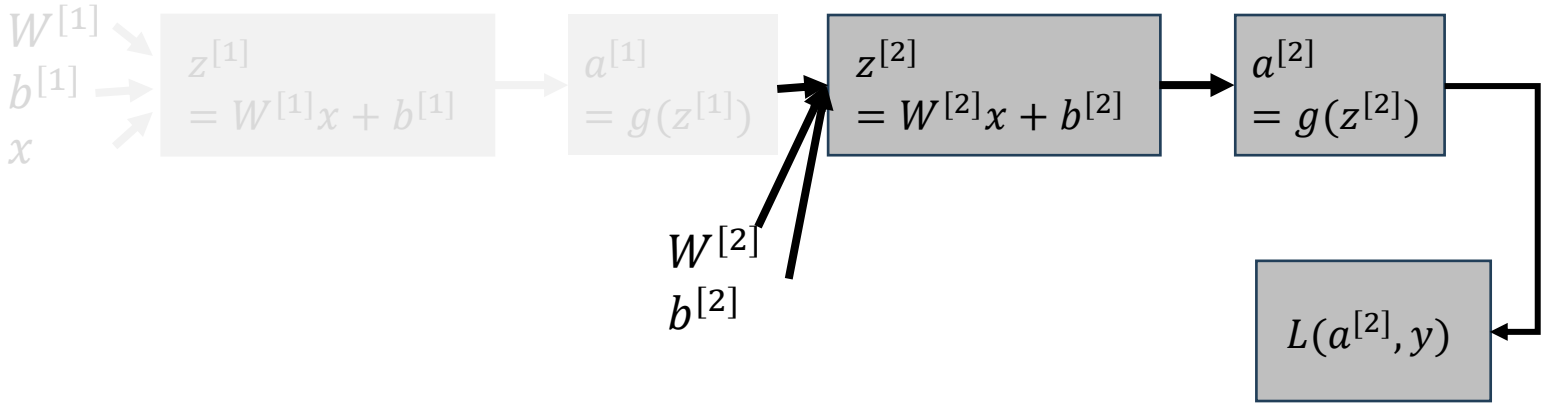
General Formulation





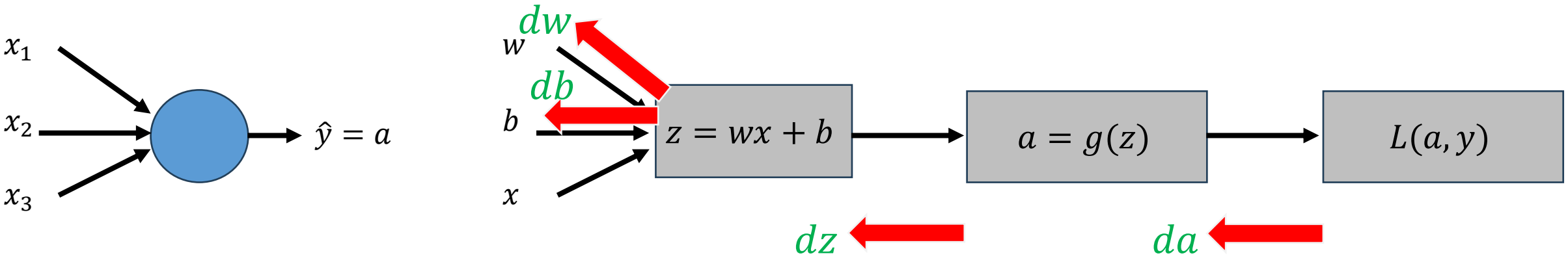
[1] [2]





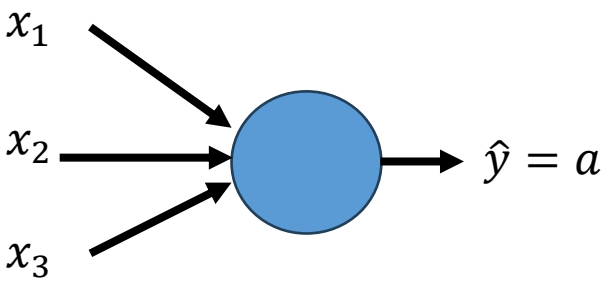
Machine Learning: *Neural Networks*

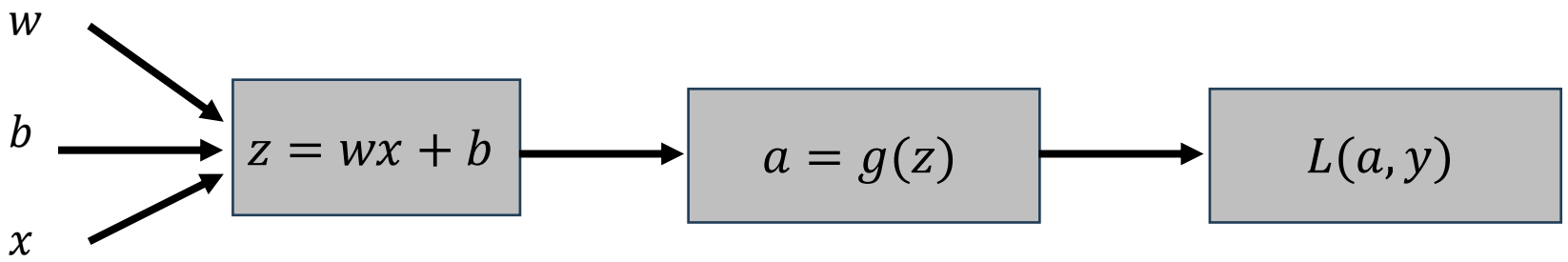
General Formulation: Backward Calculation

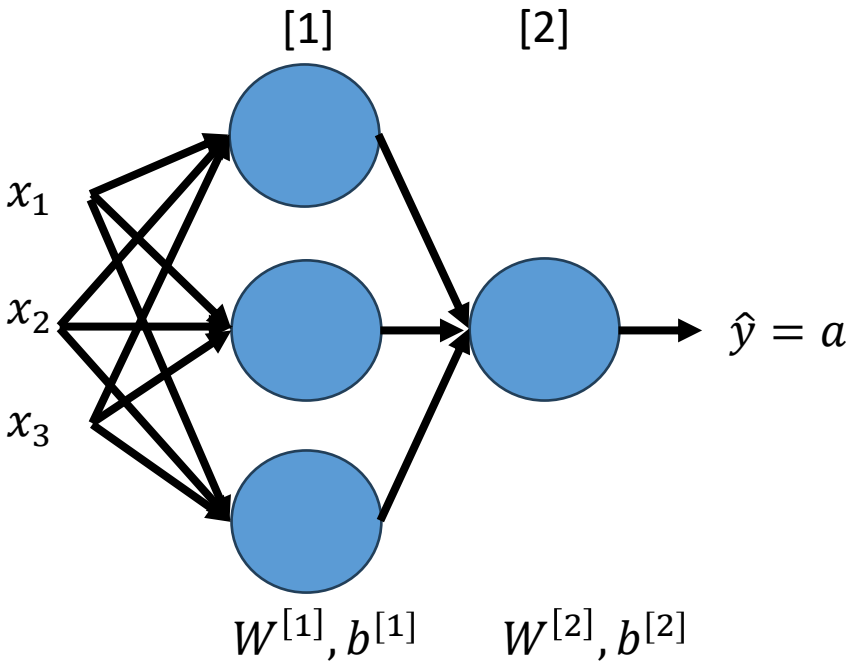


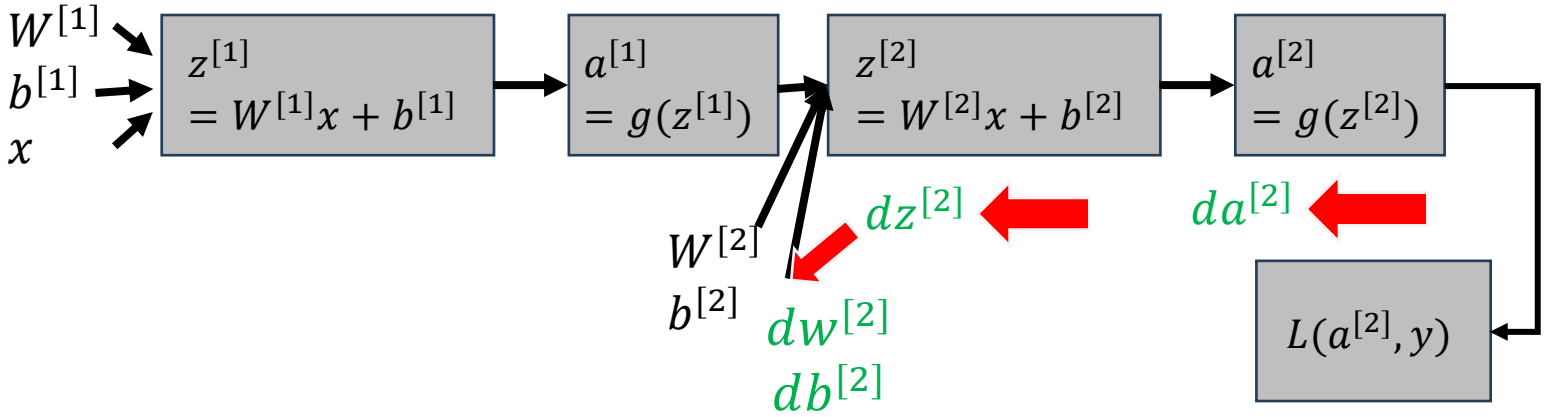
Machine Learning: *Neural Networks*

General Formulation



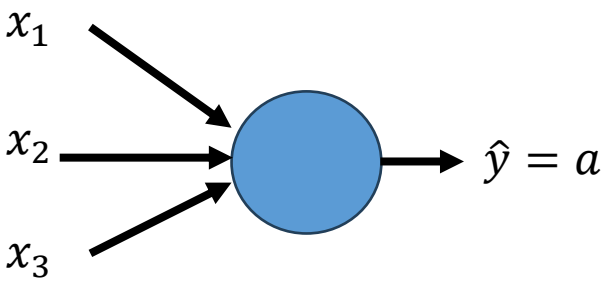


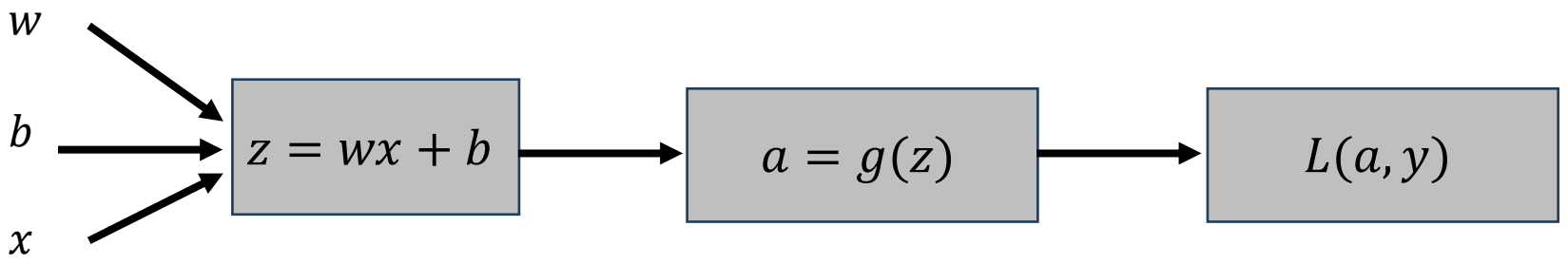


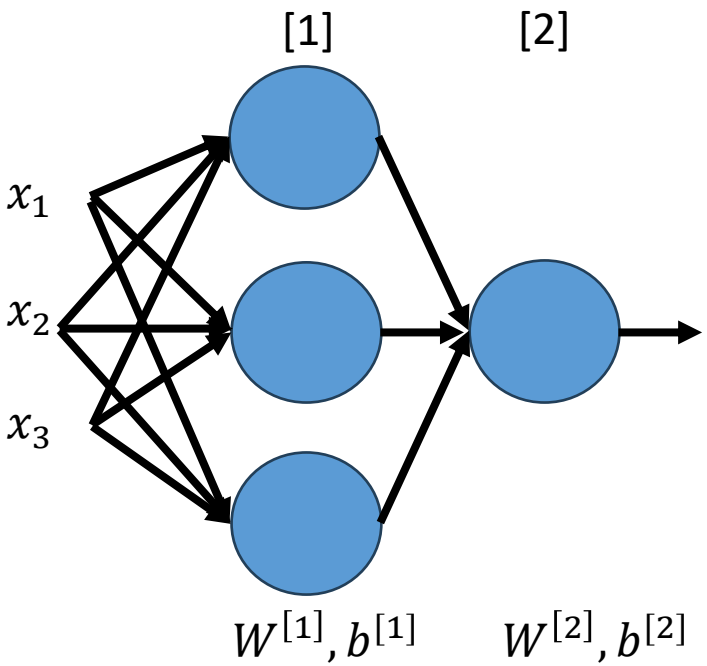


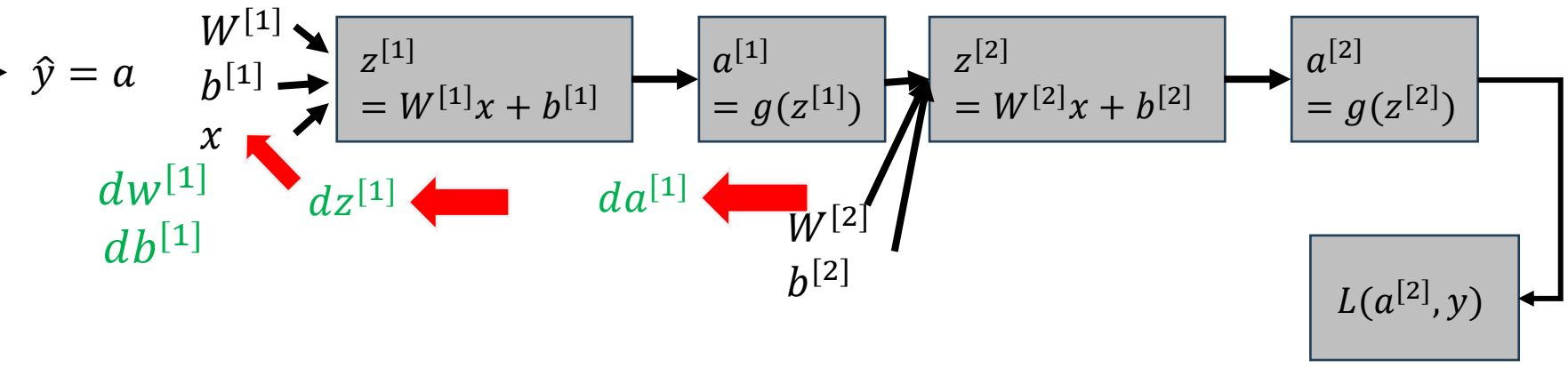
Machine Learning: *Neural Networks*

General Formulation





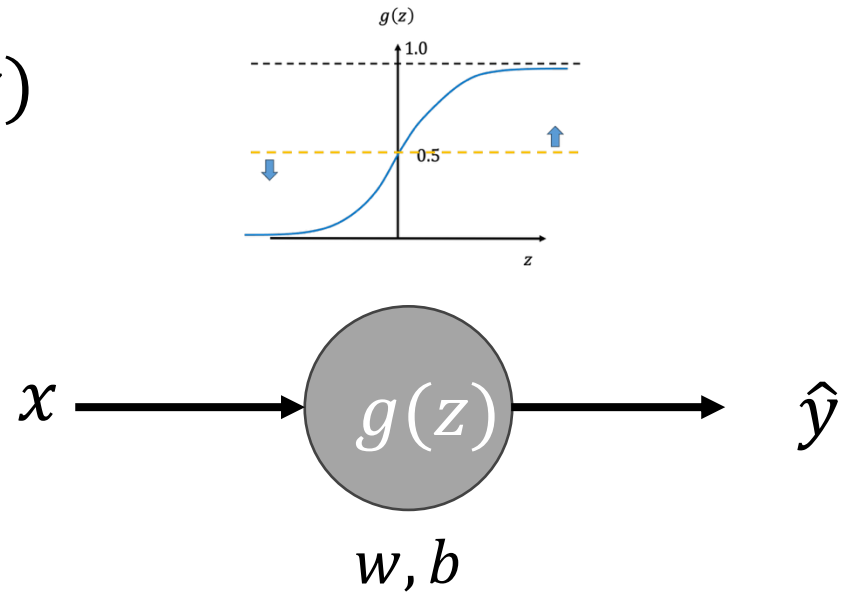




Machine Learning: *Neural Networks*

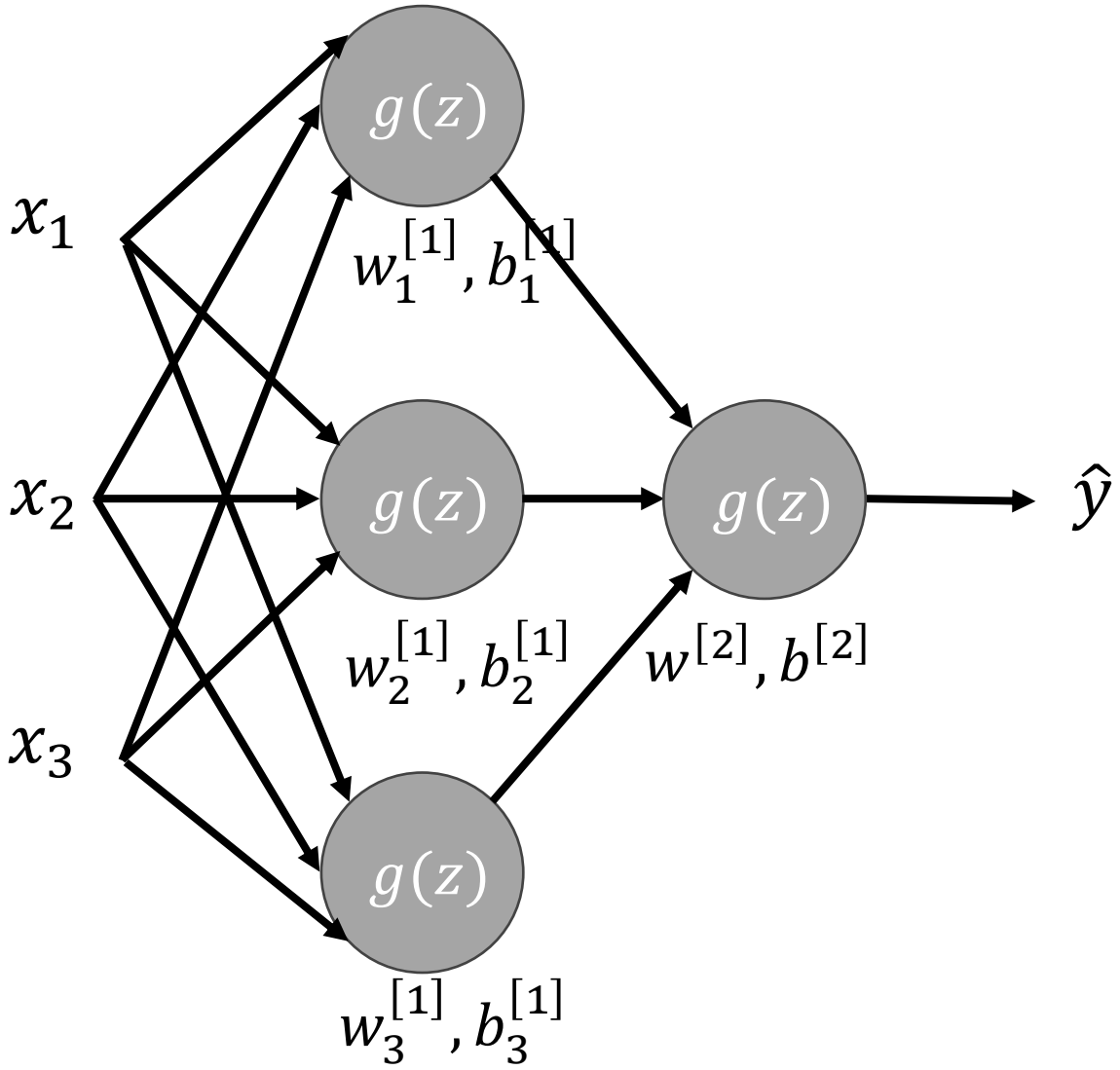
General Formulation

$$z = w.x + b$$
$$\hat{y} = a = g(z)$$



Machine Learning: *Neural Networks*

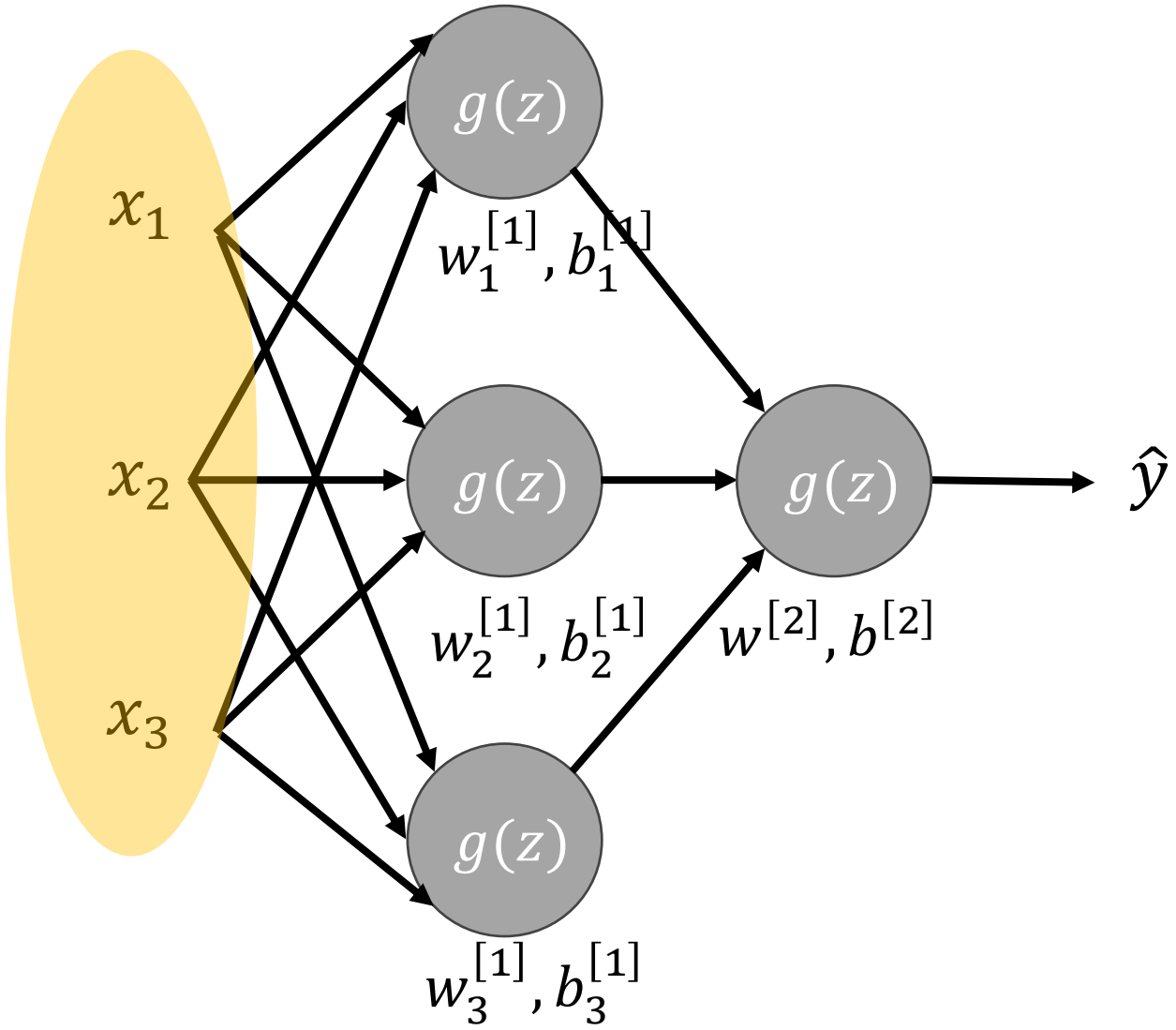
General Formulation



Machine Learning: *Neural Networks*

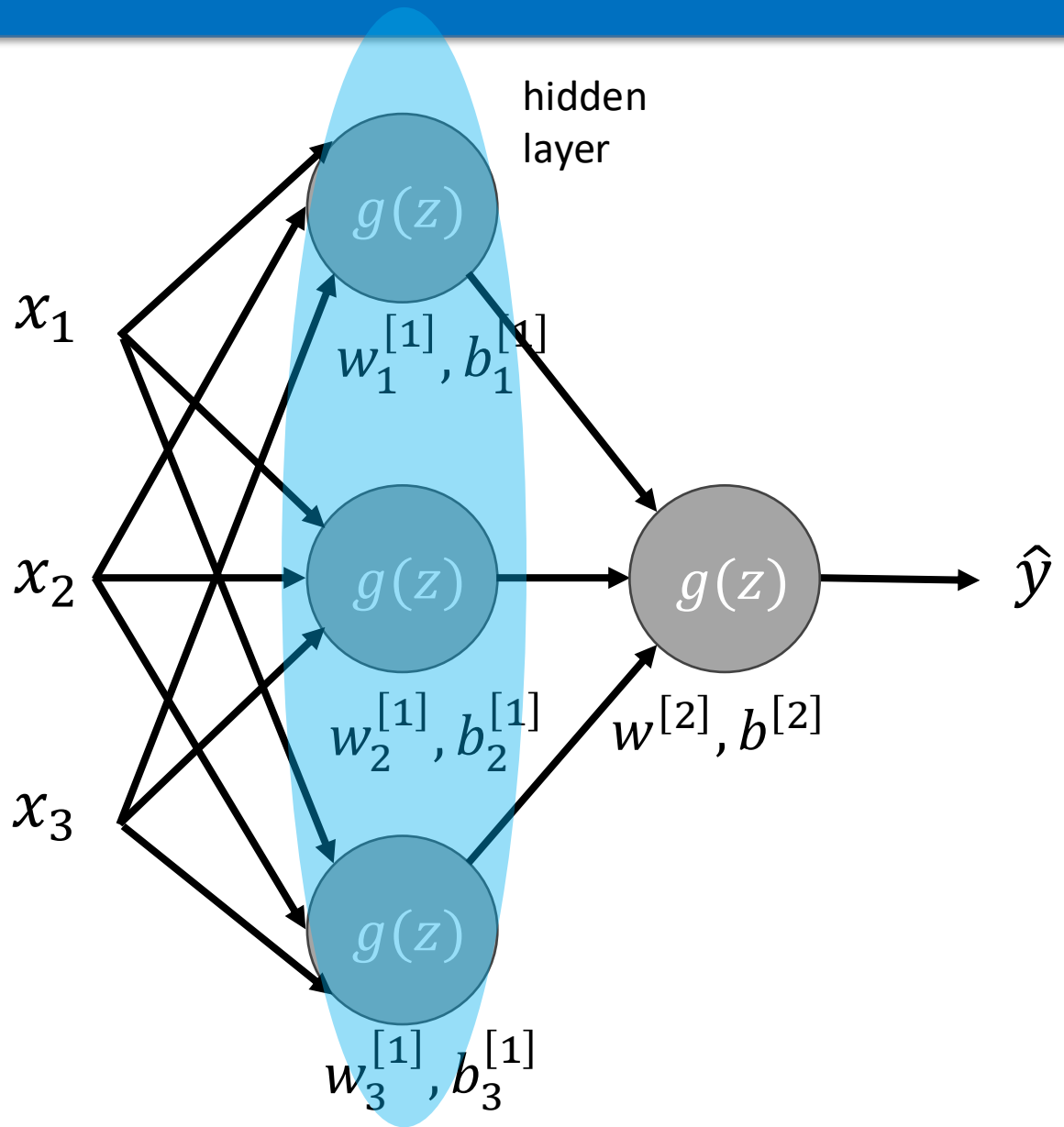
General Formulation

Input layer



Machine Learning: *Neural Networks*

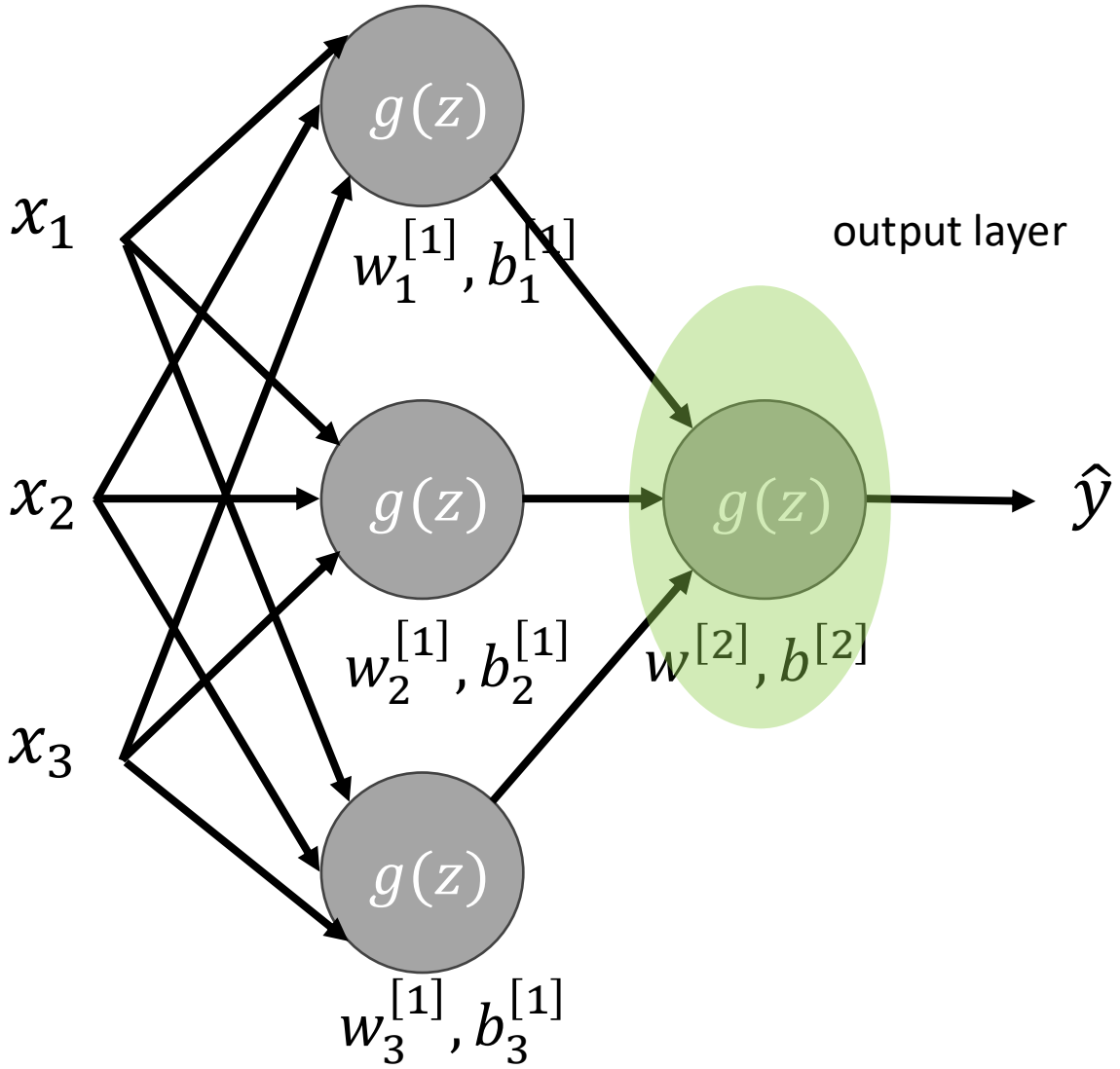
General Formulation



Machine Learning: *Neural Networks*

General Formulation

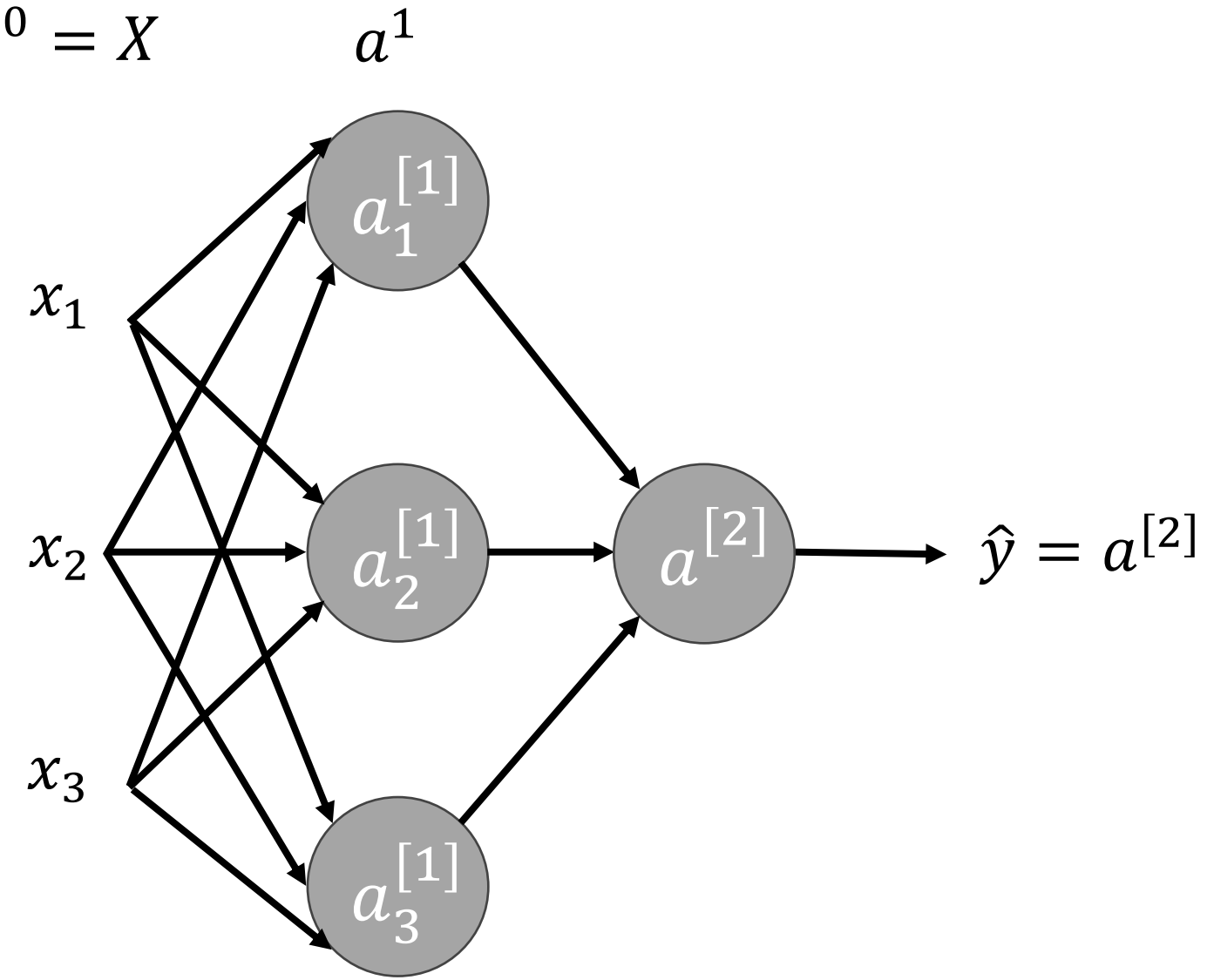
2-layer NN



Machine Learning: *Neural Networks*

General Formulation

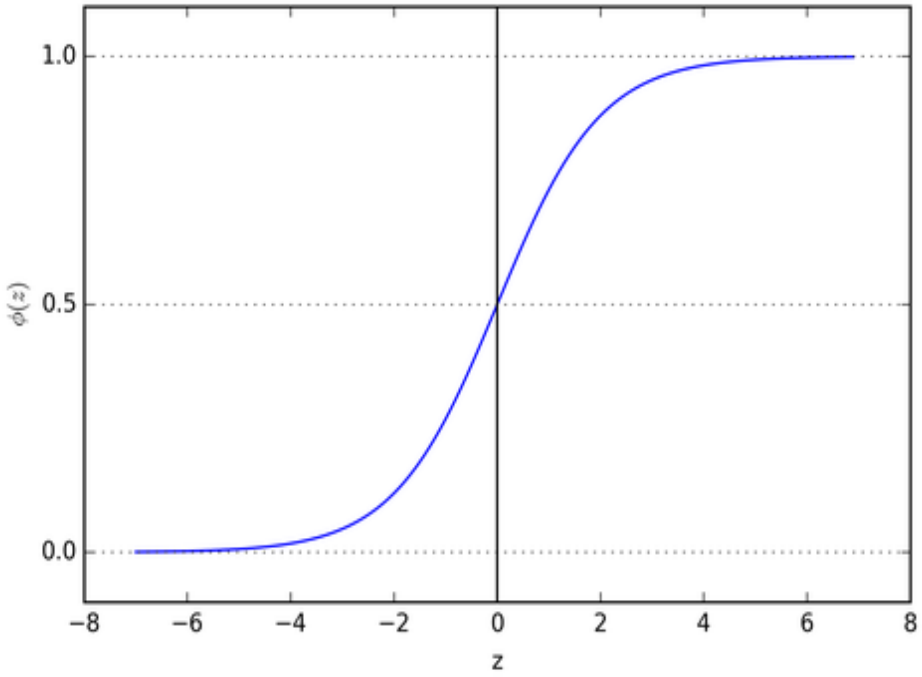
Activation notation



Machine Learning: *Neural Networks*

Activation Function: Sigmoid

$$g(z) = \frac{1}{1 + e^{-z}}$$



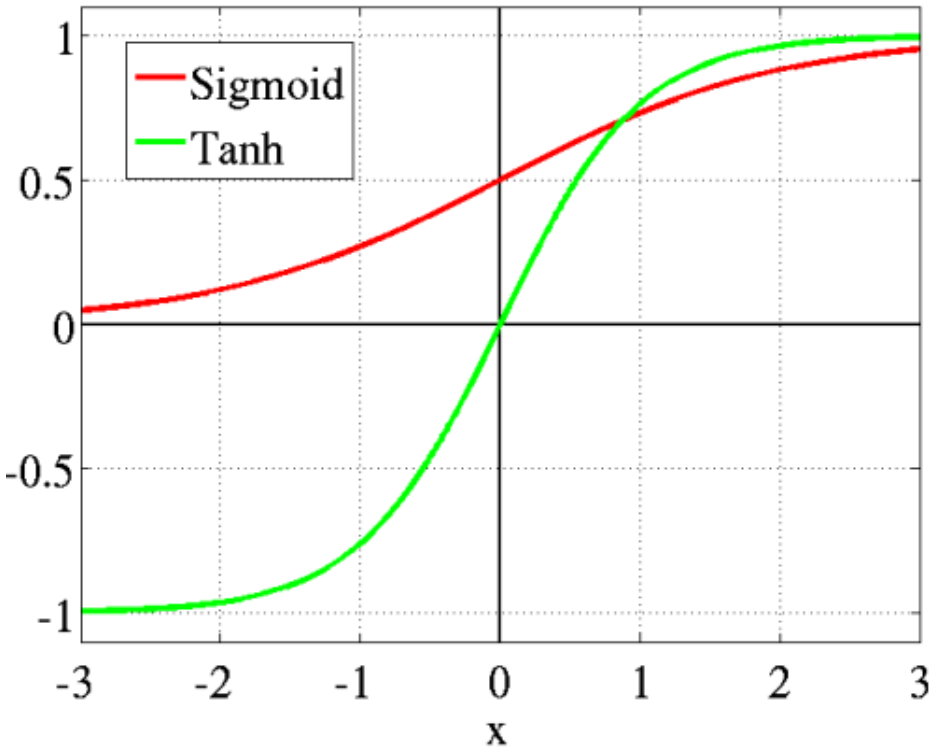
Usage: Traditionally used for binary classification in the output layer.

Characteristics: Outputs values between 0 and 1, representing probabilities. However, it suffers from the vanishing gradient problem.

Machine Learning: *Neural Networks*

Activation Function: Hyperbolic Tangent

$$g(z) = tanh(z) = \frac{e^z + e^{-z}}{e^z - e^{-z}}$$



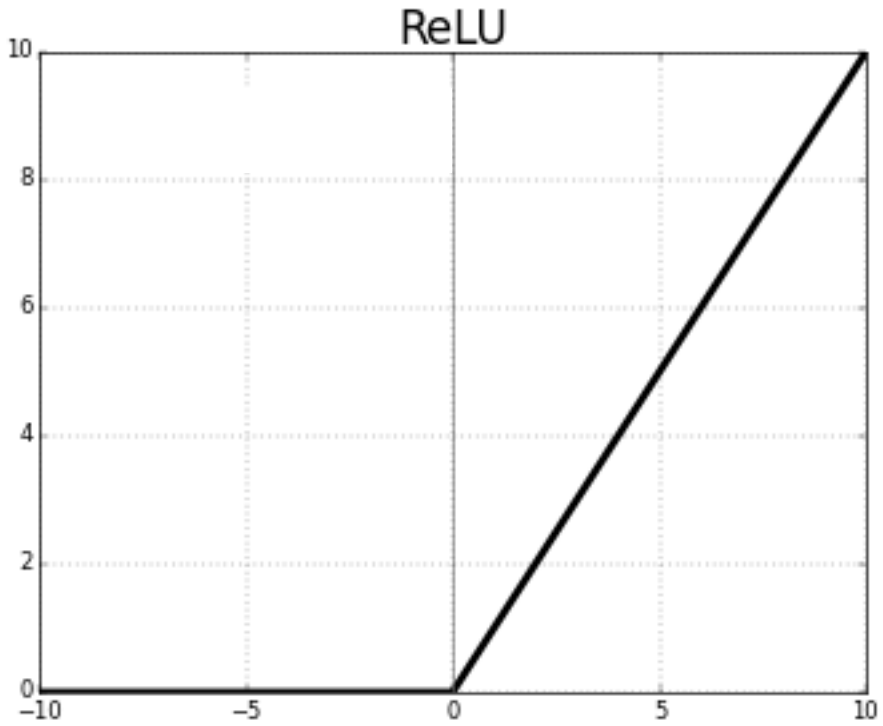
Usage: Often used in hidden layers.

Characteristics: Similar to the sigmoid but outputs values between -1 and 1. It also suffers from the vanishing gradient problem but is zero-centered, making optimization easier in some cases.

Machine Learning: *Neural Networks*

Activation Function: Rectified Linear Unit (ReLU)

$$g(z) = \begin{cases} 0, & z < 0 \\ z, & z \geq 0 \end{cases}$$



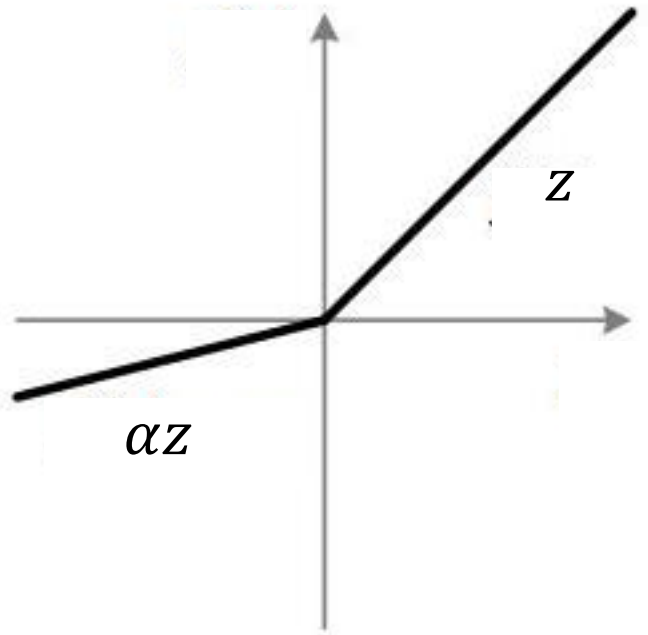
Usage: Very common in hidden layers.

Characteristics: It allows only positive values to pass through and blocks negative values. This function helps with the vanishing gradient problem but can lead to dying ReLU problem (where neurons stop firing during training).

Machine Learning: *Neural Networks*

Activation Function: Leaky Rectified Linear Unit (Leaky ReLU)

$$g(z) = \begin{cases} \alpha z, & z < 0 \\ z, & z \geq 0 \end{cases}$$ α small number



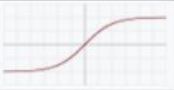
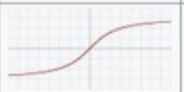
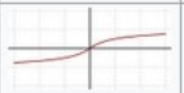
Usage: Used in hidden layers to address the dying ReLU problem.

Characteristics: Similar to ReLU but allows a small gradient when the unit is not active.

Machine Learning: *Neural Networks*

Activation Function:

Summary

Name	Plot	Equation	Derivative (with respect to x)	Range
Identity		$f(x) = x$	$f'(x) = 1$	$(-\infty, \infty)$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$	$\{0, 1\}$
Logistic (a.k.a. Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$	$(0, 1)$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$	$(-1, 1)$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$	$\left(-\frac{\pi}{2}, \frac{\pi}{2}\right)$
Softsign		$f(x) = \frac{x}{1 + \lvert x \rvert}$	$f'(x) = \frac{1}{(1 + \lvert x \rvert)^2}$	$(-1, 1)$
Rectified linear unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$[0, \infty)$
Leaky rectified linear unit (Leaky ReLU)		$f(x) = \begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0.01 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$
Parameteric rectified linear unit (PReLU)		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$

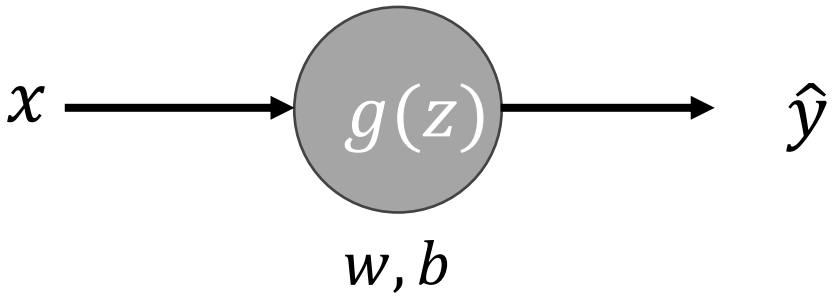
https://courses.engr.illinois.edu/ece598pv/fa2017/Lecture09_NeuralNet Architectures_MoitreyaChatterjee.pdf

Machine Learning: *Neural Networks*

Activation Function:

Why non-linear activation functions?

$$z = w.x + b$$
$$\hat{y} = a = g(z)$$

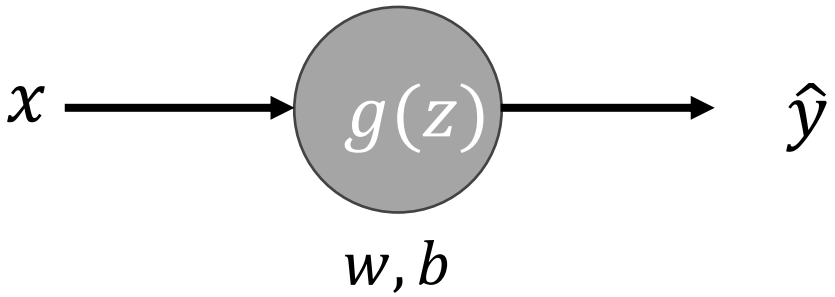


Machine Learning: *Neural Networks*

Activation Function:

Why non-linear activation functions?

$$z = w.x + b$$
$$\hat{y} = a = g(z)$$



$$z^{[1]} = W^{[1]}x + b^{[1]}$$
$$a^{[1]} = g(z^{[1]})$$

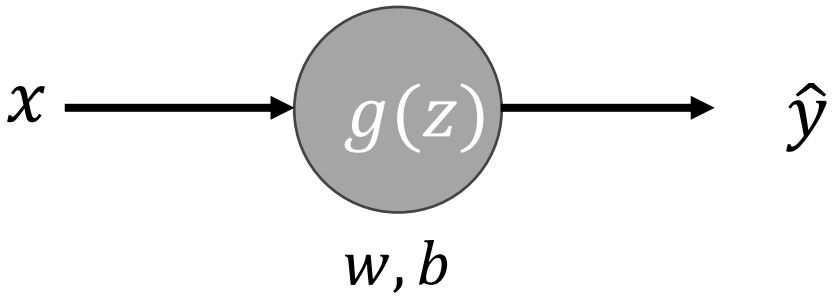
$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$
$$a^{[2]} = g(z^{[2]})$$

Machine Learning: *Neural Networks*

Activation Function:

Why non-linear activation functions?

$$z = w.x + b$$
$$\hat{y} = a = g(z)$$



$$z^{[1]} = W^{[1]}x + b^{[1]}$$
$$\cancel{a^{[1]} = g(z^{[1]})} \rightarrow a^{[1]} = z^{[1]}$$

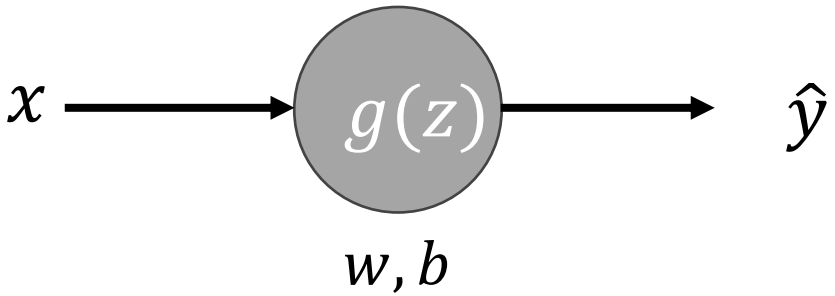
$$\begin{aligned} z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} = W^{[2]}z^{[1]} + b^{[2]} \\ &= W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]} \\ &= W^{[2]}W^{[1]}x + W^{[2]}b^{[1]} + b^{[2]} \end{aligned}$$

Machine Learning: *Neural Networks*

Activation Function:

Why non-linear activation functions?

$$z = w.x + b$$
$$\hat{y} = a = g(z)$$



$$z^{[1]} = W^{[1]}x + b^{[1]}$$
$$\cancel{a^{[1]} = g(z^{[1]})} \rightarrow a^{[1]} = z^{[1]}$$

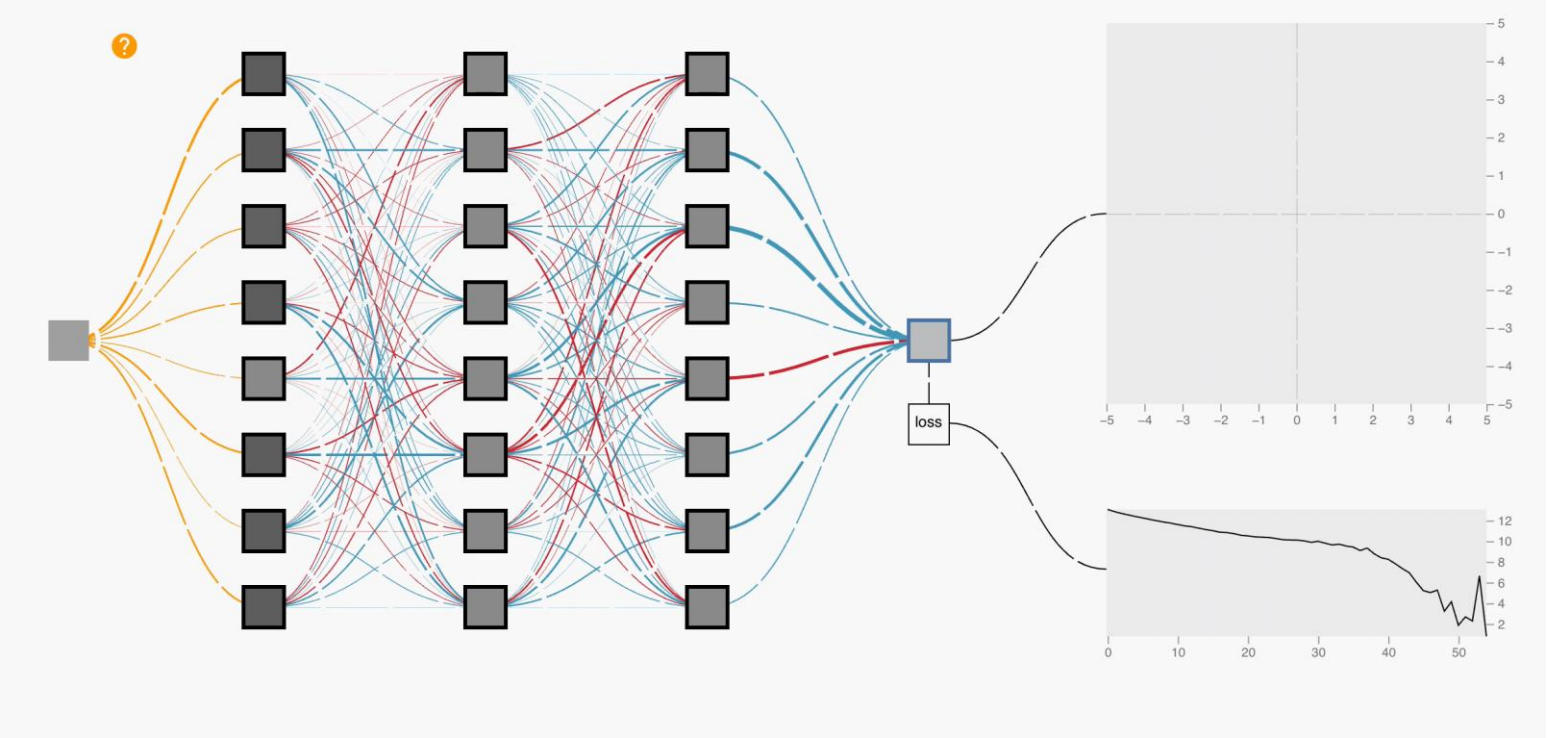
$$\begin{aligned} z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} = W^{[2]}z^{[1]} + b^{[2]} \\ &= W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]} \\ &= W^{[2]}W^{[1]}x + W^{[2]}b^{[1]} + b^{[2]} \\ &= Wx + b \end{aligned}$$

Machine Learning: *Neural Networks*

Forward and Backward Propagation:

Demo

https://xnought.github.io/backprop-explainer/



Machine Learning: *Neural Networks*

Feed Forward Neural Networks

Python

https://www.kaggle.com/code/timniven/feedforward-neural-network-tutorial

Machine Learning: *Neural Networks*

Feed Forward Neural Networks

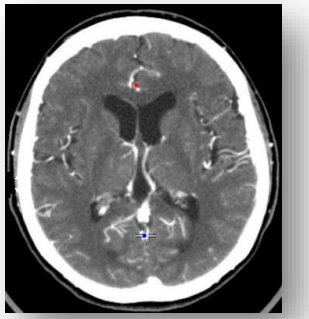
Python

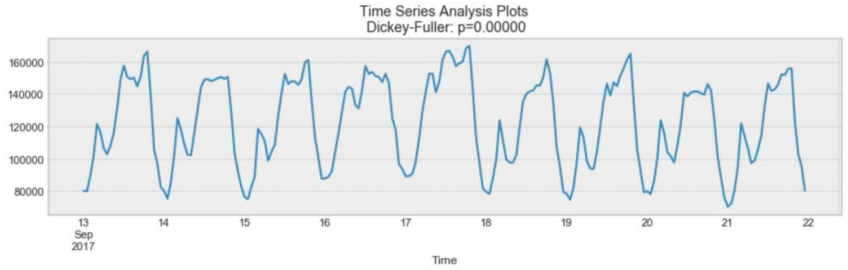
```
pip install tensorflow
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Machine Learning: *Neural Networks*

Feed Forward Neural Network







198	0.01	1
205	-0.05	0
190	0.001	0
⋮	⋮	⋮

Machine Learning: *Neural Networks*

Recurrent Neural Network (RNN):

Motivation

Named Entity Recognition (NER)

"There is a nodule in left lobe of lung."

"There is a **nodule** in left lobe of lung."

Machine Learning: *Neural Networks*

Recurrent Neural Network (RNN):

Motivation

Named Entity Recognition (NER)

"There is a nodule in left lobe of lung."

"There is a **nodule** in left lobe of lung."

$x^{<1>}$ $x^{<2>}$ ... $x^{<9>}$

$$\begin{bmatrix}0\\0\\\vdots\\1\\0\\\vdots\end{bmatrix} \quad \begin{bmatrix}0\\0\\\vdots\\1\\0\\\vdots\end{bmatrix} \quad \begin{bmatrix}0\\0\\\vdots\\1\\0\\\vdots\end{bmatrix}$$

Machine Learning: *Neural Networks*

Recurrent Neural Network (RNN):

Motivation

Named Entity Recognition (NER)

"There is a nodule in left lobe of lung."

$x^{<1>}$ $x^{<2>}$ ... $x^{<9>}$

$$\begin{bmatrix}0\\0\\\vdots\\1\\0\\\vdots\end{bmatrix} \quad \begin{bmatrix}0\\0\\\vdots\\1\\0\\\vdots\end{bmatrix} \quad \begin{bmatrix}0\\0\\\vdots\\1\\0\\\vdots\end{bmatrix}$$

"There is a **nodule** in left lobe of lung."

$y^{<1>}$ $y^{<2>}$ ... $y^{<9>}$

$0 \quad 0 \quad 0 \quad 1 \quad 0 \quad \ldots \quad 0$

Machine Learning: *Neural Networks*

Notation

$x^{(i)}$	Sample (i)
$x^{[i]}$	Layer $[i]$
$x^{<i>}$	Position in sequence $< i >$

Machine Learning: *Neural Networks*

Notation

Training sample (1)
Training sample (2)
…
Training sample (i)
…
Training sample T_x

$x^{(i)<j>}$: the j element in training sample i

Example:

100: "There is a nodule in left lobe of lung."

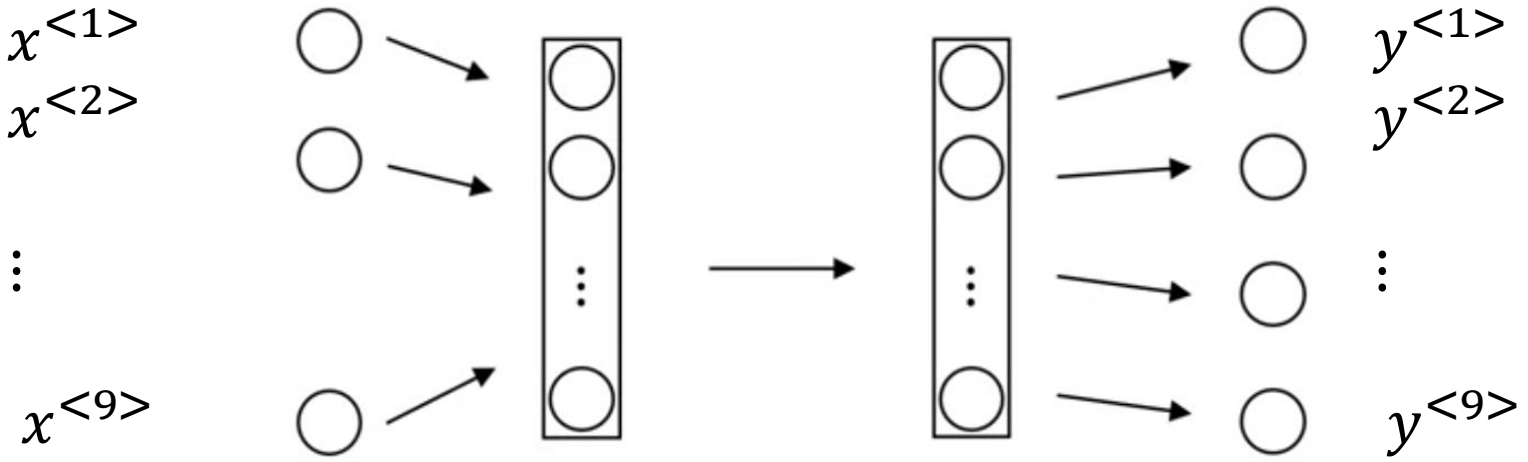
$x^{(100)<4>} = 'nodule'$

Machine Learning: *Neural Networks*

Recurrent Neural Network (RNN):

Motivation

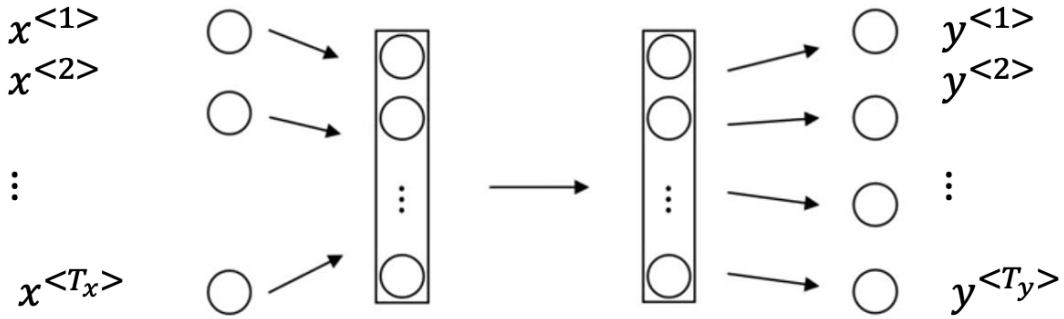
Named Entity Recognition (NER)



Machine Learning: *Neural Networks*

Recurrent Neural Network (RNN):

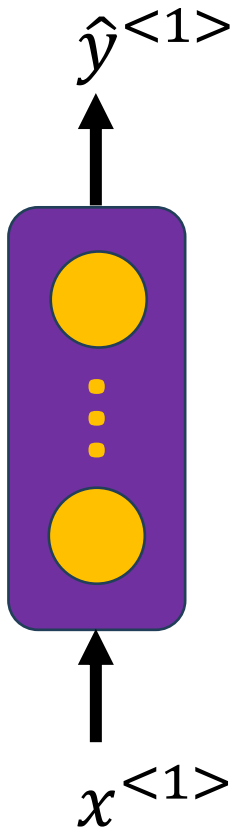
Why not FFNN for seq-2-seq?



- **Lack of Temporal Context**: treat each input independently, without considering the order and context of the data
- **Fixed Input and Output Size:** designed to work with inputs and outputs of a fixed size.
- **Inability to Capture Long-Term Dependencies**: lack a mechanism to remember previous inputs for an extended period.
- **Inefficient Representation**: have to represent the sequence in a way that captures temporal information, like including a time step with each input. However, this is typically inefficient and doesn't capture the nuances of sequential dependencies effectively.

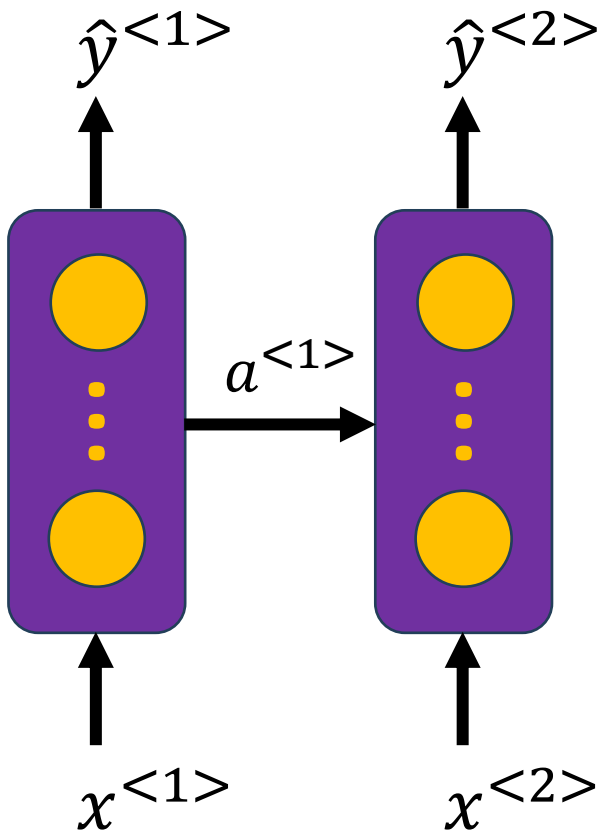
Machine Learning: *Neural Networks*

Recurrent Neural Network (RNN):



Machine Learning: *Neural Networks*

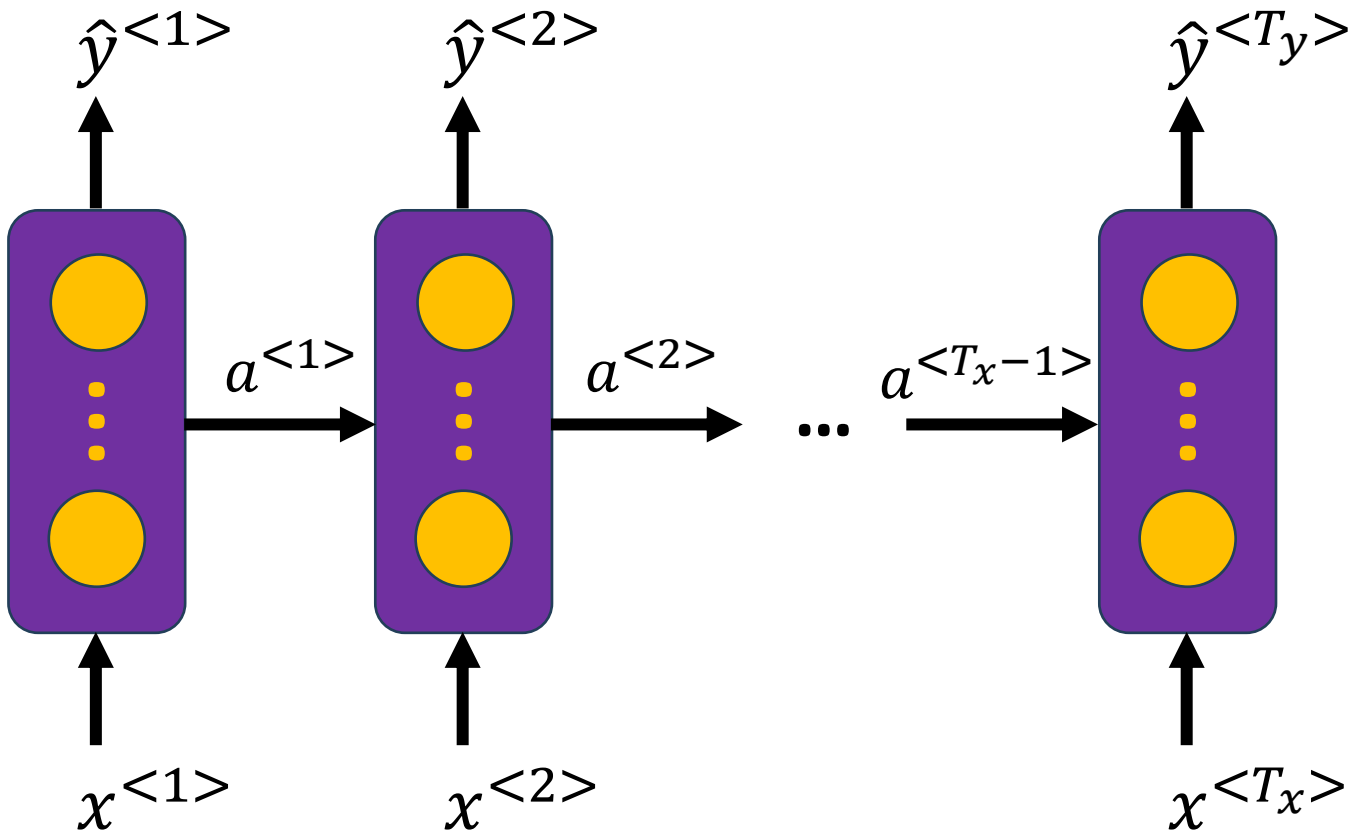
Recurrent Neural Network (RNN):



Machine Learning: *Neural Networks*

Recurrent Neural Network (RNN):

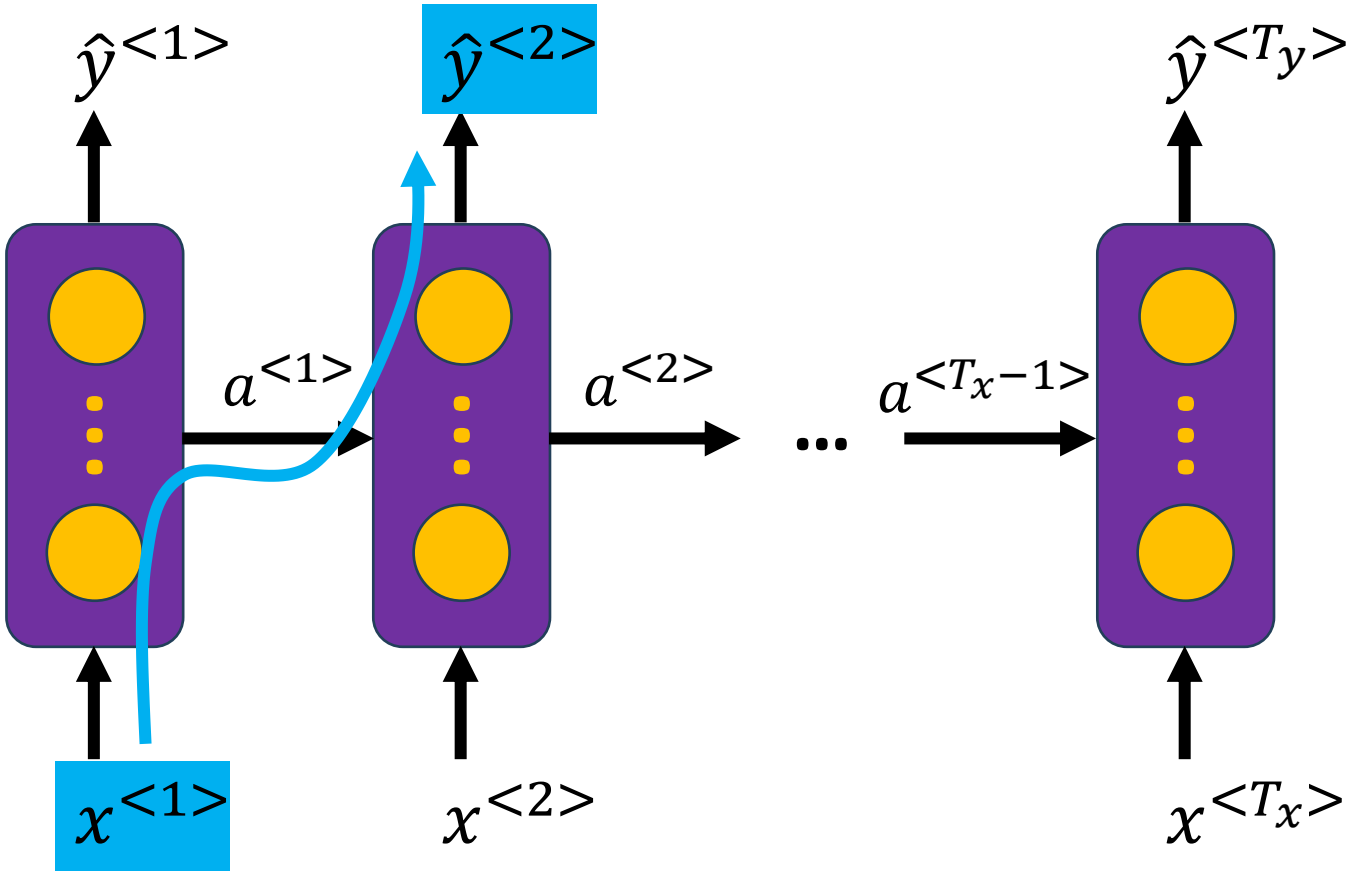
Assume $T_y = T_x$



Machine Learning: *Neural Networks*

Recurrent Neural Network (RNN):

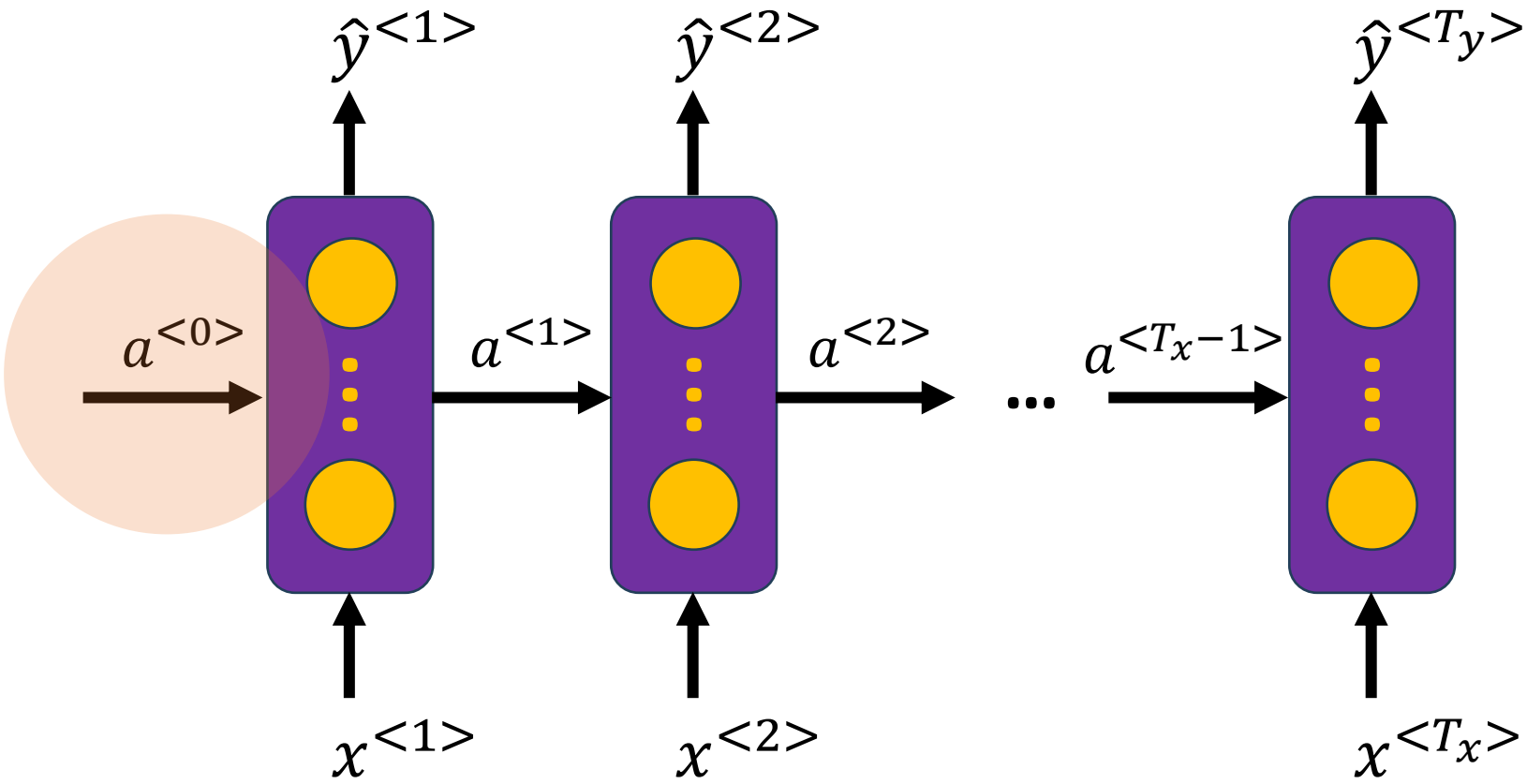
Assume $T_y = T_x$



Machine Learning: *Neural Networks*

Recurrent Neural Network (RNN):

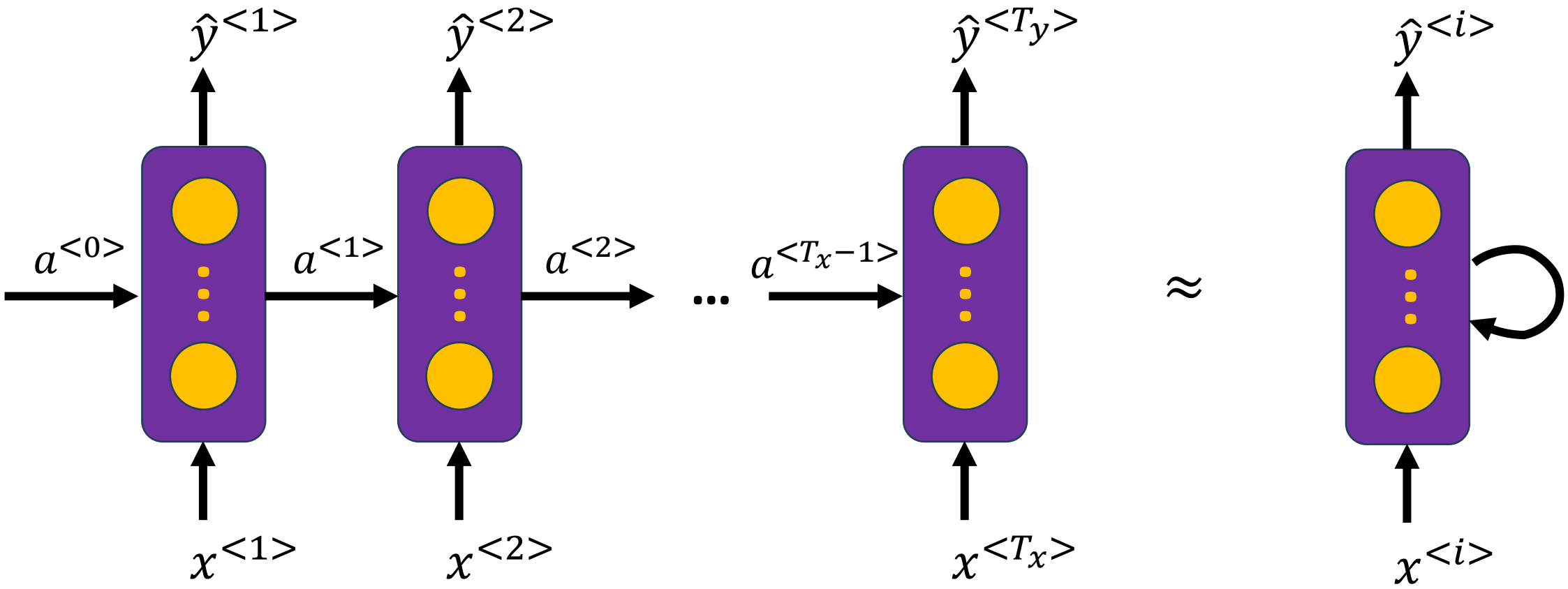
Assume $T_y = T_x$



Machine Learning: *Neural Networks*

Recurrent Neural Network (RNN):

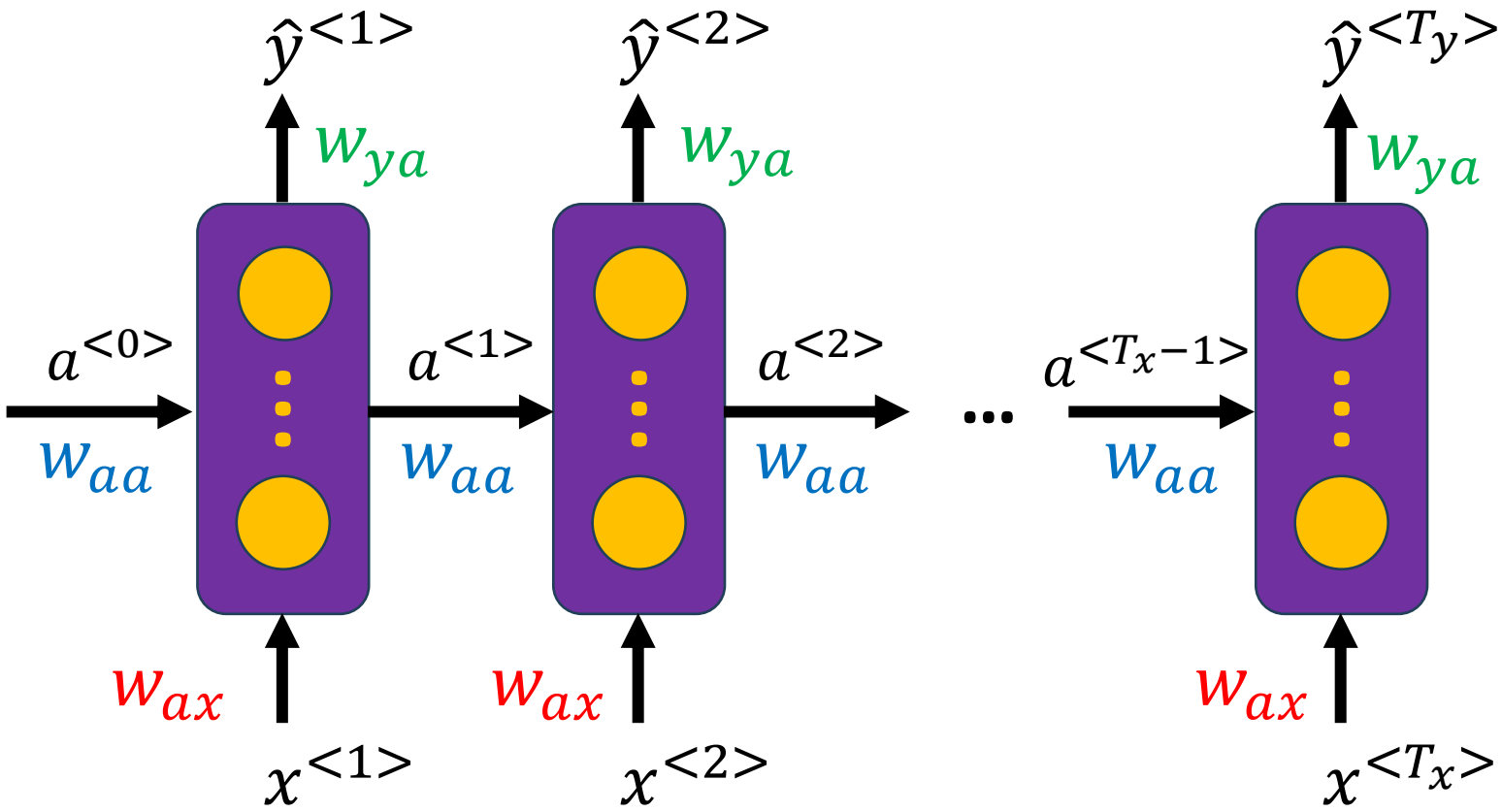
Assume $T_y = T_x$



Machine Learning: *Neural Networks*

Recurrent Neural Network (RNN):

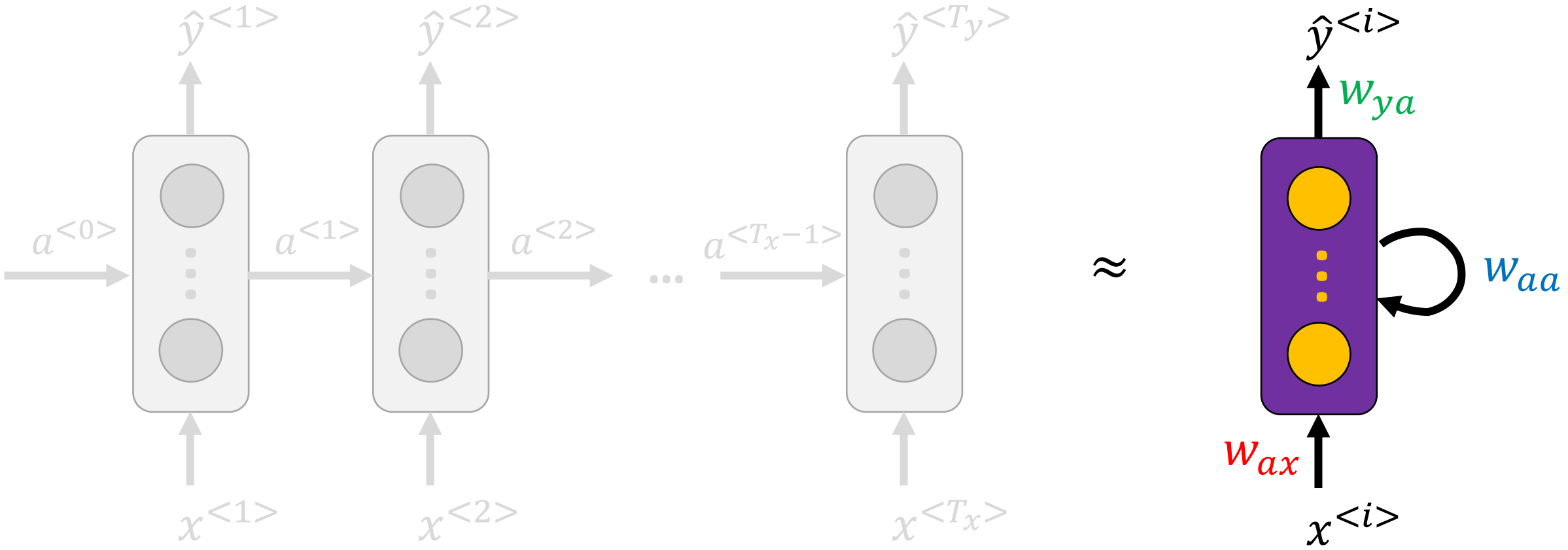
Assume $T_y = T_x$



Machine Learning: *Neural Networks*

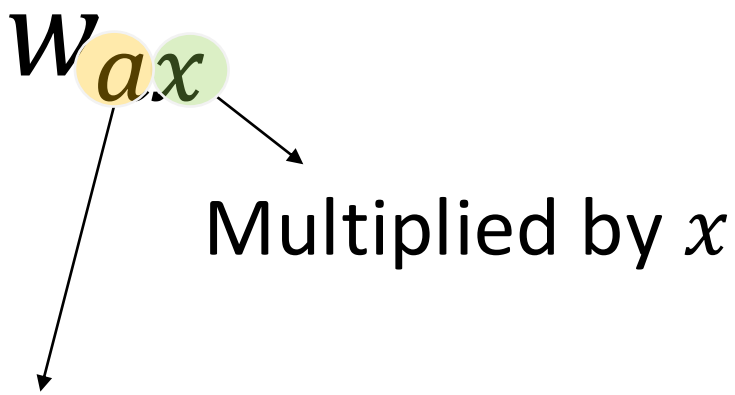
Recurrent Neural Network (RNN):

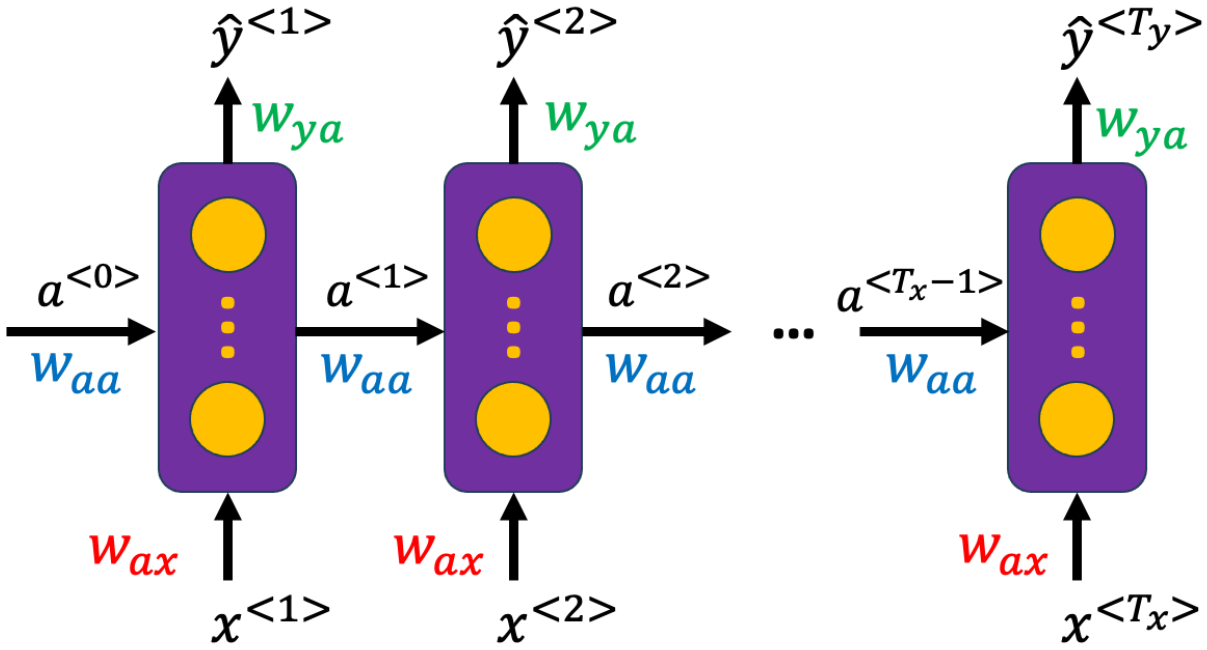
Assume $T_y = T_x$



Machine Learning: *Recurrent Neural Network (RNN)*

Forward Propagation

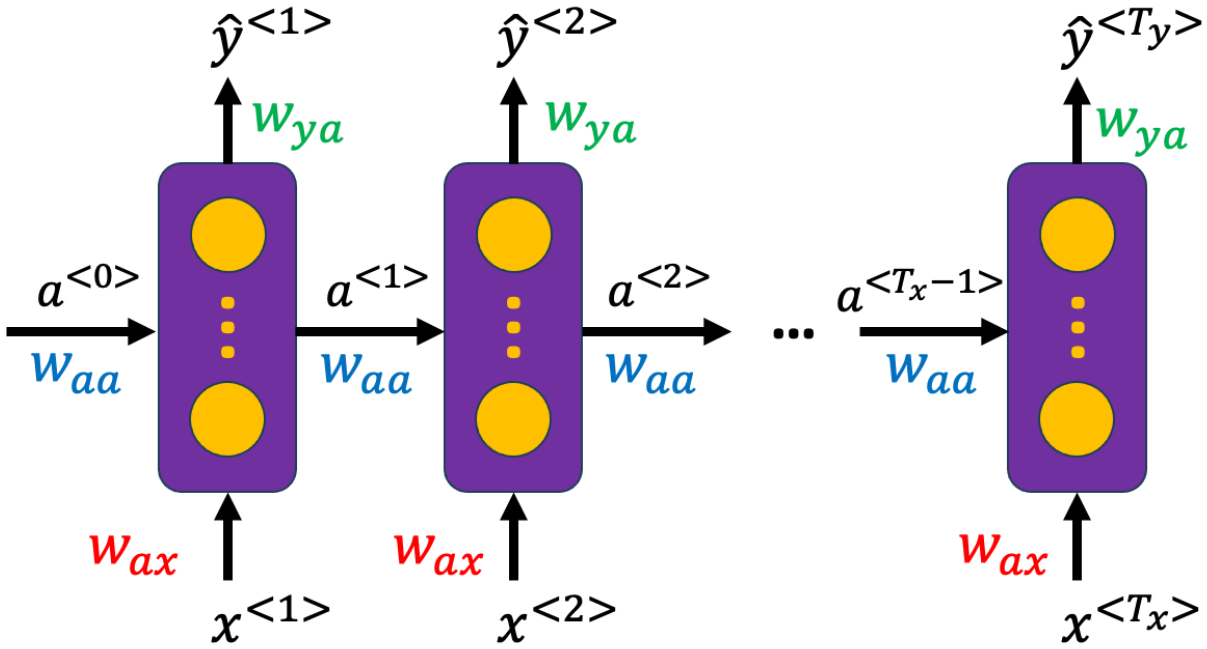




Machine Learning: *Recurrent Neural Network (RNN)*

Forward Propagation

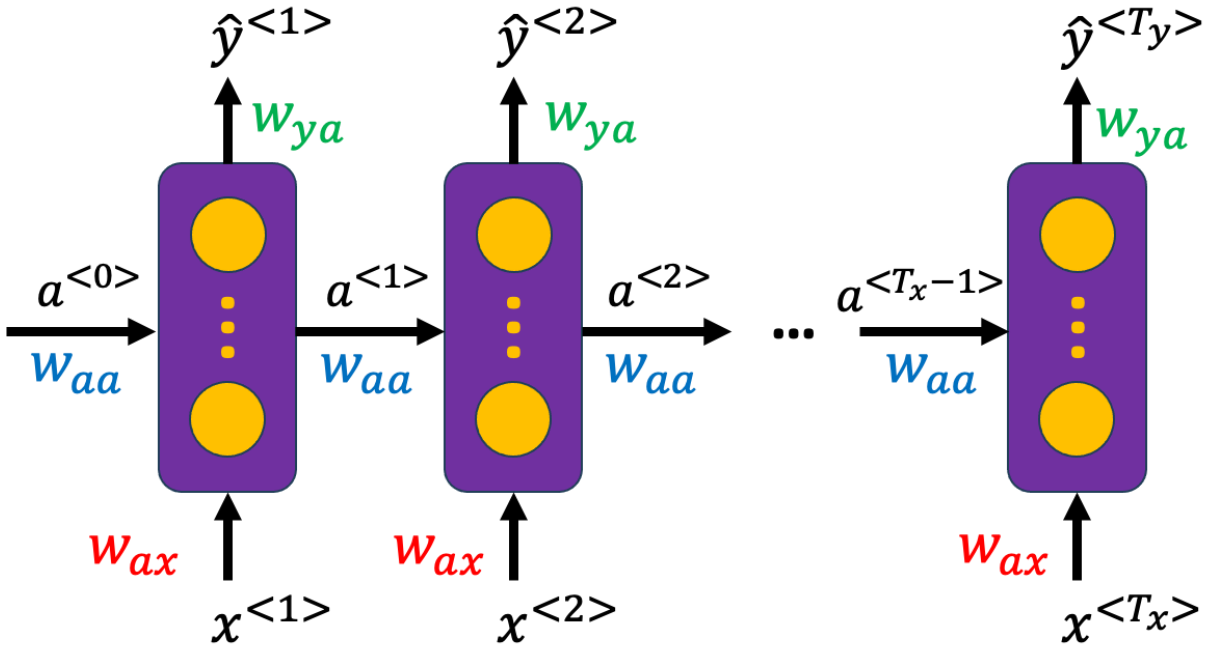
$$a^{<0>} = \vec{0}$$
$$a^{<1>} = g_1(w_{aa}.a^{<0>} + w_{ax}.x^{<1>} + b_a)$$
$$\hat{y}^{<1>} = g_2(w_{ya}.a^{<1>} + b_y)$$



Machine Learning: *Recurrent Neural Network (RNN)*

Forward Propagation

$$a^{<0>} = \vec{0}$$
$$a^{<1>} = g_1(w_{aa}.a^{<0>} + w_{ax}.x^{<1>} + b_a)$$
$$\hat{y}^{<1>} = g_2(w_{ya}.a^{<1>} + b_y)$$



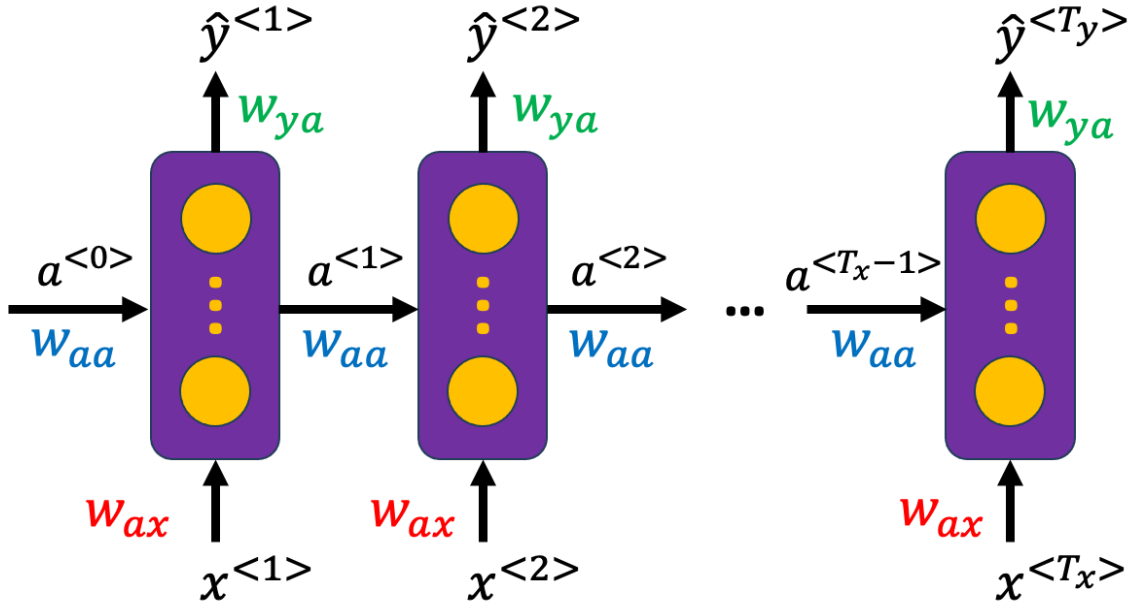
$$a^{<0>} = \vec{0}$$
$$a^{<t>} = g_1(w_{aa}.a^{<t-1>} + w_{ax}.x^{<t>} + b_a)$$
$$\hat{y}^{<t>} = g_2(w_{ya}.a^{<t>} + b_y)$$

Machine Learning: *Recurrent Neural Network (RNN)*

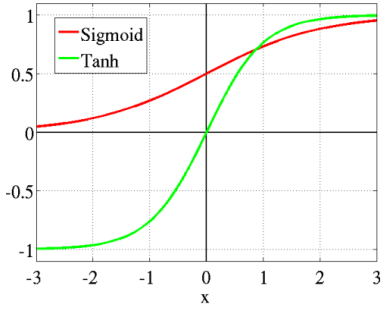
Forward Propagation

Activation Function

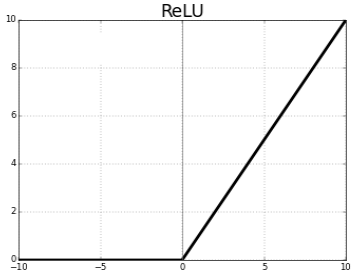
$$a^{<t>} = g_1(w_{aa}.a^{<t-1>} + w_{ax}.x^{<t>} + b_a)$$



$$g(z) = tanh(z) = \frac{e^z + e^{-z}}{e^z - e^{-z}}$$



$$g(z) = \begin{cases} 0, & z < 0 \\ z, & z \geq 0 \end{cases}$$

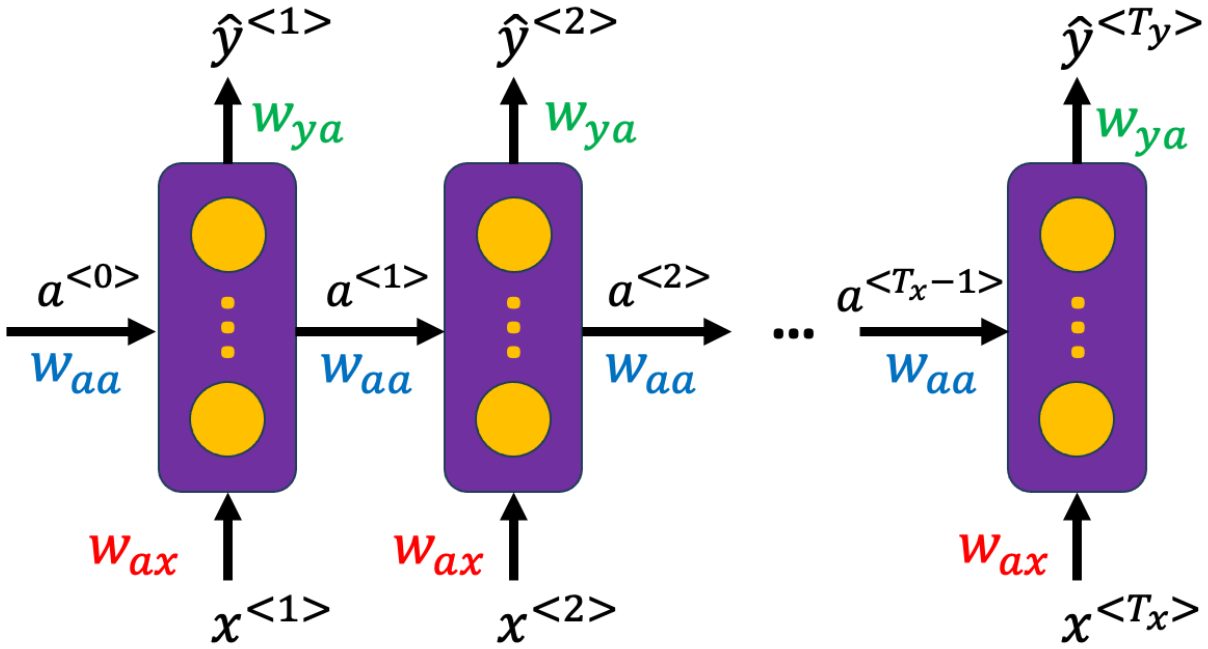


Machine Learning: *Recurrent Neural Network (RNN)*

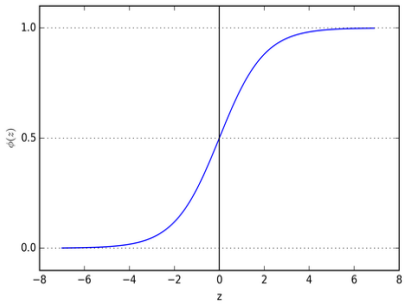
Forward Propagation

Activation Function

$$\hat{y}^{<t>} = g_2(w_{ya}.a^{<t>} + b_y)$$



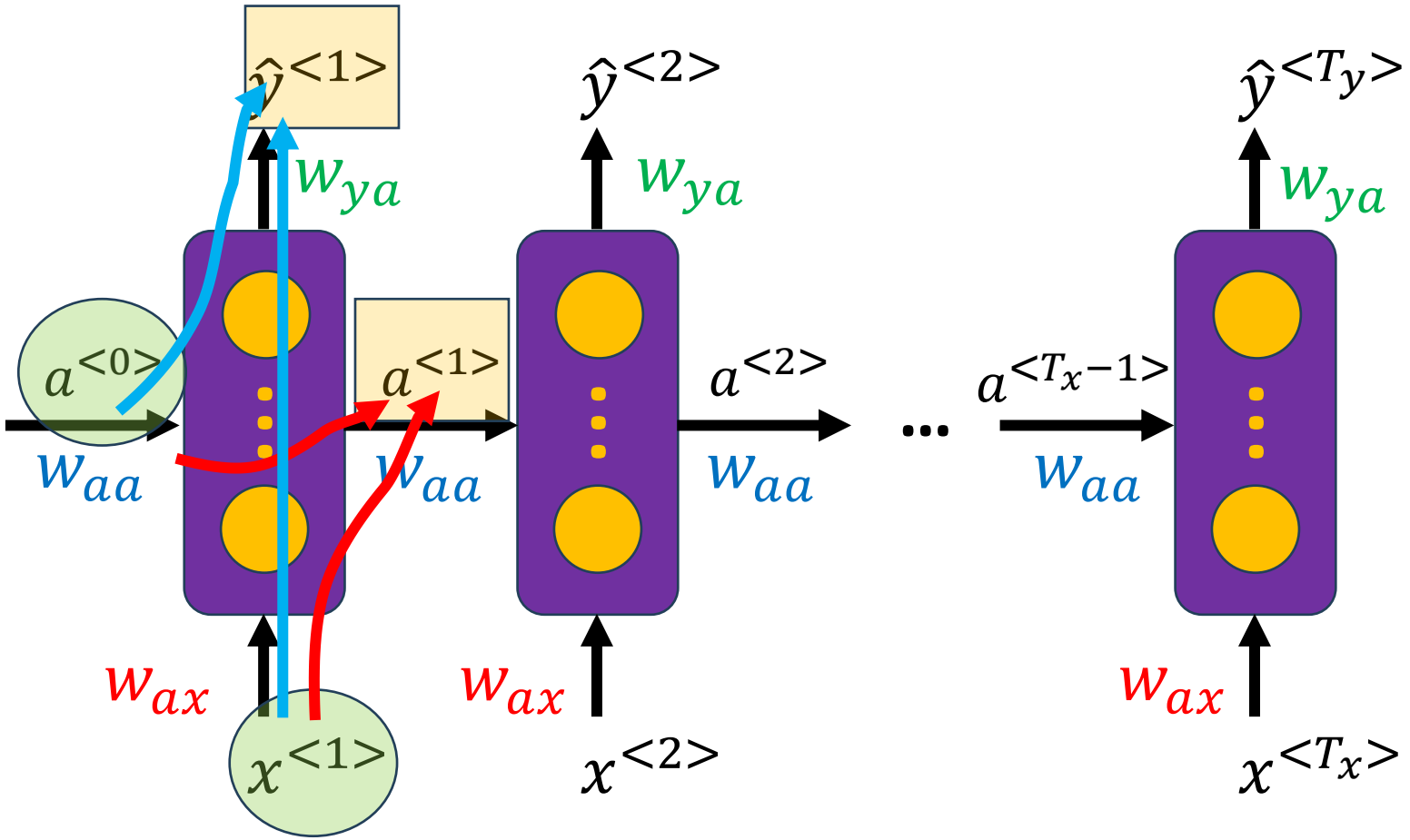
$$g(z) = \frac{1}{1 + e^{-z}}$$



Machine Learning: *Neural Networks*

Forward Propagation

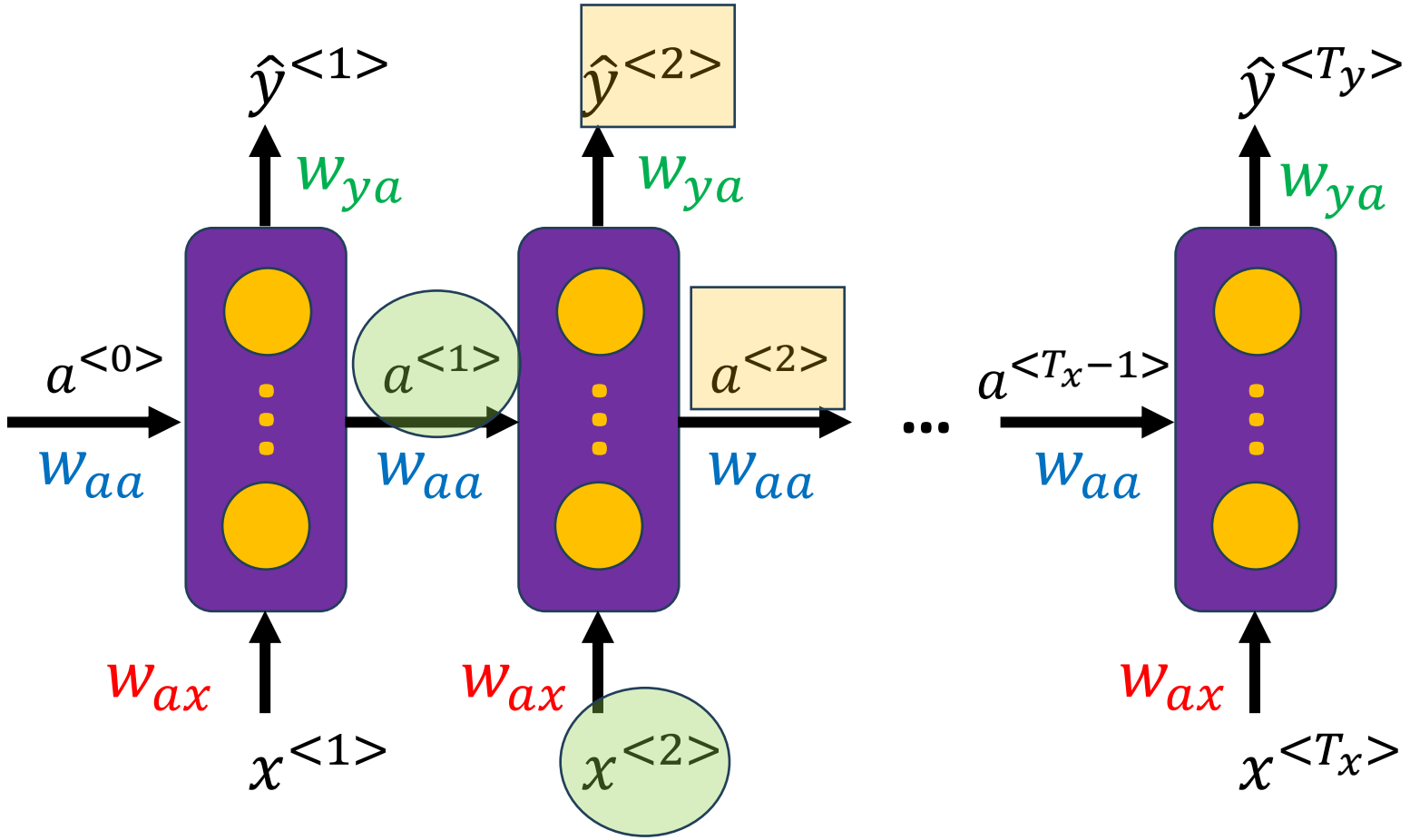
Assume $T_y = T_x$



Machine Learning: *Neural Networks*

Forward Propagation

Assume $T_y = T_x$



Machine Learning: *Recurrent Neural Network (RNN)*

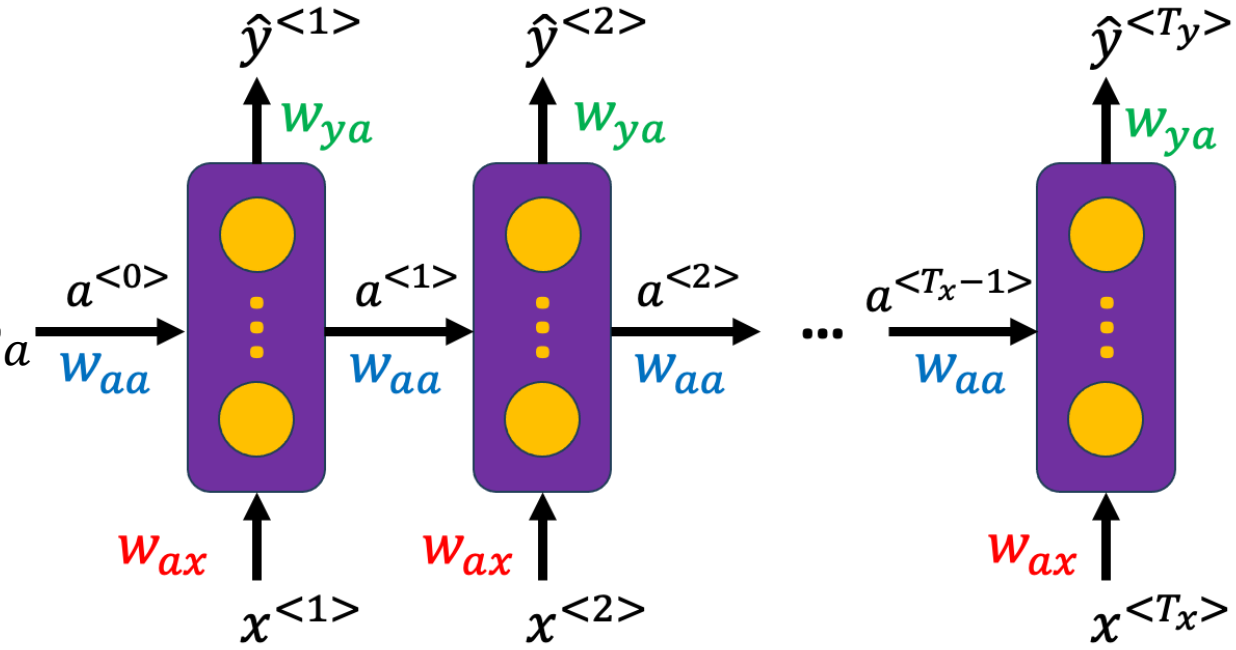
Forward Propagation:

Simplified

$$a^{<t>} = g_1(w_{aa}.a^{<t-1>} + w_{ax}.x^{<t>} + b_a$$

$$[w_{aa}|w_{ax}] = W_a$$

$$[a^{<t-1>}, x^{<t>}] = \begin{bmatrix} a^{<t-1>} \\ x^{<t>} \end{bmatrix}$$



Machine Learning: *Recurrent Neural Network (RNN)*

Forward Propagation:

Simplified

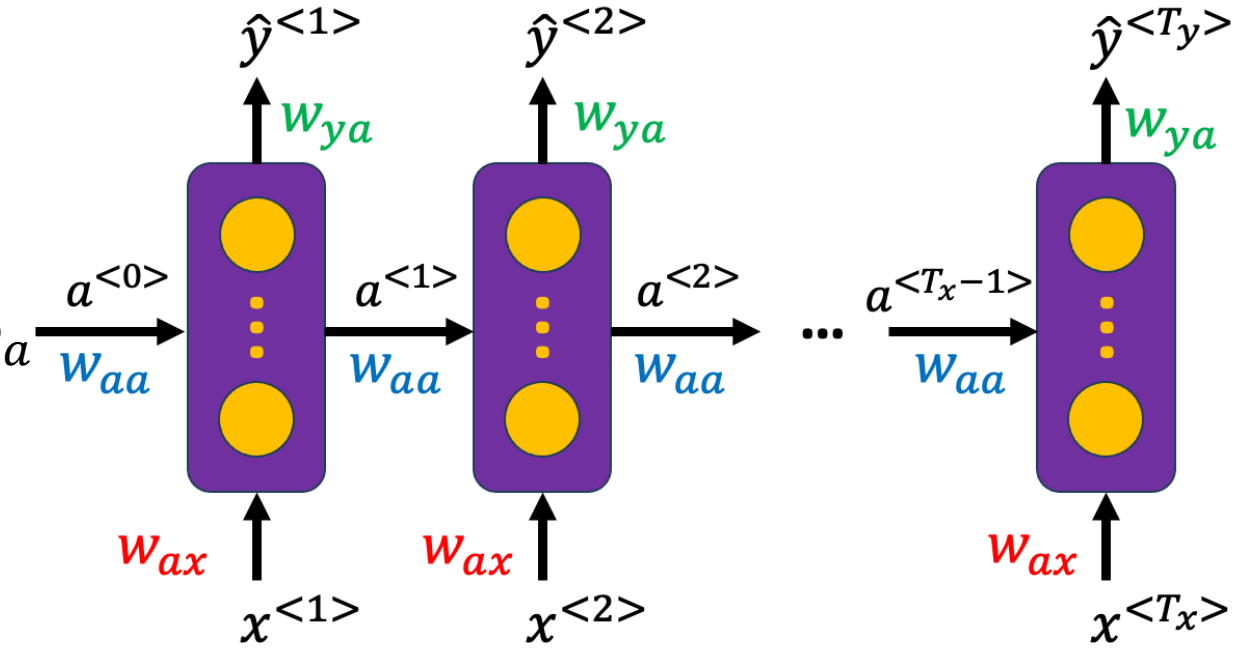
$$a^{<t>} = g_1(w_{aa}.a^{<t-1>} + w_{ax}.x^{<t>} + b_a$$

$$[w_{aa}|w_{ax}] = W_a$$

(100,100) (100,1000) (100,1100)

$$[a^{<t-1>}, x^{<t>}] = \begin{bmatrix} a^{<t-1>} \\ x^{<t>} \end{bmatrix}$$

(100) (1000) (1100)



Machine Learning: *Recurrent Neural Network (RNN)*

Forward Propagation:

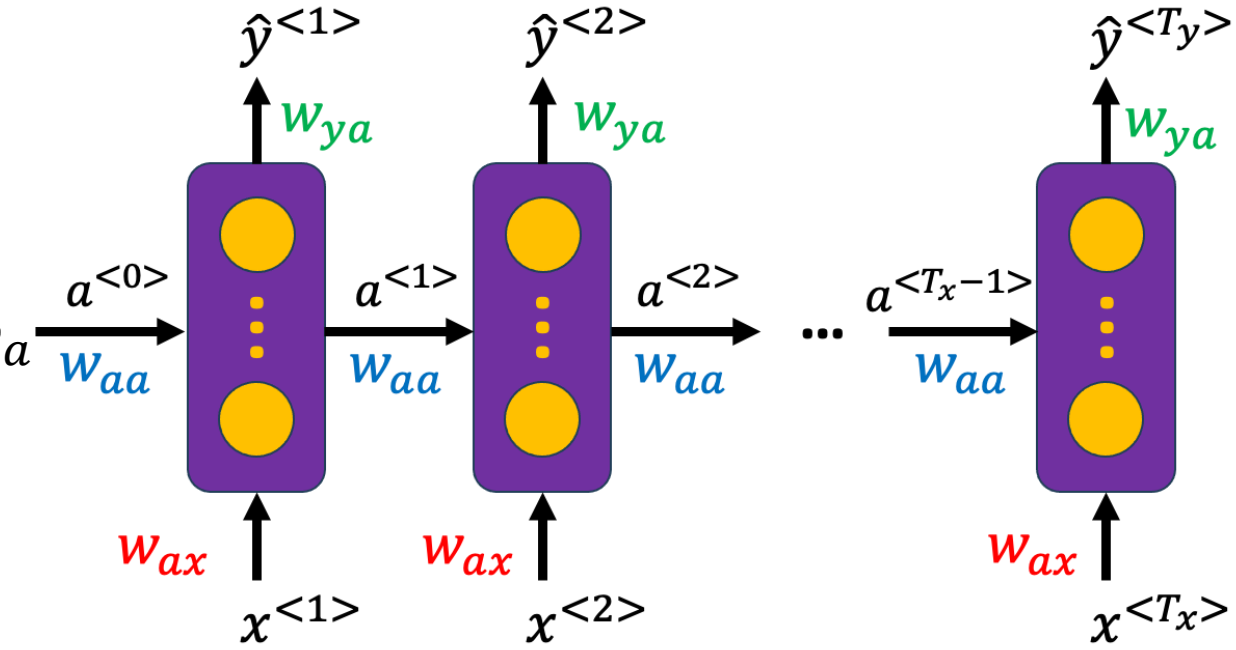
Simplified

$$a^{<t>} = g_1(w_{aa}.a^{<t-1>} + w_{ax}.x^{<t>} + b_a$$

$$[w_{aa}|w_{ax}] = W_a$$

$$[a^{<t-1>}, x^{<t>}] = \begin{bmatrix} a^{<t-1>} \\ x^{<t>} \end{bmatrix}$$

$$a^{<t>} = g_1(W_a.[a^{<t-1>}, x^{<t>}] + b_a)$$



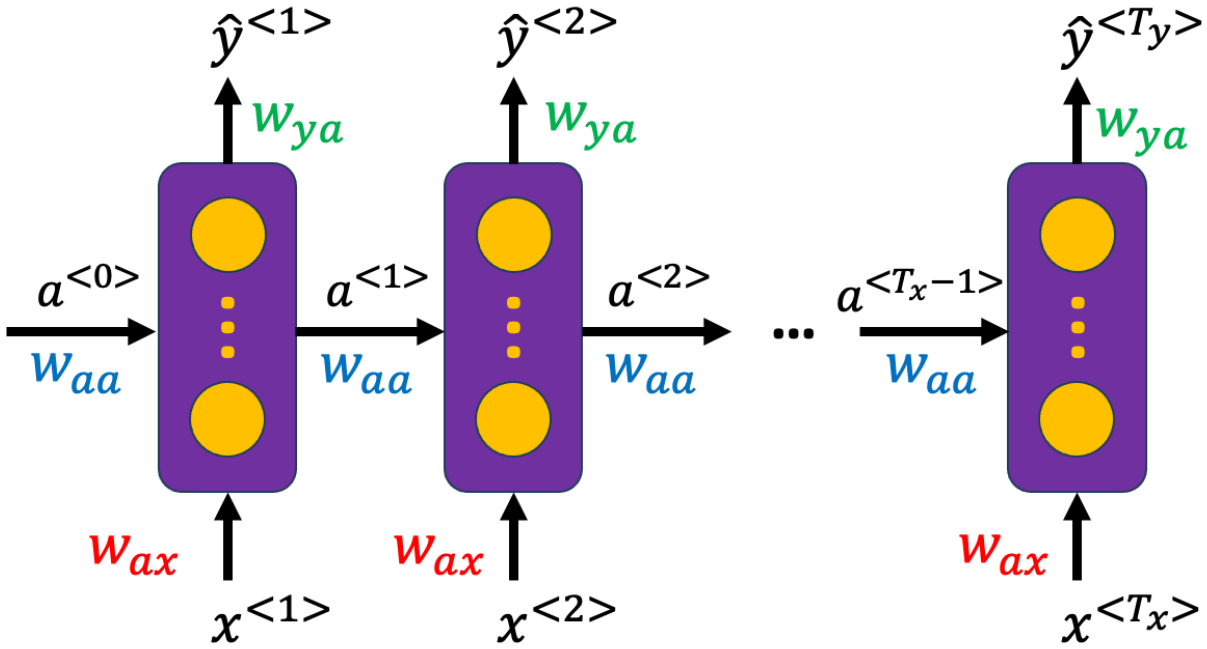
Machine Learning: *Recurrent Neural Network (RNN)*

Forward Propagation:

Simplified

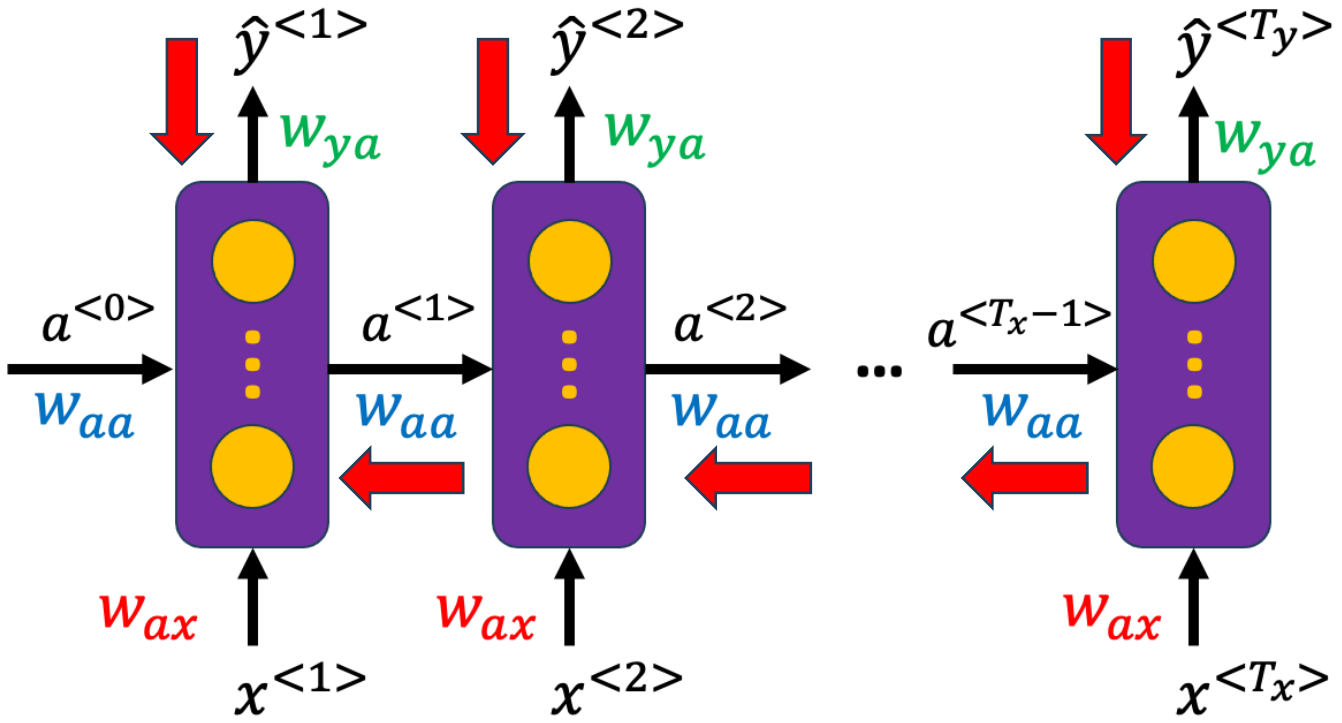
$$\hat{y}^{<t>} = g_2(w_{ya}.a^{<t>} + b_y)$$

$$\hat{y}^{<t>} = g_2(w_y.a^{<t>} + b_y)$$



Machine Learning: *Recurrent Neural Network (RNN)*

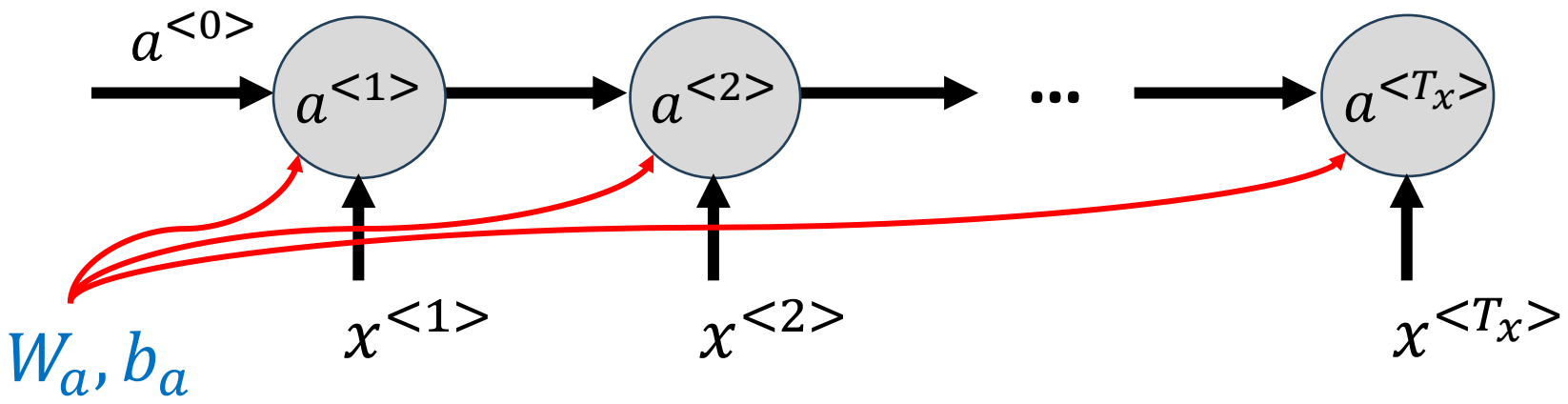
Backpropagation*:*



Machine Learning: *Recurrent Neural Network (RNN)*

Forward Propagation*:*

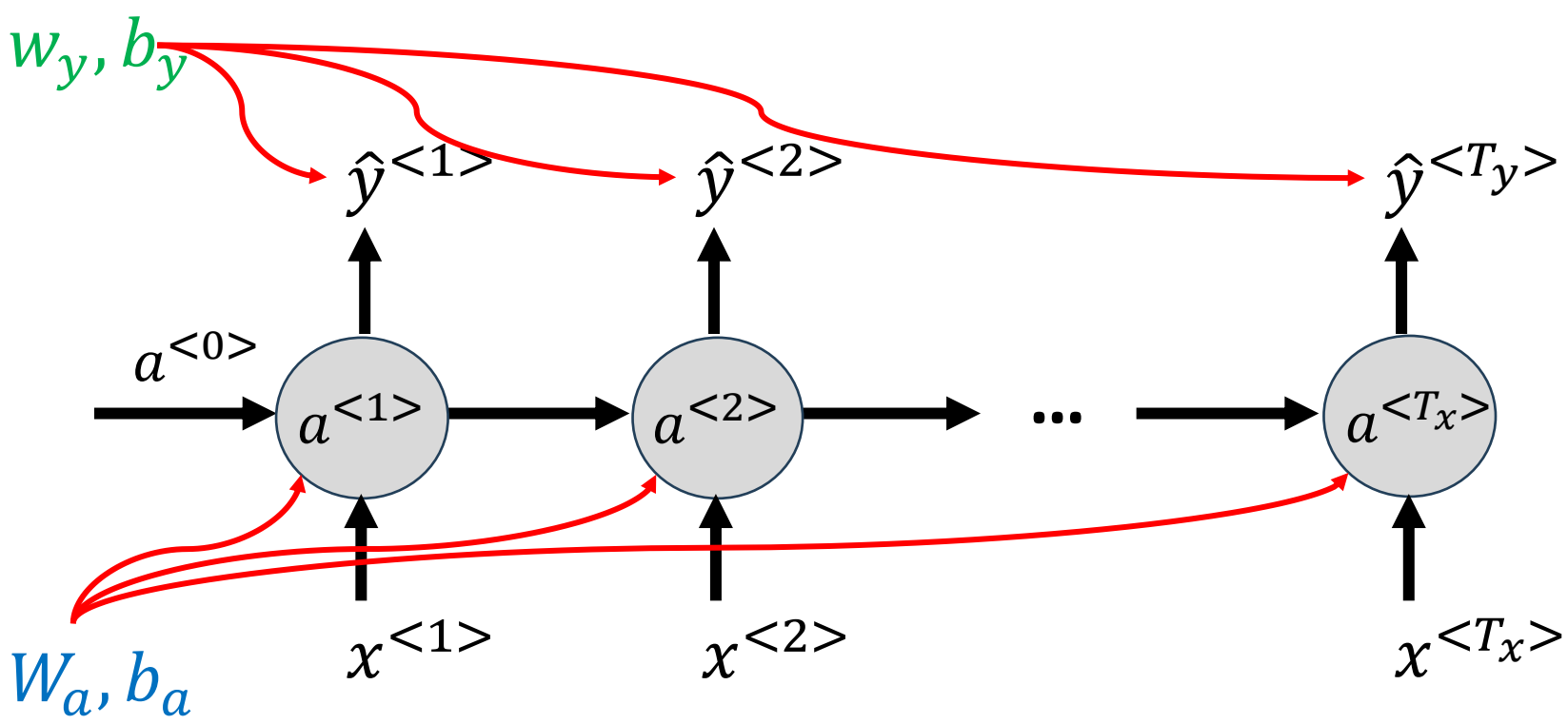
Calculation



Machine Learning: *Recurrent Neural Network (RNN)*

Forward Propagation:

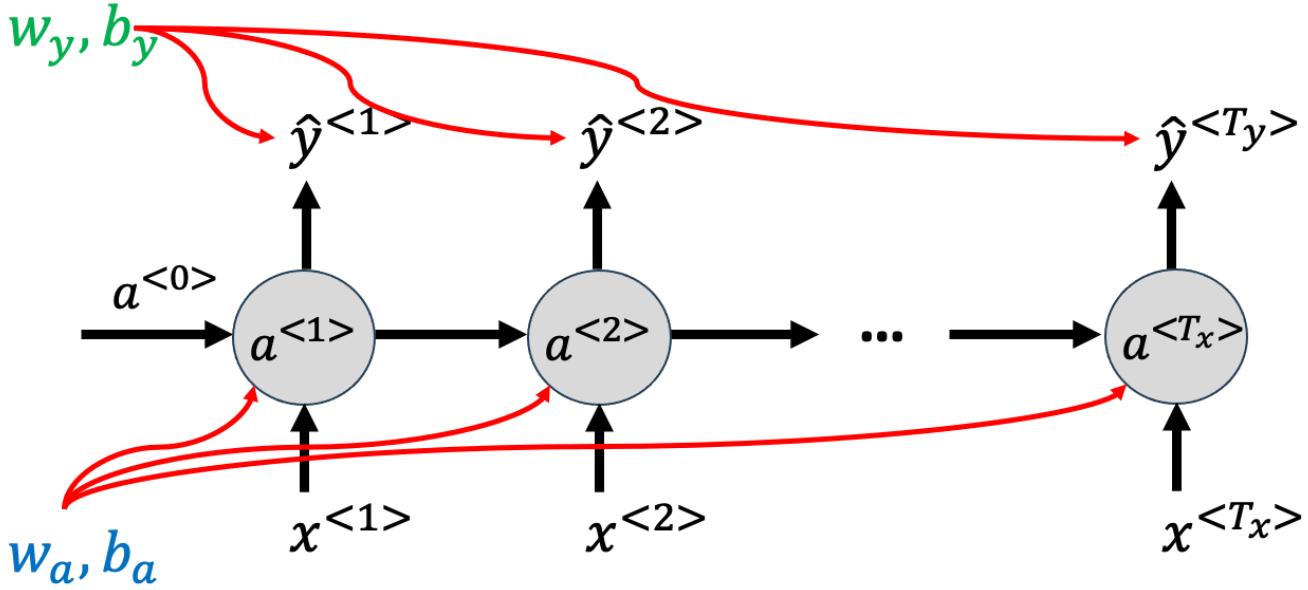
Calculation



Machine Learning: *Recurrent Neural Network (RNN)*

Backpropagation*:*

Calculation

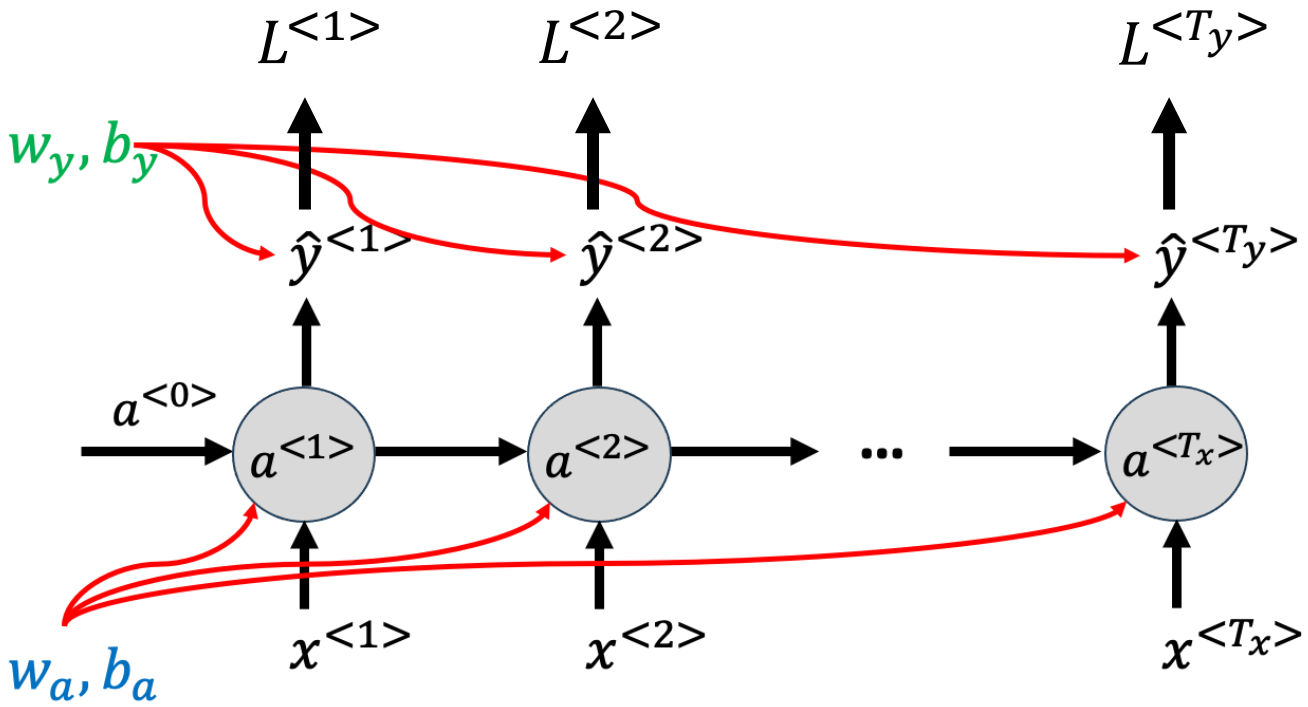


$$L^{<t>}(\hat{y}^{<t>}, y^{<t>}) = -\,y^{<t>}\log(\hat{y}^{<t>}) - \,(1 - y^{<t>})\log(1 - \hat{y}^{<t>})$$

loss for a single word

Machine Learning: *Recurrent Neural Network (RNN)*

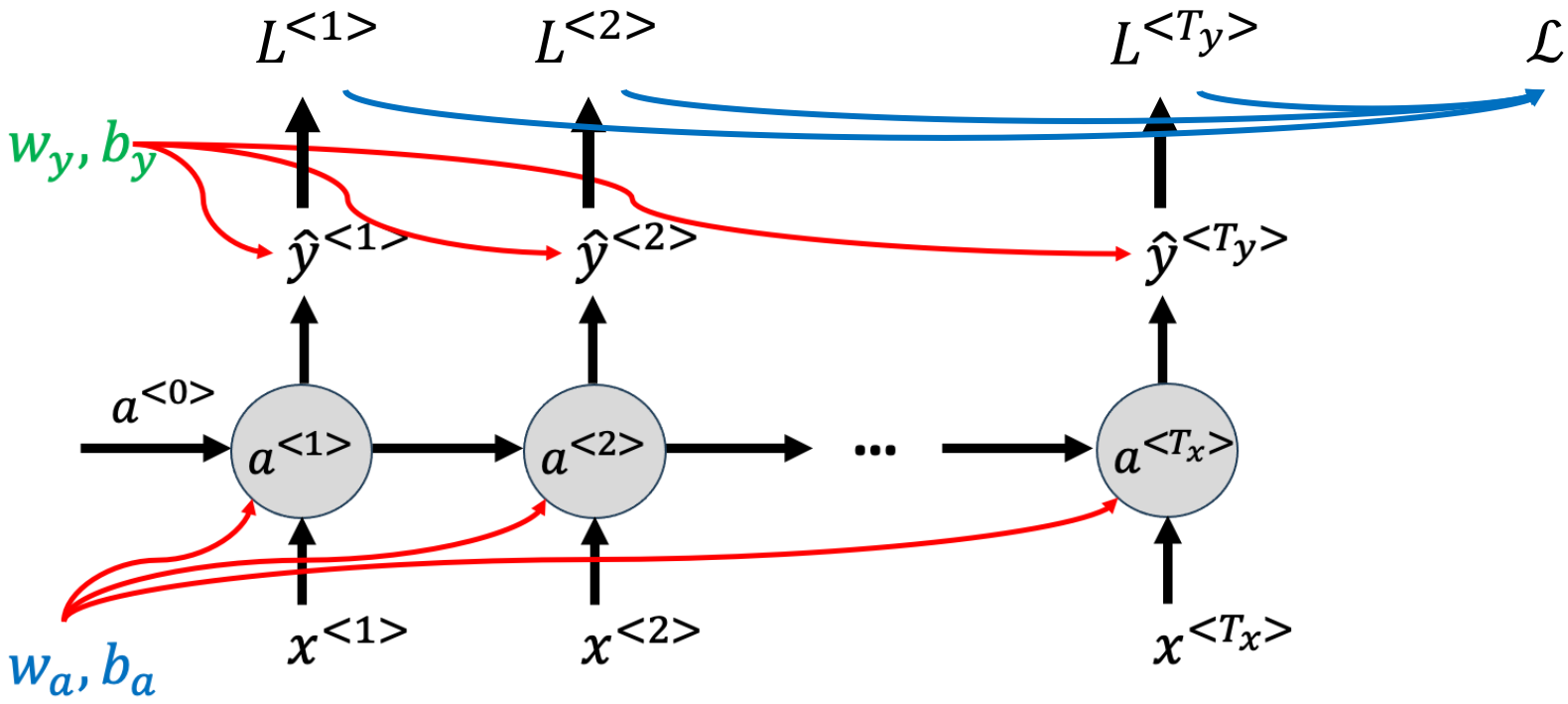
Backpropagation:
Calculation



$$L^{<t>}(\hat{y}^{<t>}, y^{<t>}) = -\,y^{<t>}\log(\hat{y}^{<t>}) - \,(1 - y^{<t>})\log(1 - \hat{y}^{<t>})$$

Machine Learning: *Recurrent Neural Network (RNN)*

Backpropagation:
Calculation



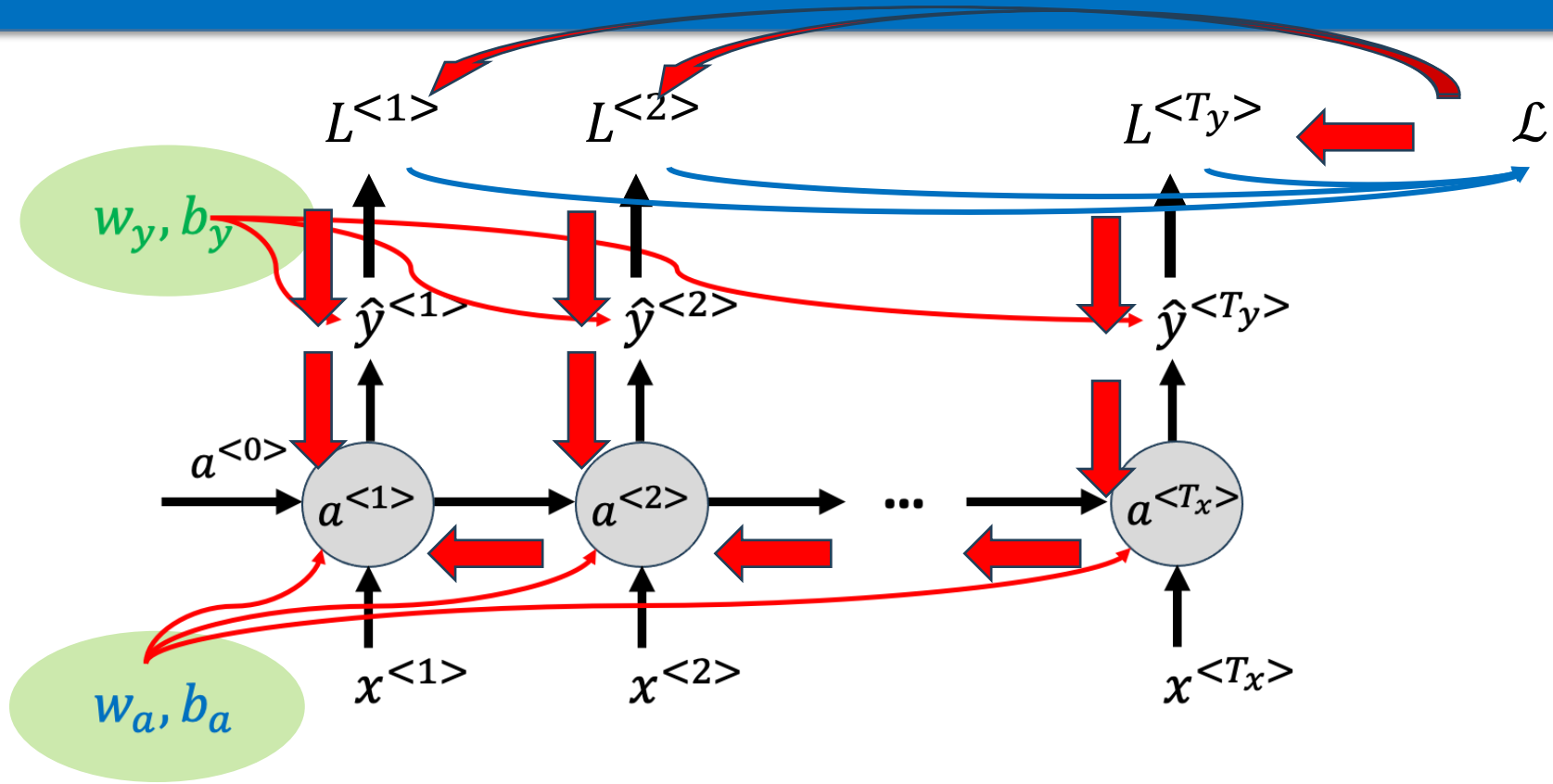
$$L^{<t>}(\hat{y}^{<t>}, y^{<t>}) = -\, y^{<t>}\log(\hat{y}^{<t>}) - \,(1 - y^{<t>})\log(1 - \hat{y}^{<t>})$$

$$\mathcal{L}(\hat{y}, y) = \sum_{t=1}^{T_y} L^{<t>}(\hat{y}^{<t>}, y^{<t>})$$

loss for the whole sequence

Machine Learning: *Recurrent Neural Network (RNN)*

Backpropagation:
Calculation



$$L^{<t>}(\hat{y}^{<t>}, y^{<t>}) = -\,y^{<t>}\log(\hat{y}^{<t>}) - \,(1 - y^{<t>})\log(1 - \hat{y}^{<t>})$$

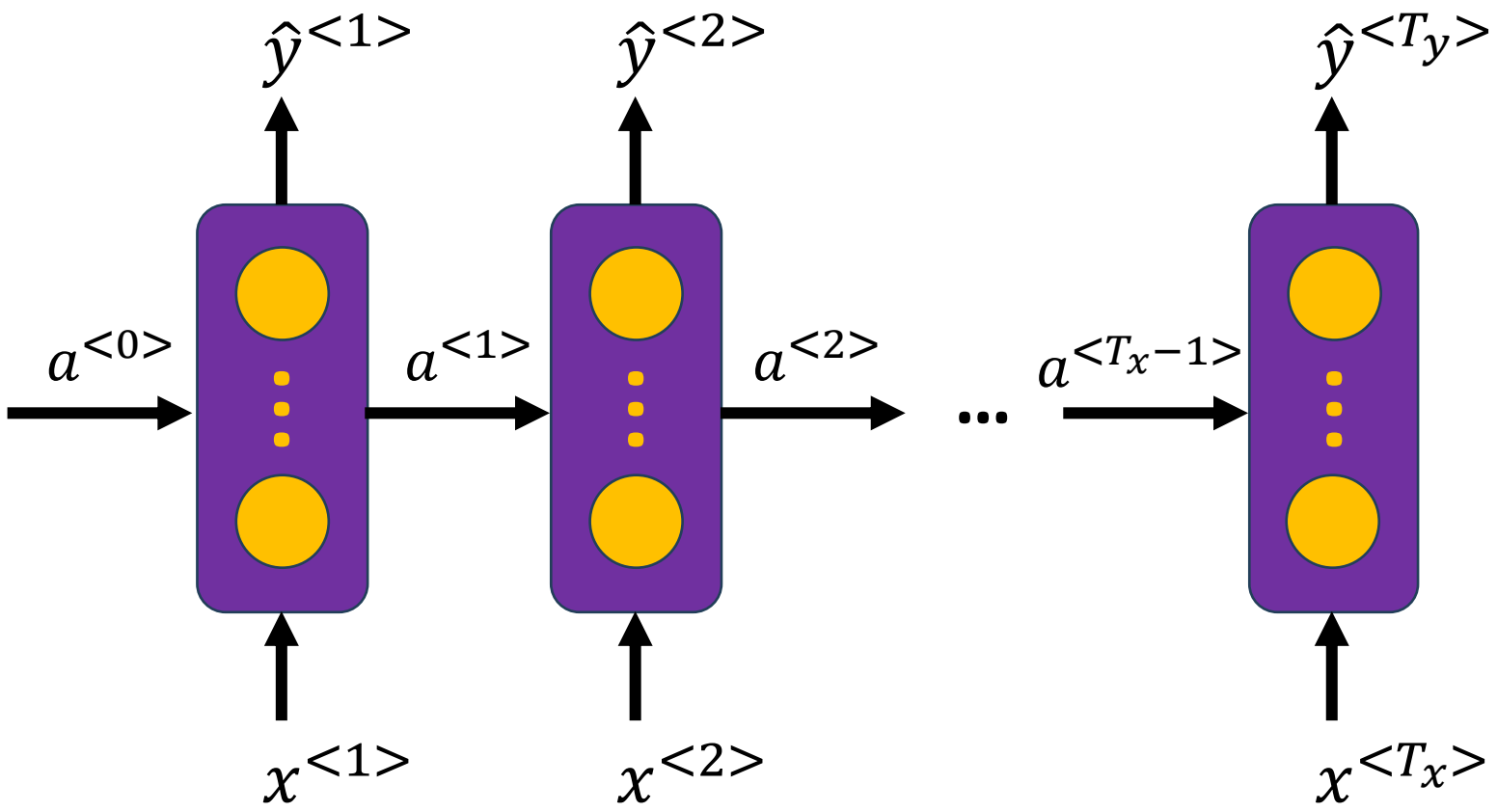
$$\mathcal{L}(\hat{y}, y) = \sum_{t=1}^{T_y} L^{<t>}(\hat{y}^{<t>}, y^{<t>})$$

Machine Learning: *Recurrent Neural Network (RNN)*

RNN Architectures

Many to Many

$T_y = T_x$

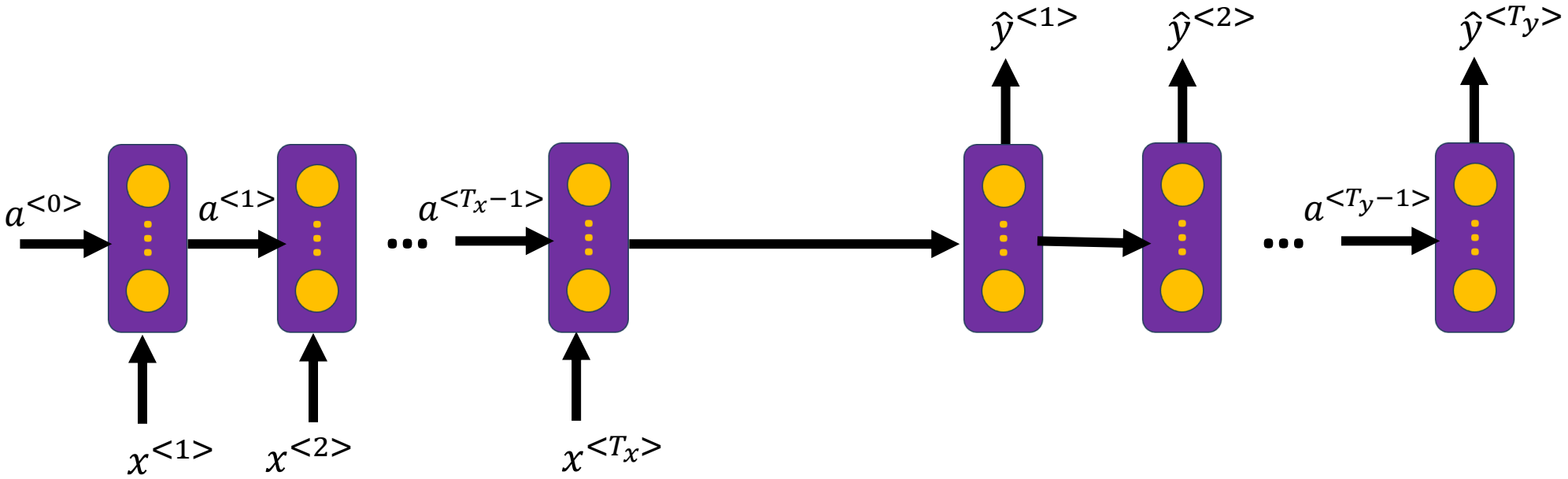


Machine Learning: *Recurrent Neural Network (RNN)*

RNN Architectures

Many to Many

$T_y \neq T_x$

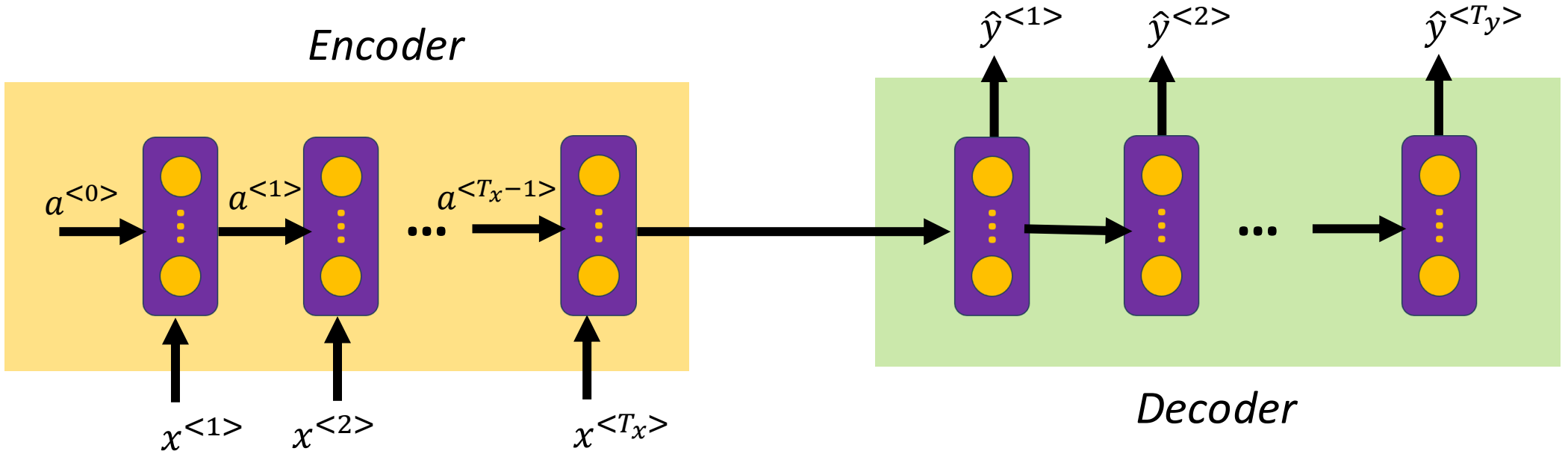


Machine Learning: *Recurrent Neural Network (RNN)*

RNN Architectures

Many to Many

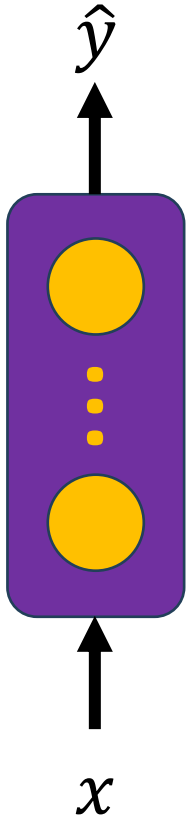
$T_y \neq T_x$



Machine Learning: *Recurrent Neural Network (RNN)*

RNN Architectures

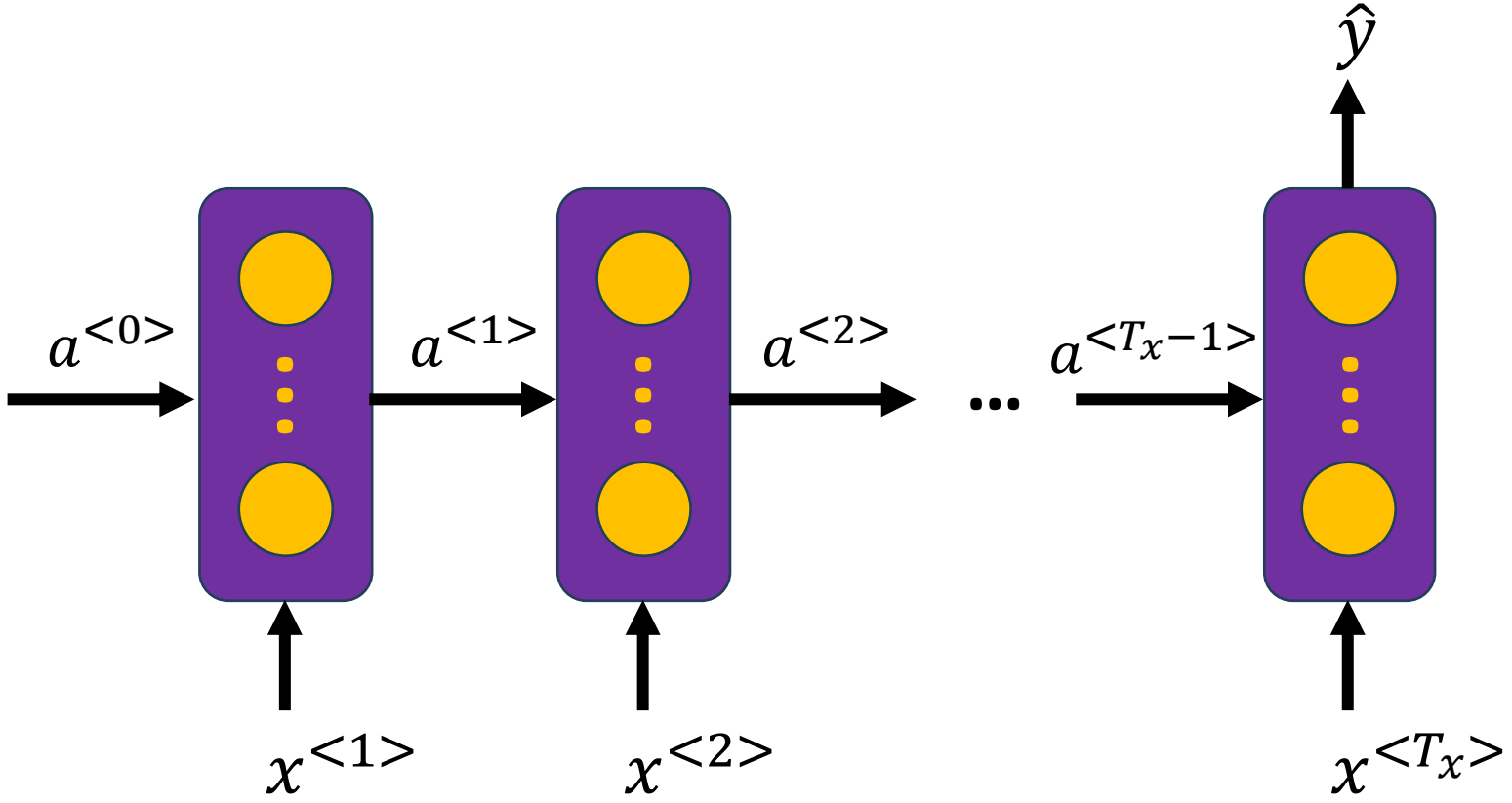
One to One



Machine Learning: *Recurrent Neural Network (RNN)*

RNN Architectures

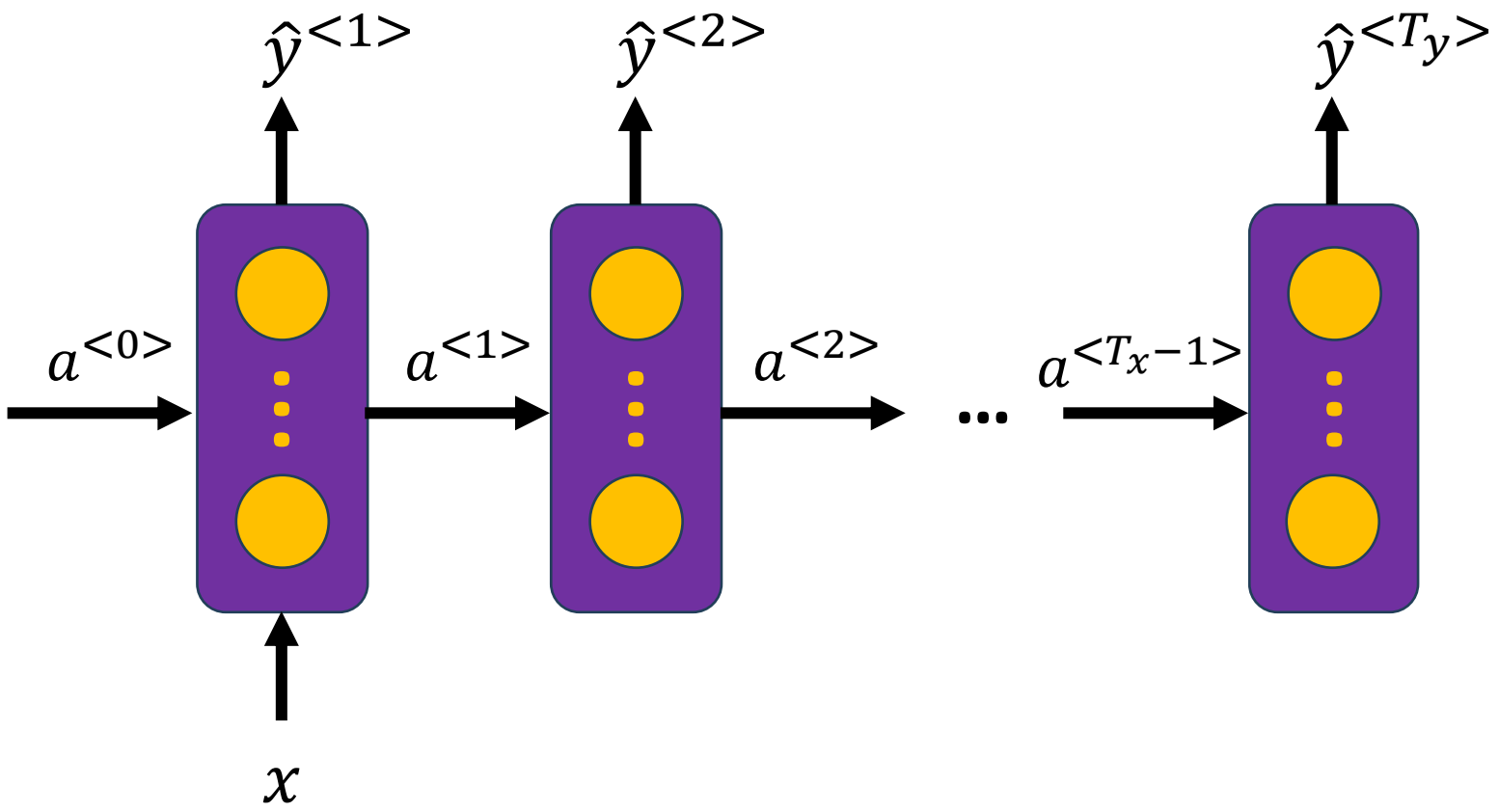
Many to One



Machine Learning: *Recurrent Neural Network (RNN)*

RNN Architectures

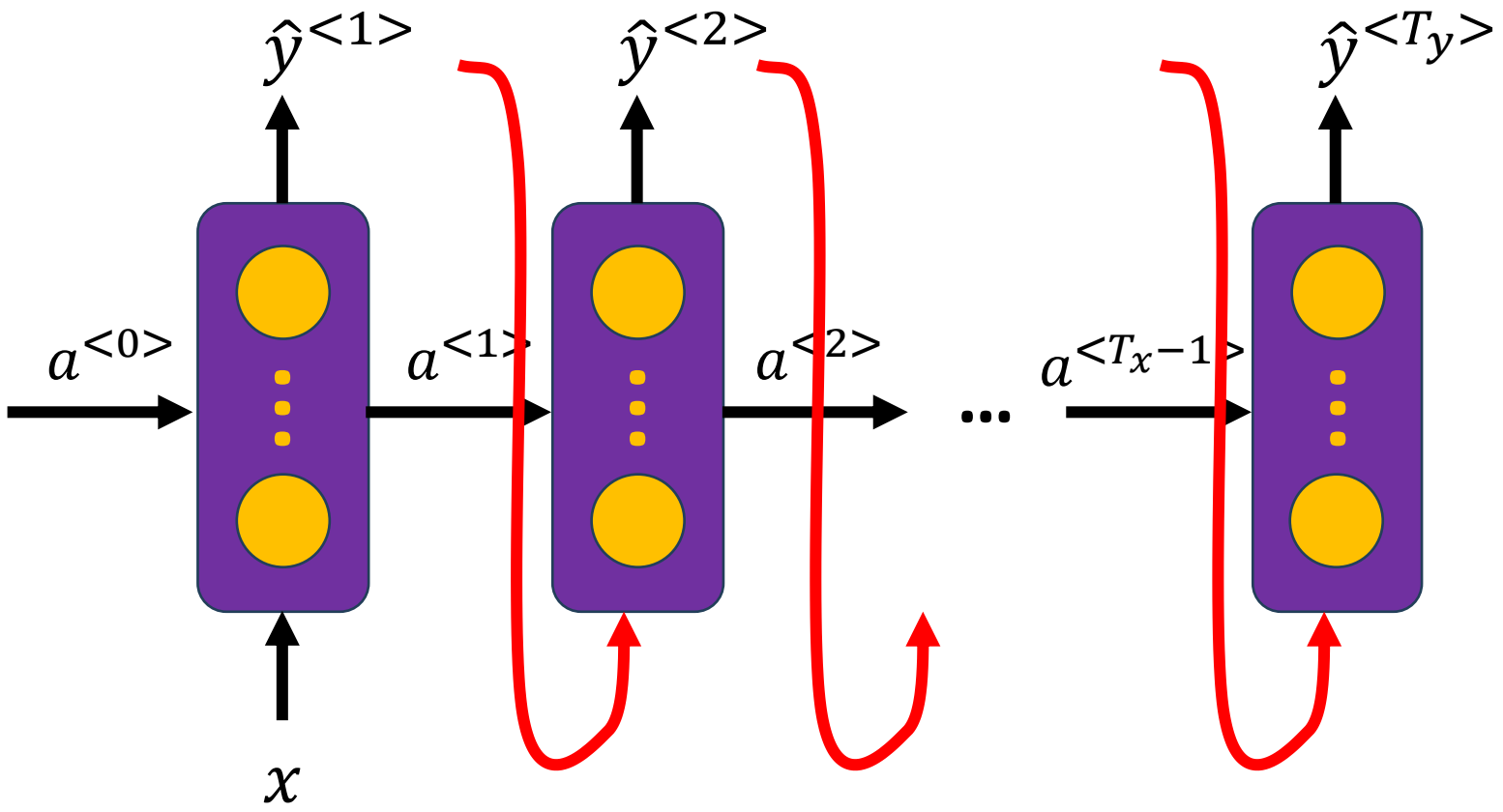
One to Many



Machine Learning: *Recurrent Neural Network (RNN)*

RNN Architectures

One to Many



Machine Learning: *Recurrent Neural Network (RNN)*

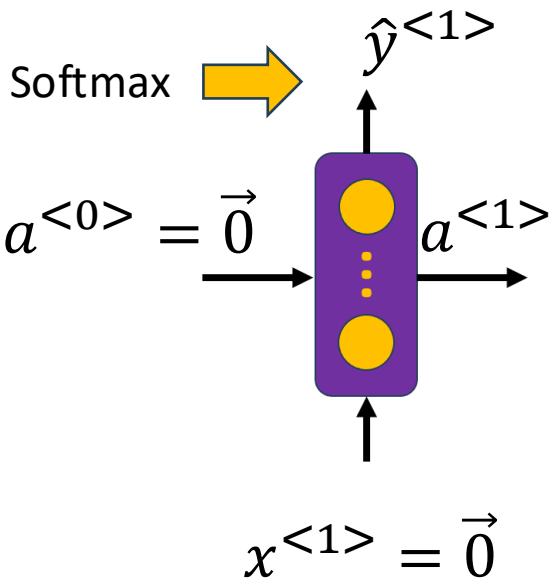
RNN for Language Modeling

“There is a nodule in the left lobe of lung <EOS>”

Machine Learning: *Recurrent Neural Network (RNN)*

RNN for Language Modeling

"There is a nodule in the left lobe of lung <EOS>"

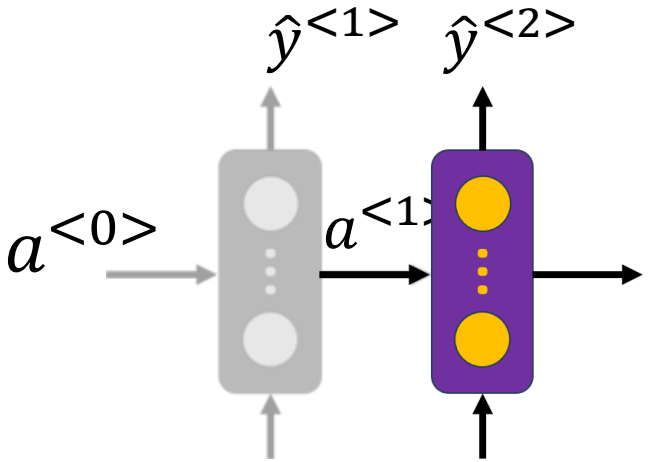


$\hat{y}^{<1>}$: probability of the next word being any of the words in the dictionary

Machine Learning: *Recurrent Neural Network (RNN)*

RNN for Language Modeling

"There is a nodule in the left lobe of lung <EOS>"



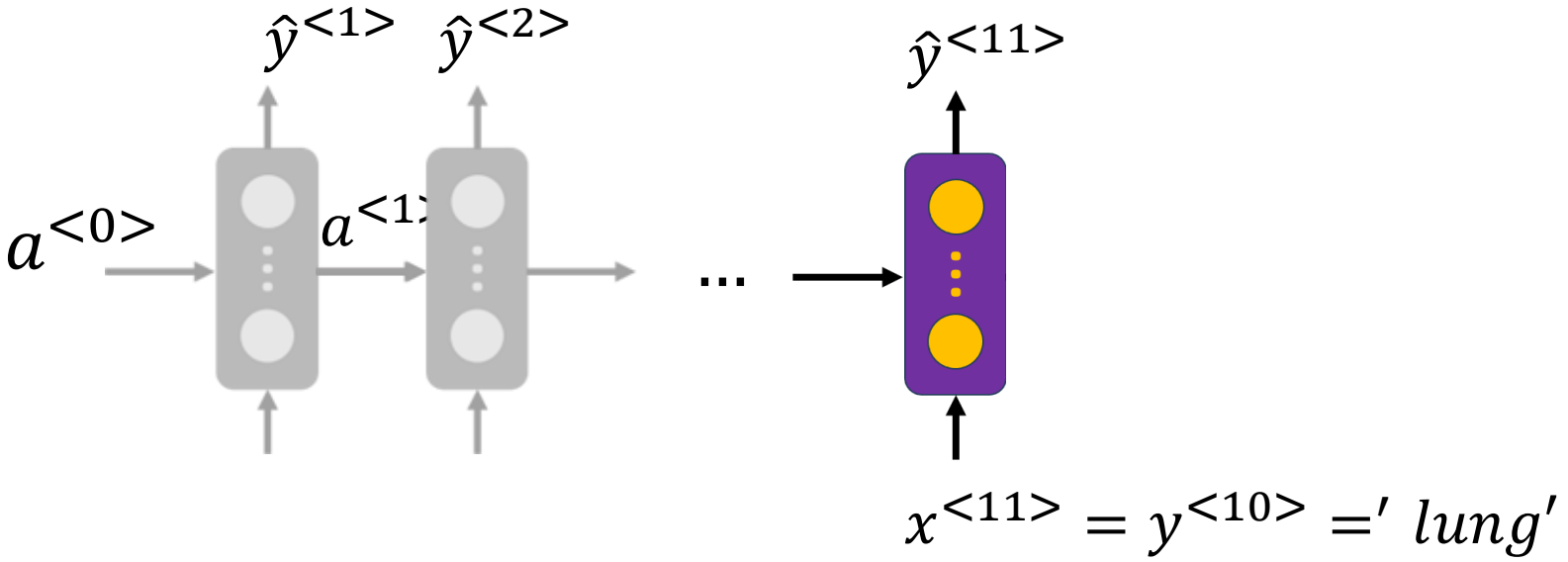
$\hat{y}^{<2>}$: probability of the next word given previous known word(s)

$$x^{<2>} = y^{<1>} = 'there'$$

Machine Learning: *Recurrent Neural Network (RNN)*

RNN for Language Modeling

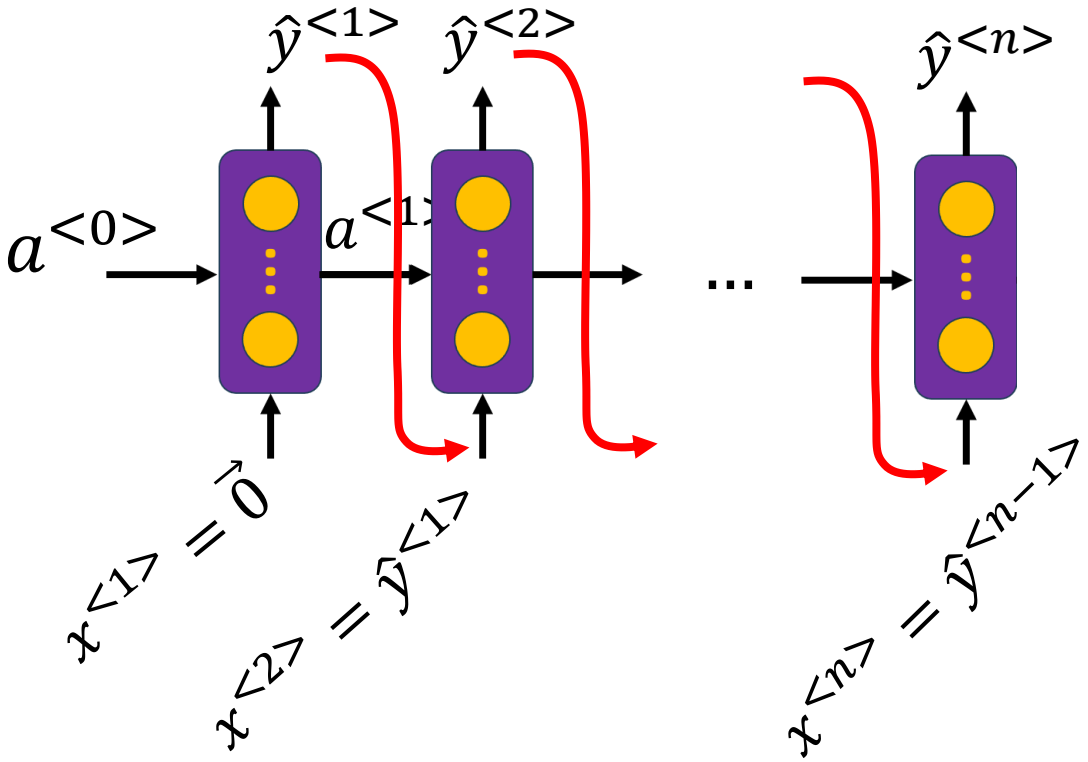
"There is a nodule in the left lobe of lung <EOS>"



Machine Learning: *Recurrent Neural Network (RNN)*

RNN for Language Modeling

Inference



Machine Learning: *Recurrent Neural Network (RNN)*

RNN for Language Modeling

DEMO

https://transformer.huggingface.co/

Machine Learning: *Recurrent Neural Network (RNN)*

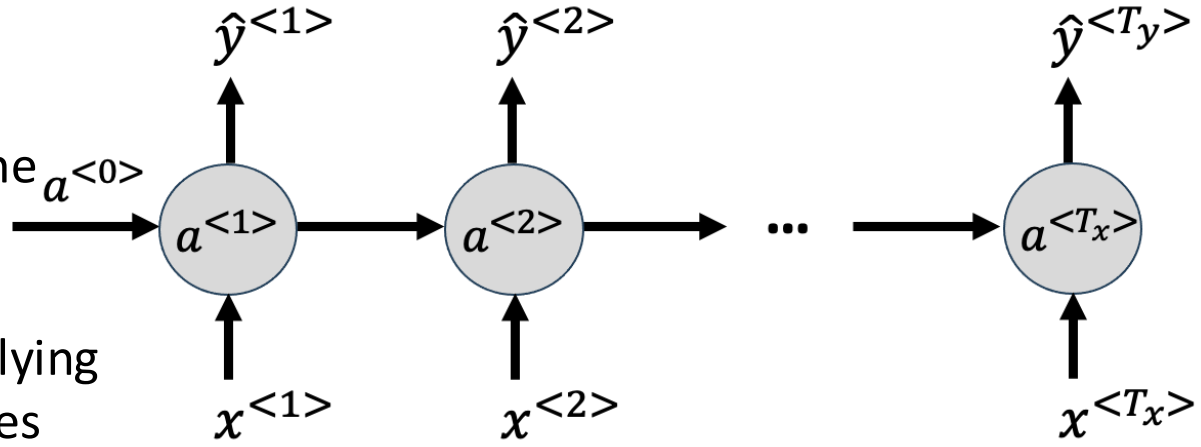
Vanishing Gradients

What: a challenge encountered in training deep neural networks, particularly those using gradient-based learning methods and backpropagation.

When: Occurs when the gradients of the network's weights become very small, effectively preventing the weights from changing their values.

Why: Gradients are propagated backward by multiplying the local derivatives layer by layer. If these derivatives are consistently less than 1 (e.g., with certain activation functions like the sigmoid), repeated multiplication causes the gradient to diminish exponentially.

So: As a result, the network stops learning or learns very slowly, making it difficult to train deep networks effectively.

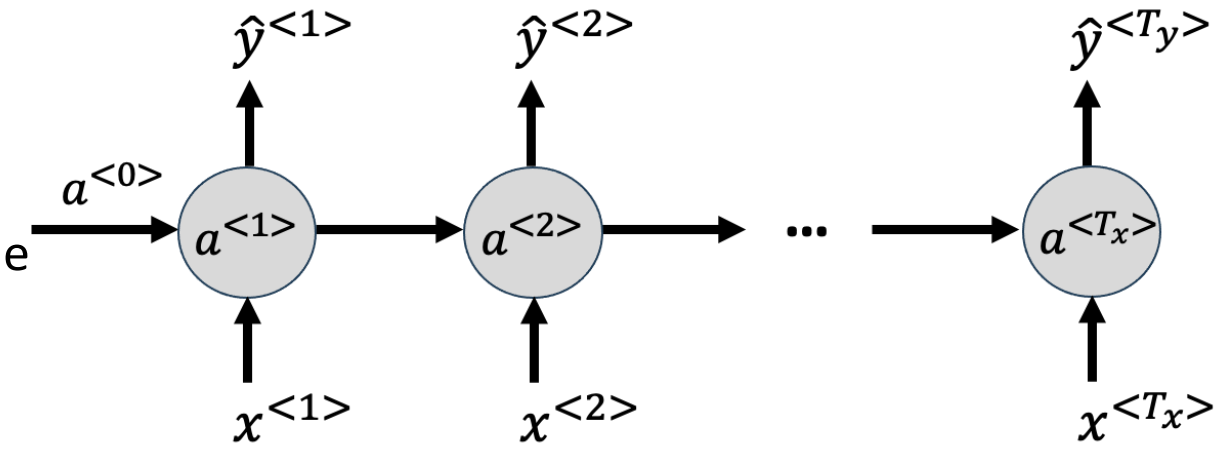


Machine Learning: *Recurrent Neural Network (RNN)*

Exploding Gradients

What: a challenge encountered in training deep neural networks, particularly those using gradient-based learning methods and backpropagation.

When: The opposite of vanishing gradient, where gradients become excessively large, causing extreme weight updates.



Why:

- Multiplying a series of derivatives greater than 1 can make gradients explode exponentially.
- High learning rates can exacerbate this issue.

So: Training can become wildly unstable, with loss and parameters oscillating or diverging.

Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU):

Example

….

There is a nodule in the left lobe of lung. …… The right lobe is clear.

….

The right lobe of breast…..

Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU):

Example

….

There is a nodule in the left lobe of lung. …… The right lobe is clear.

….

The right lobe of breast…..

term	context
lobe	lung
. . .	No change
lobe	No change
. . .	No change
lobe	breast
. . .	

Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU):

Cho, Kyunghyun, Bengio, Yoshua et al. (2014). "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation". *Association for Computational Linguistics*.
Link: https://arxiv.org/abs/1406.1078

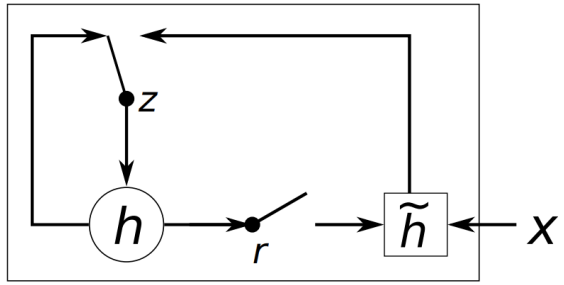


Figure 2: An illustration of the proposed hidden activation function. The update gate z selects whether the hidden state is to be updated with a new hidden state $\tilde{h}$. The reset gate r decides whether the previous hidden state is ignored. See Eqs. (5)–(8) for the detailed equations of r, z, h and $\tilde{h}$.

Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU):

Simplified

$c^{<t>} = a^{<t>}$ Memory Cell

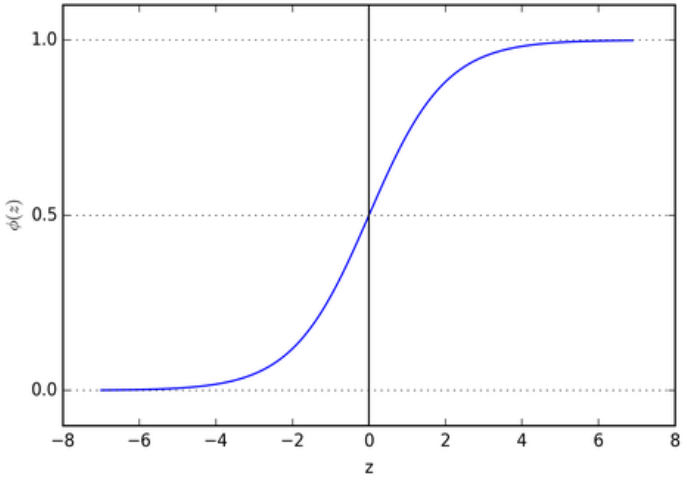
Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU):

Simplified

Gate $\Gamma_u = \sigma(W_u.[c^{<t-1>}, x^{<t>}] + b_u)$

Update



Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU):

Simplified

$$\Gamma_u$$

$$c^{<t>} = 1$$

There is a nodule in the left lobe of lung. …… The right lobe is clear.

Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU):

Simplified

There is a nodule in the left lobe of lung. …… The right lobe is clear.

….

$$c^{<t>} = 0$$

The right lobe of breast…..

Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU):

Simplified

$$\Gamma_u = \sigma(W_u.[c^{<t-1>}, x^{<t>}] + b_u)$$

$$\tilde{c}^{<t>} = \tanh(W_c.[c^{<t-1>}, x^{<t>}] + b_c)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

Elementwise Multiplication

Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU)

Intuition

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$\Gamma_u = 0 \rightarrow c^{<t>} = c^{<t-1>}$$

REMEMBER YOUR MEMORY

Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU)

Intuition

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$\Gamma_u = 0 \rightarrow c^{<t>} = c^{<t-1>}$ *REMEMBER YOUR MEMORY*

$\Gamma_u = 1 \rightarrow c^{<t>} = \tilde{c}^{<t>}$ *FORGET YOUR MEMORY and UPDATE*

Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU):

Intuition

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$\Gamma_u = 1 \rightarrow c^{<t>} = \tilde{c}^{<t>}$$

No update

$\Gamma_u = 1$ $\quad$ $\Gamma_u = 0$

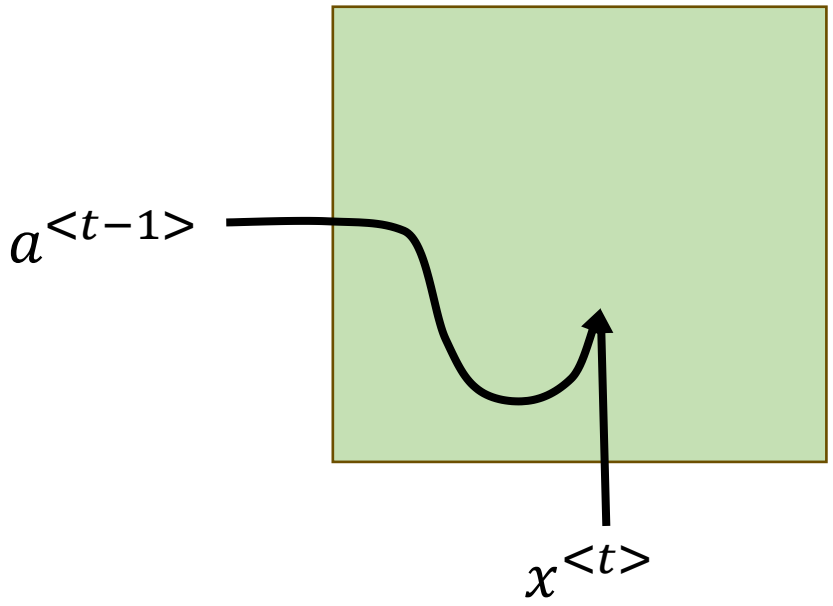
$c^{<t>} = 1$ $\longrightarrow$

There is a nodule in the left lobe of lung. …… The right lobe is clear.

Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU)

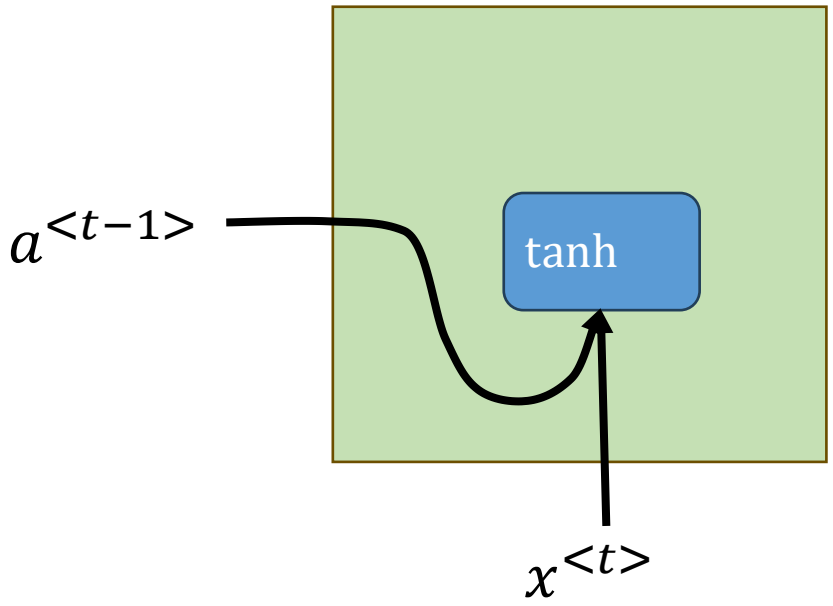
$$a^{<t>} = g_1(W_a.[a^{<t-1>}, x^{<t>}] + b_a)$$



Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU)

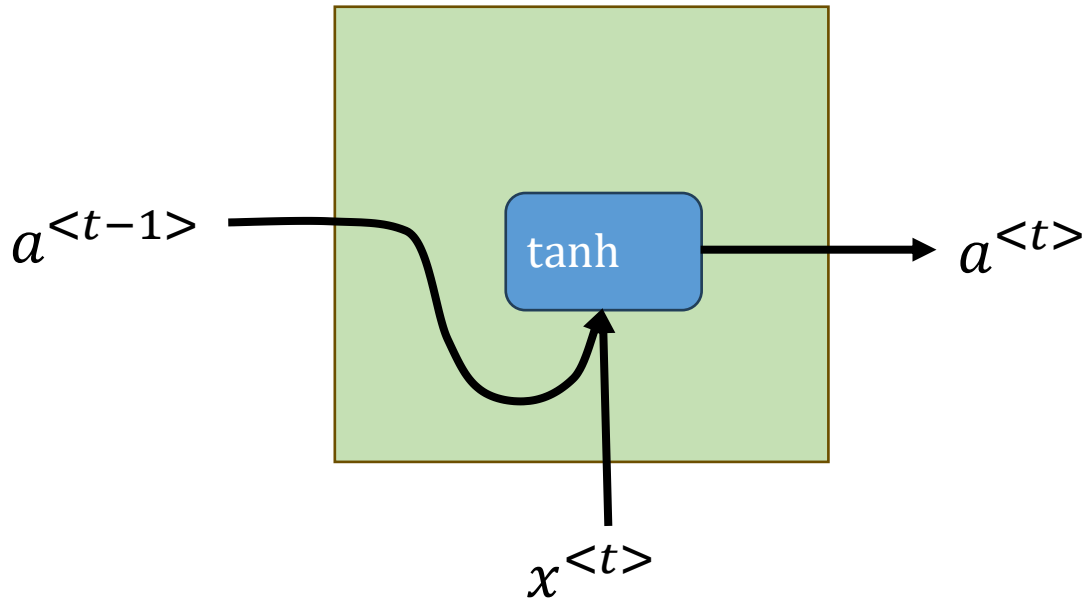
$$a^{<t>} = g_1(W_a.[a^{<t-1>}, x^{<t>}] + b_a)$$



Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU)

$$a^{<t>} = g_1(W_a.[a^{<t-1>}, x^{<t>}] + b_a)$$

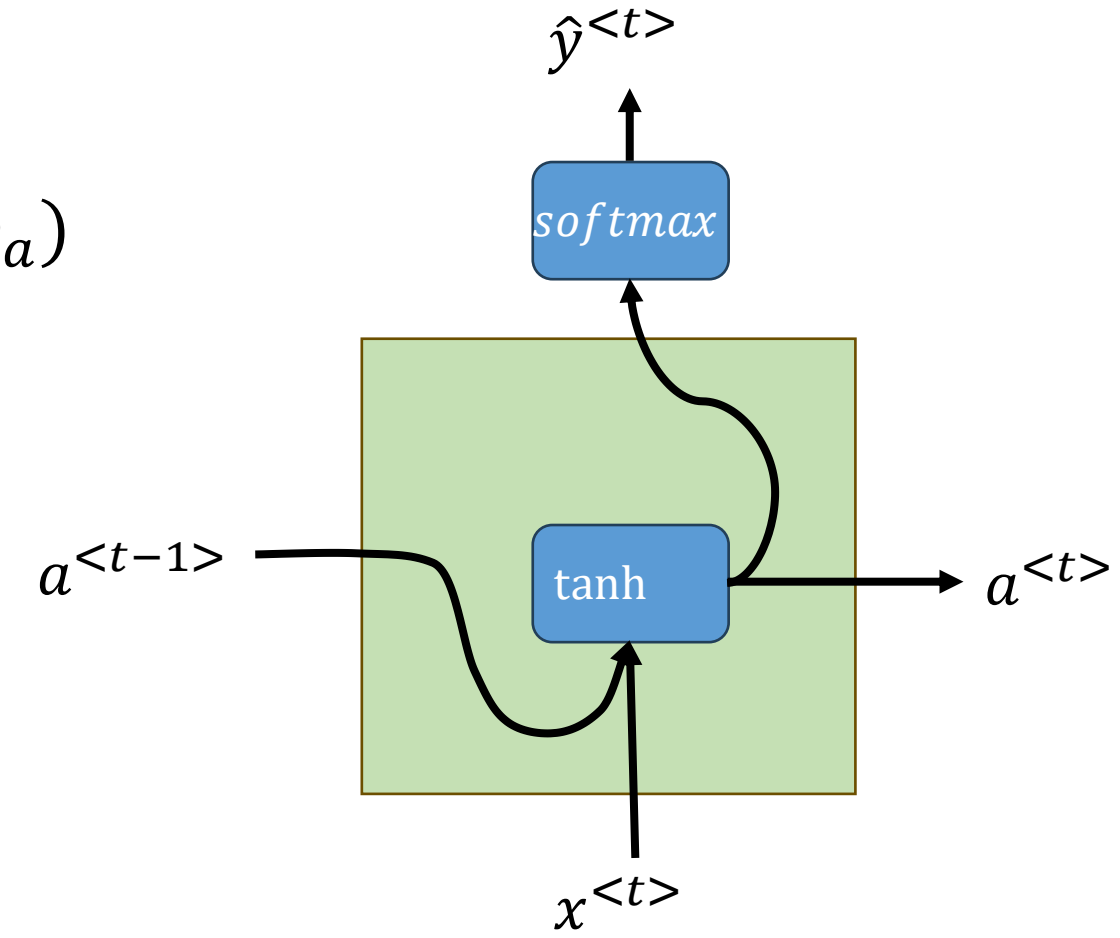


Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU)

$$a^{<t>} = g_1(W_a.[a^{<t-1>}, x^{<t>}] + b_a)$$

$$\hat{y}^{<t>} = g_2(w_y.a^{<t>} + b_y)$$



Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU):

Simplified

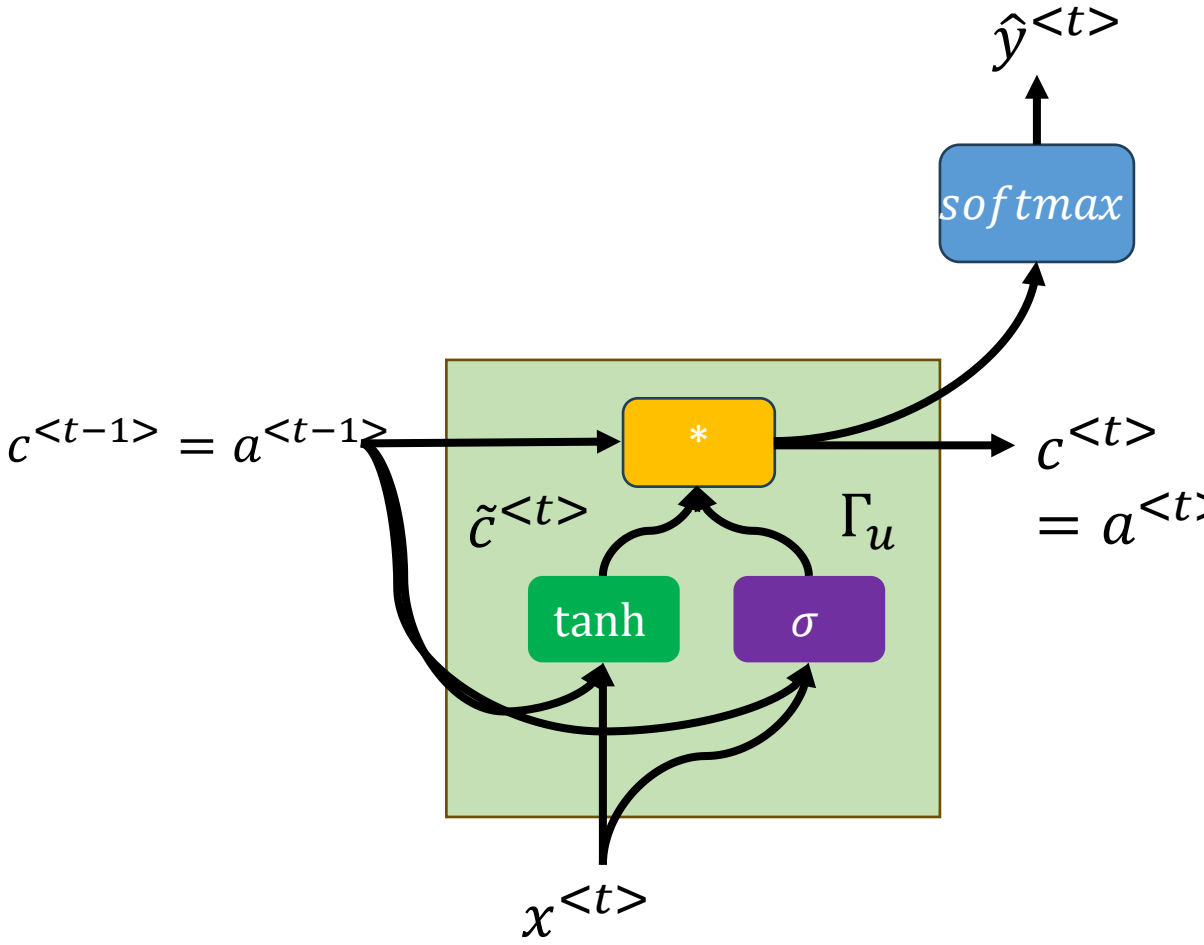
$$c^{<t>} = a^{<t>}$$

$$\tilde{c}^{<t>} = \tanh(W_c.[c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u.[c^{<t-1>}, x^{<t>}] + b_u)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$



Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU):

Full Version

$$c^{<t>} = a^{<t>}$$

$$\tilde{c}^{<t>} = \tanh(W_c.[c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u.[c^{<t-1>}, x^{<t>}] + b_u)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

Machine Learning: *Recurrent Neural Network (RNN)*

Gated Recurrent Unit (GRU):

Full Version

$$c^{<t>} = a^{<t>}$$

$$\tilde{c}^{<t>} = \tanh(W_c.[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

How relevant $c^{<t-1>}$ is to $c^{<t>}$

$$\Gamma_u = \sigma(W_u.[c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r.[c^{<t-1>}, x^{<t>}] + b_r)$$

Relevant

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

Machine Learning: *Recurrent Neural Network (RNN)*

Long-Short-Term Memory (LSTM)

Sepp Hochreiter, Jurgen Schmidhuber, 1997. "Long Short-Term Memory"
Link: https://deeplearning.cs.cmu.edu/F23/document/readings/LSTM.pdf

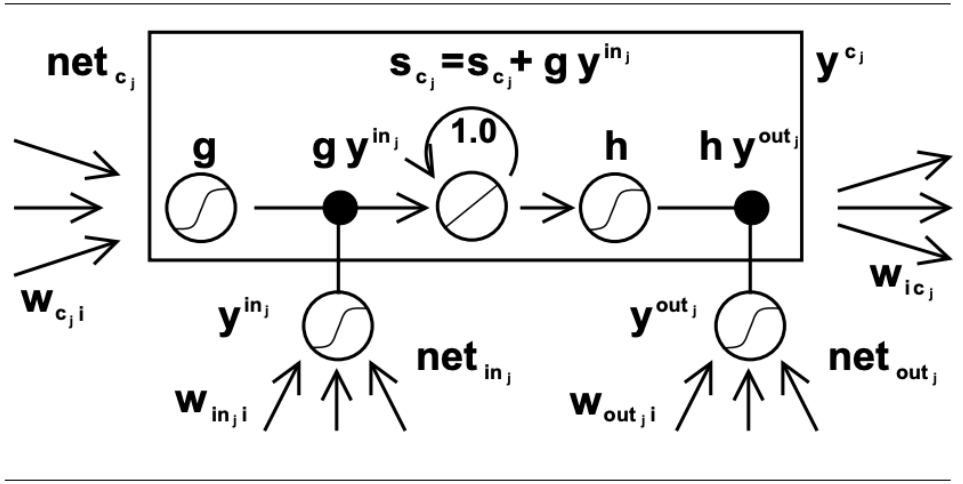


Figure 1: Architecture of memory cell c_j (the box) and its gate units in_j, out_j. The self-recurrent connection (with weight 1.0) indicates feedback with a delay of one time step. It builds the basis of the CEC. The gate units open and close access to CEC. See text and appendix A.1 for details.

Machine Learning: *Recurrent Neural Network (RNN)*

Long-Short-Term Memory (LSTM)

GRU

$$\tilde{c}^{<t>} = \tanh(W_c.[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u.[c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r.[c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

Machine Learning: *Recurrent Neural Network (RNN)*

Long-Short-Term Memory (LSTM)

GRU

$$\tilde{c}^{<t>} = \tanh(W_c.[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u.[c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r.[c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

LSTM

$$\tilde{c}^{<t>} = \tanh(W_c.[a^{<t-1>}, x^{<t>}] + b_c)$$

Machine Learning: *Recurrent Neural Network (RNN)*

Long-Short-Term Memory (LSTM)

GRU

$$\tilde{c}^{<t>} = \tanh(W_c.[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u.[c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r.[c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

LSTM

$$\tilde{c}^{<t>} = \tanh(W_c.[a^{<t-1>}, x^{<t>}] + b_c)$$

Update $$\Gamma_u = \sigma(W_u.[a^{<t-1>}, x^{<t>}] + b_u)$$

Machine Learning: *Recurrent Neural Network (RNN)*

Long-Short-Term Memory (LSTM)

GRU

$$\tilde{c}^{<t>} = \tanh(W_c.[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u.[c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r.[c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

LSTM

$$\tilde{c}^{<t>} = \tanh(W_c.[a^{<t-1>}, x^{<t>}] + b_c)$$

Update $\Gamma_u = \sigma(W_u.[a^{<t-1>}, x^{<t>}] + b_u)$

Forget $\Gamma_f = \sigma(W_f.[a^{<t-1>}, x^{<t>}] + b_f)$

Machine Learning: *Recurrent Neural Network (RNN)*

Long-Short-Term Memory (LSTM)

GRU

$$\tilde{c}^{<t>} = \tanh(W_c.[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u.[c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r.[c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

LSTM

$$\tilde{c}^{<t>} = \tanh(W_c.[a^{<t-1>}, x^{<t>}] + b_c)$$

Update $$\Gamma_u = \sigma(W_u.[a^{<t-1>}, x^{<t>}] + b_u)$$

Forget $$\Gamma_f = \sigma(W_f.[a^{<t-1>}, x^{<t>}] + b_f)$$

Output $$\Gamma_o = \sigma(W_o.[a^{<t-1>}, x^{<t>}] + b_o)$$

Machine Learning: *Recurrent Neural Network (RNN)*

Long-Short-Term Memory (LSTM)

	GRU		LSTM
	$\tilde{c}^{<t>} = \tanh(W_c.[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$		$\tilde{c}^{<t>} = \tanh(W_c.[a^{<t-1>}, x^{<t>}] + b_c)$
	$\Gamma_u = \sigma(W_u.[c^{<t-1>}, x^{<t>}] + b_u)$	Update	$\Gamma_u = \sigma(W_u.[a^{<t-1>}, x^{<t>}] + b_u)$
	$\Gamma_r = \sigma(W_r.[c^{<t-1>}, x^{<t>}] + b_r)$	Forget	$\Gamma_f = \sigma(W_f.[a^{<t-1>}, x^{<t>}] + b_f)$
		Output	$\Gamma_o = \sigma(W_o.[a^{<t-1>}, x^{<t>}] + b_o)$
	$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$	⇒	$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$
	$a^{<t>} = c^{<t>}$		

Machine Learning: *Recurrent Neural Network (RNN)*

Long-Short-Term Memory (LSTM)

GRU

$$\tilde{c}^{<t>} = \tanh(W_c.[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u.[c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r.[c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

LSTM

$$\tilde{c}^{<t>} = \tanh(W_c.[a^{<t-1>}, x^{<t>}] + b_c)$$

Update $$\Gamma_u = \sigma(W_u.[a^{<t-1>}, x^{<t>}] + b_u)$$

Forget $$\Gamma_f = \sigma(W_f.[a^{<t-1>}, x^{<t>}] + b_f)$$

Output $$\Gamma_o = \sigma(W_o.[a^{<t-1>}, x^{<t>}] + b_o)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_o * \tanh(c^{<t>})$$

Machine Learning: *Recurrent Neural Network (RNN)*

Long-Short-Term Memory (LSTM)

$$\tilde{c}^{<t>} = \tanh(W_c.[\Gamma_r * a^{<t-1>}, x^{<t>}] \text{ -}$$

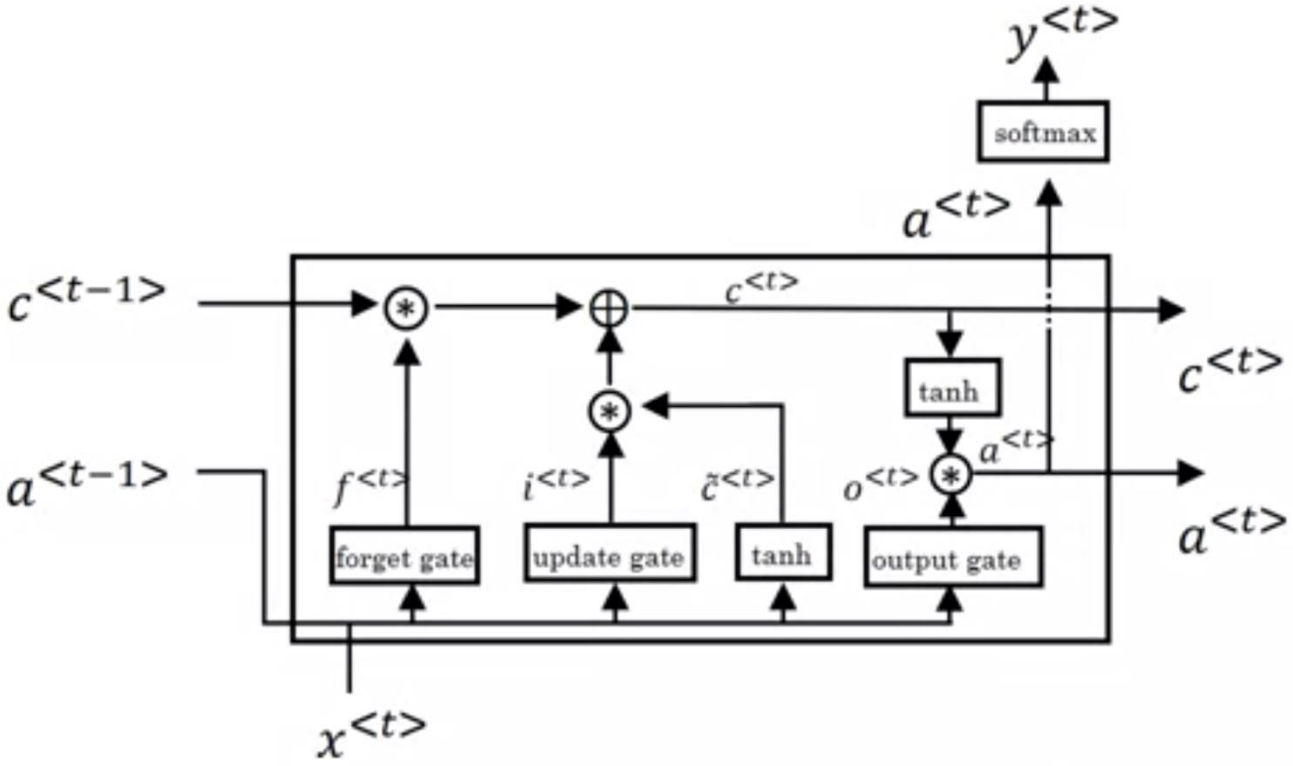
$$\Gamma_u = \sigma(W_u.[a^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_f = \sigma\big(W_f.[a^{<t-1>}, x^{<t>}] + b_f\big)$$

$$\Gamma_o = \sigma(W_o.[a^{<t-1>}, x^{<t>}] + b_o)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

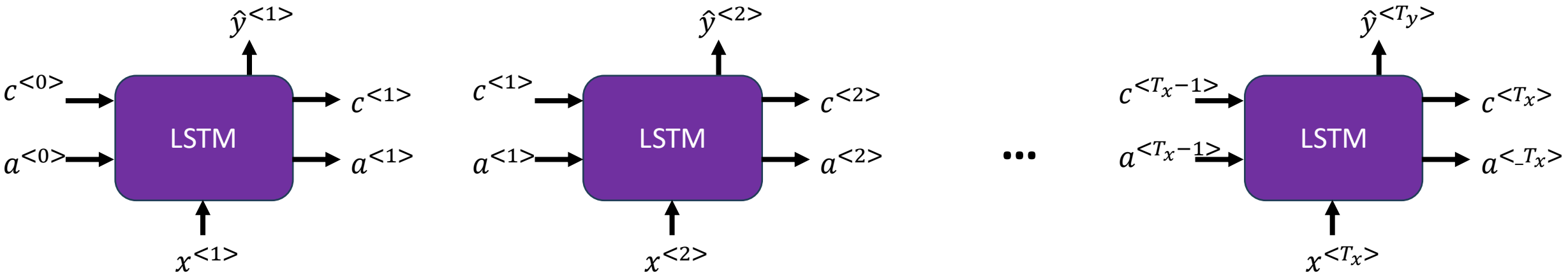
$$a^{<t>} = \Gamma_o * \tanh(c^{<t>})$$



Machine Learning: *Recurrent Neural Network (RNN)*

Long-Short-Term Memory (LSTM)

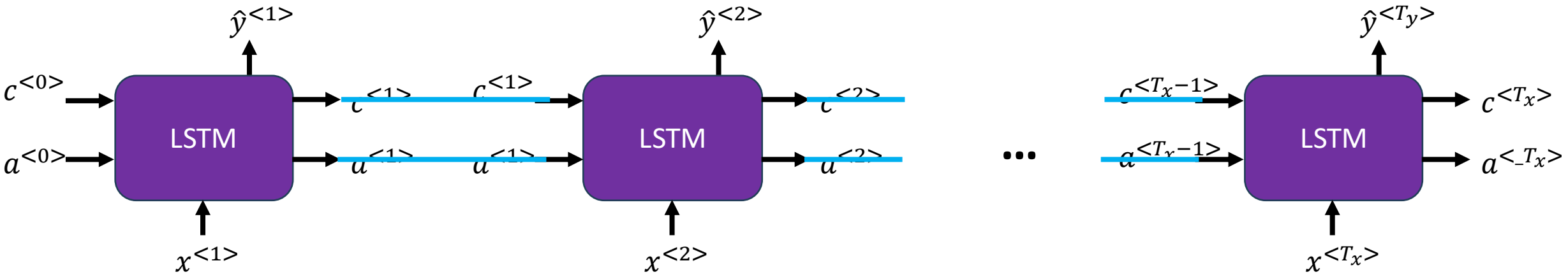
Cascade



Machine Learning: *Recurrent Neural Network (RNN)*

Long-Short-Term Memory (LSTM)

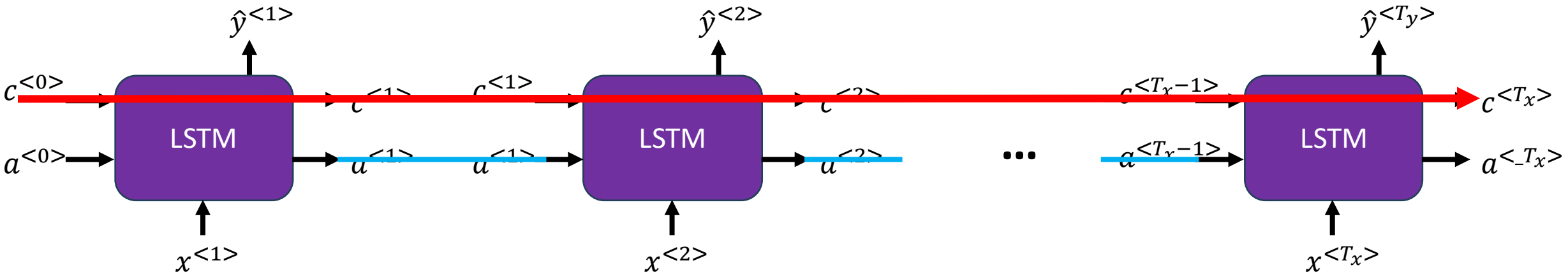
Cascade



Machine Learning: *Recurrent Neural Network (RNN)*

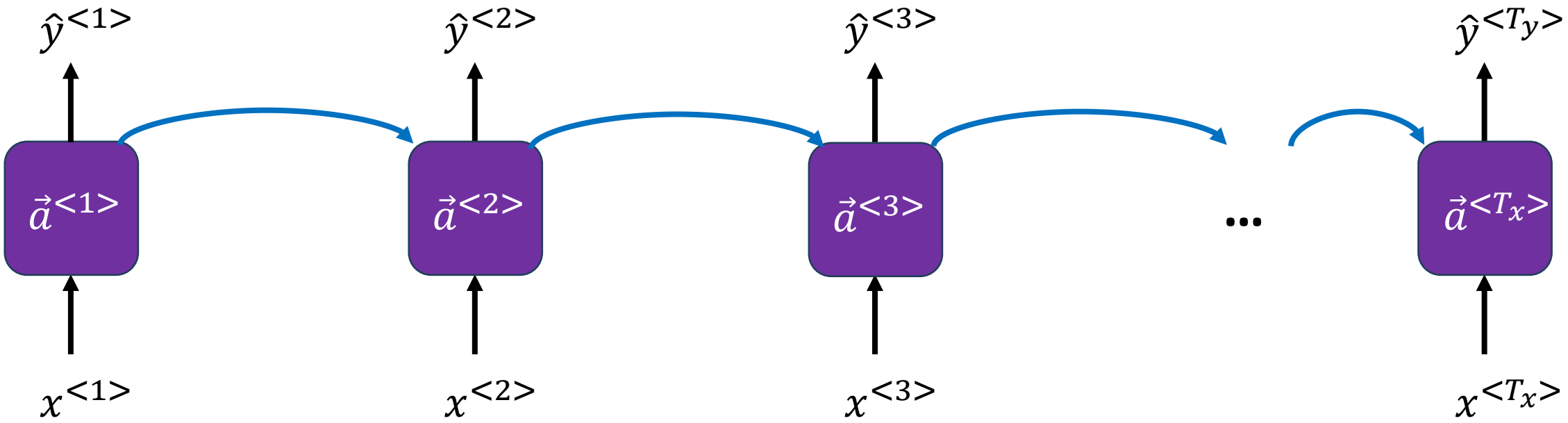
Long-Short-Term Memory (LSTM)

Cascade



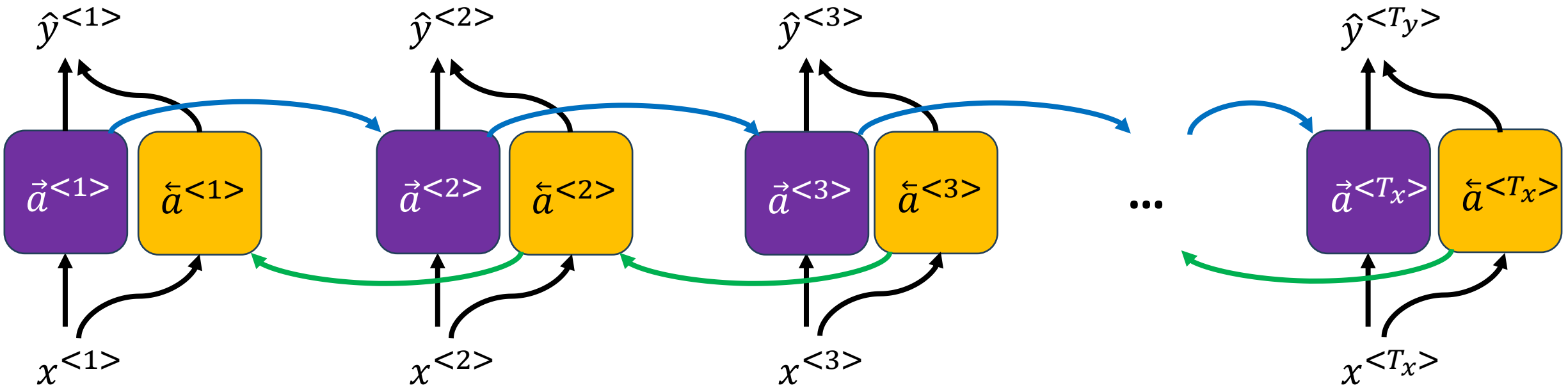
Machine Learning: *Recurrent Neural Network (RNN)*

Bi-directional RNN



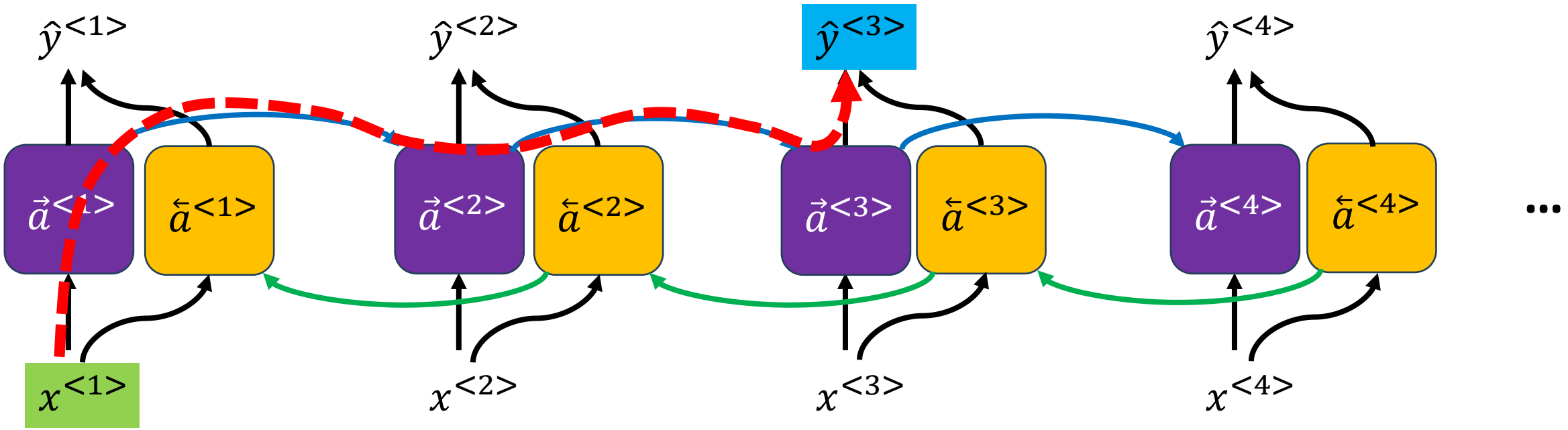
Machine Learning: *Recurrent Neural Network (RNN)*

Bi-directional RNN



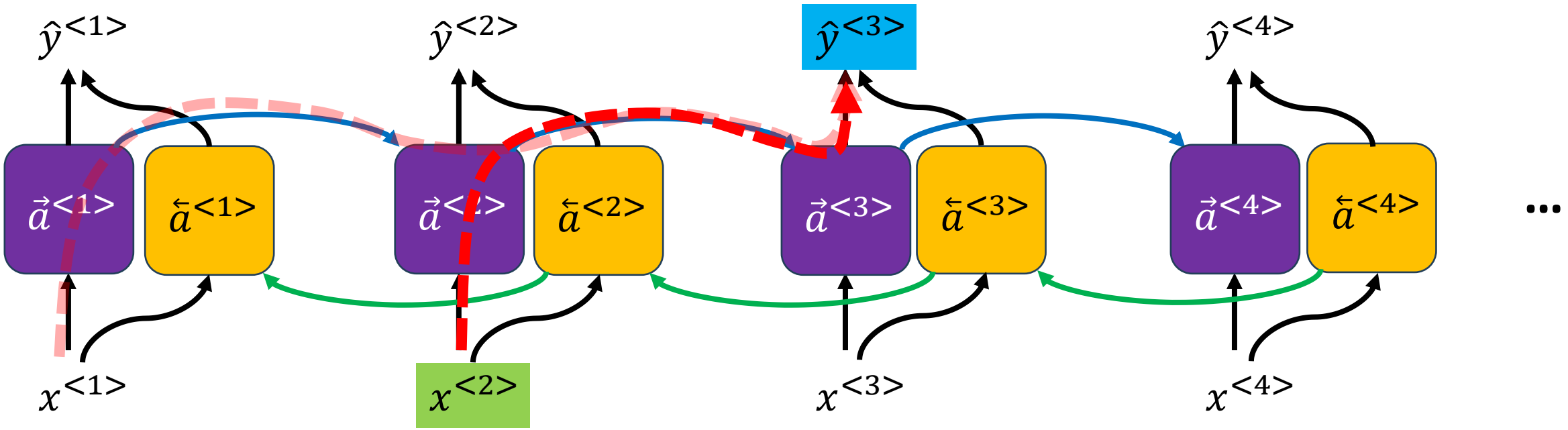
Machine Learning: *Recurrent Neural Network (RNN)*

Bi-directional RNN



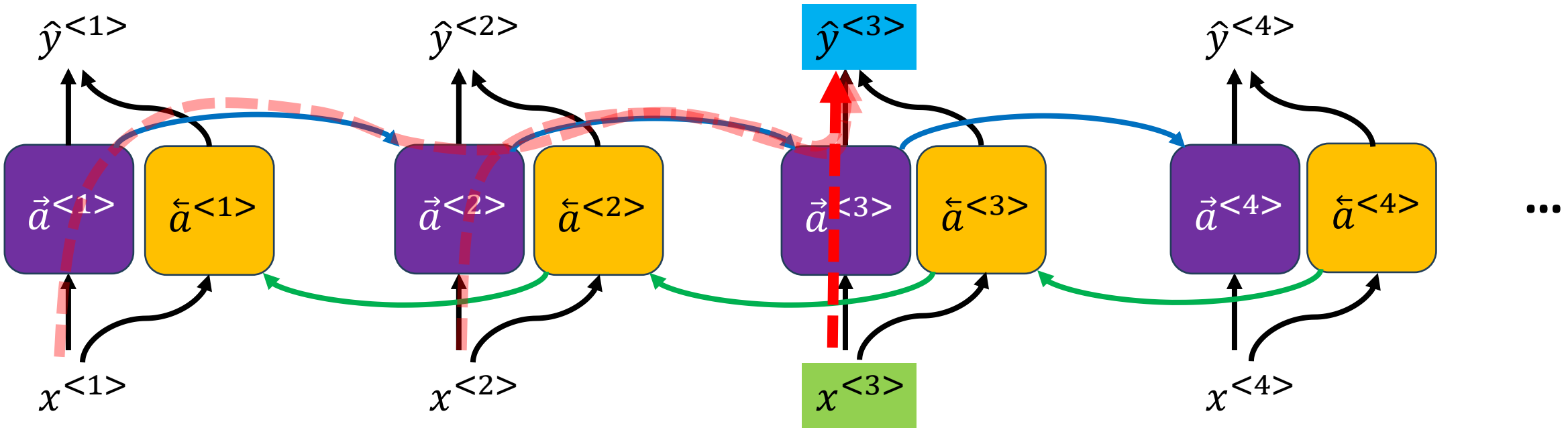
Machine Learning: *Recurrent Neural Network (RNN)*

Bi-directional RNN



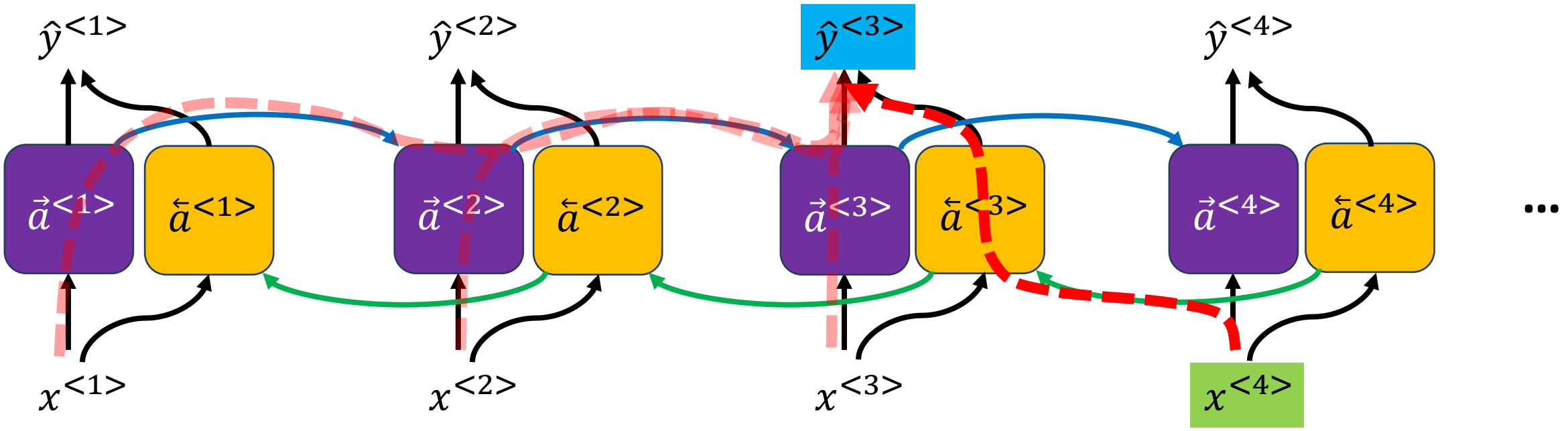
Machine Learning: *Recurrent Neural Network (RNN)*

Bi-directional RNN



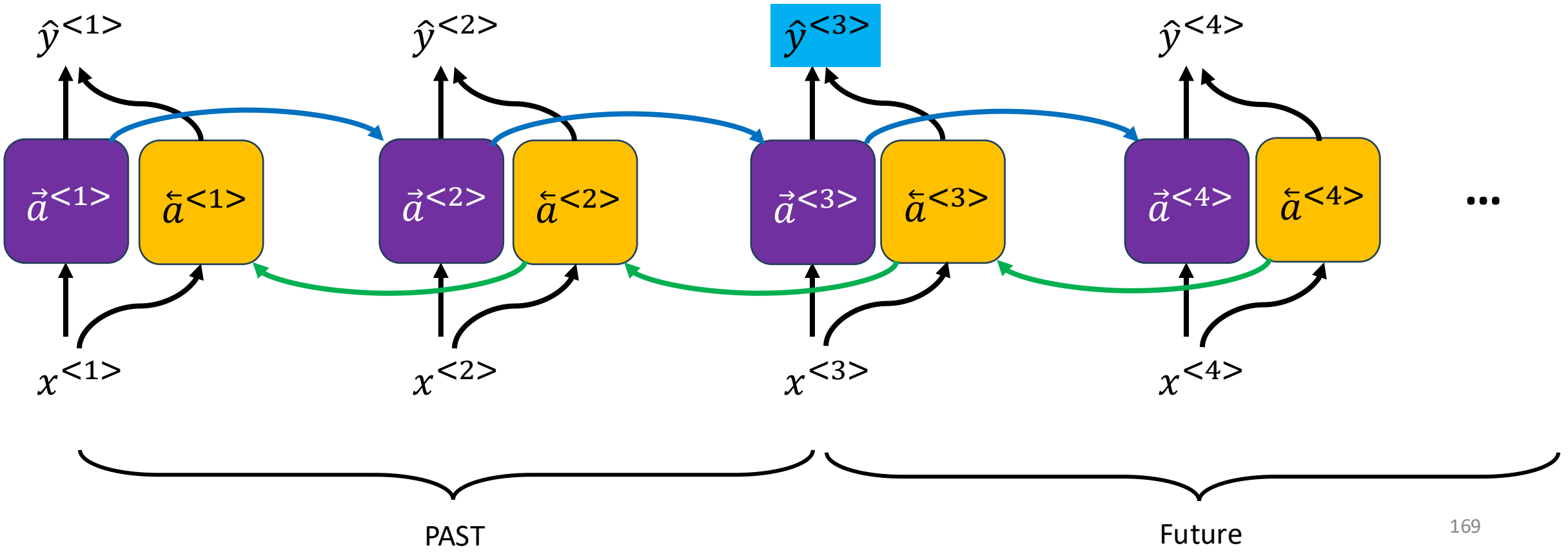
Machine Learning: *Recurrent Neural Network (RNN)*

Bi-directional RNN



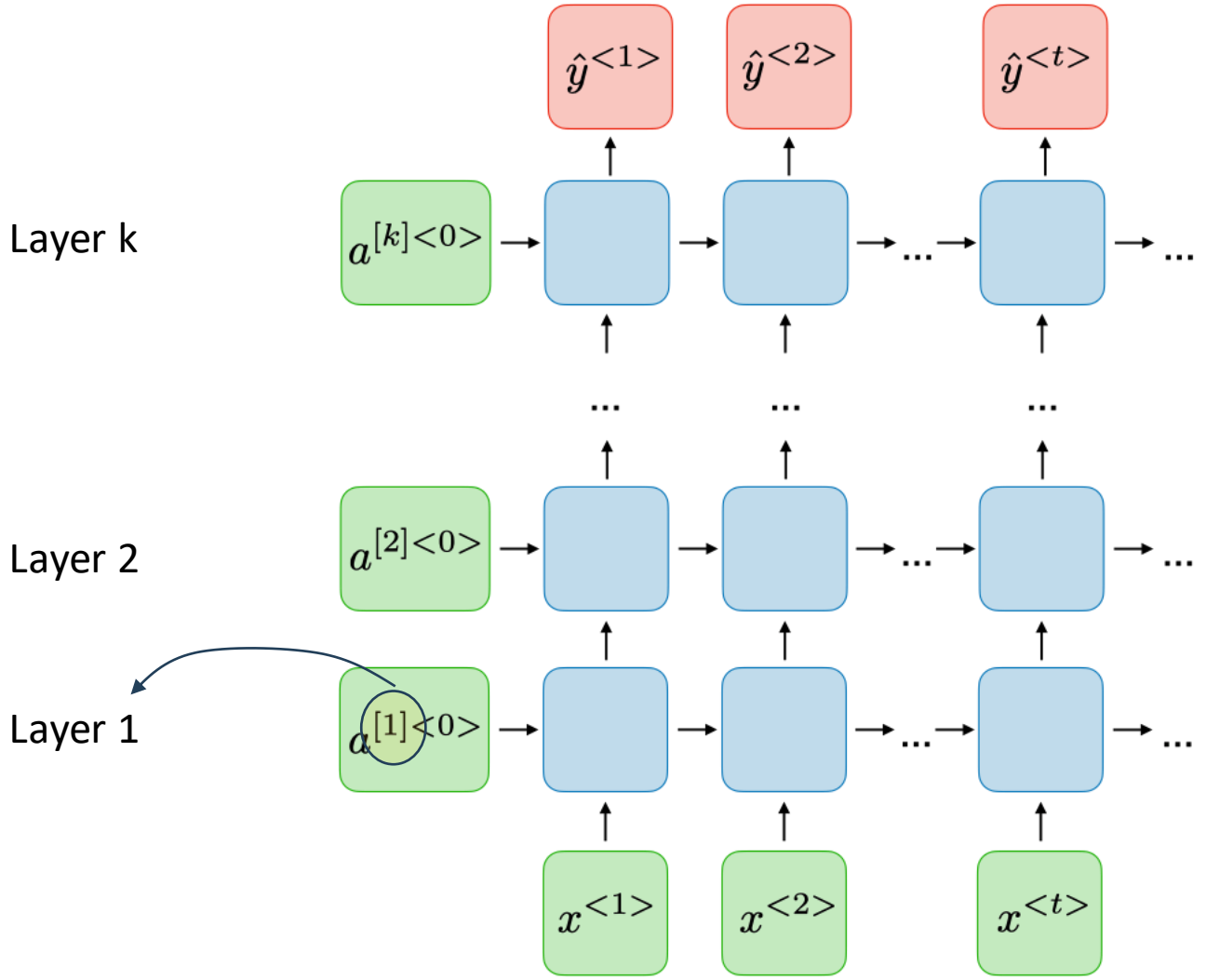
Machine Learning: *Recurrent Neural Network (RNN)*

Bi-directional RNN



Machine Learning: *Recurrent Neural Network (RNN)*

Deep RNN



Machine Learning: *Neural Networks*

RNN

Python

```
pip install tensorflow
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
```

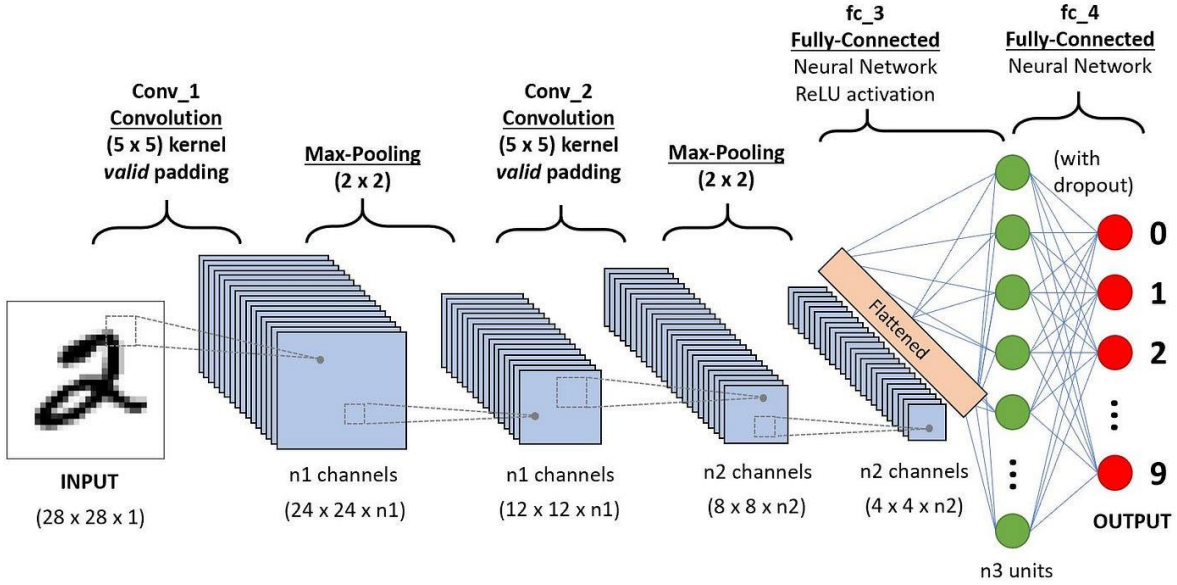
```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU, Dense
```

Machine Learning: *Neural Networks*

Convolutional Neural Network (CNN)

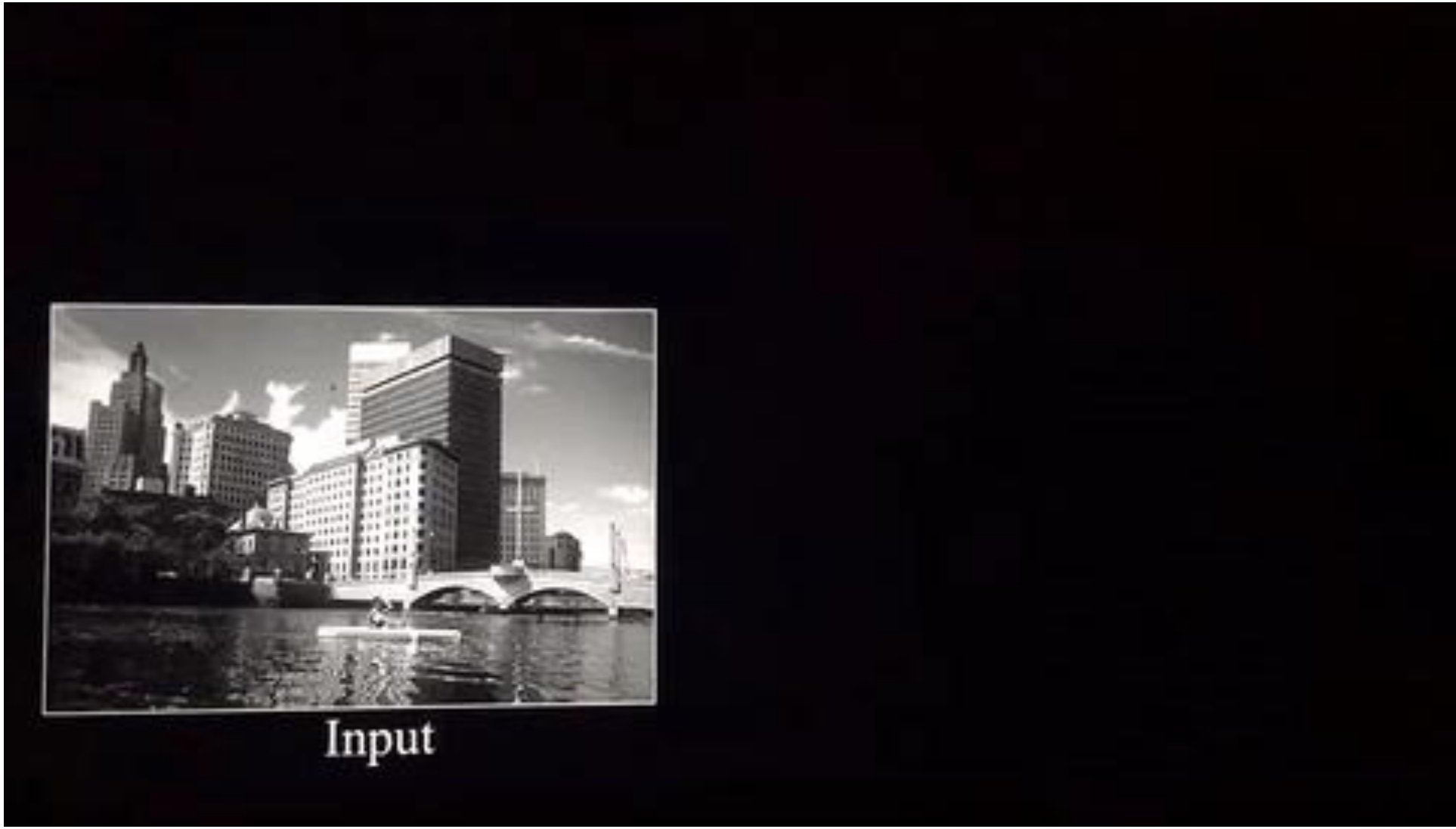
- derived from how the human visual system processes visual information.
- derive its name from the “convolution” operator.
- designed to automatically and efficiently recognize spatial hierarchies of features (mostly in images).



https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

Machine Learning: *Convolutional Neural Network (CNN)*

Features



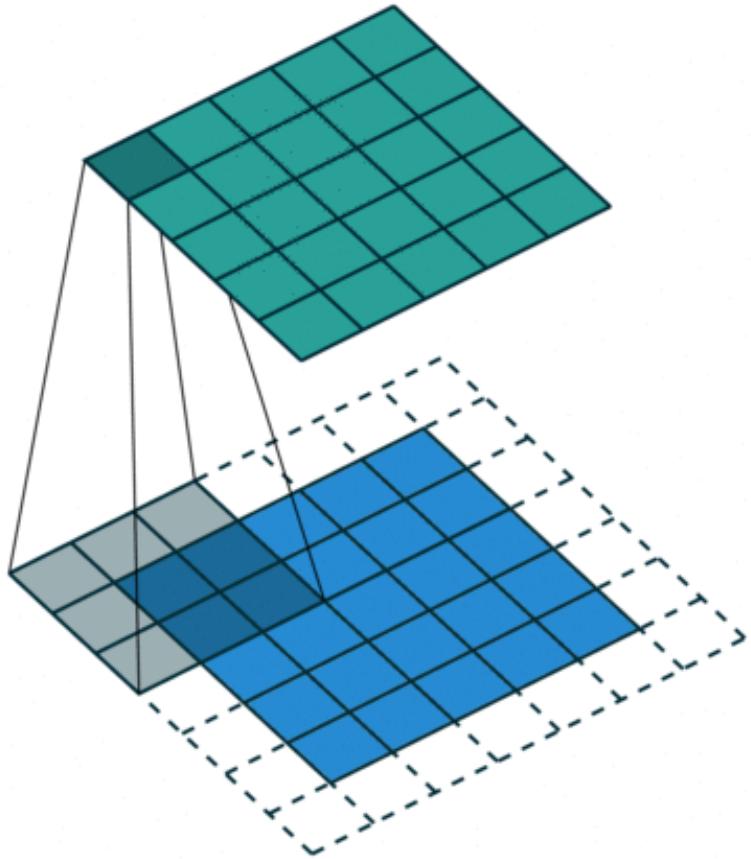
https://cs.nyu.edu/~fergus/tutorials/deep_learning_cvpr12/

Machine Learning: *Convolutional Neural Network (CNN)*

Convolution

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

4		

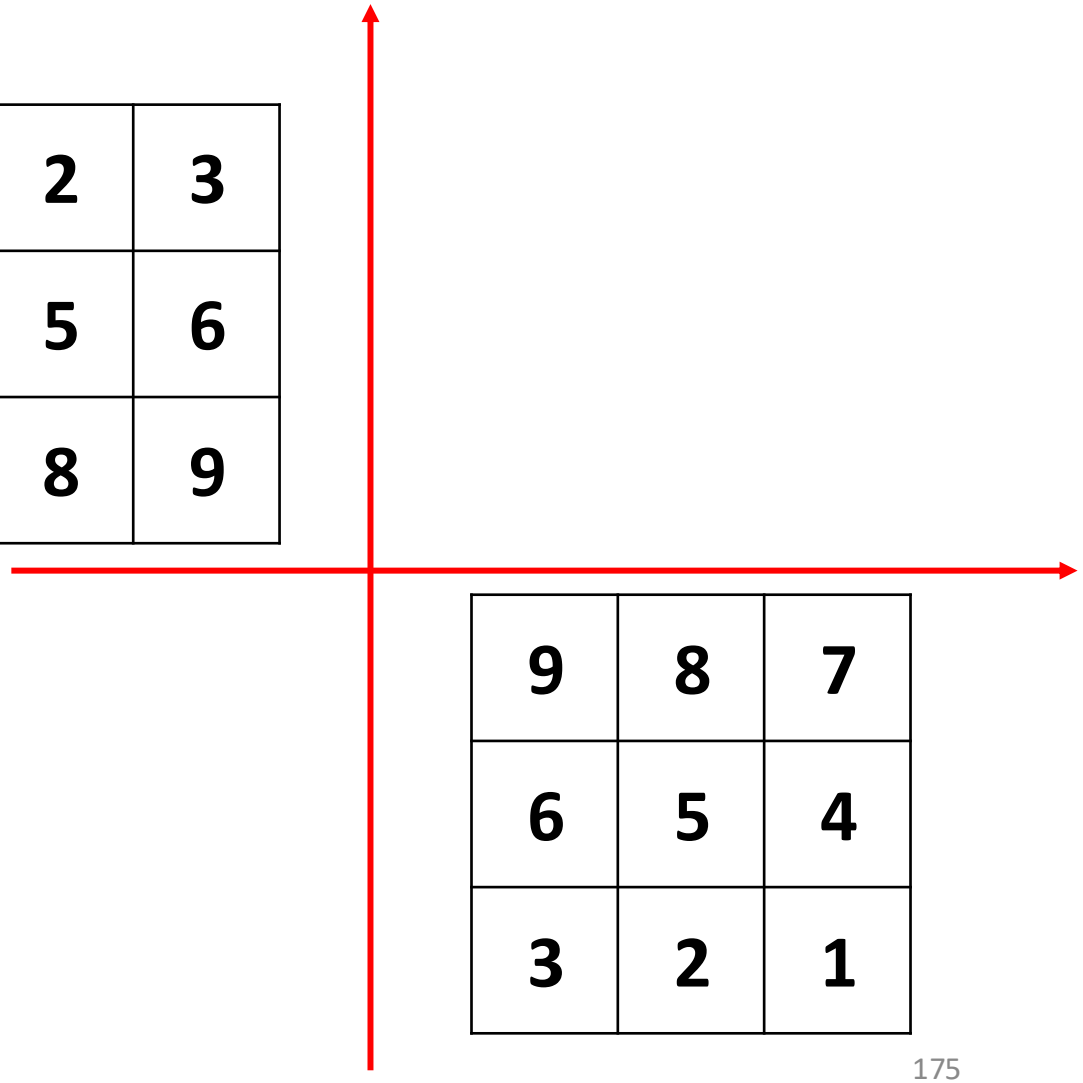


Machine Learning: *Convolutional Neural Network (CNN)*

Convolution vs. Cross-Correlation

*

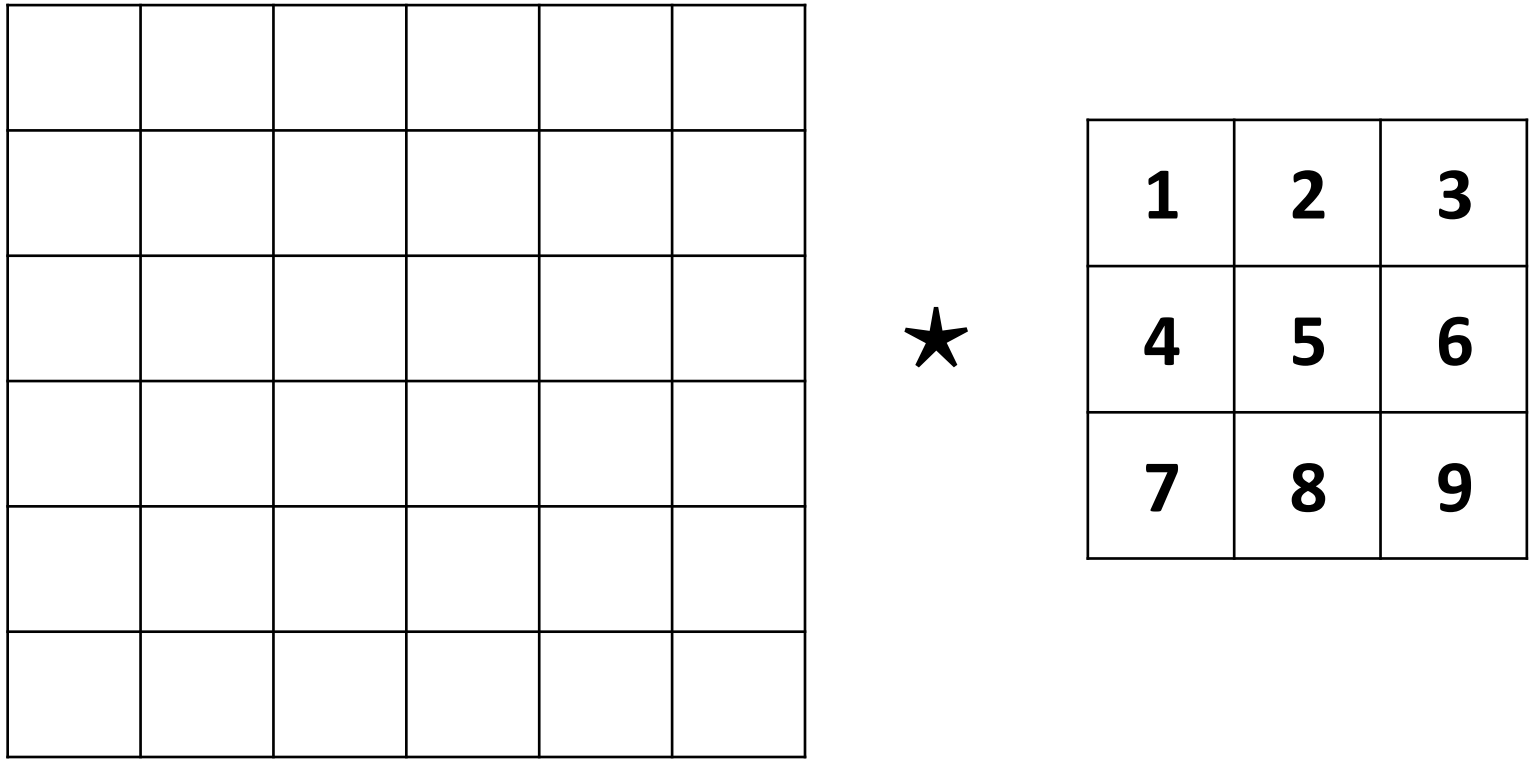
1	2	3
4	5	6
7	8	9



9	8	7
6	5	4
3	2	1

Machine Learning: *Convolutional Neural Network (CNN)*

Convolution vs. Cross-Correlation

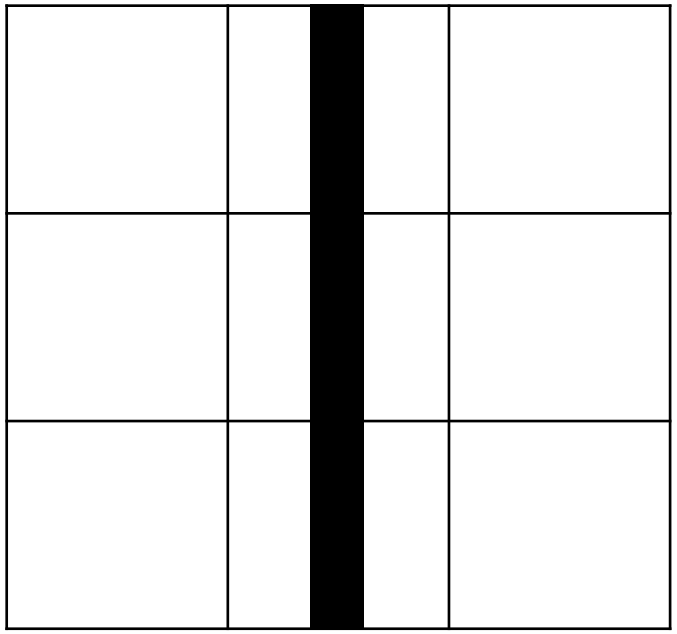


$\star$

1	2	3
4	5	6
7	8	9

Machine Learning: *Convolutional Neural Network (CNN)*

Design Edge Detection



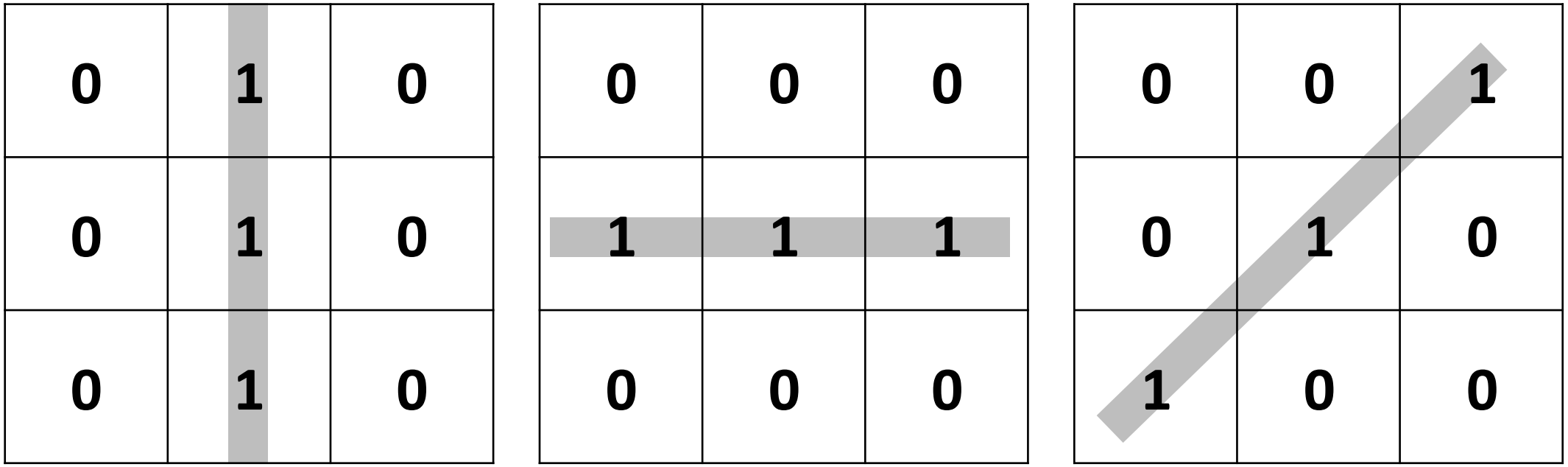
Machine Learning: *Convolutional Neural Network (CNN)*

Design Edge Detection

0	1	0
0	1	0
0	1	0

Machine Learning: *Convolutional Neural Network (CNN)*

Design Edge Detection



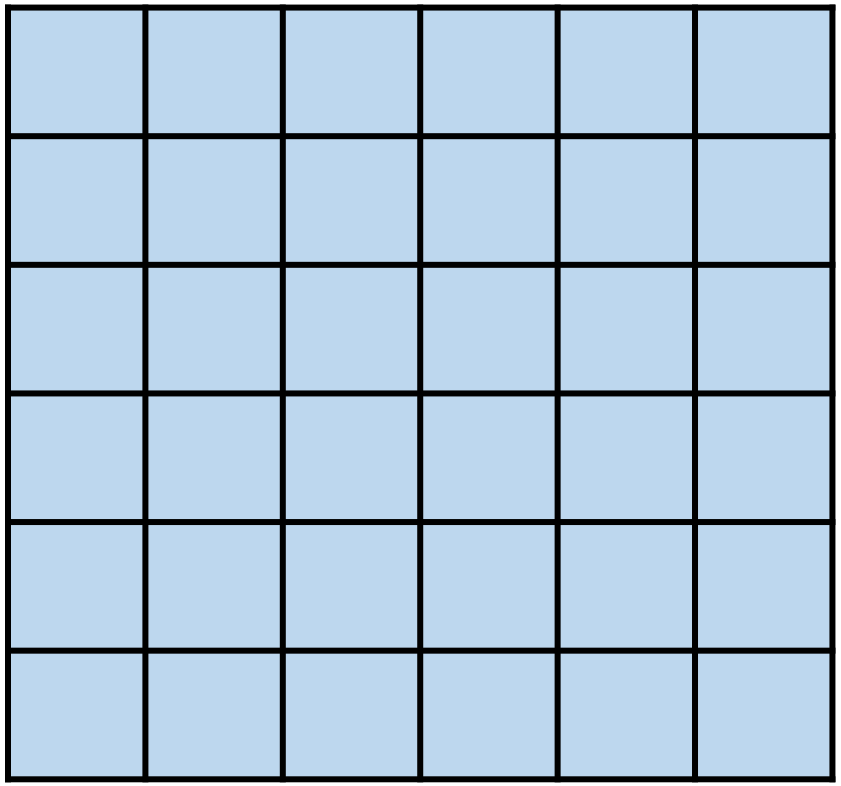
Machine Learning: *Convolutional Neural Network (CNN)*

Learn Edge Detection

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

Machine Learning: *Convolutional Neural Network (CNN)*

Padding



6×6

$*$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

3×3

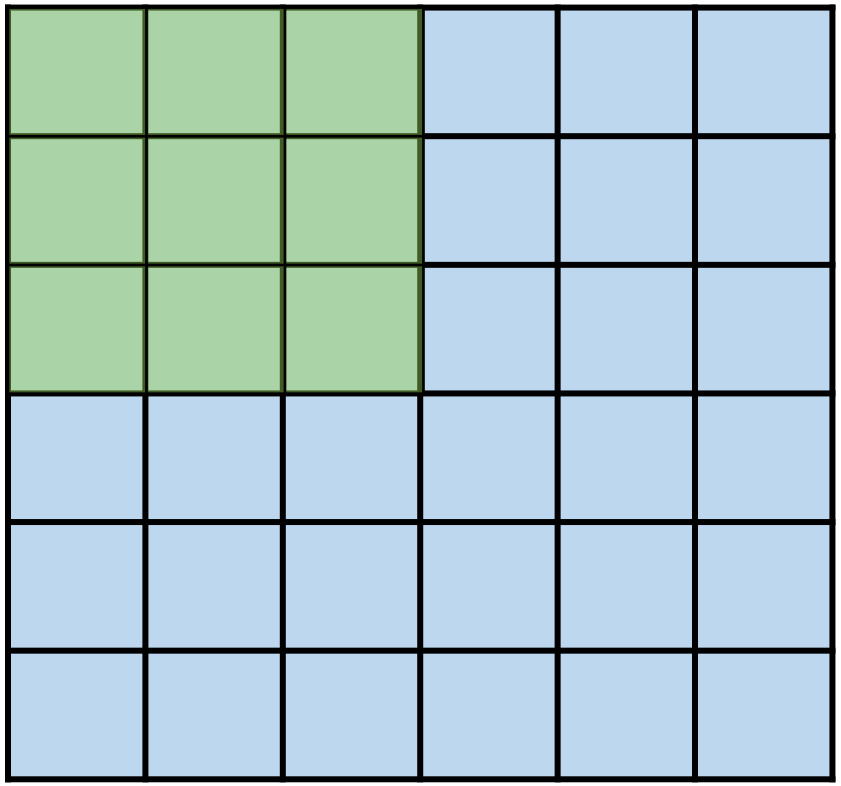
$=$

?

$?\times?$

Machine Learning: *Convolutional Neural Network (CNN)*

Padding



$*$

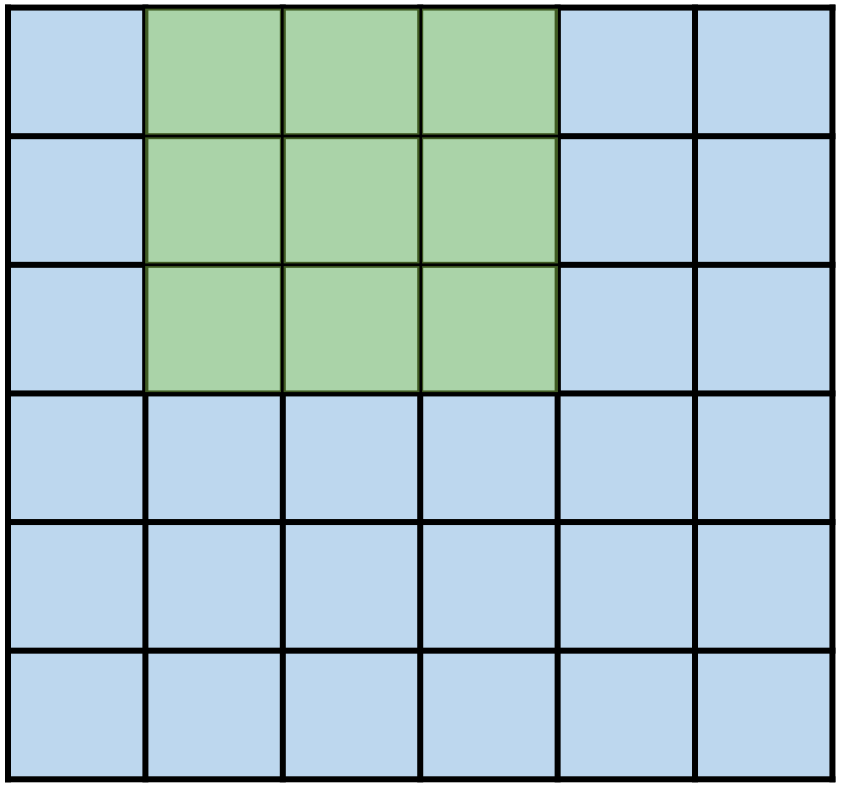
w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

$=$



Machine Learning: *Convolutional Neural Network (CNN)*

Padding



$*$

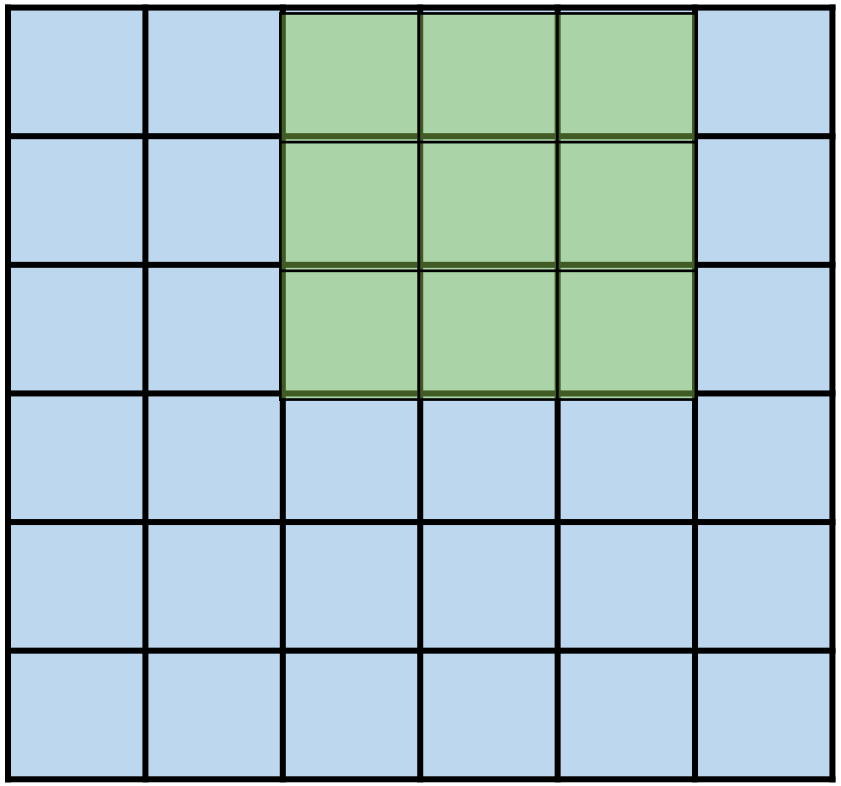
w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

$=$



Machine Learning: *Convolutional Neural Network (CNN)*

Padding



$*$

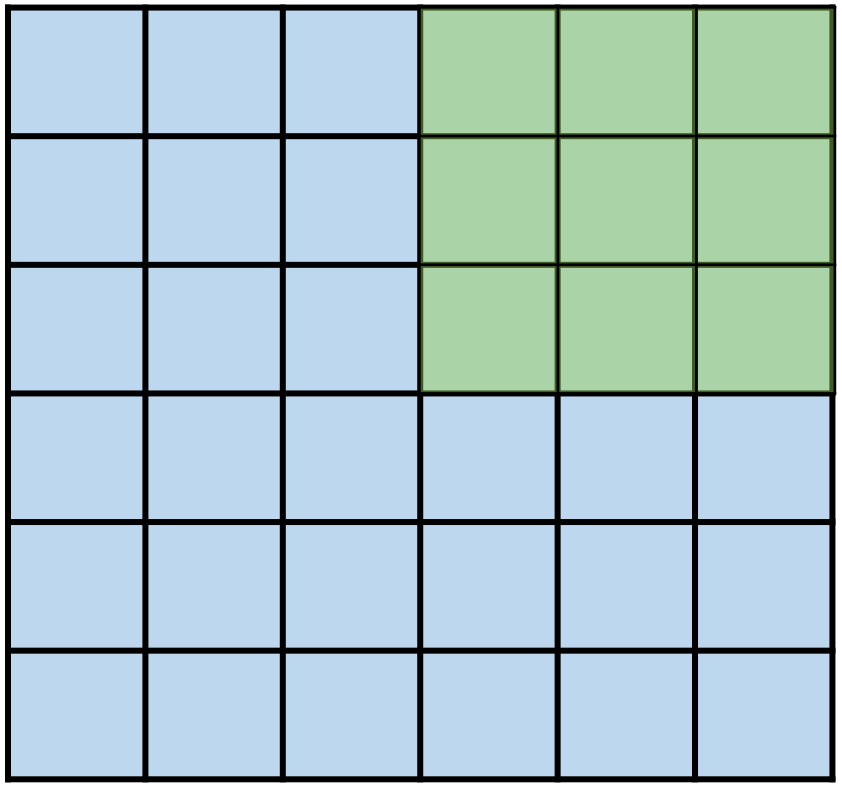
w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

$=$



Machine Learning: *Convolutional Neural Network (CNN)*

Padding



$*$

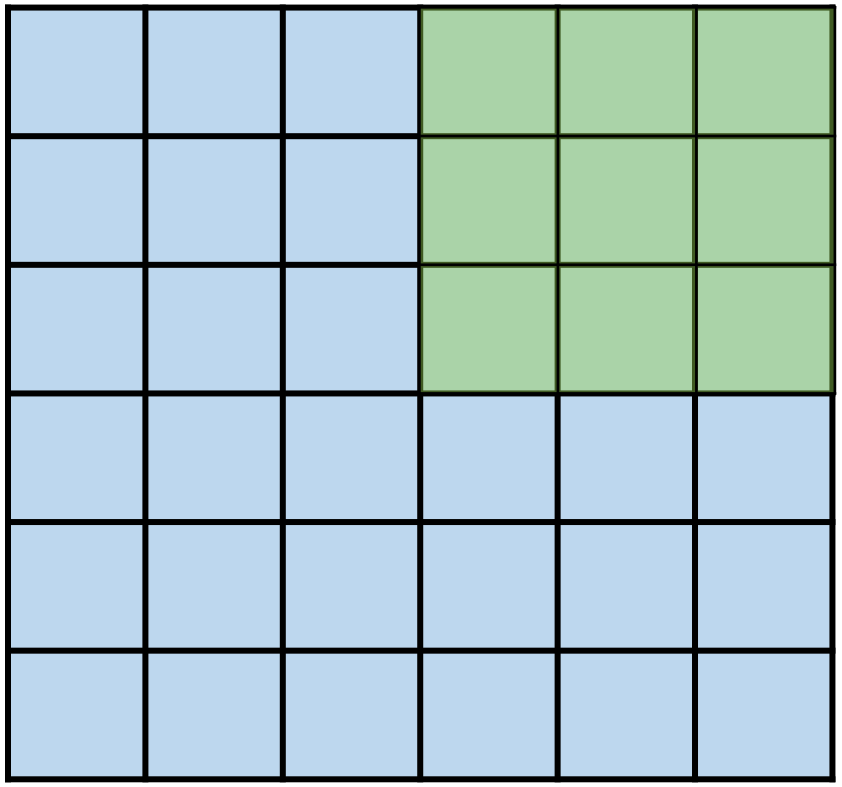
w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

$=$



Machine Learning: *Convolutional Neural Network (CNN)*

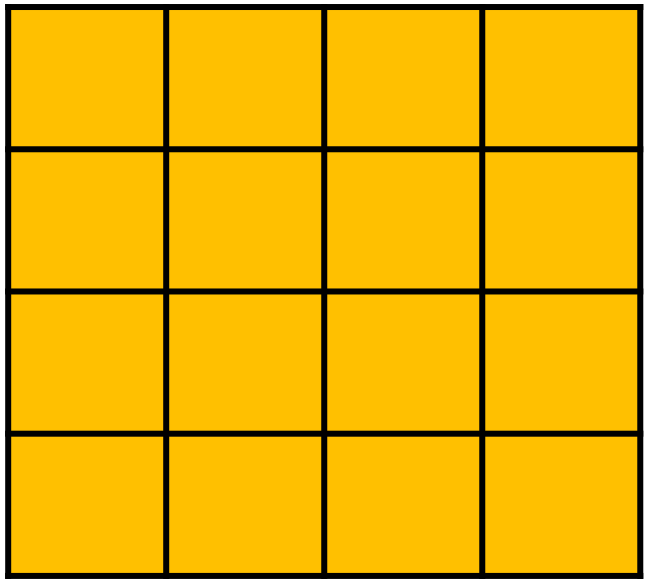
Padding



$*$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

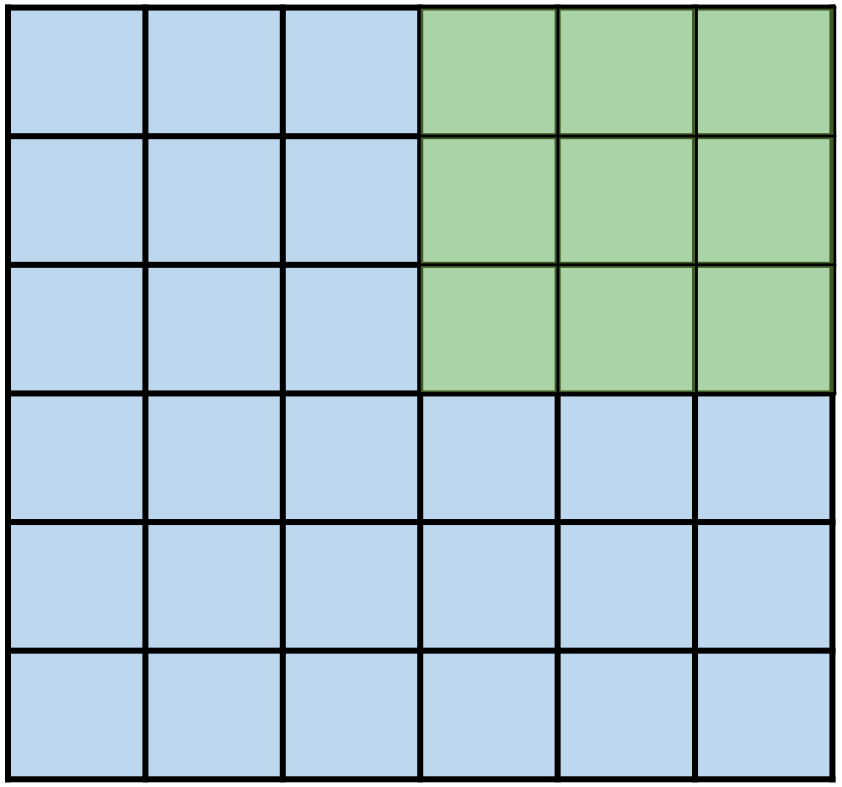
$=$



4×4

Machine Learning: *Convolutional Neural Network (CNN)*

Padding



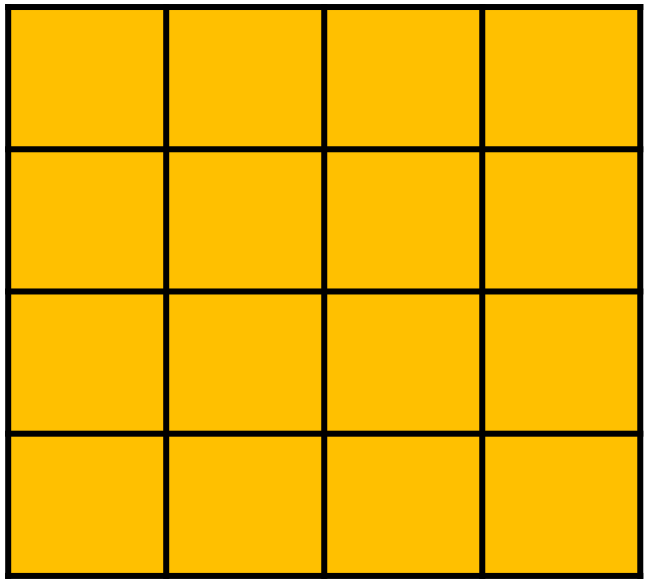
$n \times n$

$*$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

$m \times m$

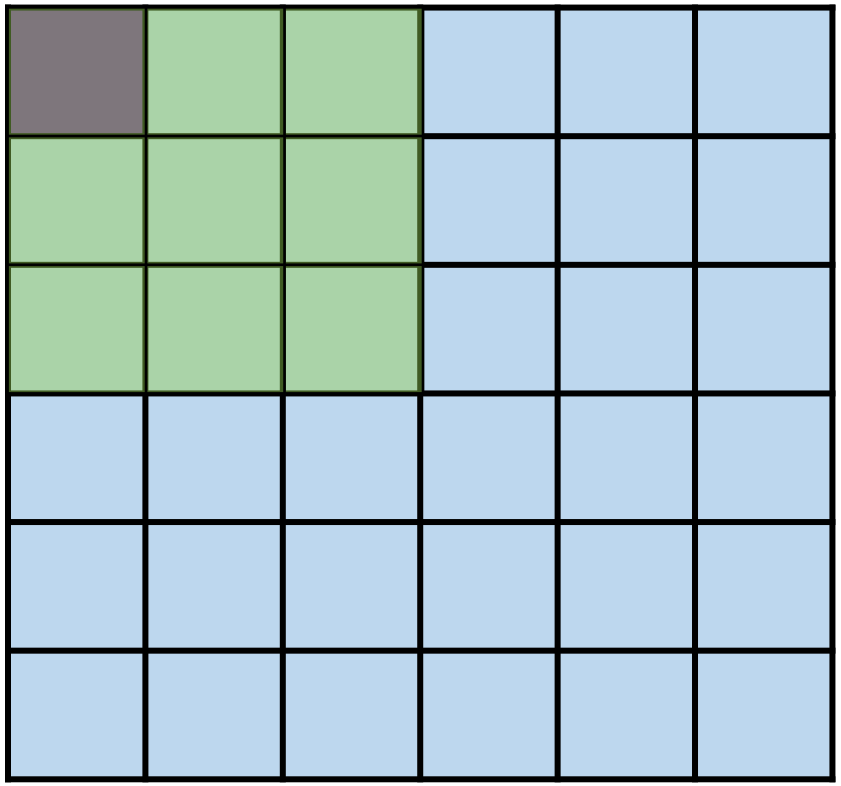
$=$



$(n-m+1) \times (n-m+1)$

Machine Learning: *Convolutional Neural Network (CNN)*

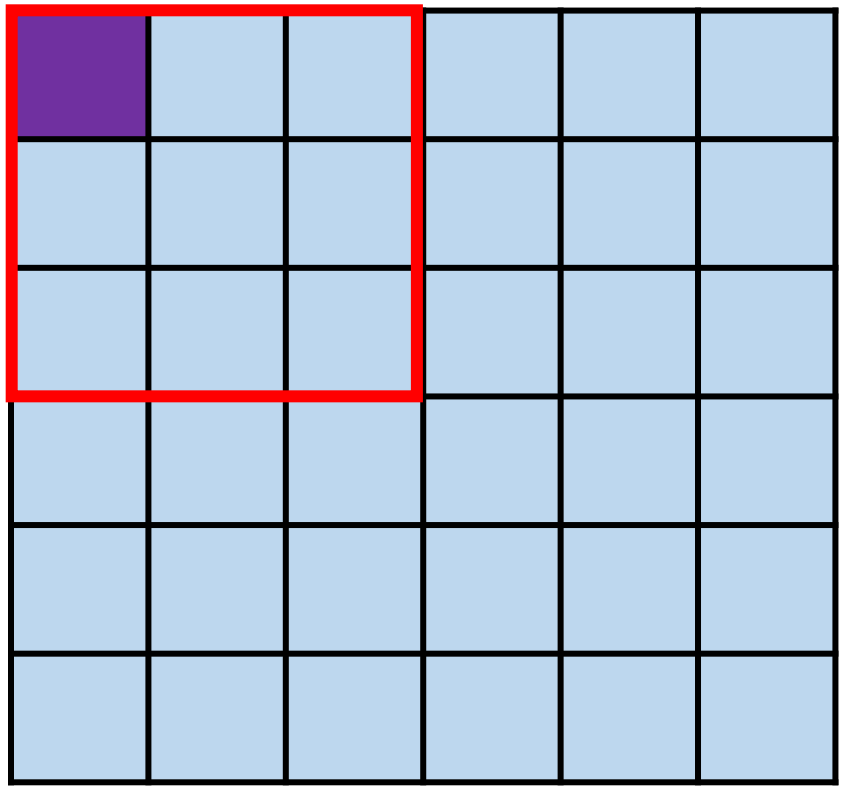
Padding



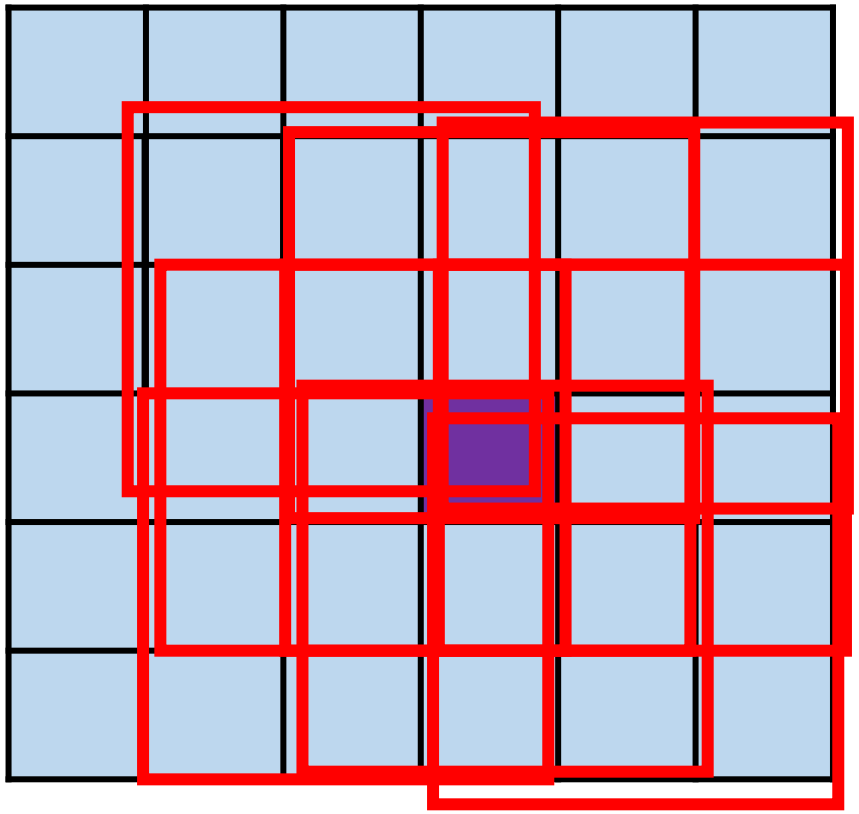
$n \times n$

Machine Learning: *Convolutional Neural Network (CNN)*

Padding

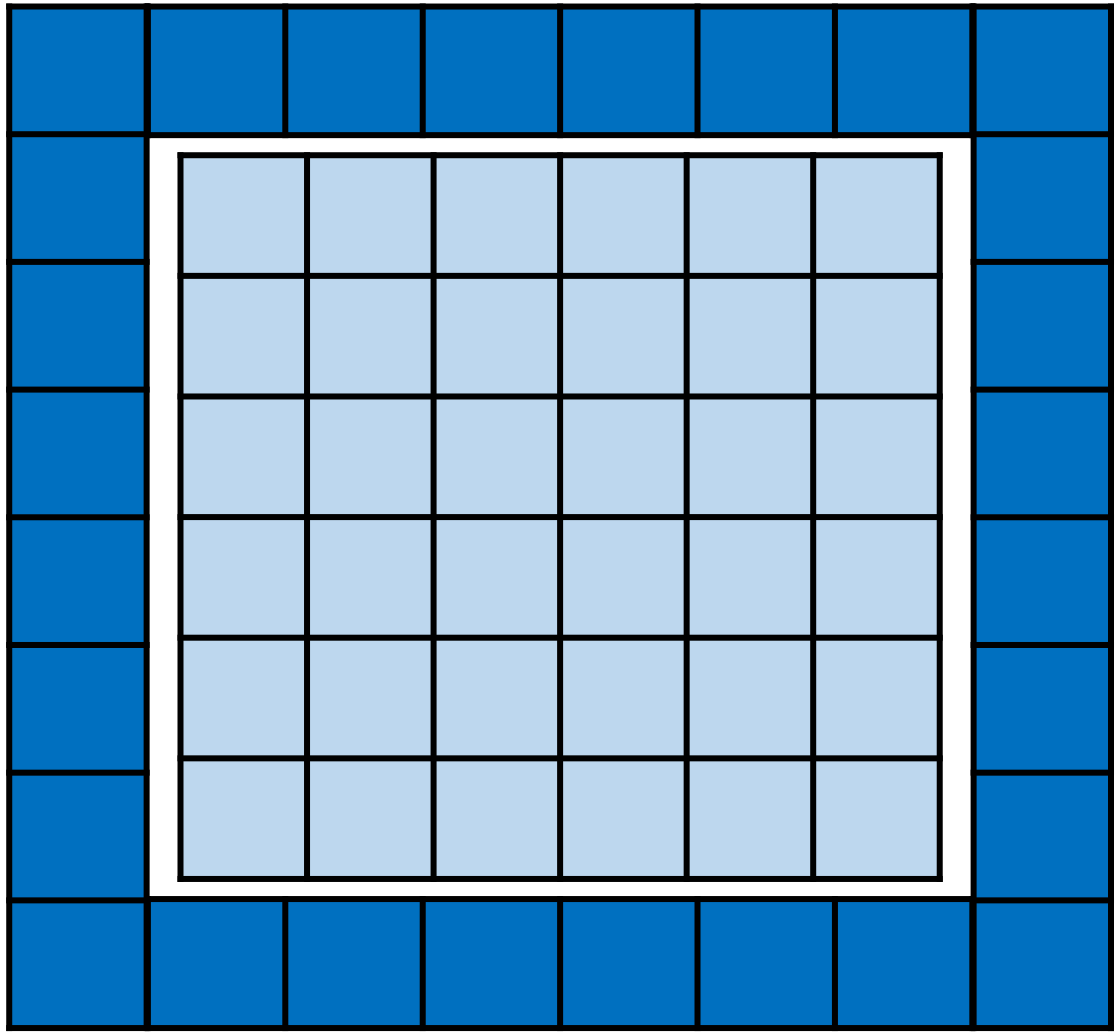


$n \times n$



Machine Learning: *Convolutional Neural Network (CNN)*

Padding



$6 \times 6 \rightarrow 8 \times 8$

Machine Learning: *Convolutional Neural Network (CNN)*

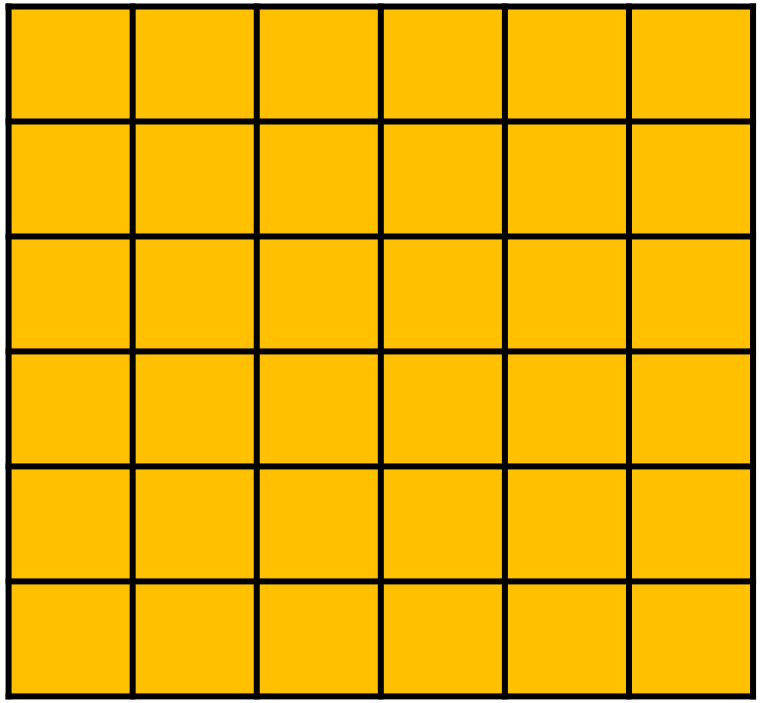
Padding

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

$*$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

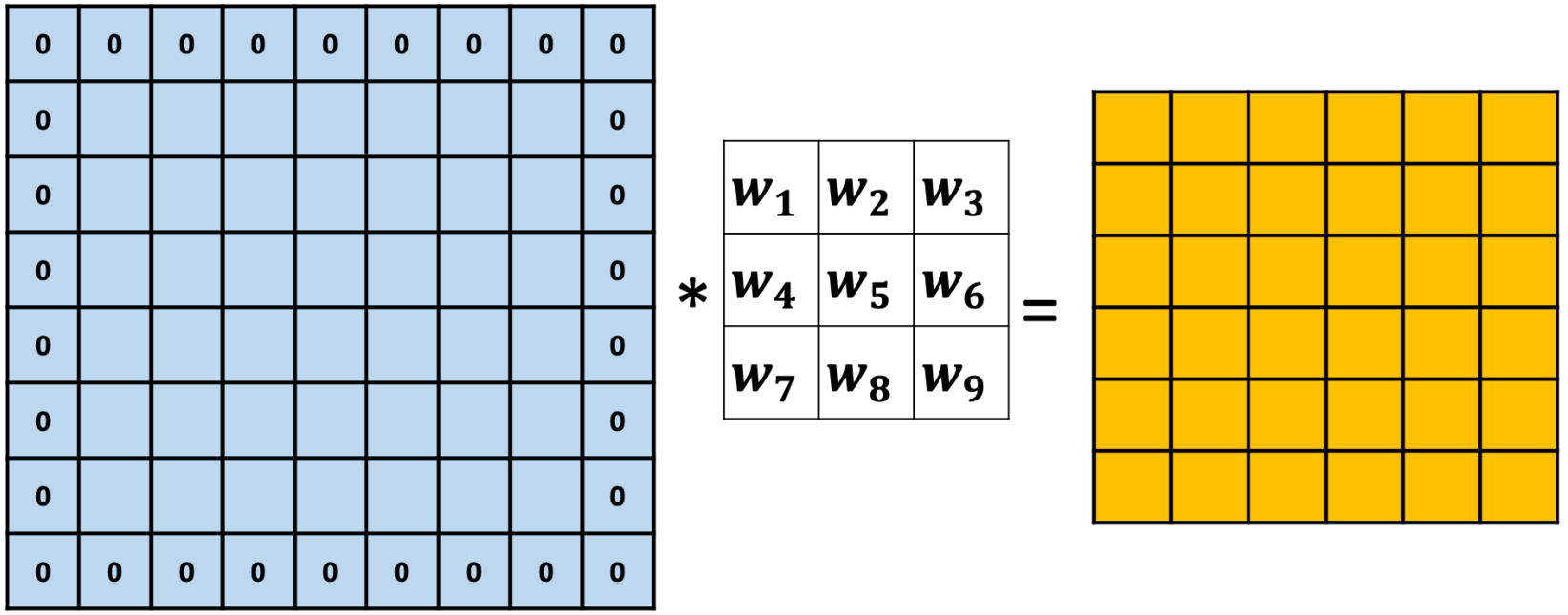
$=$



Machine Learning: *Convolutional Neural Network (CNN)*

Padding

Padding by p on each side



$n \times n$ $\quad$ $m \times m$ $\quad$ $(n + 2p - m + 1) \times (n + 2p - m + 1)$

Machine Learning: *Convolutional Neural Network (CNN)*

Padding

Valid:
no padding

$$(n\times n) * (m\times m) \rightarrow (n-m+1)\times(n-m+1)$$

Same:
output size same as input

$$(n\times n) * (m\times m) \rightarrow (n+2p-m+1)\times(n+2p-m+1)$$

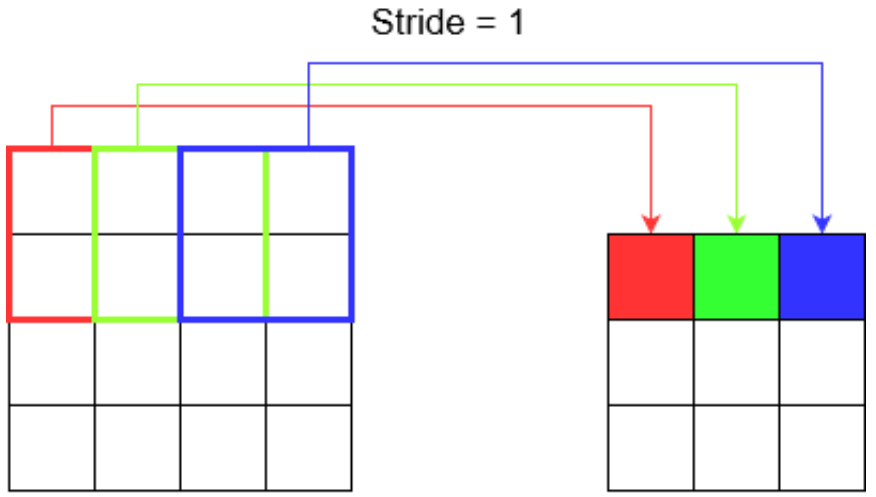
$$n+2p-m+1=n \rightarrow \boxed{p=\frac{m-1}{2}}$$

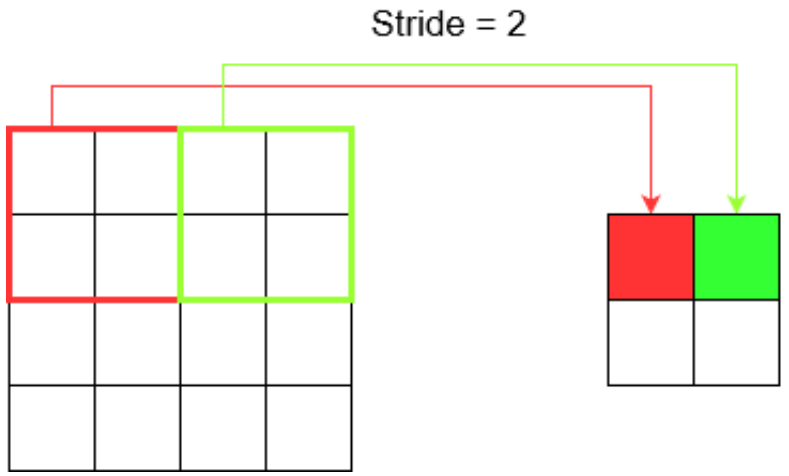
odd number

Machine Learning: *Convolutional Neural Network (CNN)*

Striding

number of pixels by which the filter (or kernel) is moved across the input image during the convolution operation.





https://stackoverflow.com/questions/42883547/intuitive-understanding-of-1d-2d-and-3d-convolutions-in-convolutional-neural-n

Machine Learning: *Convolutional Neural Network (CNN)*

Striding

Impact of Stride:

Stride = 1:

The filter moves across every pixel of the image, producing a larger output.

Stride > 1:

The filter skips pixels, resulting in a smaller output size, reducing the computational load and the dimensionality of the output, but at the cost of potentially missing some fine details.

Machine Learning: *Convolutional Neural Network (CNN)*

Striding

With padding p and stride of s:

$$(n \times n) * (m \times m) \rightarrow \left(\left\lfloor \frac{n + 2p - m}{s} \right\rfloor + 1\right) \times \left(\left\lfloor \frac{n + 2p - m}{s} \right\rfloor + 1\right)$$

$\lfloor x \rfloor$: $floor(x)$

Machine Learning: *Convolutional Neural Network (CNN)*

Layers

Convolution (CONV)

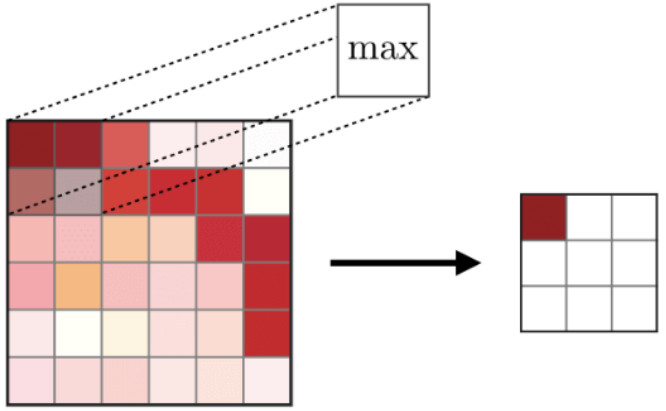
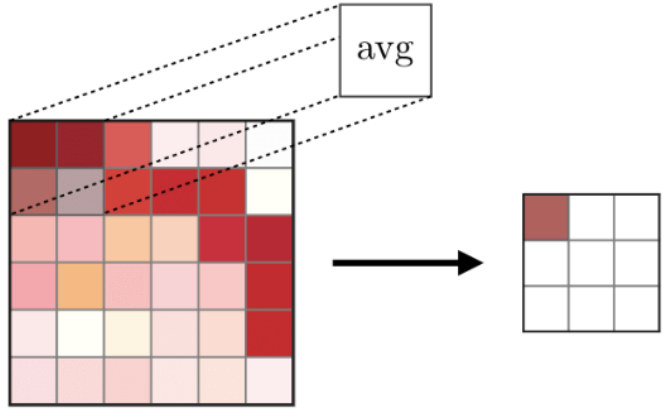
Pooling (POOL)

Fully connected (FC)

Machine Learning: *Convolutional Neural Network (CNN)*

Pooling

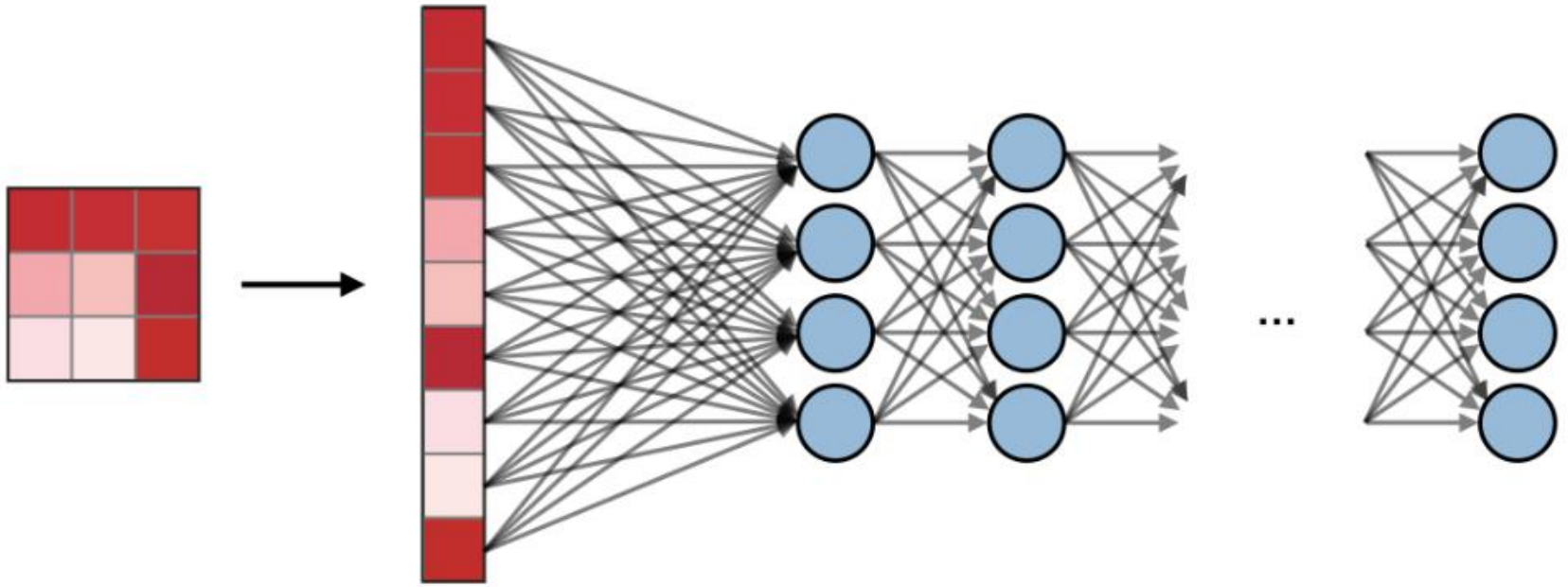
a downsampling operation, typically applied after a convolution layer, which does some spatial invariance.

Type	Max pooling	Average pooling
Purpose	Each pooling operation selects the maximum value of the current view	Each pooling operation averages the values of the current view
Illustration		
Comments	• Preserves detected features • Most commonly used	• Downsamples feature map • Used in LeNet

Machine Learning: *Convolutional Neural Network (CNN)*

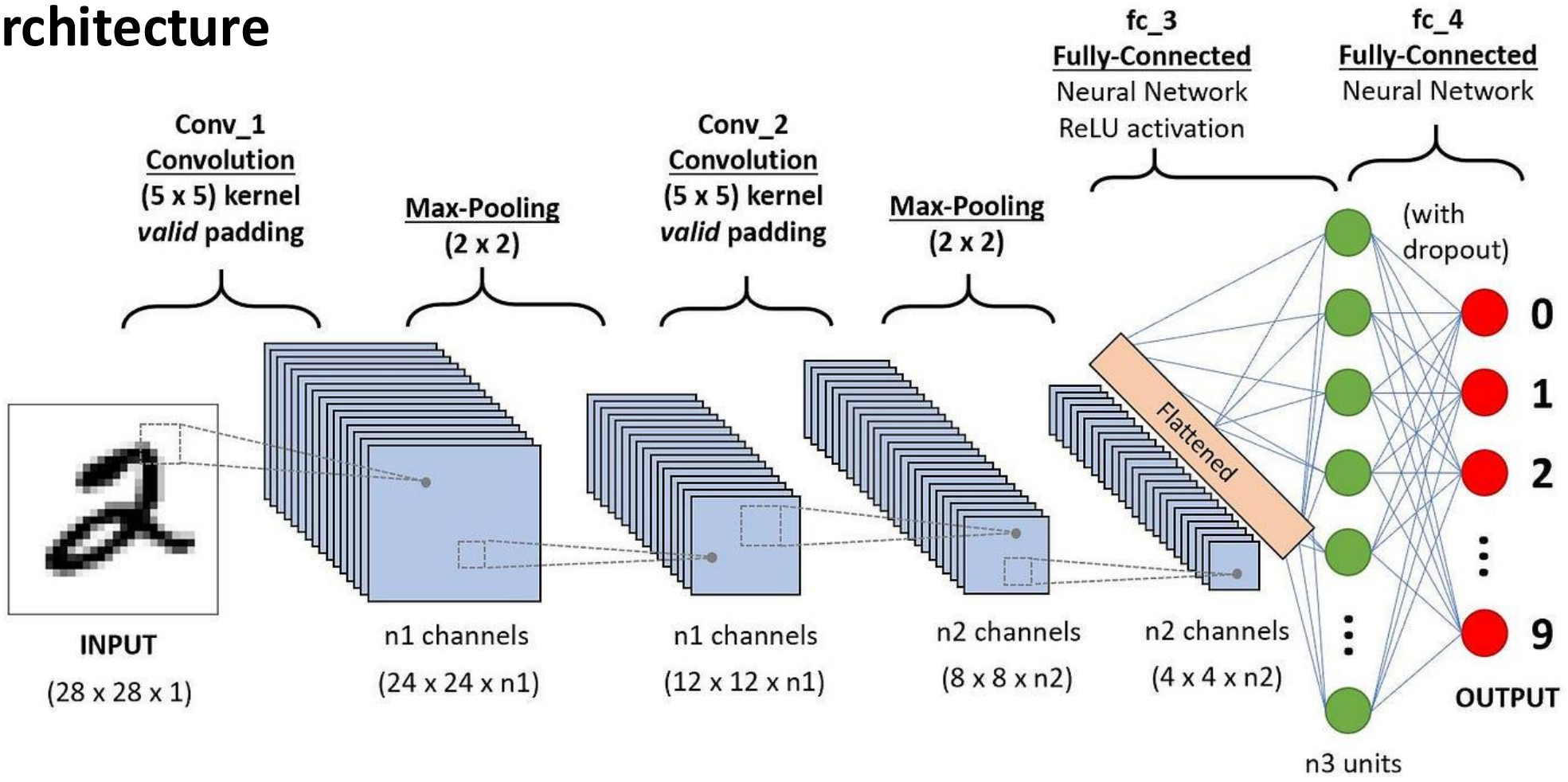
Fully Connected

operates on a flattened input where each input is connected to all neurons.



Machine Learning: *Convolutional Neural Network (CNN)*

Complete Architecture



Machine Learning: *Neural Networks*

CNN

Python

```
pip install tensorflow
```

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))

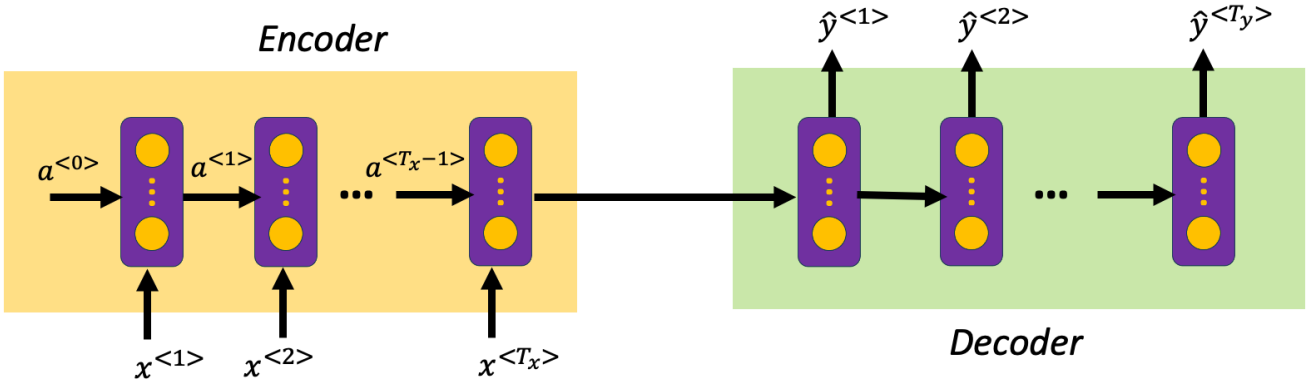
# Add Dense layers on top
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))
```

Machine Learning: *Neural Networks*

Attention

RNN: Many-to-Many

Example: Language Translation



Reference:

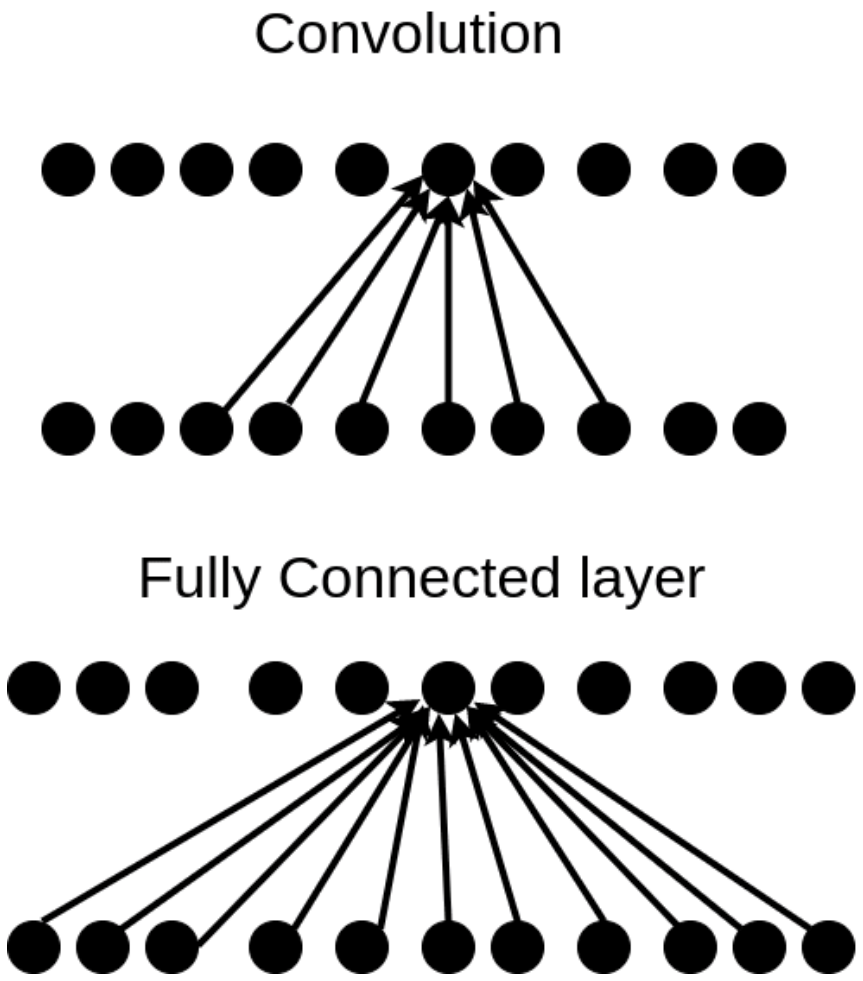
Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio, (2014) “Neural Machine Translation by Jointly Learning to Align and Translate”.
URL: https://arxiv.org/pdf/1409.0473.pdf

Kelvin Xu et al. (2015) “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention”
URL: https://proceedings.mlr.press/v37/xuc15.pdf

Machine Learning: *Neural Networks*

Attention

Intuition

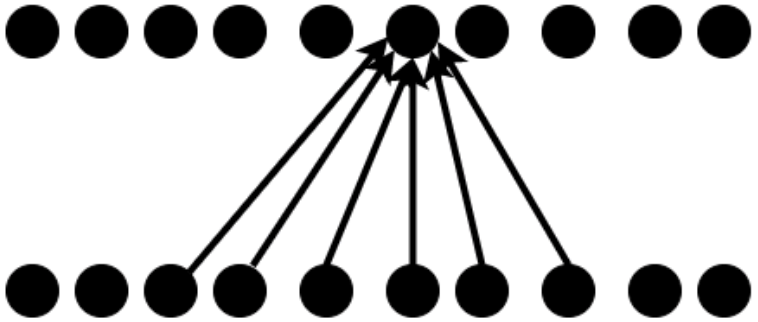


Machine Learning: *Neural Networks*

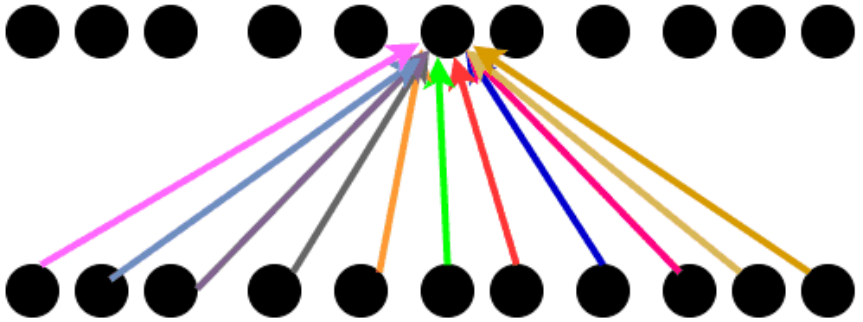
Attention

Intuition

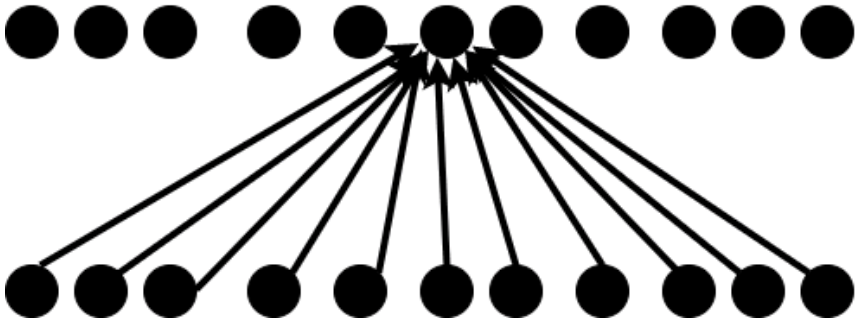
Convolution



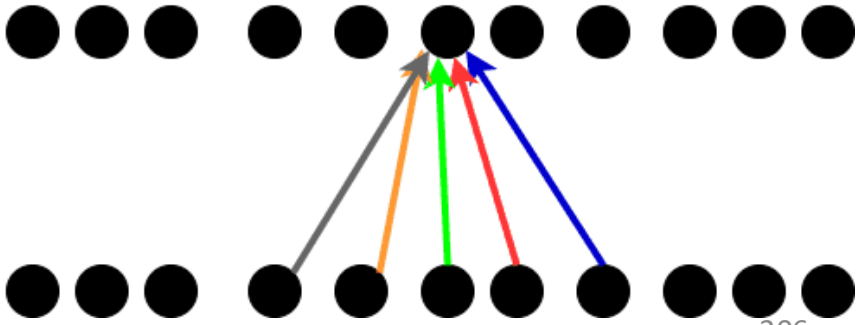
Global attention



Fully Connected layer



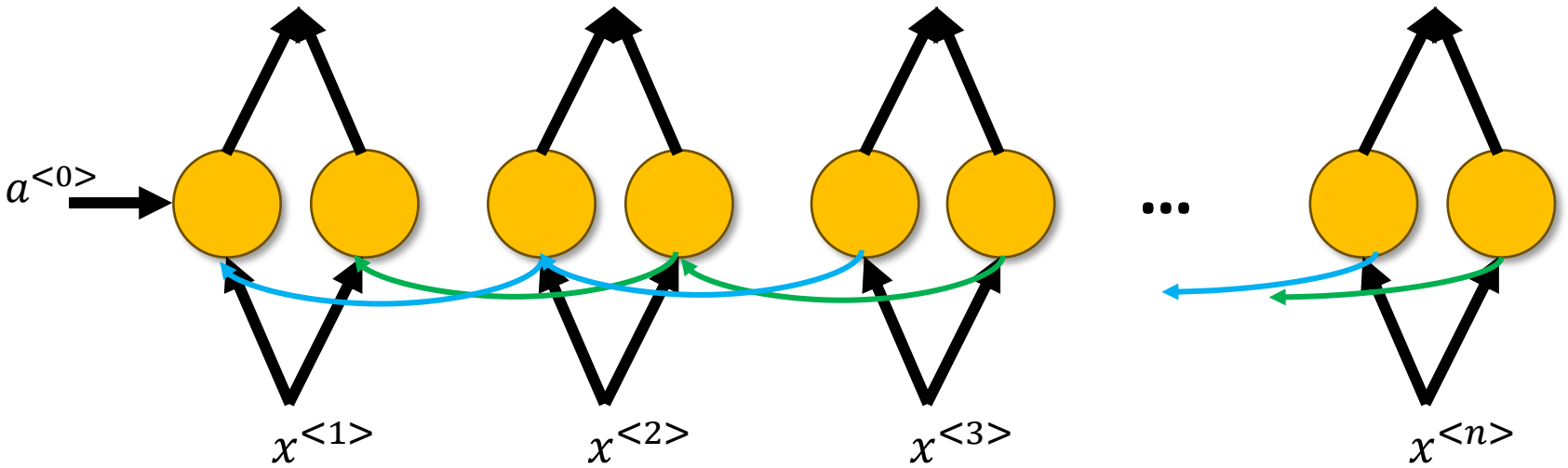
Local attention



Machine Learning: *Neural Networks*

Attention

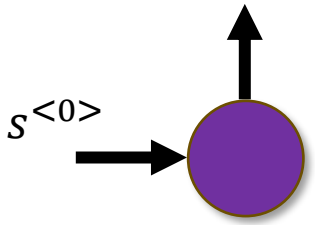
Intuition

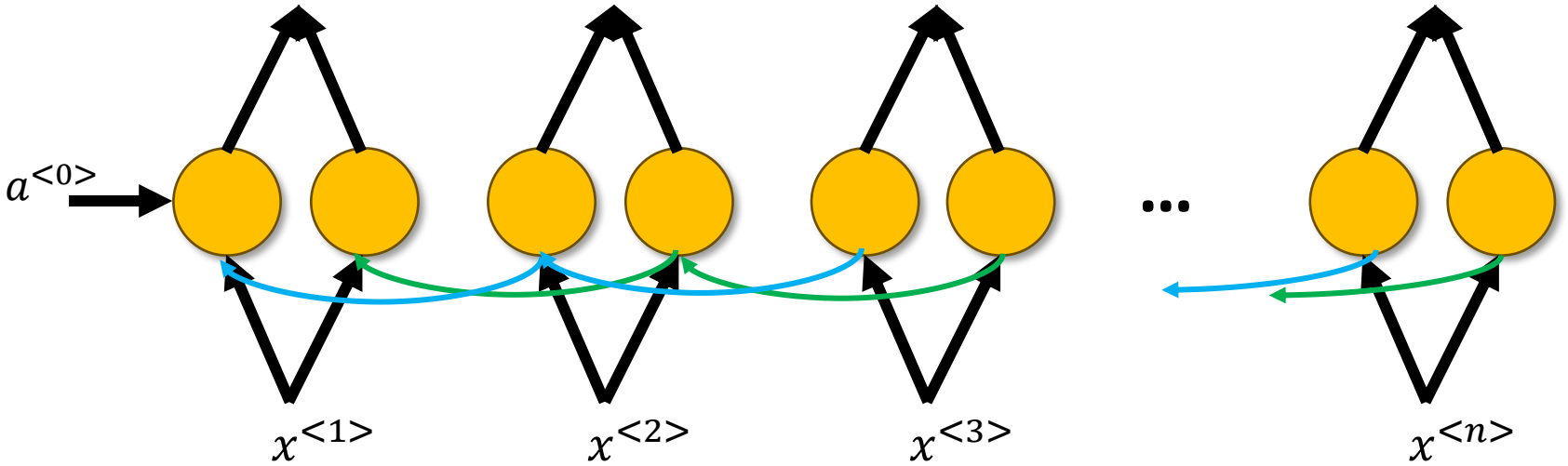


Machine Learning: *Neural Networks*

Attention

Intuition

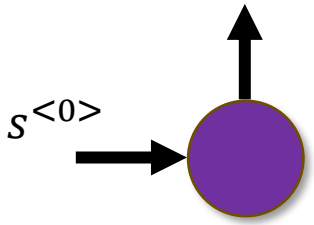




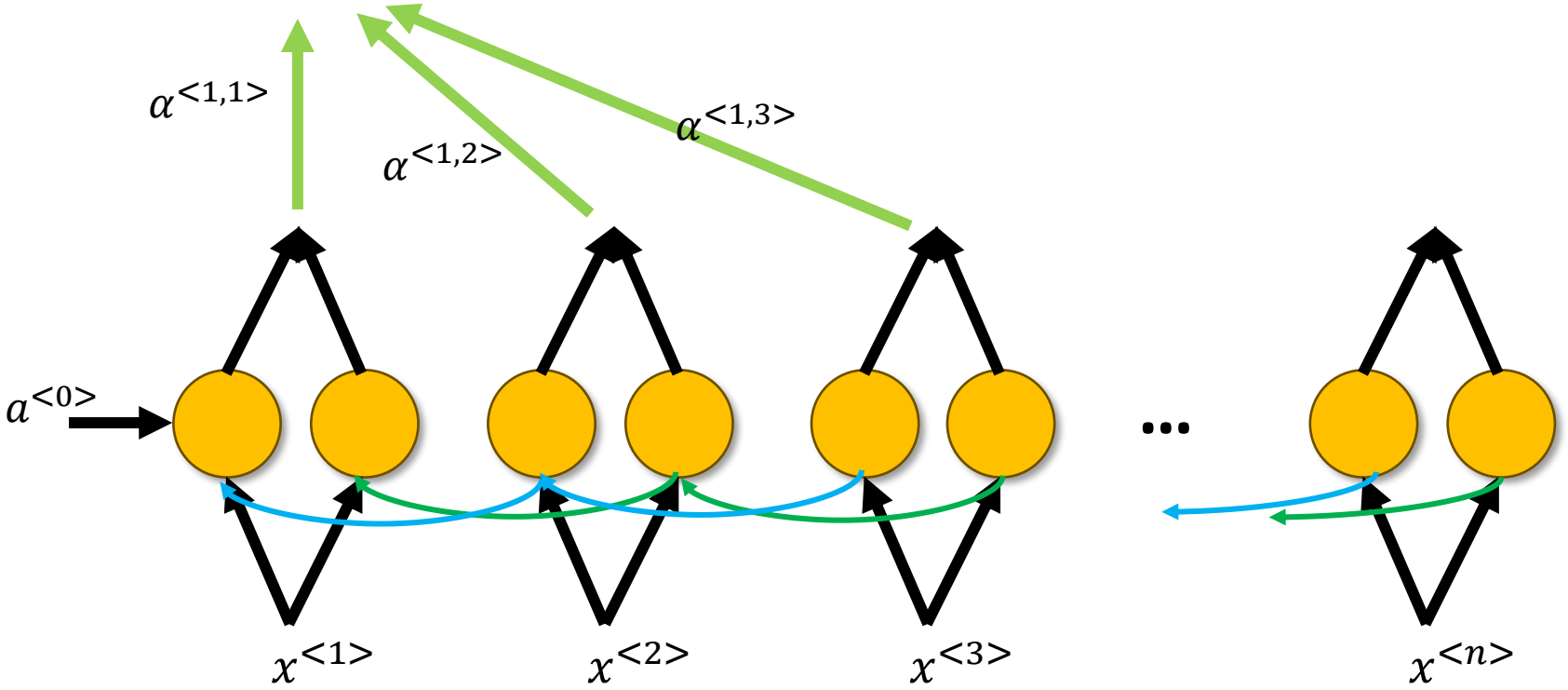
Machine Learning: *Neural Networks*

Attention

Intuition



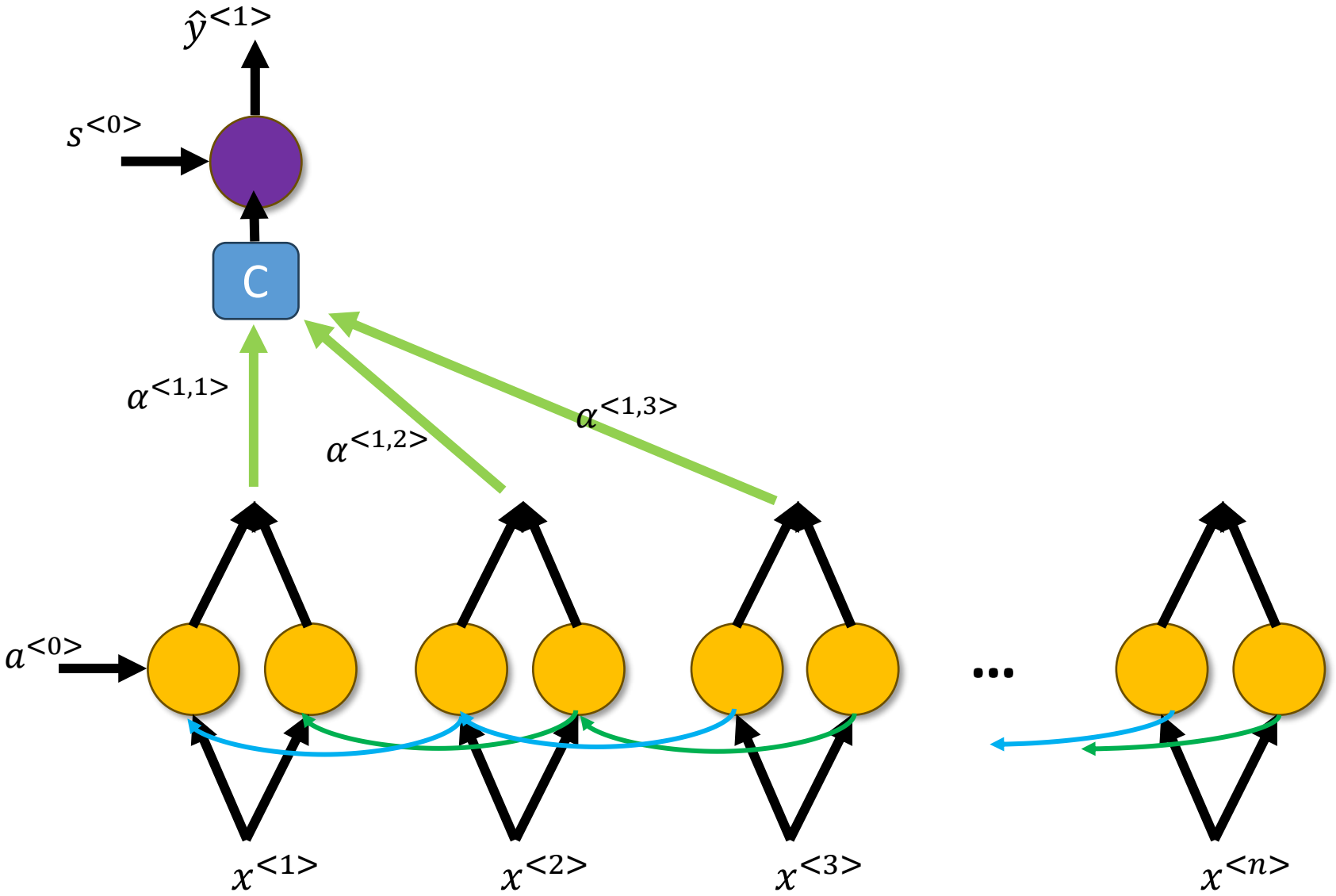
"Which input tokens should I focus on, and how much?"



Machine Learning: *Neural Networks*

Attention

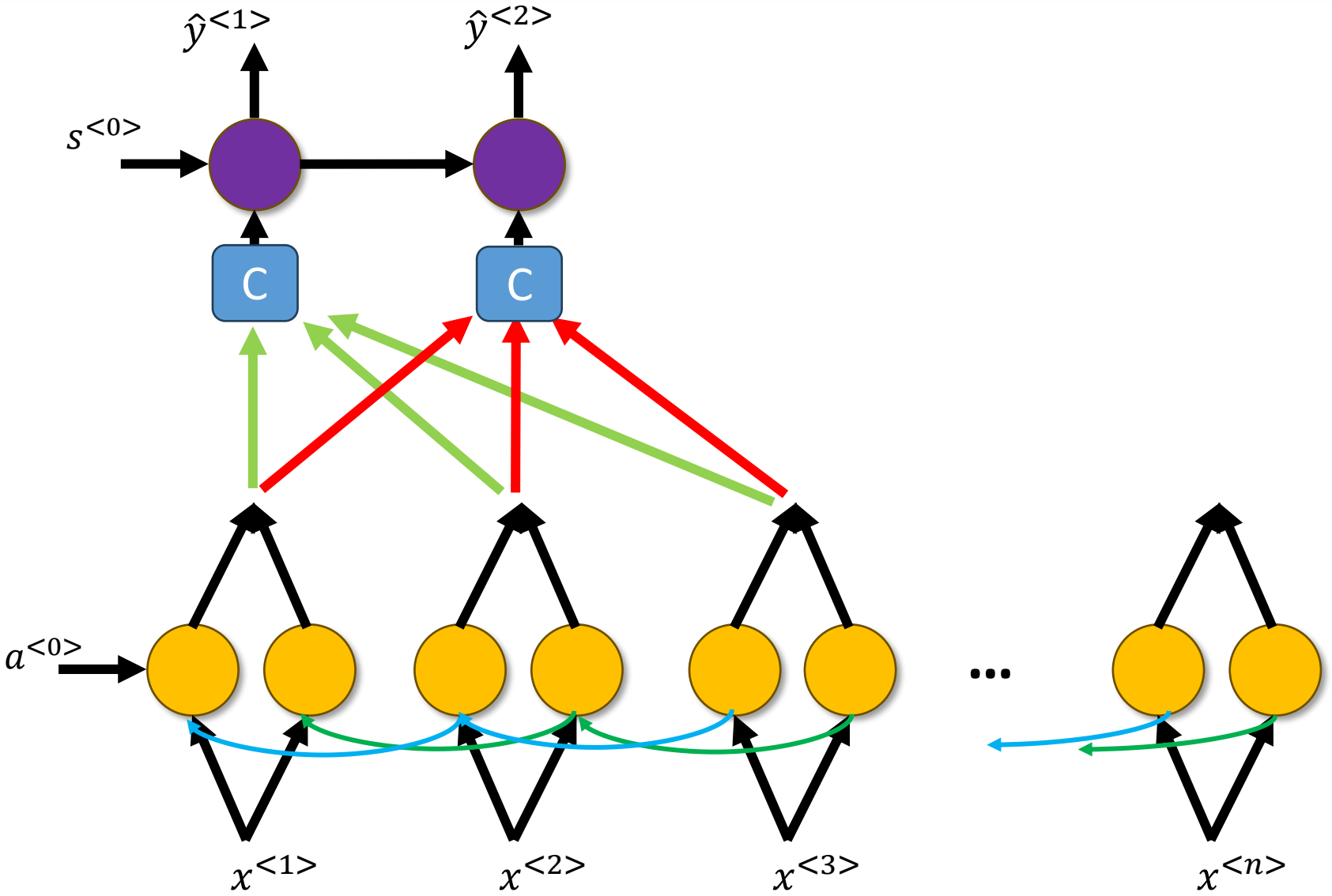
Intuition



Machine Learning: *Neural Networks*

Attention

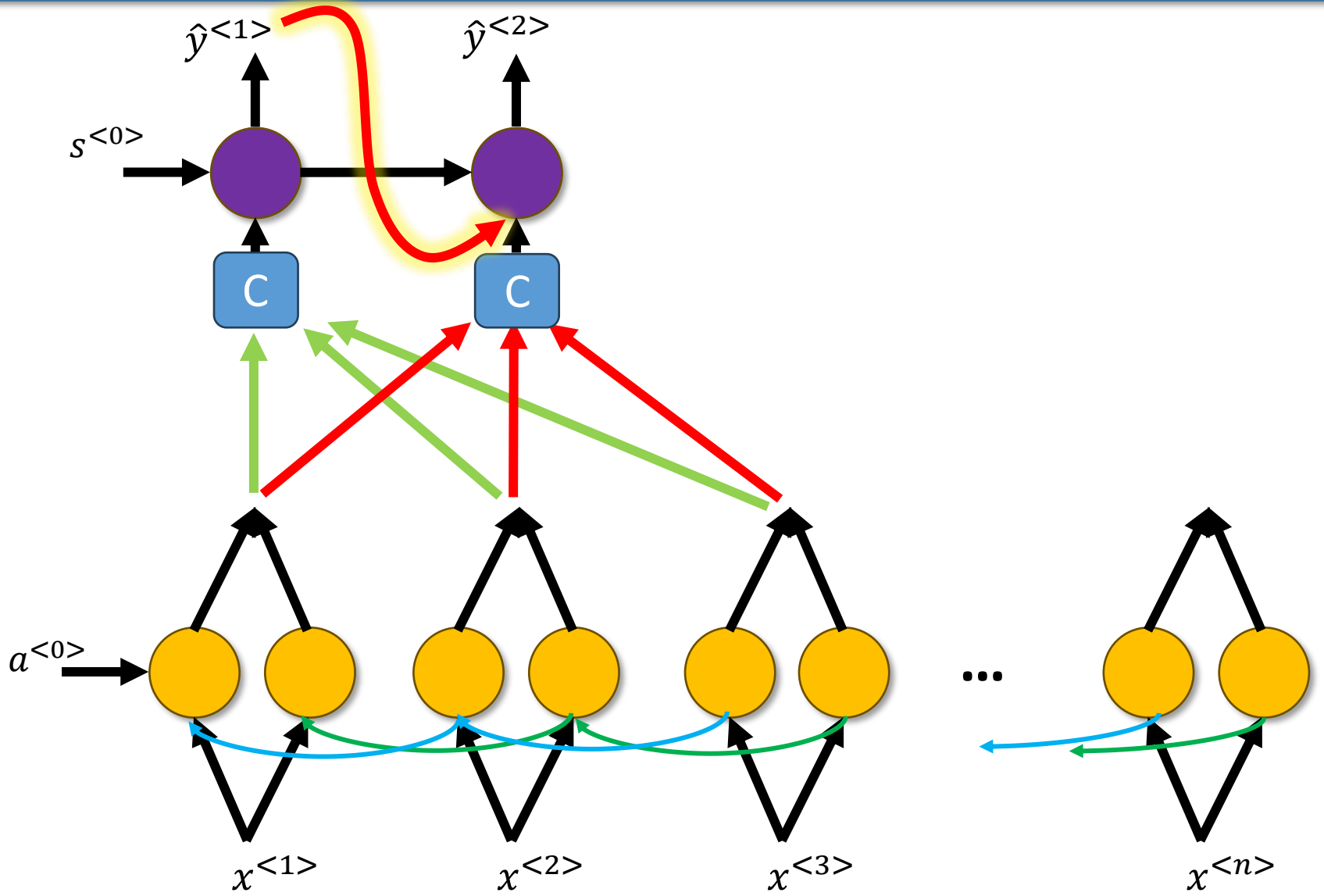
Intuition



Machine Learning: *Neural Networks*

Attention

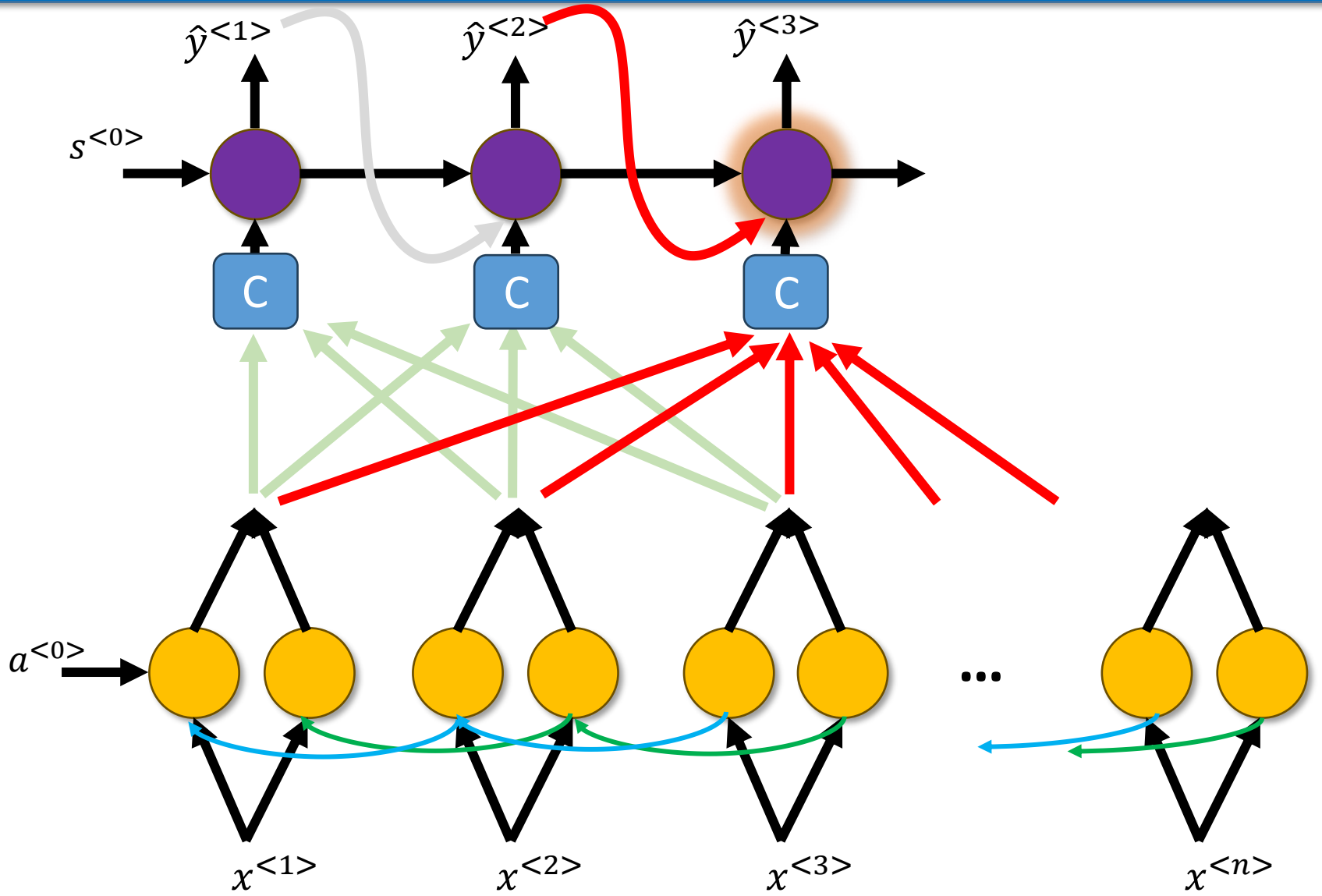
Intuition



Machine Learning: *Neural Networks*

Attention

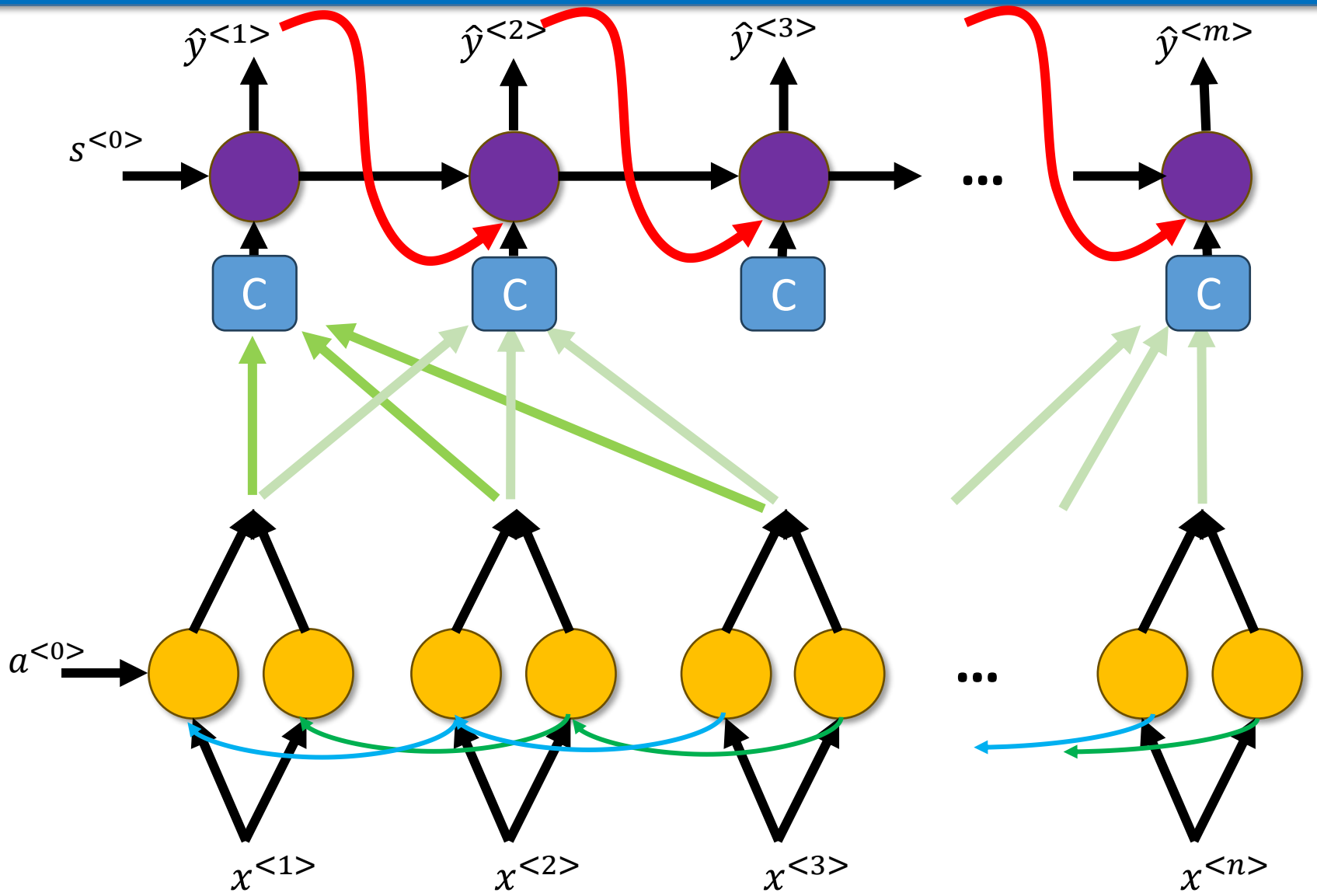
Intuition



Machine Learning: *Neural Networks*

Attention

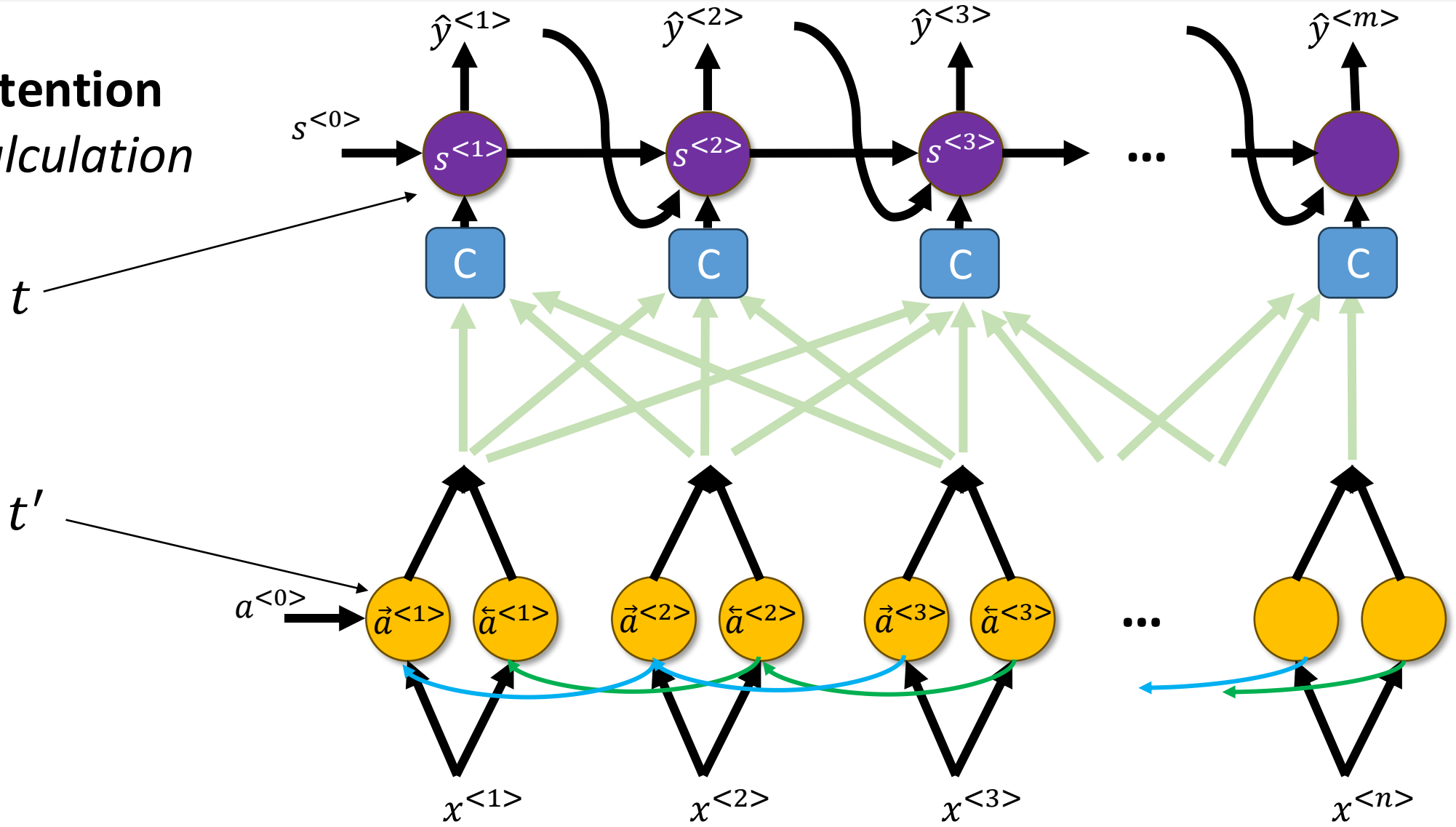
Intuition



Machine Learning: *Neural Networks*

Attention

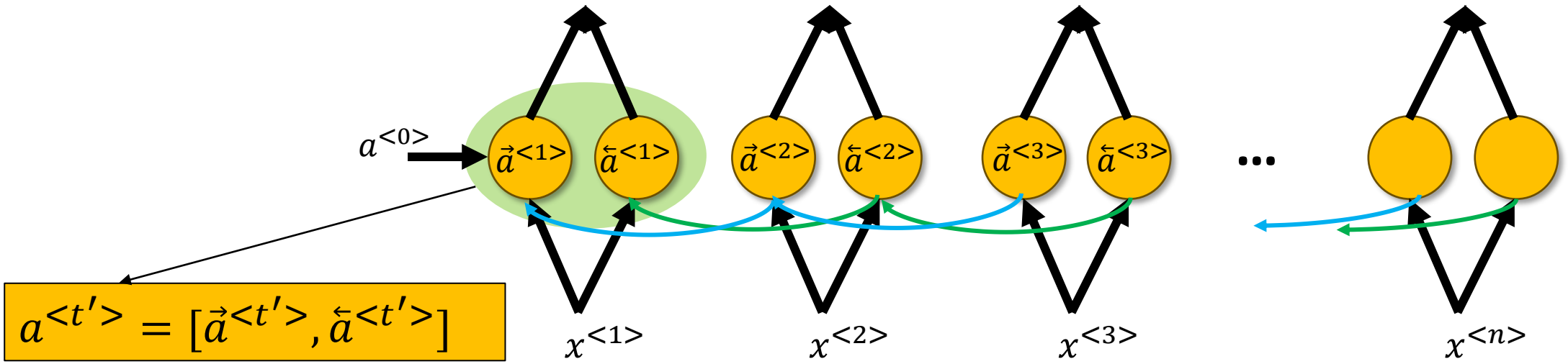
Calculation



Machine Learning: *Neural Networks*

Attention

Calculation



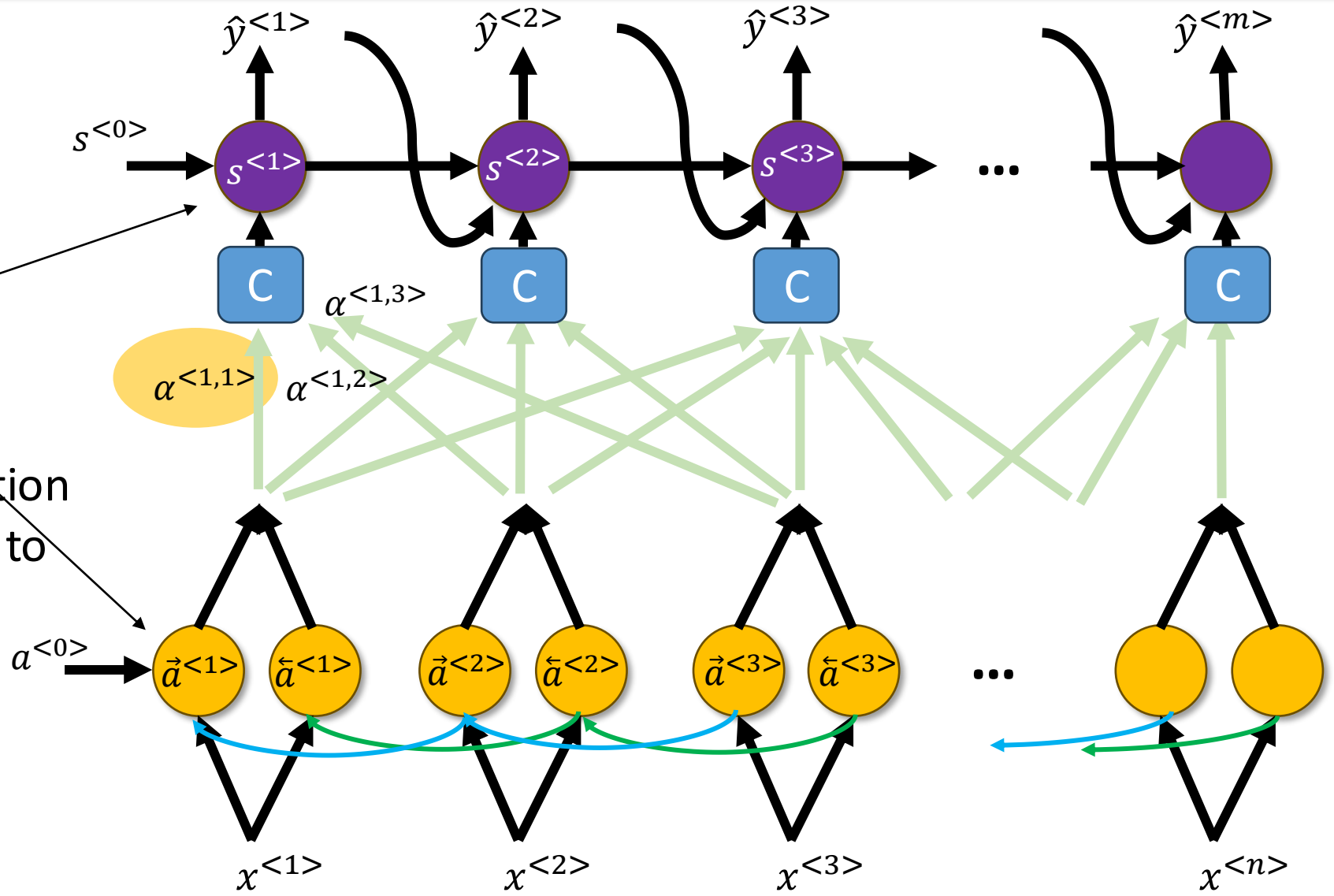
Machine Learning: *Neural Networks*

Attention

Calculation

$\alpha^{<t,t'>}$

amount of attention $y^{<t>}$ should pay to $a^{<t'>}$



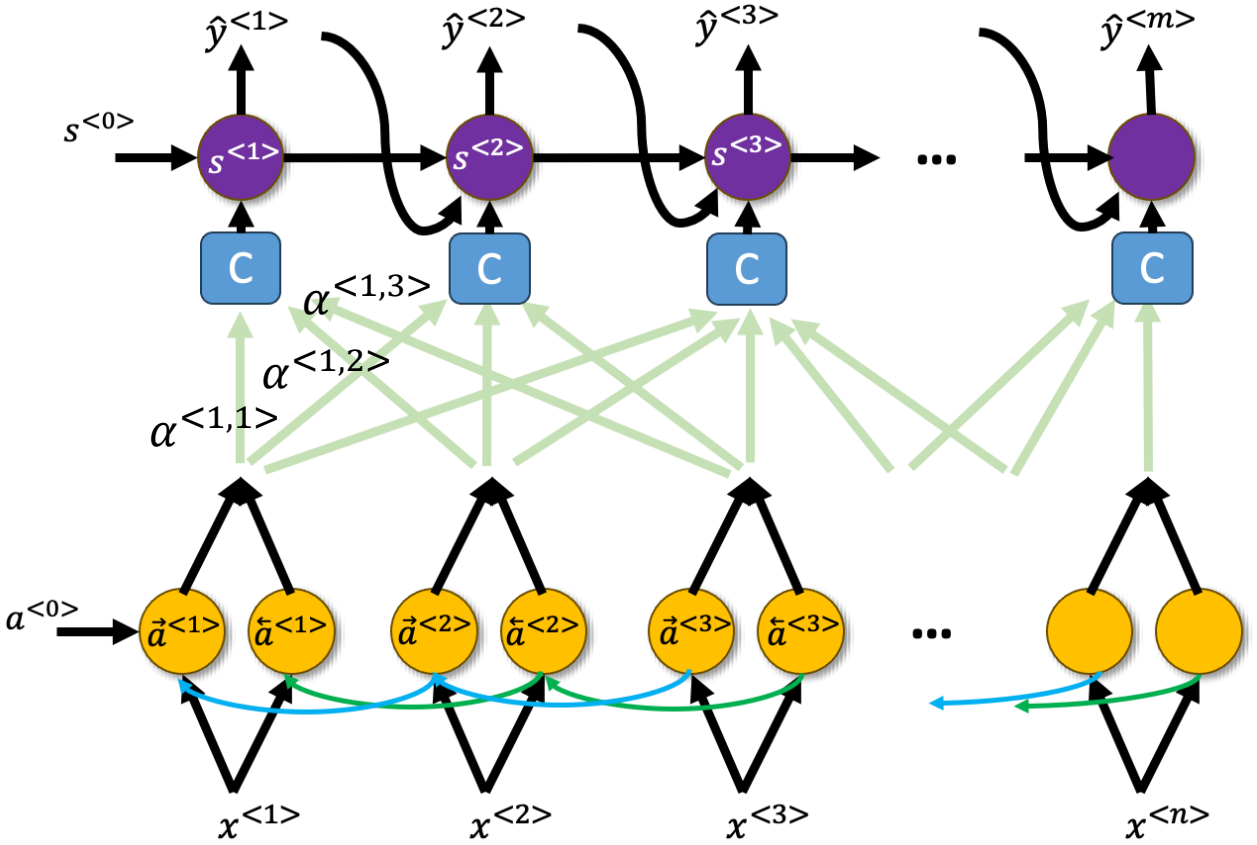
Machine Learning: *Neural Networks*

Attention

Calculation

The goal is calculate the context C:

$$c^{<t>} = \sum_{t'=1}^{n} \alpha^{<t,t'>} a^{<t'>}$$



$$a^{<t'>} = [\vec{a}^{<t'>}, \overleftarrow{a}^{<t'>}]$$

Machine Learning: *Neural Networks*

Attention

Calculation

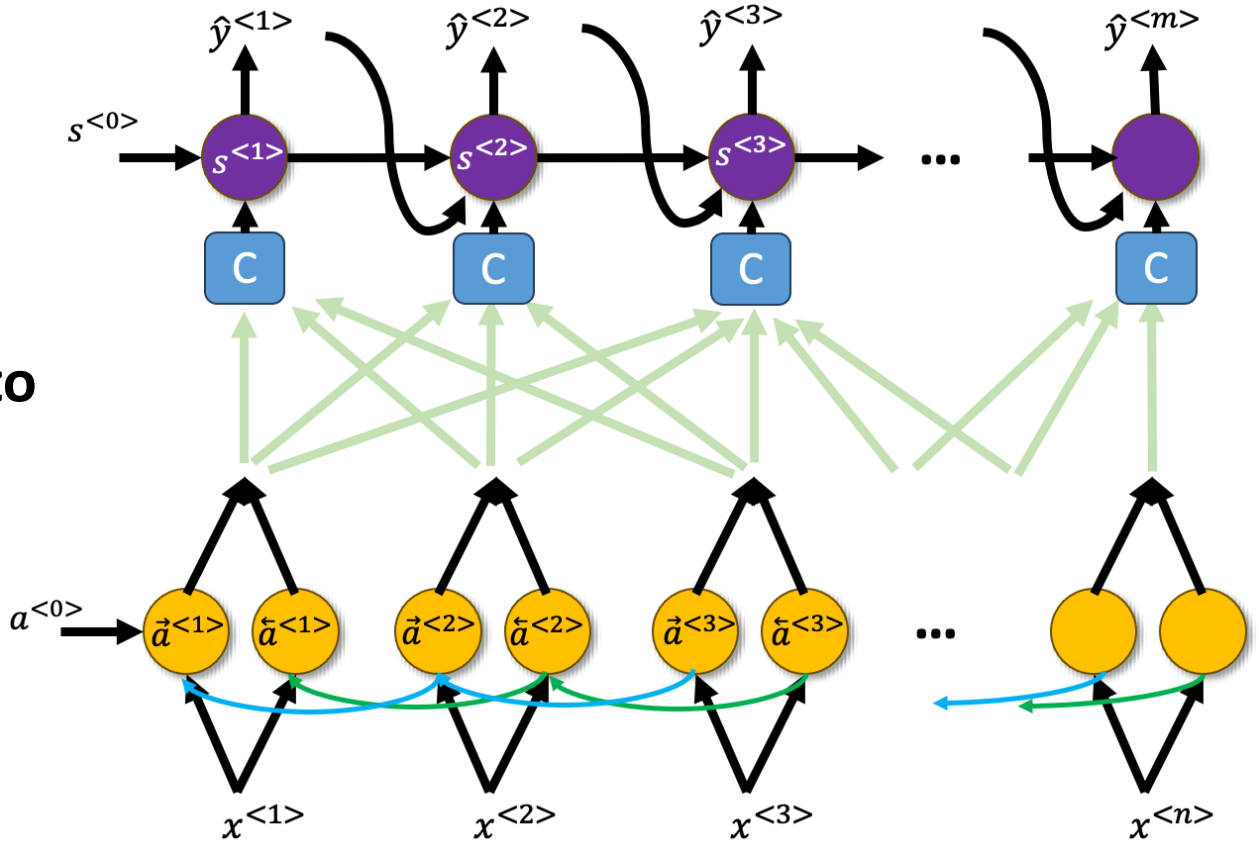
We want to know:

Which encoder state $a^{<t'>}$* is relevant to produce $s^{<t>}$?

So we compute a **score** between

$s^{<t-1>}$ and $a^{<t'>}$

$$e^{<t,t'>} = \text{score}\left(s^{<t-1>}, a^{<t'>}\right)$$



$* \; a^{<t'>} = [\vec{a}^{<t'>}, \overleftarrow{a}^{<t'>}]$

Machine Learning: *Neural Networks*

Attention

Calculation

$$e^{<t,t'>} = \text{score}\left(s^{<t-1>}, a^{<t'>}\right)$$

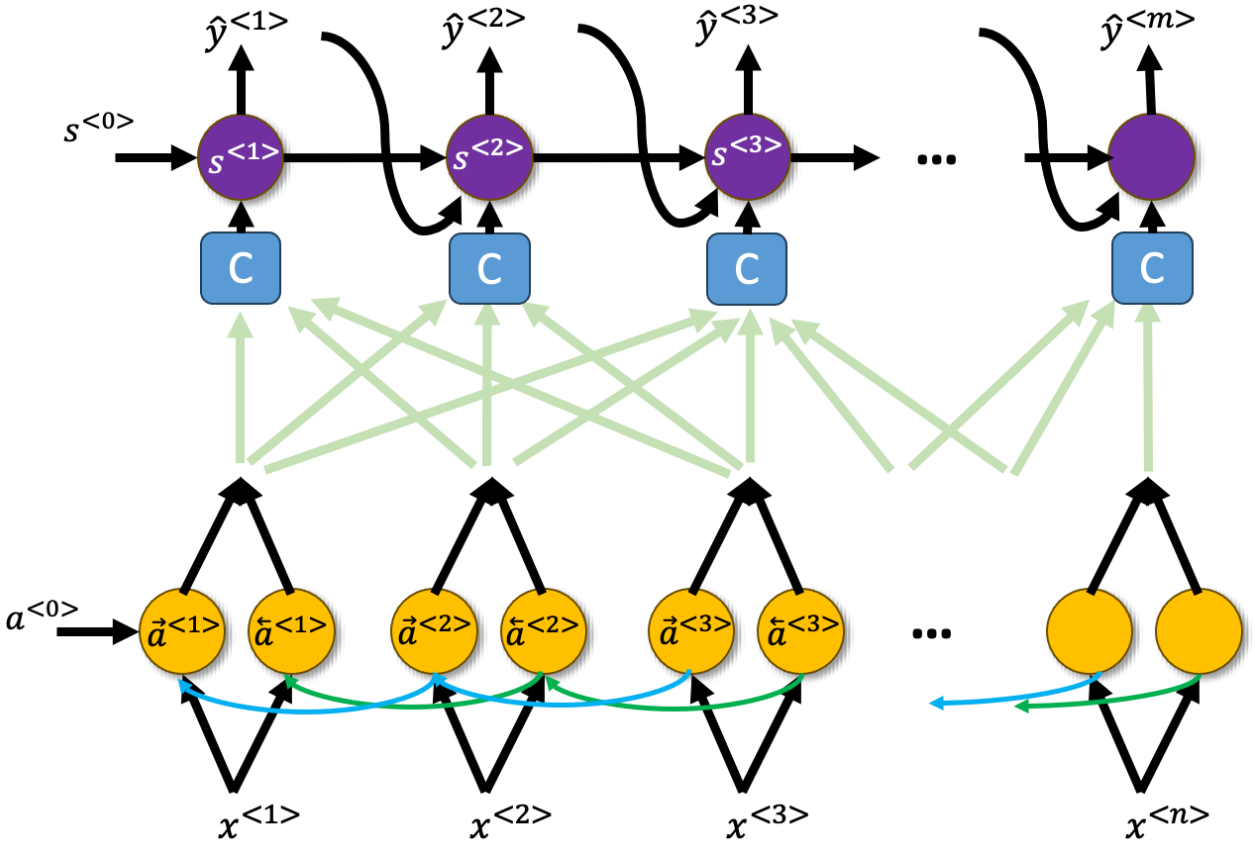
But these e's are just **similarity scores**.

Examples

$e^{<t,1>} = 2.3$

$e^{<t,1>} = -0.5$

$e^{<t,3>} = 4.1$



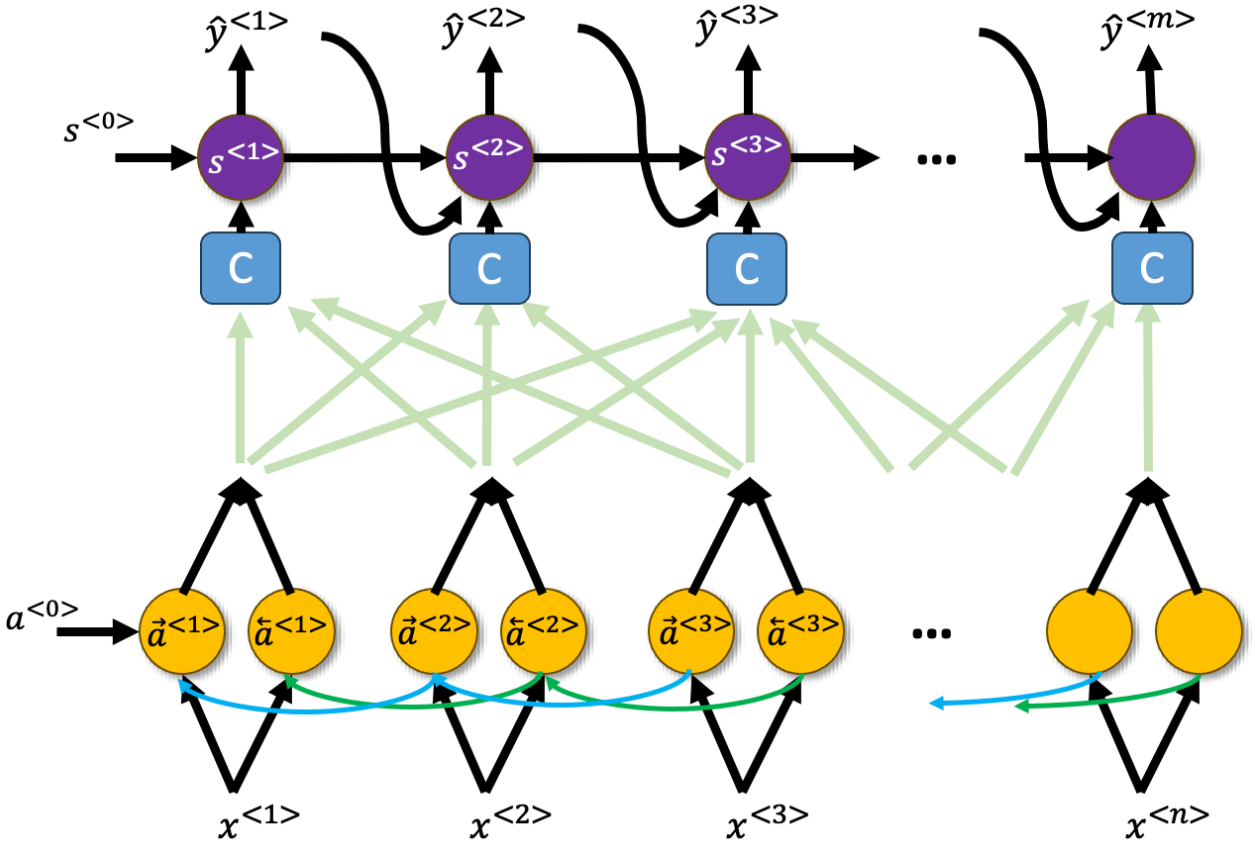
Machine Learning: *Neural Networks*

Attention

Calculation

$$e^{<t,t'>} = \text{score}\left(s^{<t-1>}, a^{<t'>}\right)$$

$$c^{<t>} = \sum_{t'=1}^{n} \alpha^{<t,t'>} a^{<t'>}$$



For this weighted sum to make sense:

- weights should be **positive**
- weights should **sum to 1**
- they should represent **relative importance**

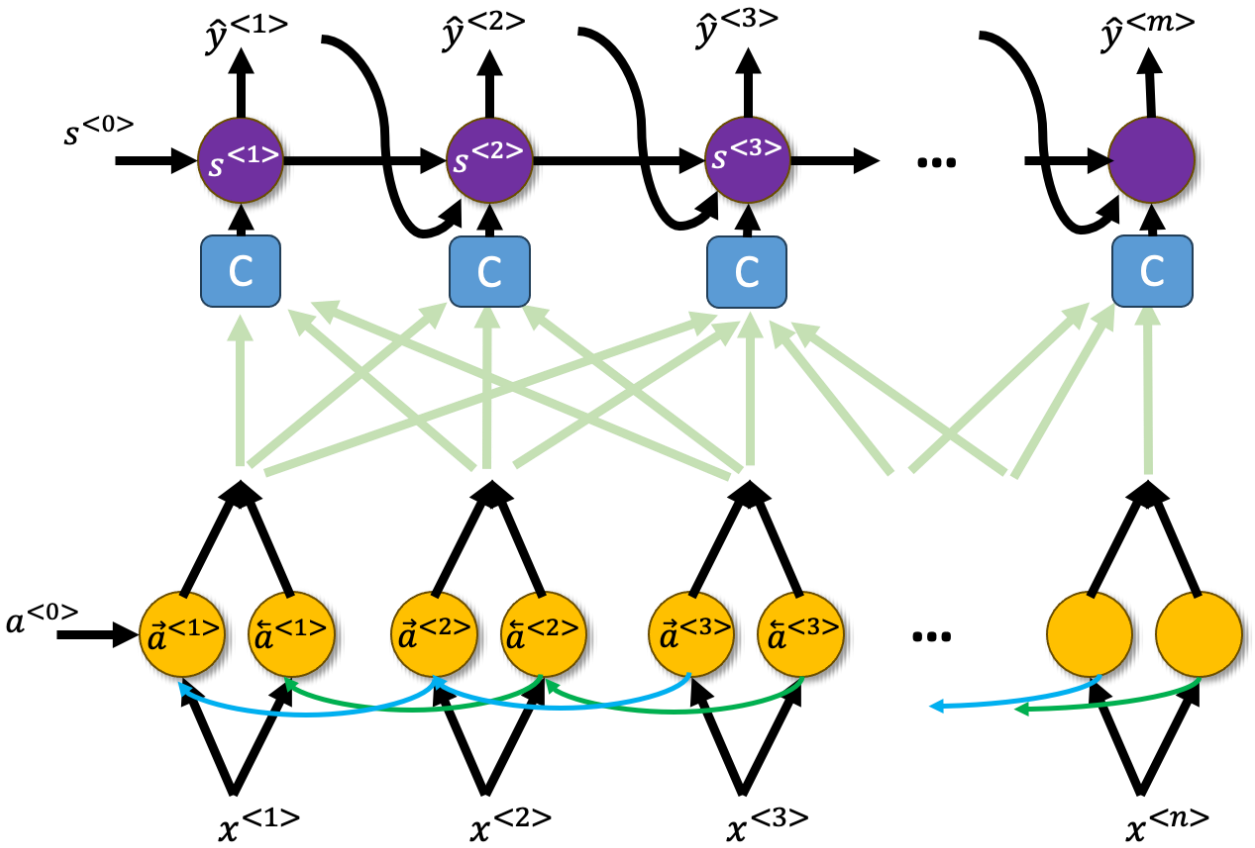
Machine Learning: *Neural Networks*

Attention

Calculation

$$e^{<t,t'>} = \text{score}\left(s^{<t-1>}, a^{<t'>}\right)$$

These are **not usable yet as weights**.



Machine Learning: *Neural Networks*

Attention

Calculation

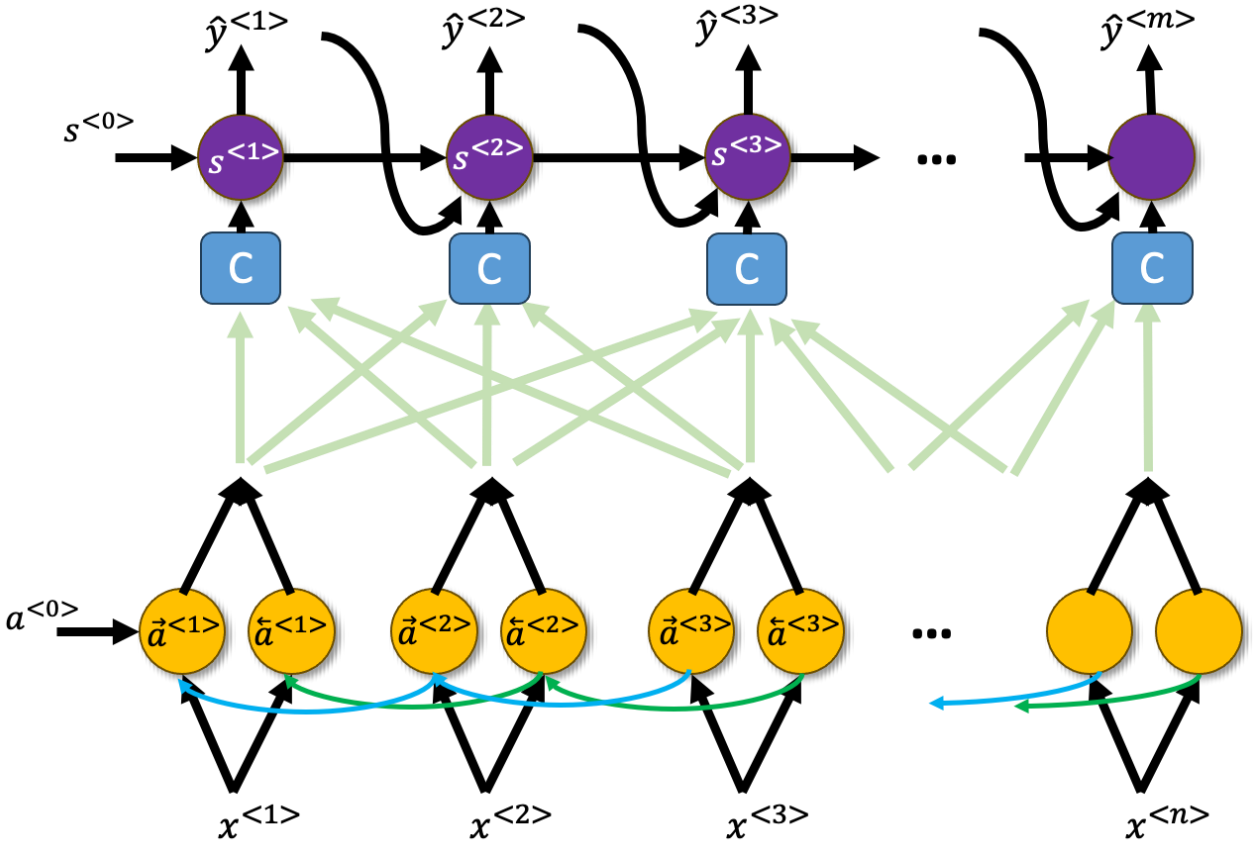
$$e^{<t,t'>} = \text{score}\left(s^{<t-1>}, a^{<t'>}\right)$$

Solution:

Convert scores into attention weights

$$\alpha^{<t,t'>} = \frac{e^{e^{<t,t'>}}}{\sum_{t'=1}^{n} e^{e^{<t,t'>}}}$$

Softmax normalization



Machine Learning: *Neural Networks*

Attention

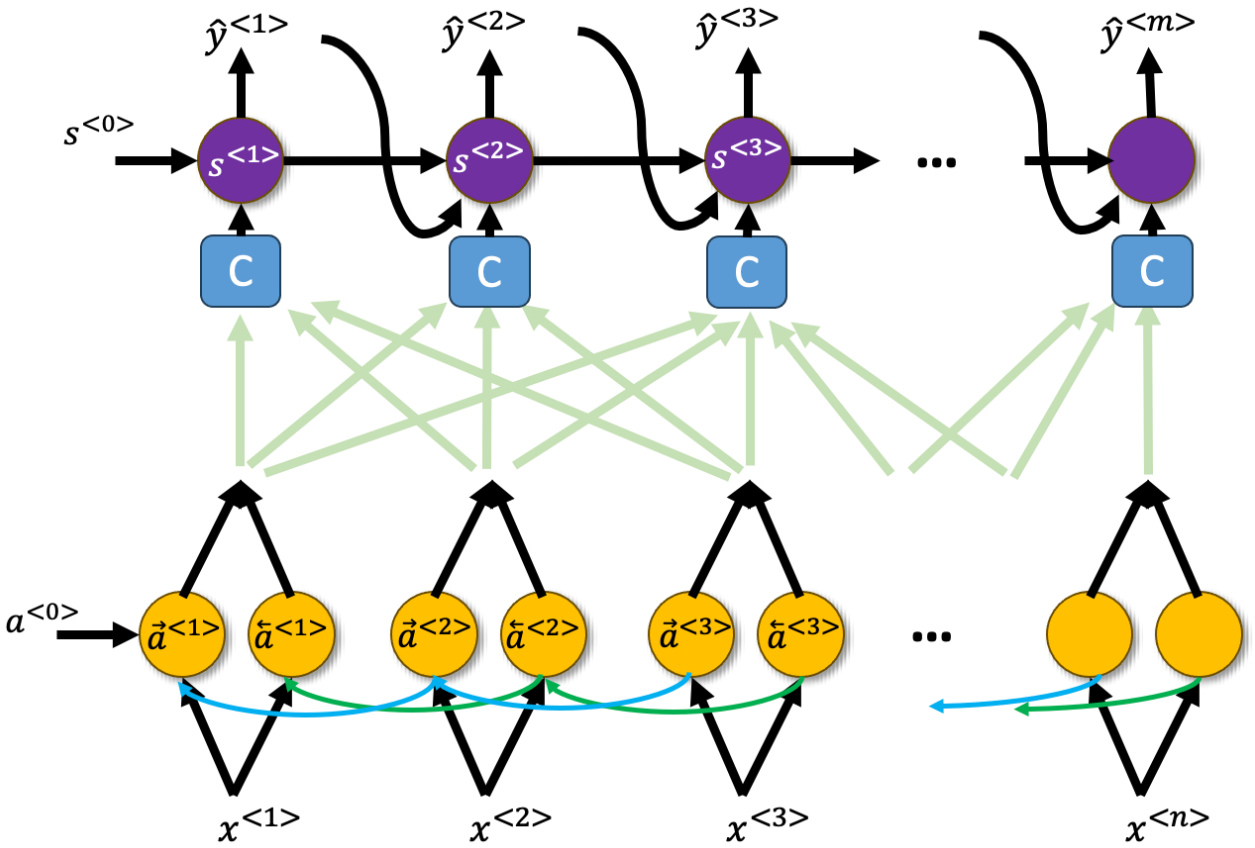
Calculation

For a specific t:

$$\sum_{t'=1}^{n} \alpha^{<t,t'>} = 1$$

$$c^{<t>} = \sum_{t'=1}^{n} \alpha^{<t,t'>} a^{<t'>}$$

weighted sum of all activations



Machine Learning: *Neural Networks*

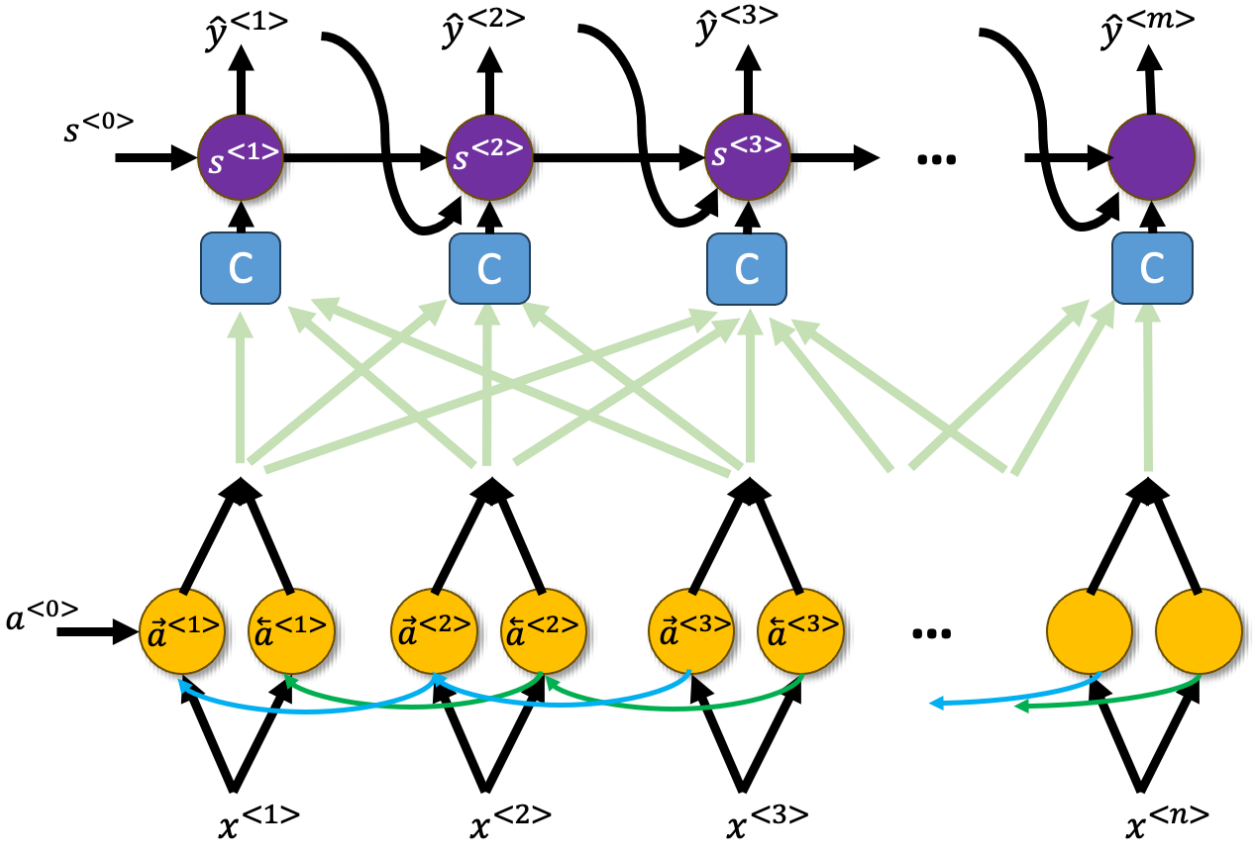
Attention

Calculation

$$\alpha^{<t,t'>} = \frac{e^{e^{<t,t'>}}}{\sum_{t'=1}^{n} e^{e^{<t,t'>}}}$$

Softmax normalization

$$e^{<t,t'>} = ?$$



Machine Learning: *Neural Networks*

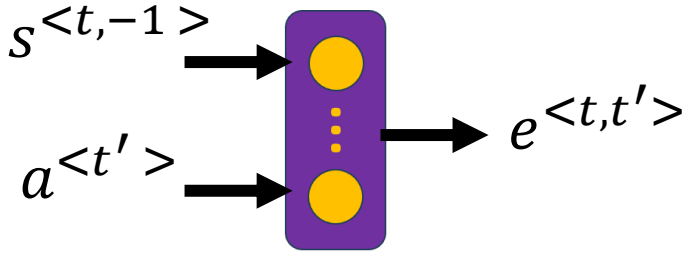
Attention

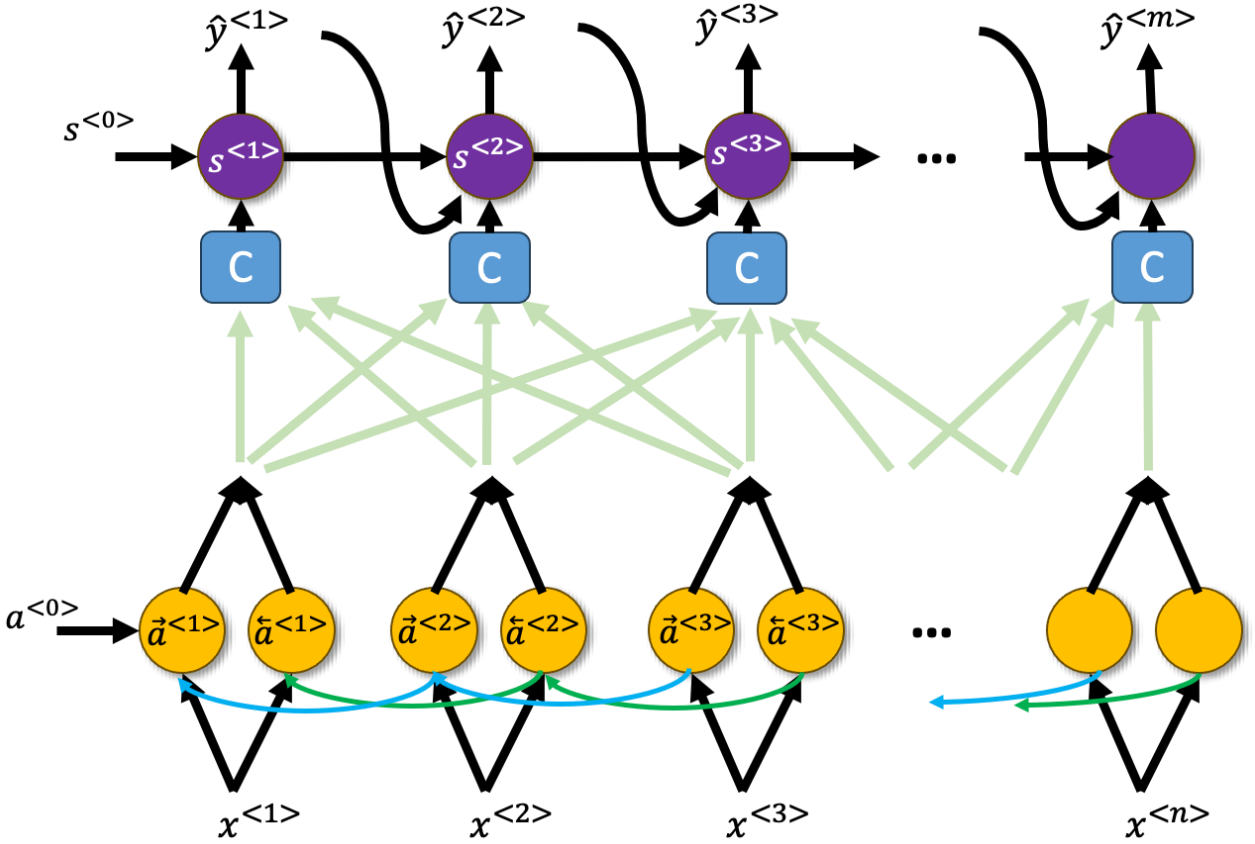
Calculation

$$\alpha^{<t,t'>} = \frac{e^{e^{<t,t'>}}}{\sum_{t'=1}^{n} e^{e^{<t,t'>}}}$$

previous state of the output

activation of current input

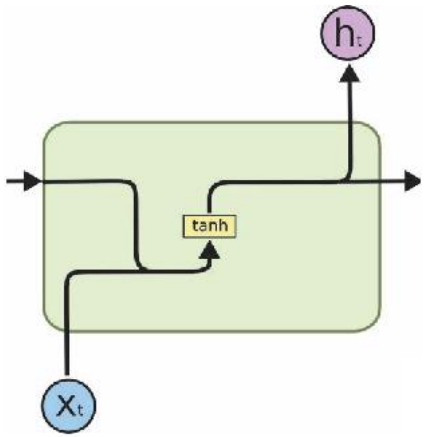




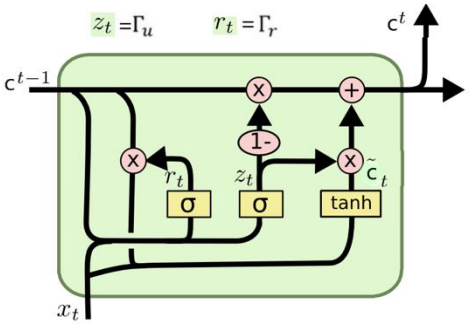
Machine Learning: *Neural Networks*

Summary

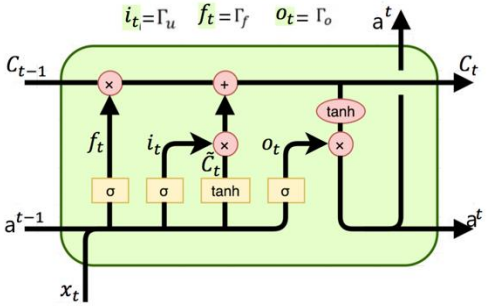
RNN



GRU



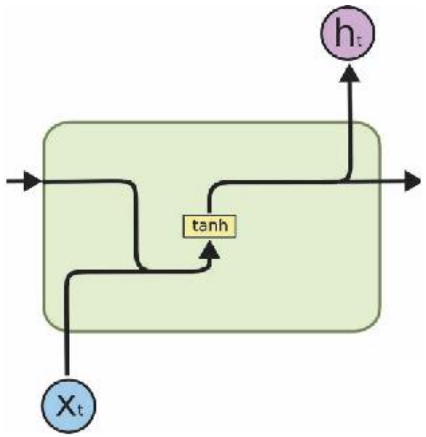
LSTM



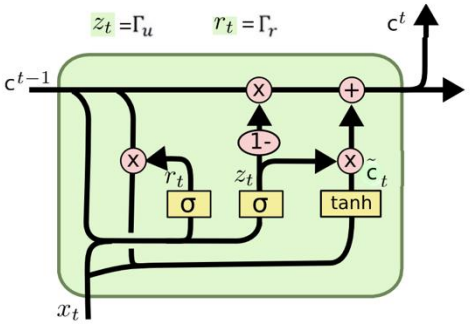
Machine Learning: *Neural Networks*

Summary

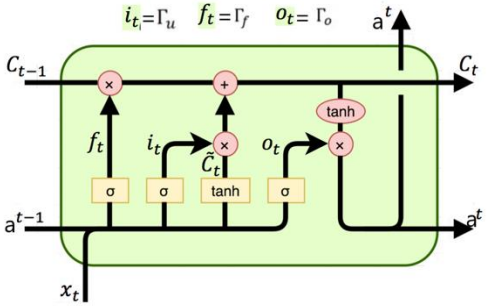
RNN



GRU



LSTM

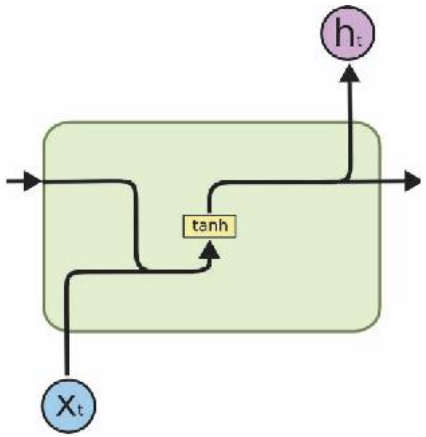


complexity

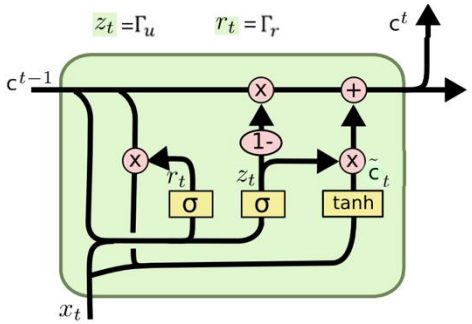
Machine Learning: *Neural Networks*

Summary

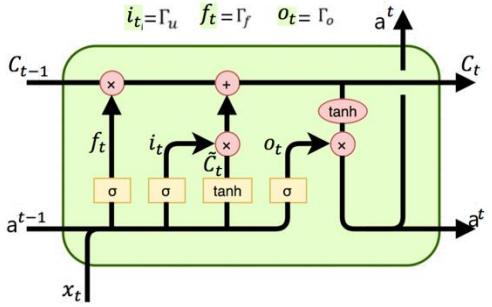
RNN



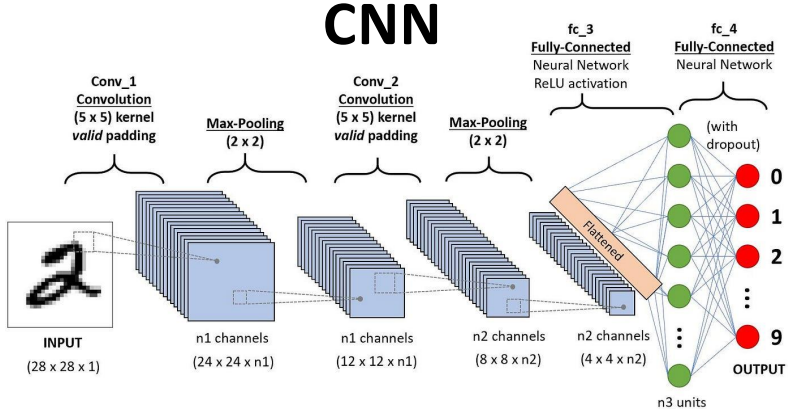
GRU



LSTM



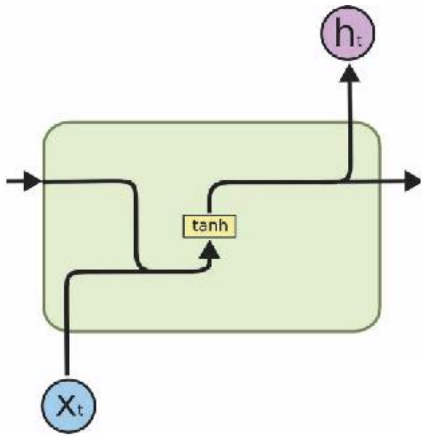
CNN



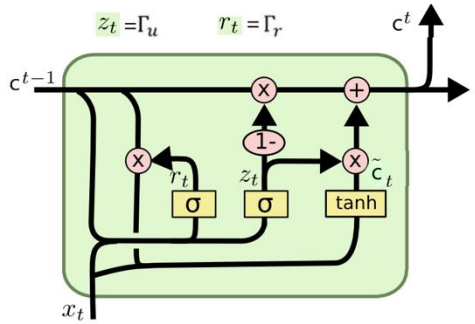
Machine Learning: *Neural Networks*

Summary

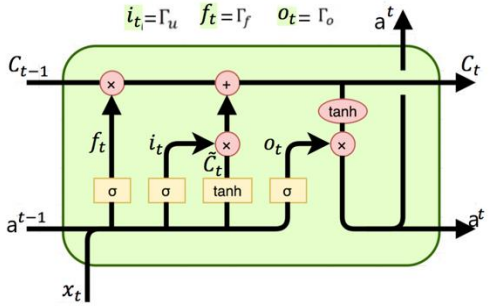
RNN



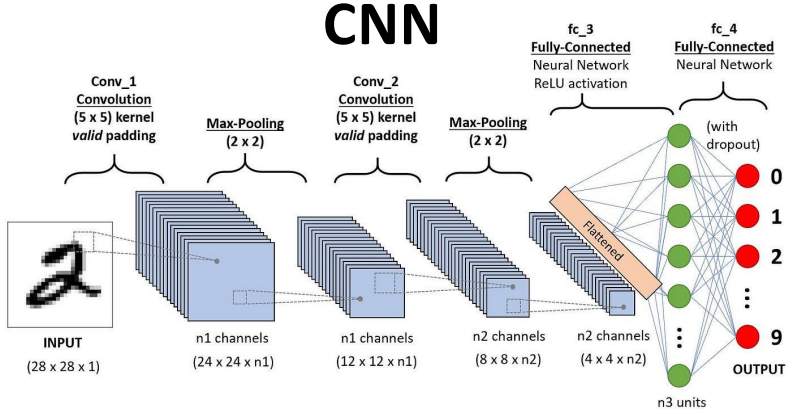
GRU



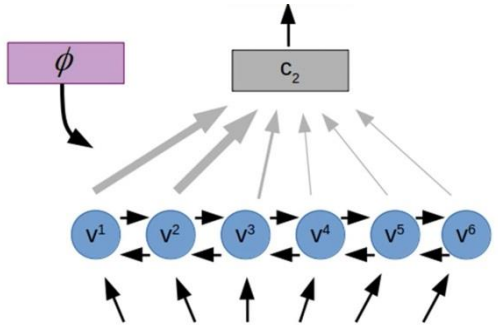
LSTM



CNN



Attention



Machine Learning: *Transformers*

Transformers

Ashish Vaswani, et al., (2017) “Attention Is All You Need”.
URL: https://arxiv.org/pdf/1706.03762.pdf

Attention + CNN

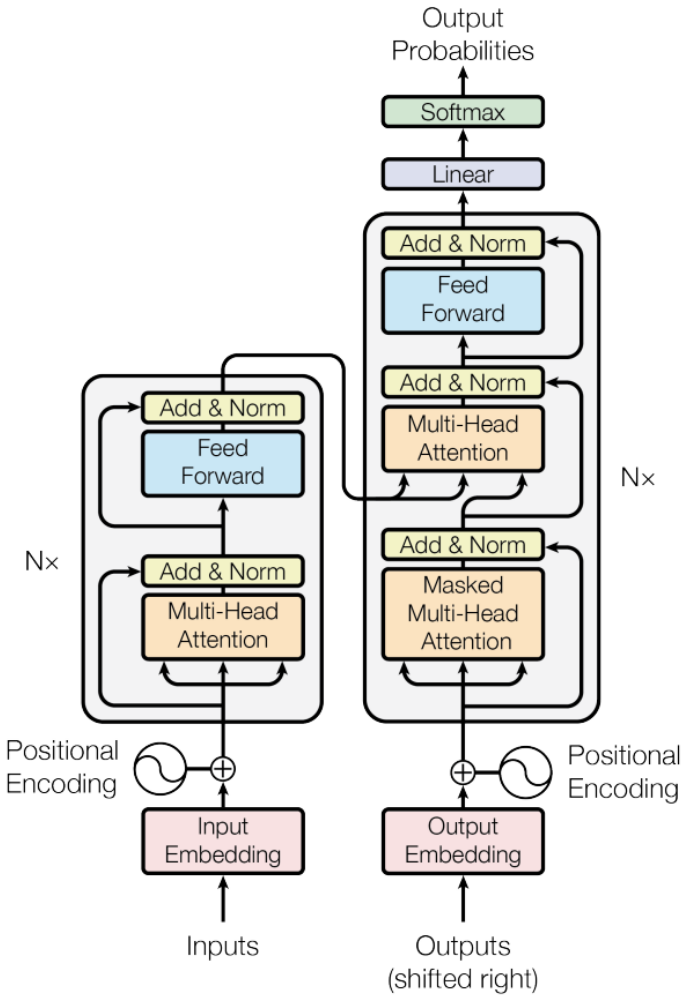


Figure 1: The Transformer - model architecture.

Machine Learning: *Transformers*

Transformers

- Self-attention
- Multi-head attention

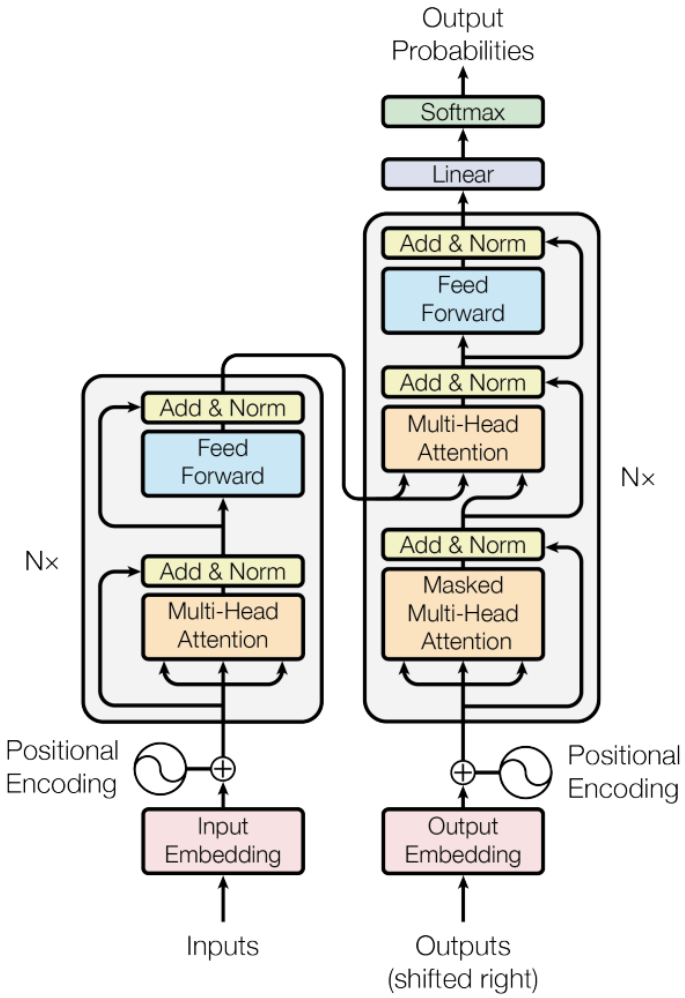


Figure 1: The Transformer - model architecture.

Machine Learning: *Transformers*

Transformers

- **Self-attention**
- Multi-head attention

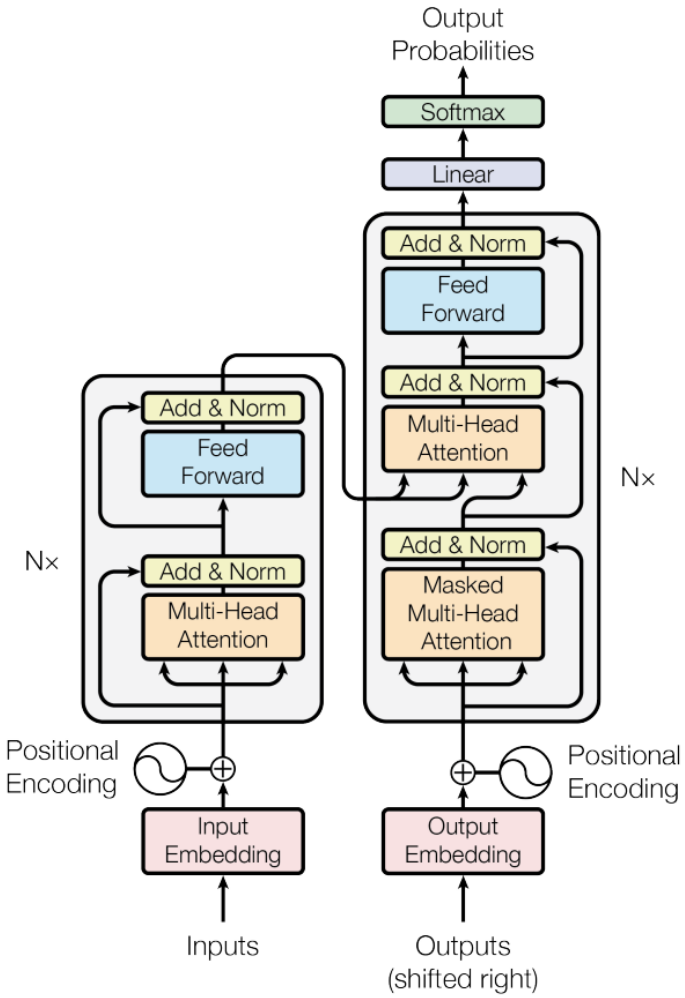


Figure 1: The Transformer - model architecture.

Machine Learning: *Transformers*

Self-Attention

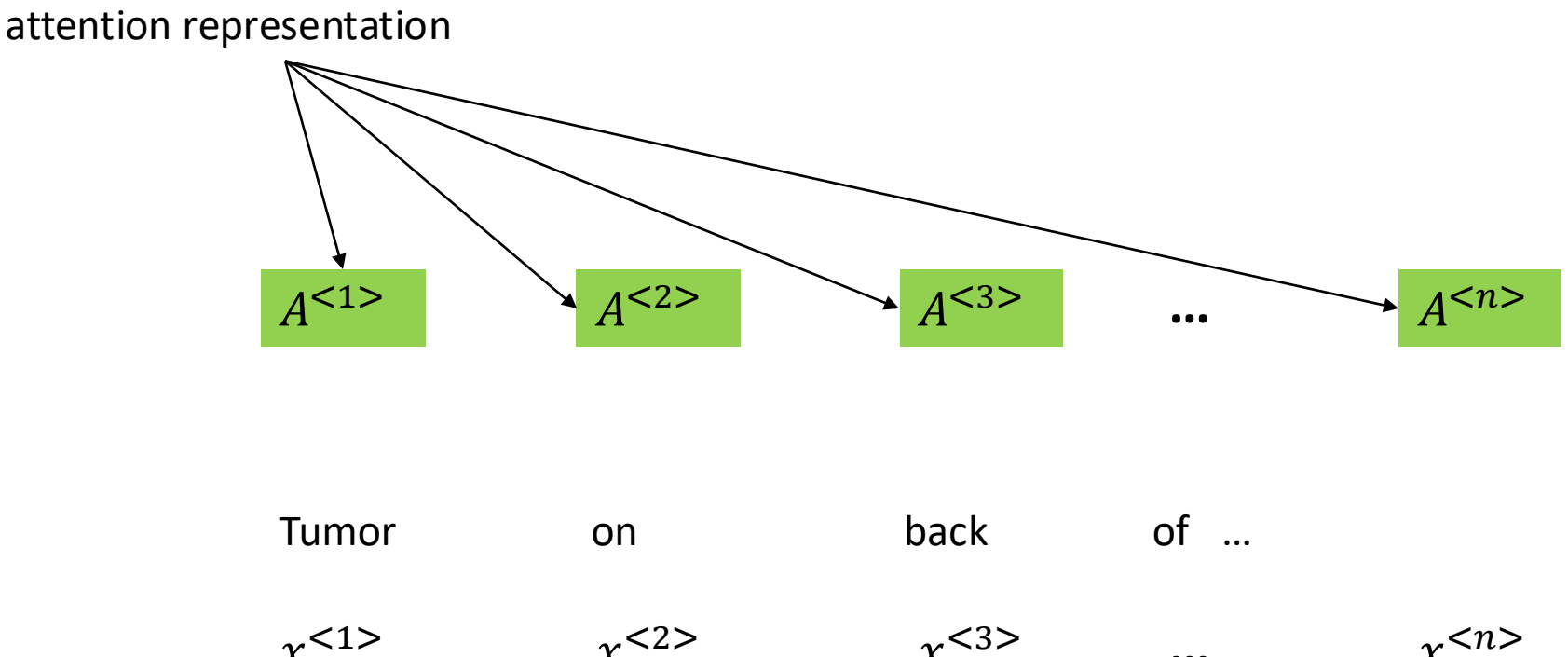
The goal is to create an attention-based representation

Tumor	on	back	of	…	
$x^{<1>}$	$x^{<2>}$	$x^{<3>}$	...		$x^{<n>}$

Machine Learning: *Transformers*

Self-Attention

The goal is to create an attention-based representation



Machine Learning: *Transformers*

Self-Attention

Tumor	on	back	of …		
$x^{<1>}$	$x^{<2>}$	$x^{<3>}$	…	$x^{<n>}$	

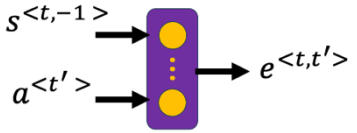
Machine Learning: *Transformers*

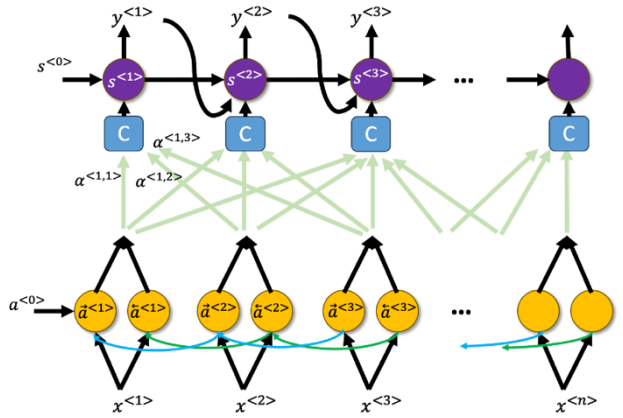
Self-Attention

Machine Learning: *Neural Networks*

Attention
Calculation

$$\alpha^{<t,t'>} = \frac{e^{e^{<t,t'>}}}{\sum_{t'=1}^{n} e^{e^{<t,t'>}}}$$





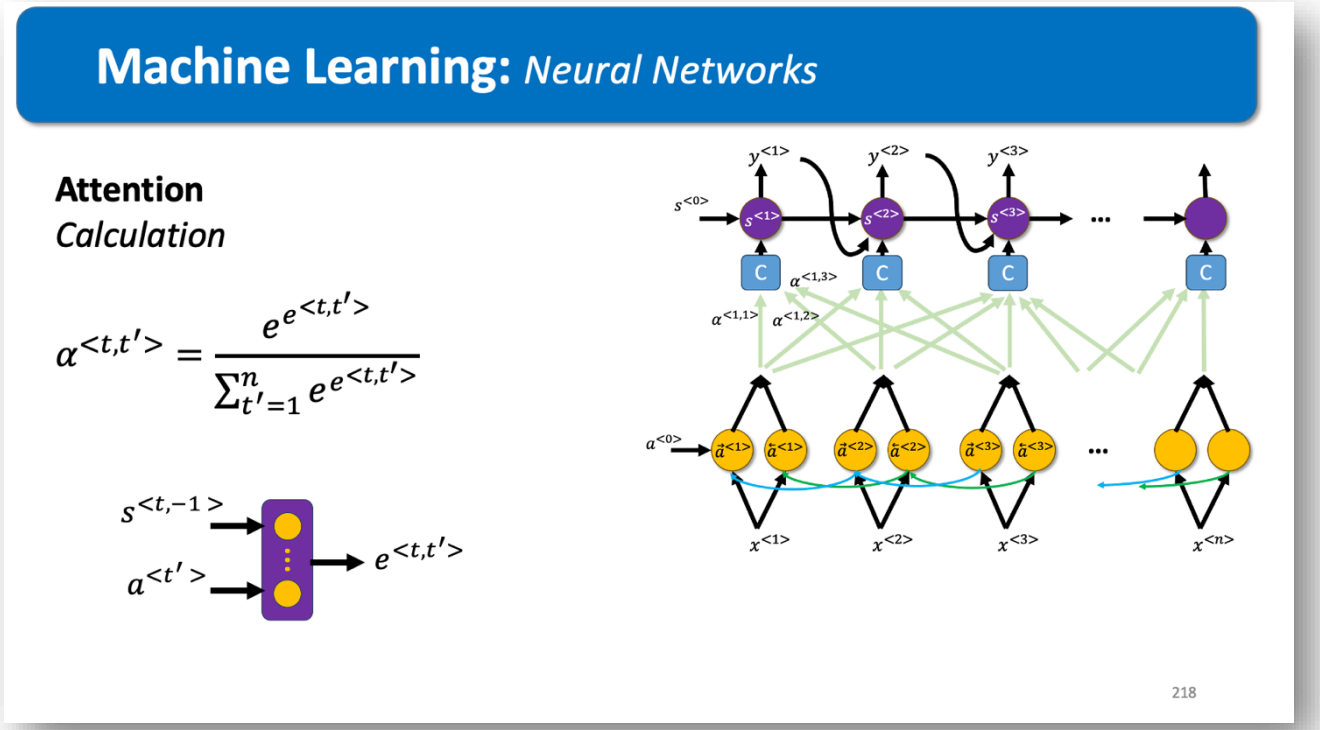
218

Tumor	on	back	of …	
$x^{<1>}$	$x^{<2>}$	$x^{<3>}$	…	$x^{<n>}$

Machine Learning: *Transformers*

Self-Attention

$$A(q^{<i>}, k^{<i>}, v^{<i>}) = \frac{e^{q^{<i>} \cdot k^{<i>}}}{\sum_j e^{q^{<j>} \cdot k^{<j>}}} v^{<i>}$$



Tumor	on	back	of …	
$x^{<1>}$	$x^{<2>}$	$x^{<3>}$	…	$x^{<n>}$

Machine Learning: *Transformers*

Self-Attention

$$A(q^{<3>}, k^{<i>}, v^{<i>}) = \frac{e^{q^{<3>} . k^{<i>}}}{\sum_j e^{q^{<3>} . k^{<j>}}} v^{<i>}$$

Q: Query
K: Key
V: Value

Q	K	V
$q^{<1>}$	$k^{<1>}$	$v^{<1>}$
$q^{<2>}$	$k^{<2>}$	$v^{<2>}$
$q^{<3>}$	$k^{<3>}$	$v^{<3>}$
…	..	..

Tumor	on	back	of …		
$x^{<1>}$	$x^{<2>}$	$x^{<3>}$	…	$x^{<n>}$	

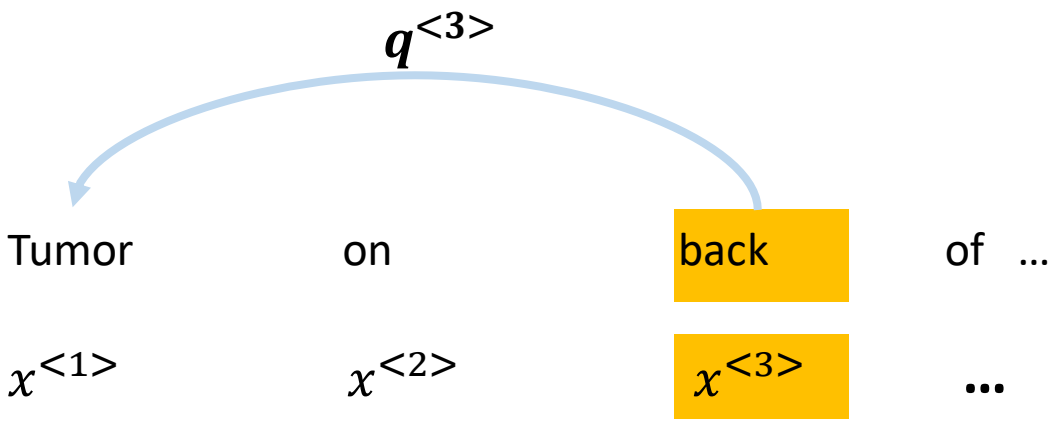
Machine Learning: *Transformers*

Self-Attention

Intuition

$$A(q^{<3>}, k^{<i>}, v^{<i>}) = \frac{e^{q^{<3>} \cdot k^{<i>}}}{\sum_j e^{q^{<3>} \cdot k^{<j>}}} v^{<i>}$$

Q	K	V
$q^{<1>}$	$k^{<1>}$	$v^{<1>}$
$q^{<2>}$	$k^{<2>}$	$v^{<2>}$
$q^{<3>}$	$k^{<3>}$	$v^{<3>}$
…	..	..



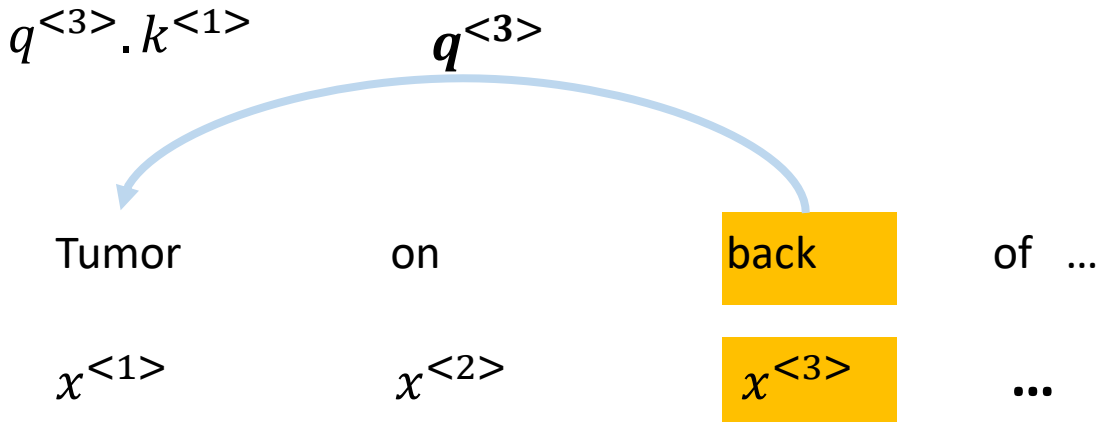
Machine Learning: *Transformers*

Self-Attention

Intuition

$$A(q^{<3>}, k^{<i>}, v^{<i>}) = \frac{e^{q^{<3>} . k^{<i>}}}{\sum_j e^{q^{<3>} . k^{<j>}}} v^{<i>}$$

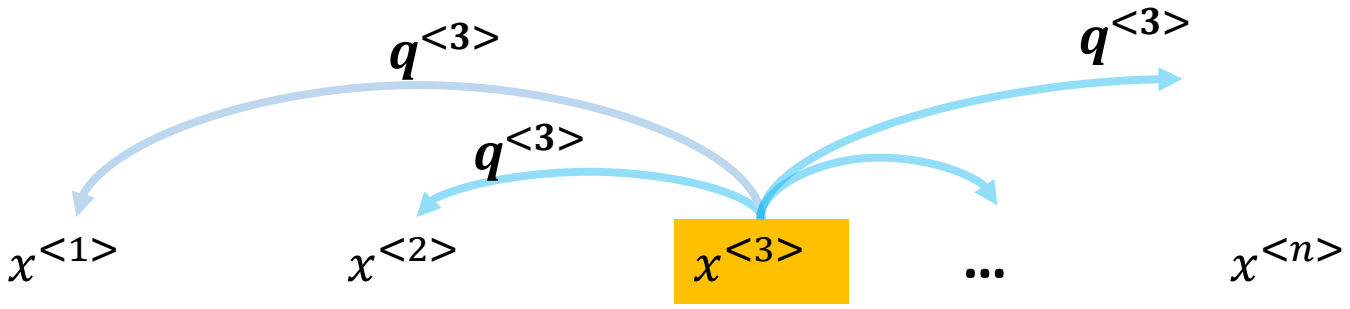
Q	K	V
$q^{<1>}$	$k^{<1>}$	$v^{<1>}$
$q^{<2>}$	$k^{<2>}$	$v^{<2>}$
$q^{<3>}$	$k^{<3>}$	$v^{<3>}$
…	..	..



Machine Learning: *Transformers*

Self-Attention

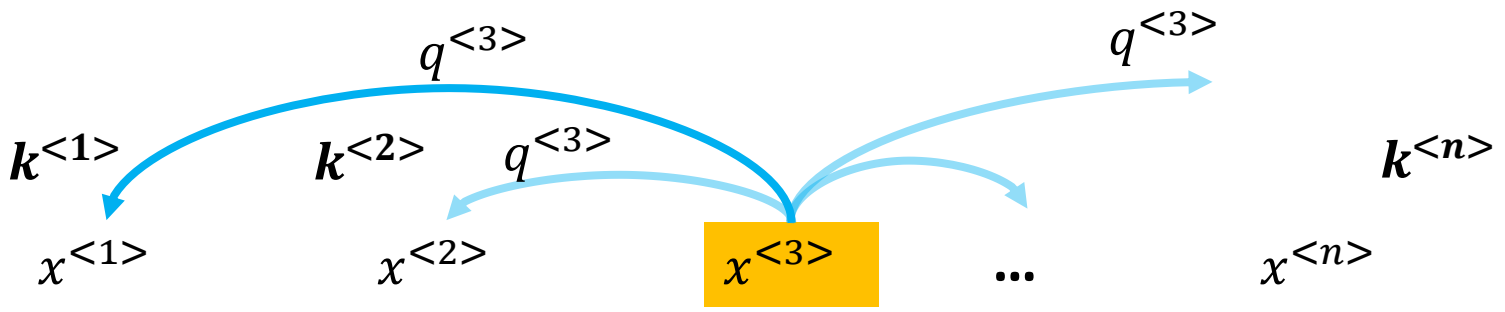
$$A(q^{<3>}, k^{<i>}, v^{<i>}) = \frac{e^{q^{<3>} . k^{<i>}}}{\sum_j e^{q^{<3>} . k^{<j>}}} v^{<i>}$$



Machine Learning: *Transformers*

Self-Attention

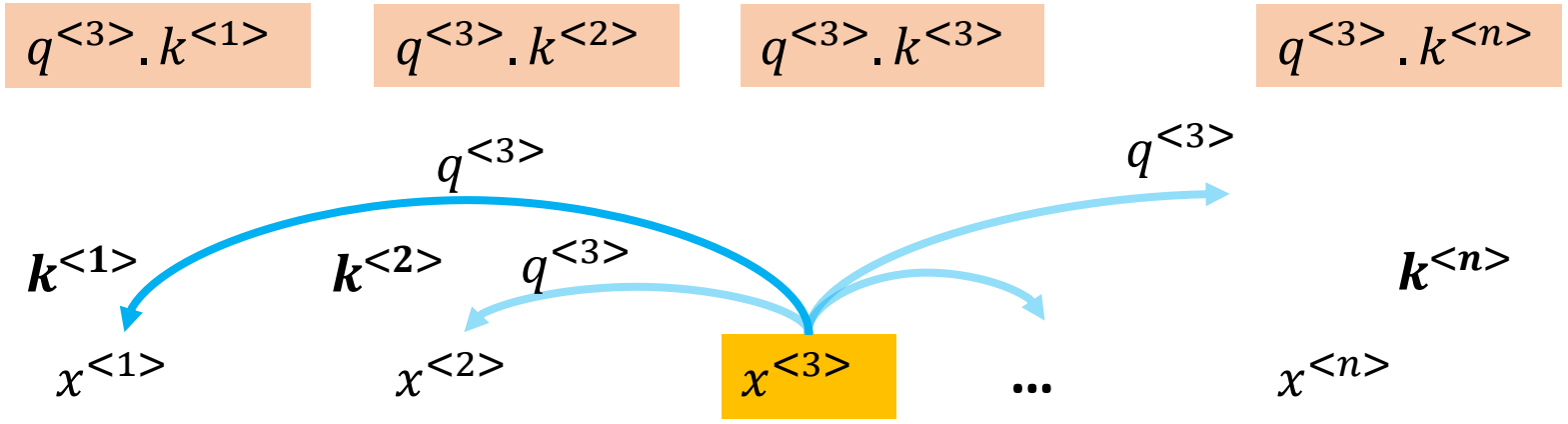
$$A(q^{<3>}, k^{<i>}, v^{<i>}) = \frac{e^{q^{<3>} . k^{<i>}}}{\sum_j e^{q^{<3>} . k^{<j>}}} v^{<i>}$$



Machine Learning: *Transformers*

Self-Attention

$$A(q^{<3>}, k^{<i>}, v^{<i>}) = \frac{e^{q^{<3>} . k^{<i>}}}{\sum_j e^{q^{<3>} . k^{<j>}}} v^{<i>}$$

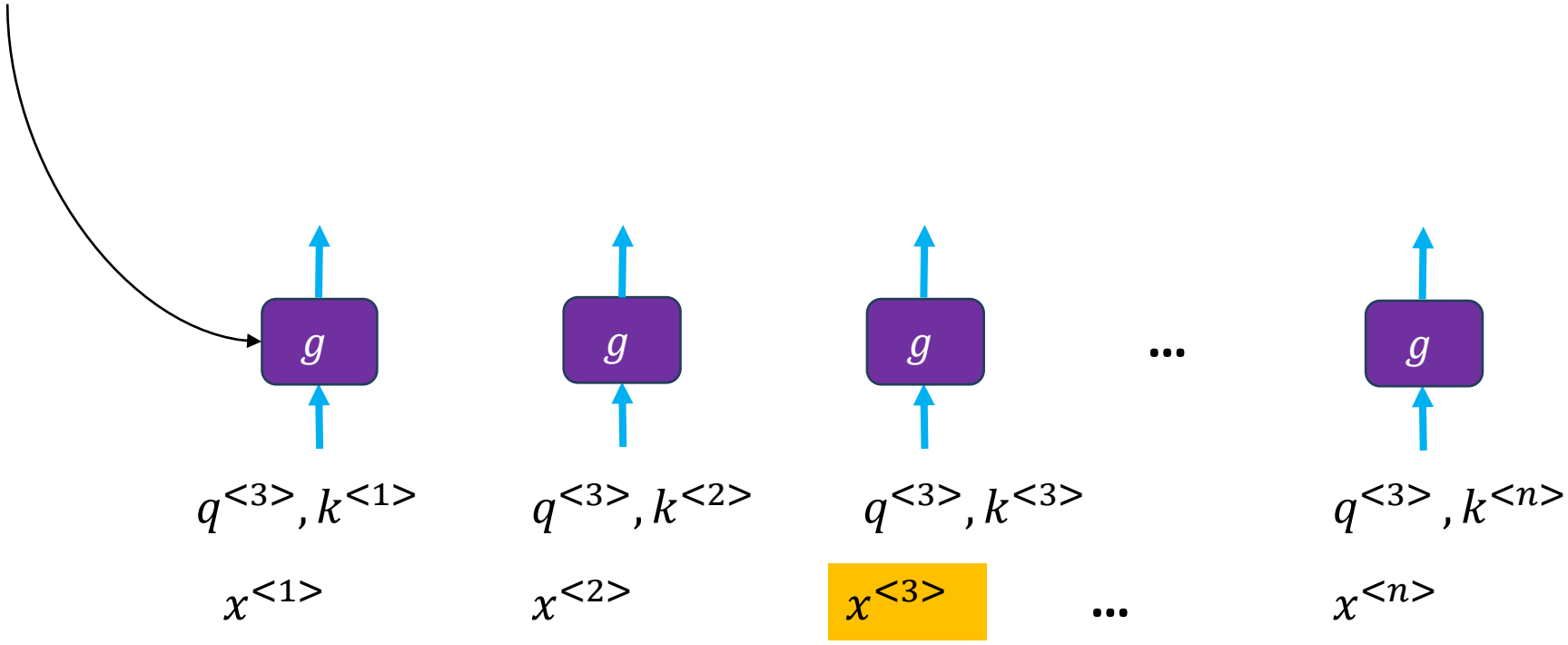


Machine Learning: *Transformers*

Self-Attention

$$A(q^{<3>}, k^{<i>}, v^{<i>}) = \frac{e^{q^{<3>}.k^{<i>}}}{\sum_j e^{q^{<3>}.k^{<j>}}} v^{<i>}$$

$$g(q, k^{<i>}) = \frac{e^{q.k^{<i>}}}{\sum_j e^{q.k^{<j>}}}$$

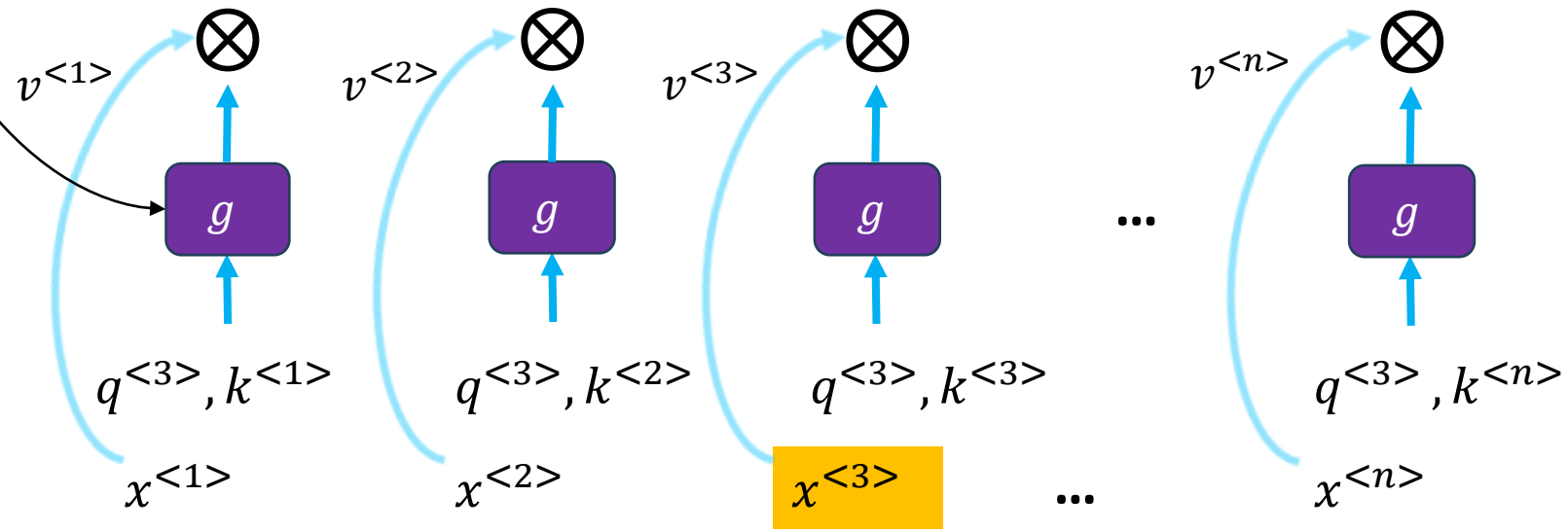


Machine Learning: *Transformers*

Self-Attention

$$g(q, k^{<i>}) = \frac{e^{q.k^{<i>}}}{\sum_j e^{q.k^{<j>}}}$$

$$A(q^{<3>}, k^{<i>}, v^{<i>}) = \frac{e^{q^{<3>}.k^{<i>}}}{\sum_j e^{q^{<3>}.k^{<j>}}} v^{<i>}$$

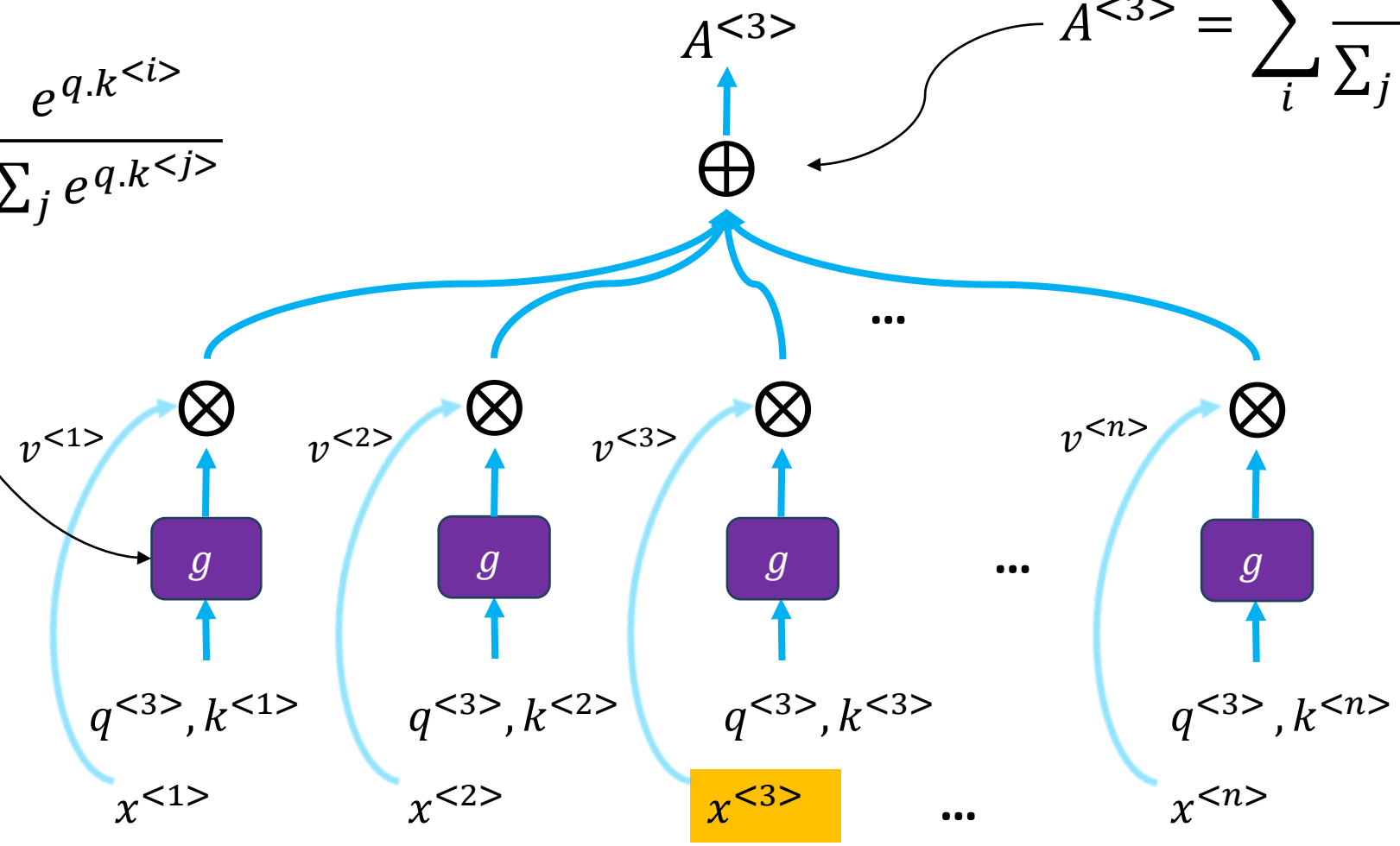


Machine Learning: *Transformers*

Self-Attention

$$g(q, k^{<i>}) = \frac{e^{q.k^{<i>}}}{\sum_j e^{q.k^{<j>}}}$$

$$A^{<3>} = \sum_i \frac{e^{q^{<3>}.k^{<i>}}}{\sum_j e^{q^{<3>}.k^{<j>}}} v^{<i>}$$



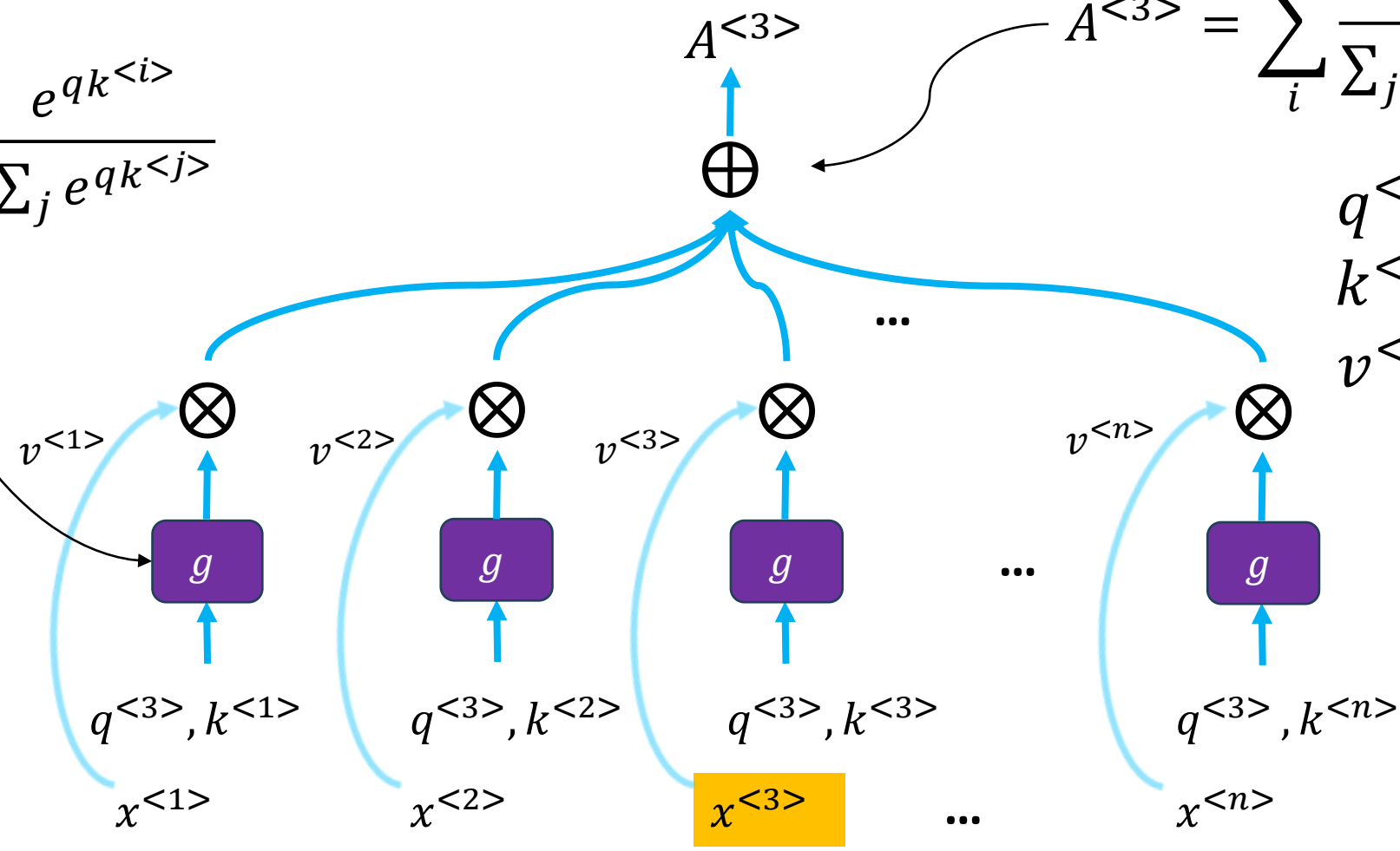
Machine Learning: *Transformers*

Self-Attention

$$g(q, k^{<i>}) = \frac{e^{qk^{<i>}}}{\sum_j e^{qk^{<j>}}}$$

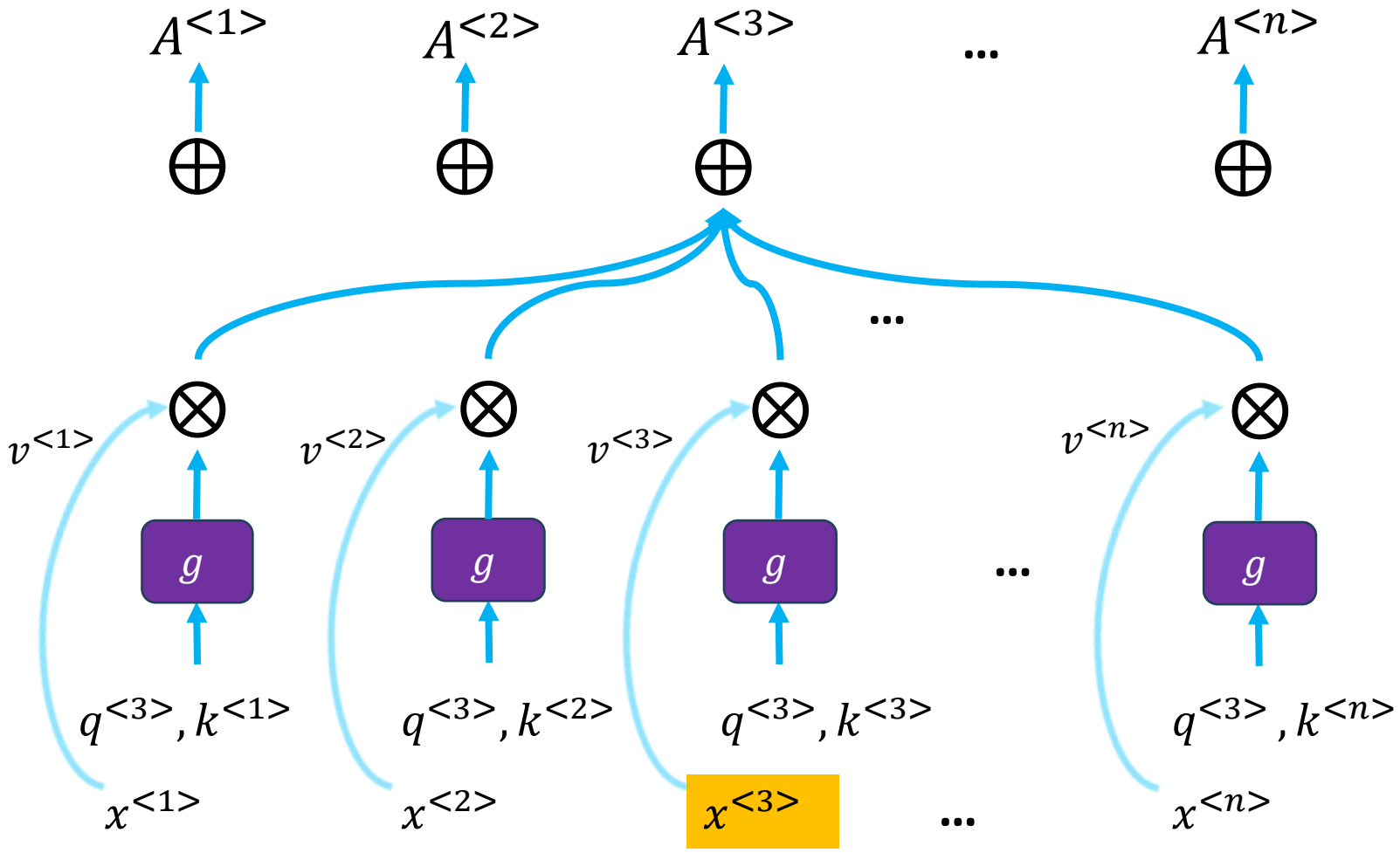
$$A^{<3>} = \sum_i \frac{e^{q^{<3>}.k^{<i>}}}{\sum_j e^{q^{<3>}.k^{<j>}}} v^{<i>}$$

$$q^{<i>} = W^Q . x^{<i>}$$
$$k^{<i>} = W^K . x^{<i>}$$
$$v^{<i>} = W^V . x^{<i>}$$



Machine Learning: *Transformers*

Self-Attention



Machine Learning: *Transformers*

Self-Attention

$$A(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$A^{<3>} = \sum_i \frac{e^{q^{<3>}.k^{<i>}}}{\sum_j e^{q^{<3>}.k^{<j>}}} v^{<i>}$$

Machine Learning: *Transformers*

Self-Attention

$$A(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaling

$$A^{<3>} = \sum_i \frac{e^{q^{<3>}.k^{<i>}}}{\sum_j e^{q^{<3>}.k^{<j>}}} v^{<i>}$$

Machine Learning: *Transformers*

Transformers

- Self-attention
- **Multi-head attention**

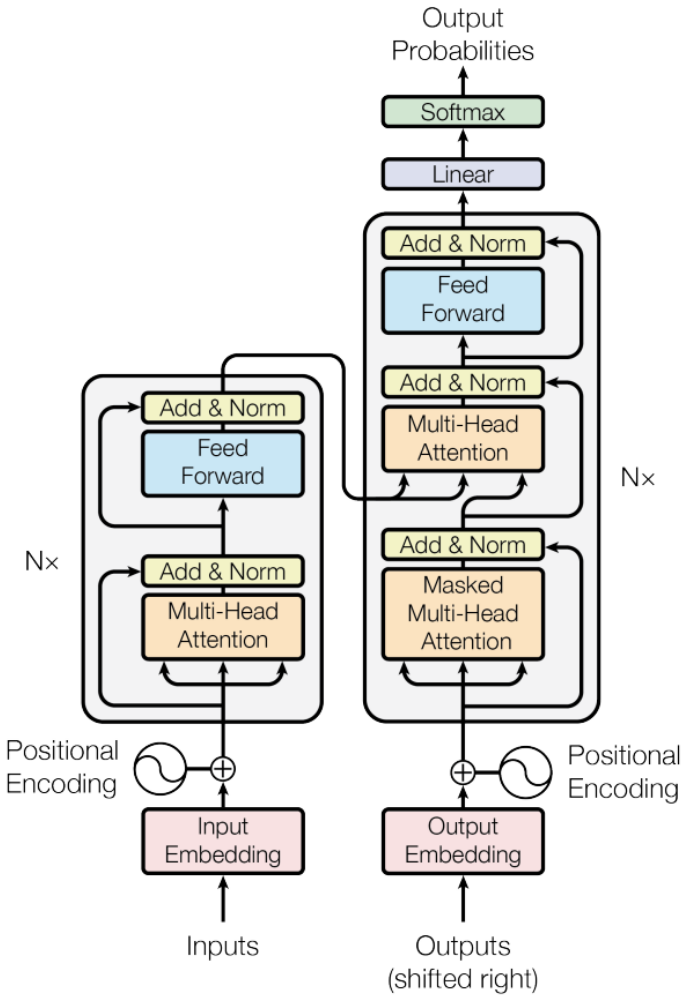
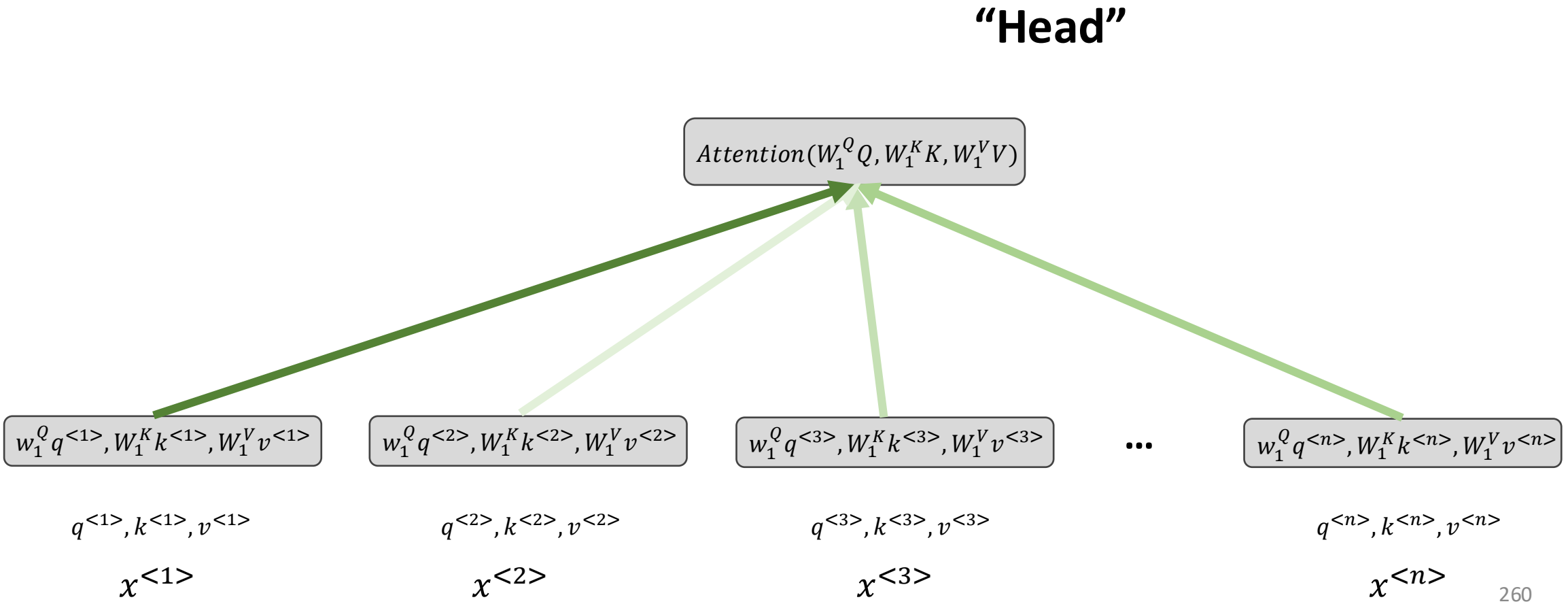


Figure 1: The Transformer - model architecture.

Machine Learning: *Transformers*

Multi-Head Attention

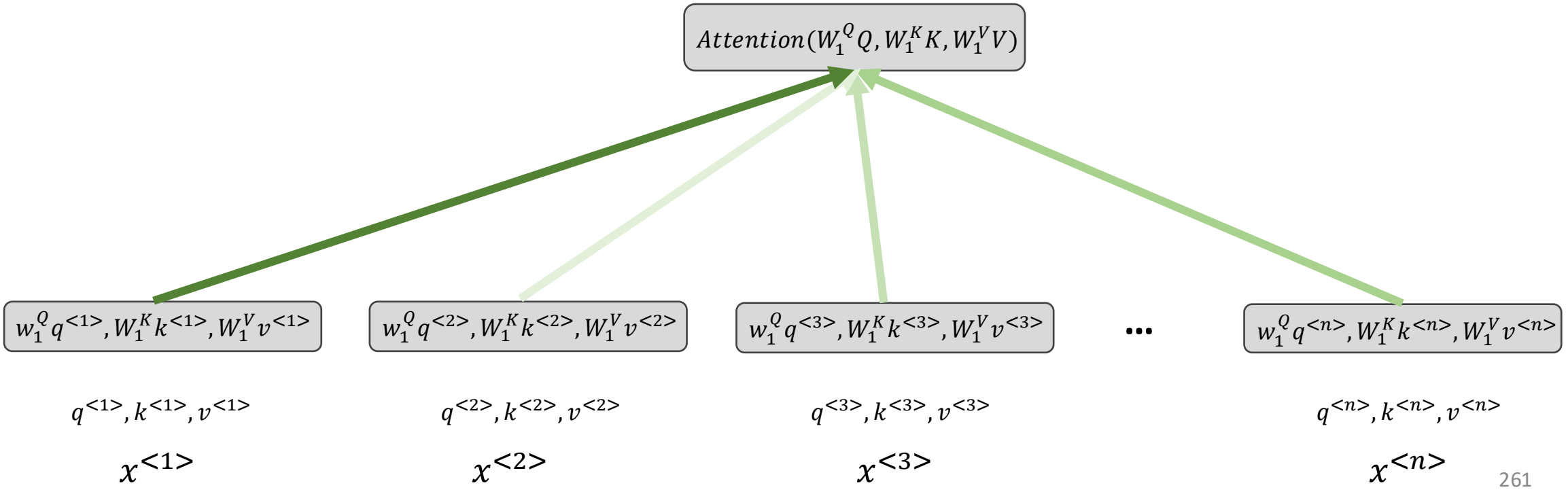


Machine Learning: *Transformers*

Multi-Head Attention

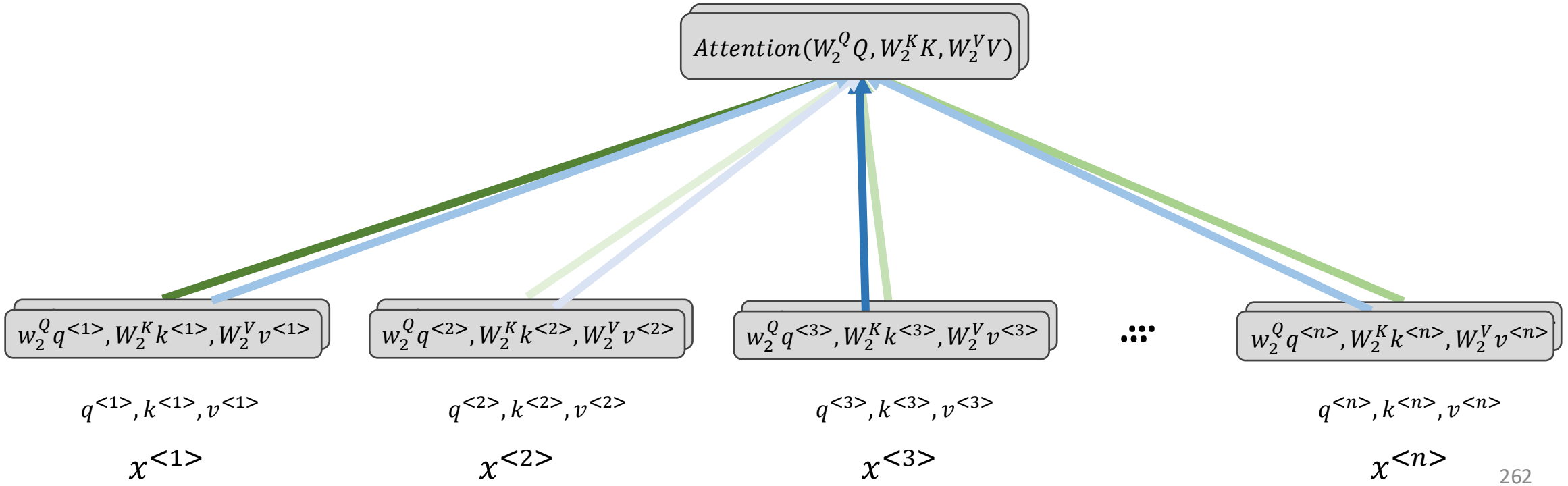
$Attention(W_1^Q Q, W_1^K K, W_1^V V)$

"Head"



Machine Learning: *Transformers*

Multi-Head Attention

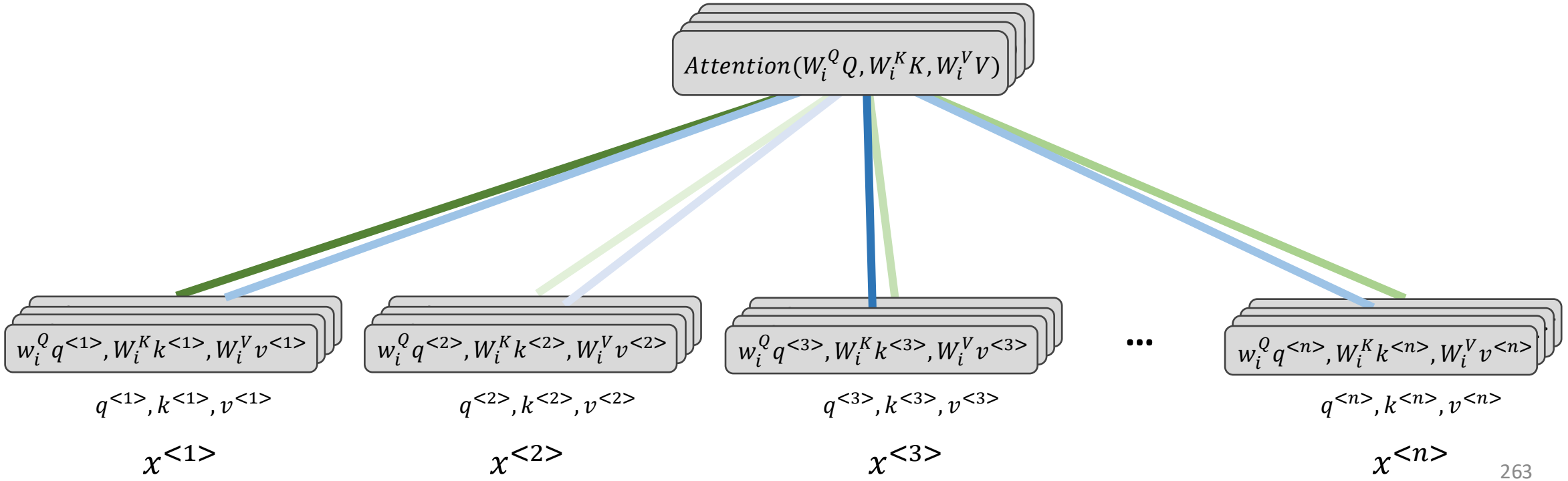


Machine Learning: *Transformers*

Multi-Head Attention

$$A(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

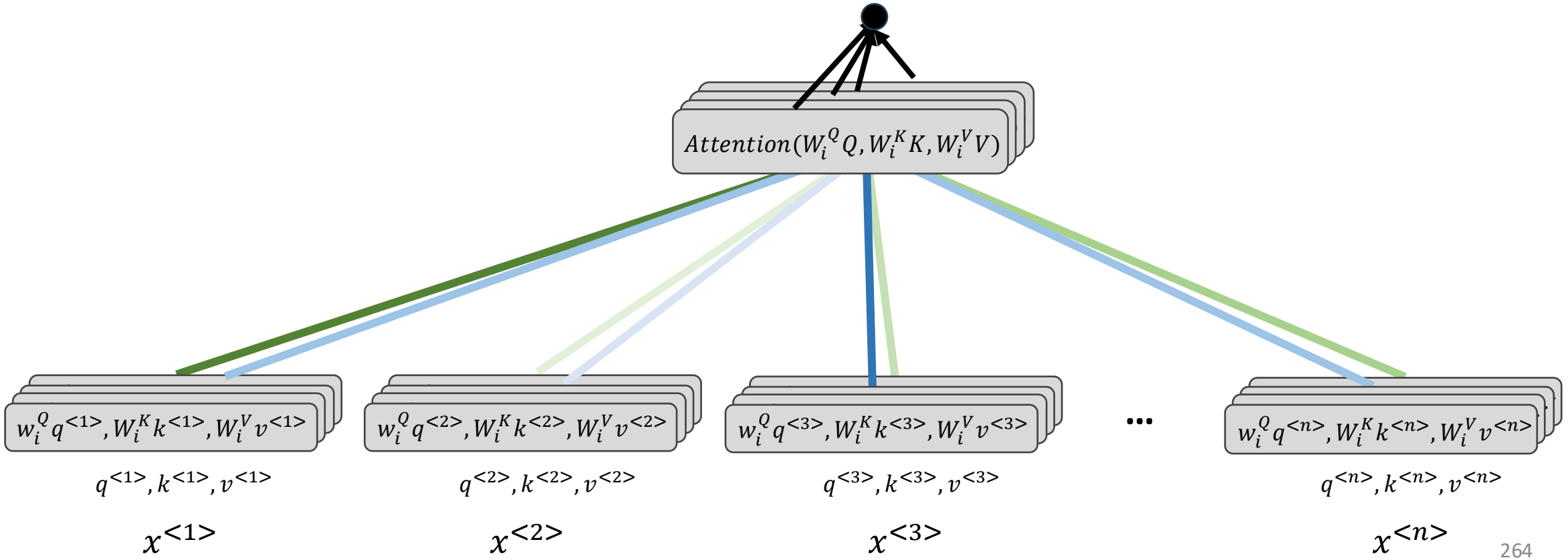
$$head_i = Attention(W_i^Q Q, W_i^K K, W_i^V V)$$



Machine Learning: *Transformers*

Multi-Head Attention

$$Multihead(Q, K, V) = concat(head_1, head_2, \ldots).W_o$$



Machine Learning: *Transformers*

TRANSFORMERS



Machine Learning: *Transformers*

Architecture

$x^{<1>}$ $x^{<2>}$ $x^{<3>}$... $x^{<n>}$

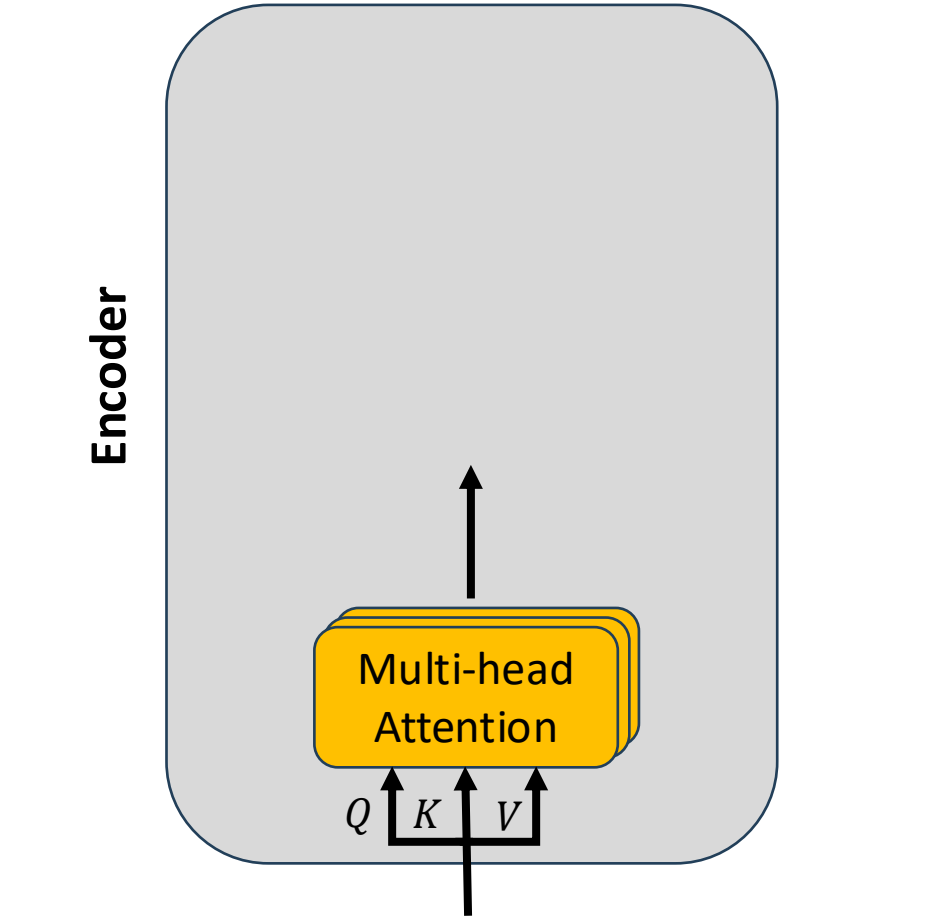
Machine Learning: *Transformers*

Architecture

$< SOS >$ $x^{<1>}$ $x^{<2>}$ $x^{<3>}$... $x^{<n>}$ $< EOS >$

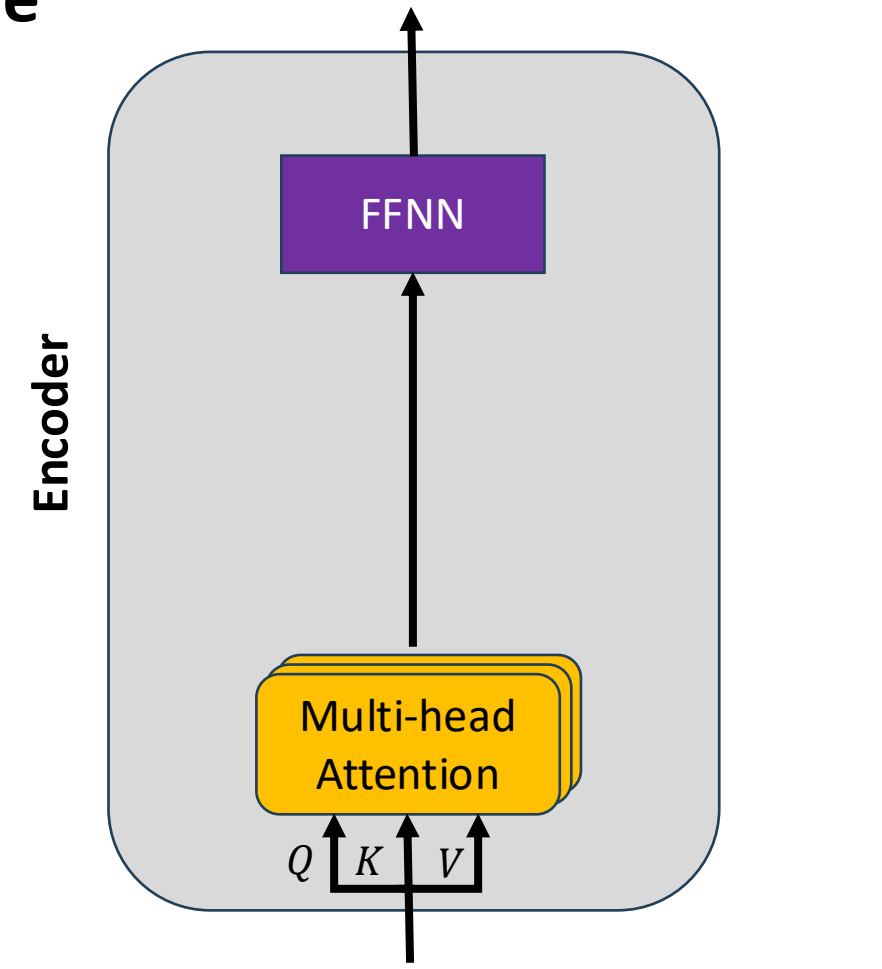
Machine Learning: *Transformers*

Architecture



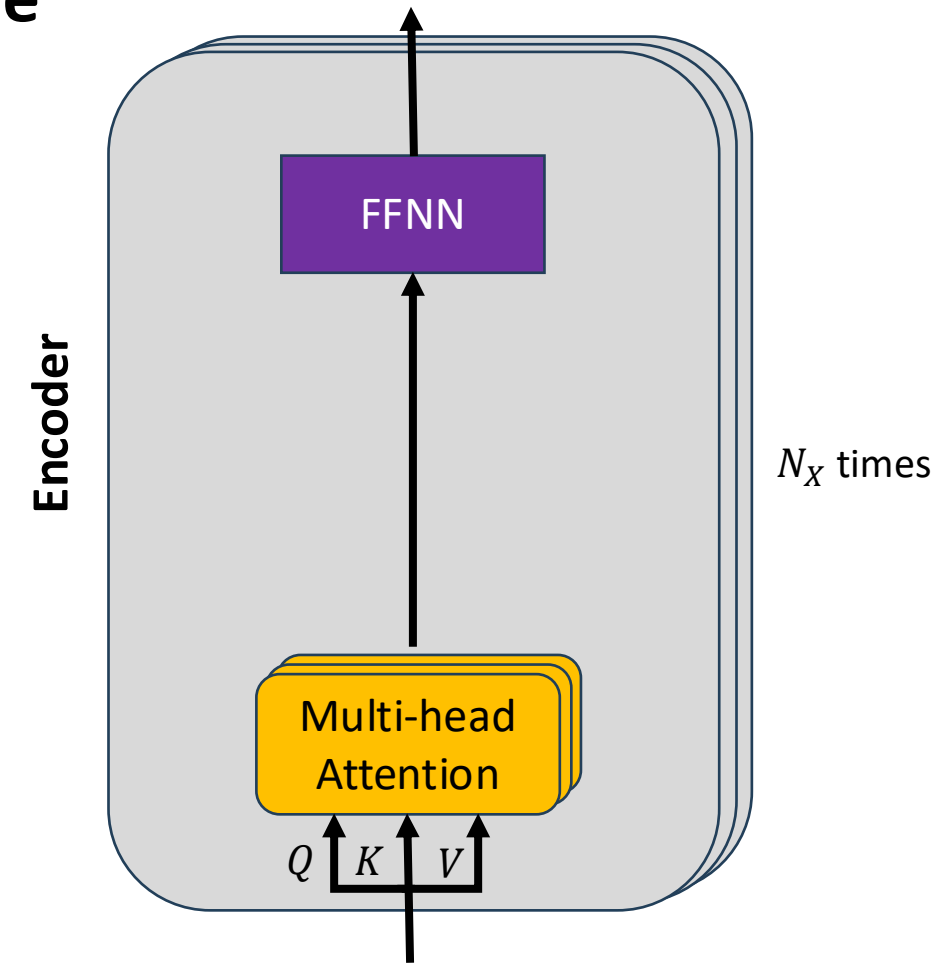
Machine Learning: *Transformers*

Architecture



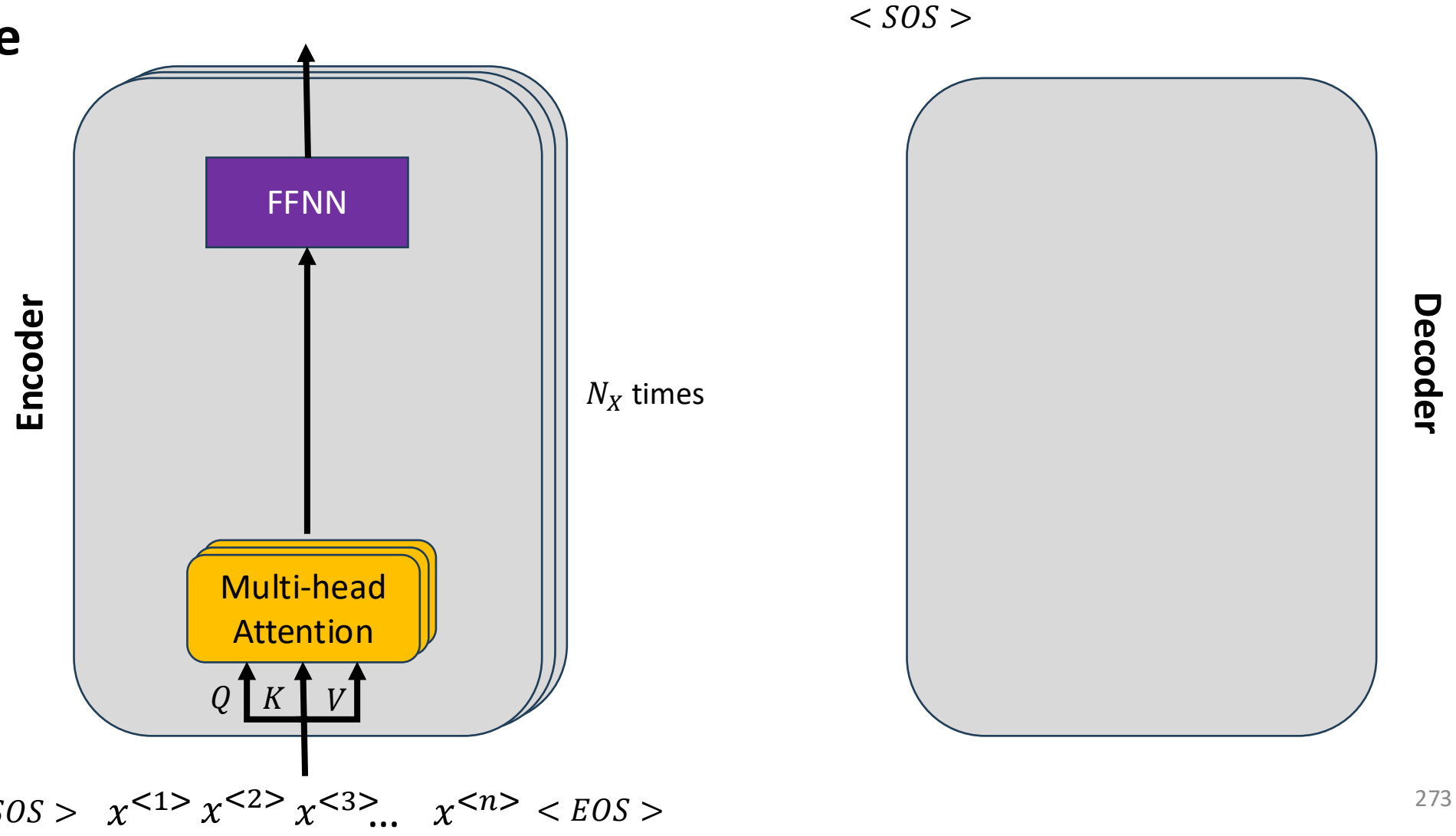
Machine Learning: *Transformers*

Architecture



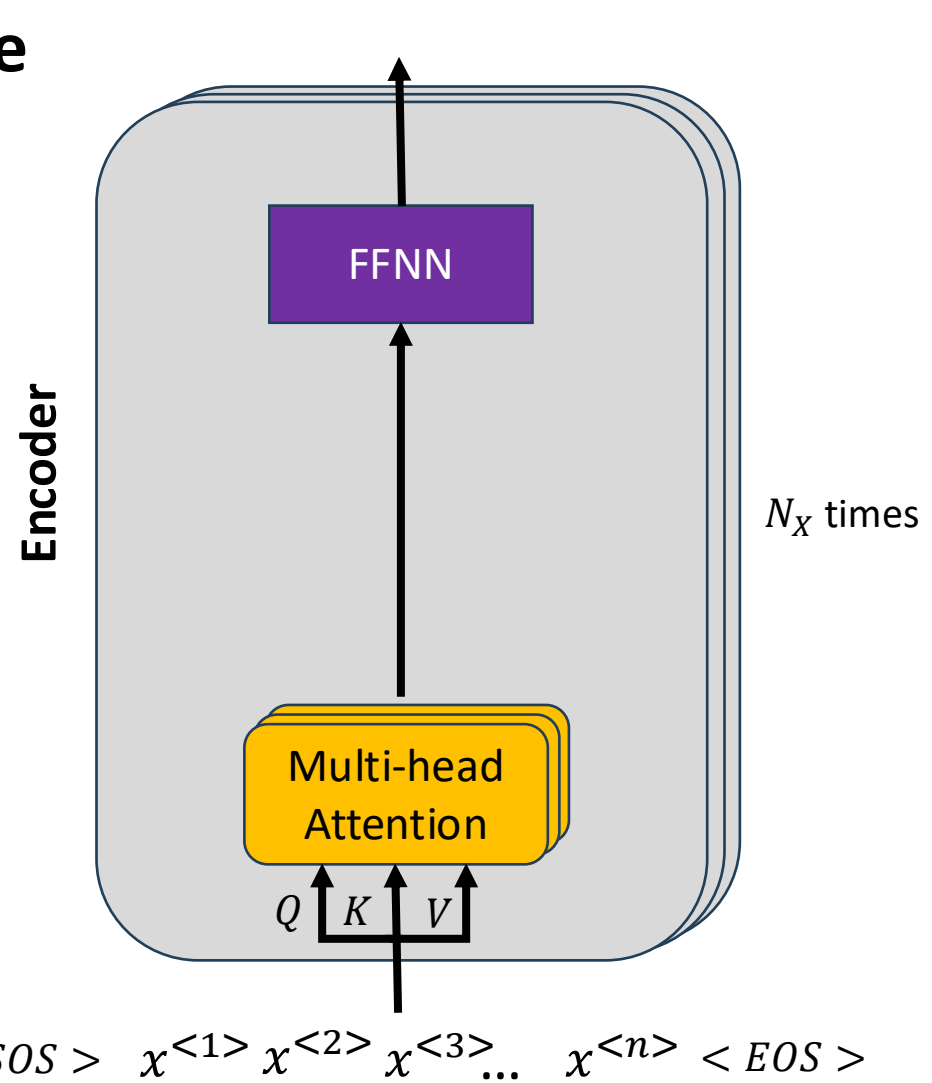
Machine Learning: *Transformers*

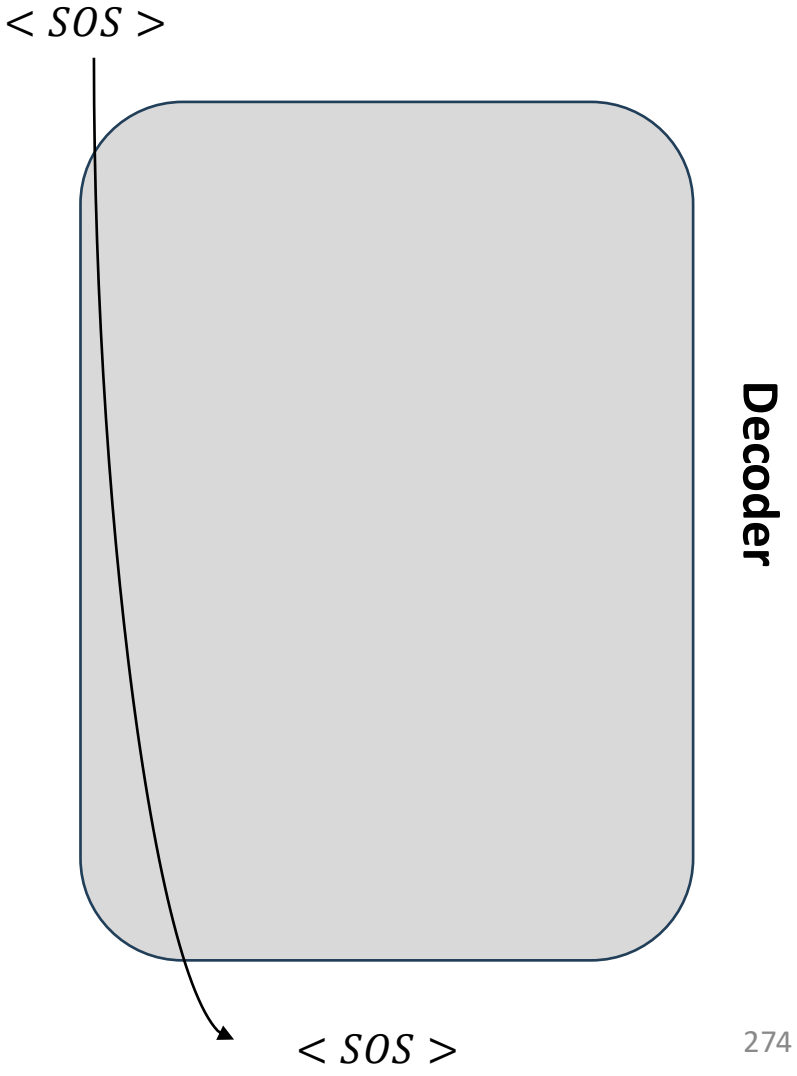
Architecture



Machine Learning: *Transformers*

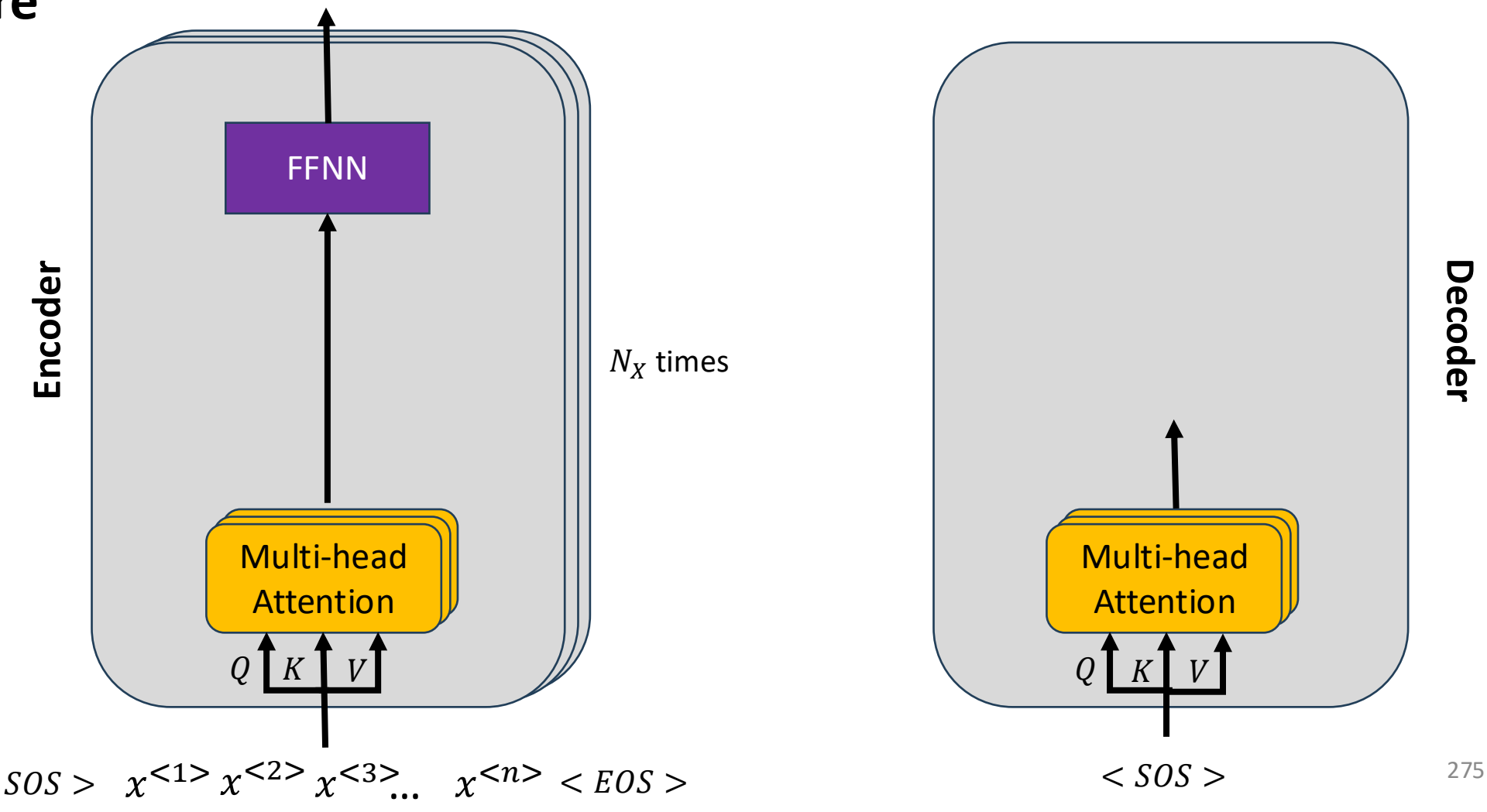
Architecture





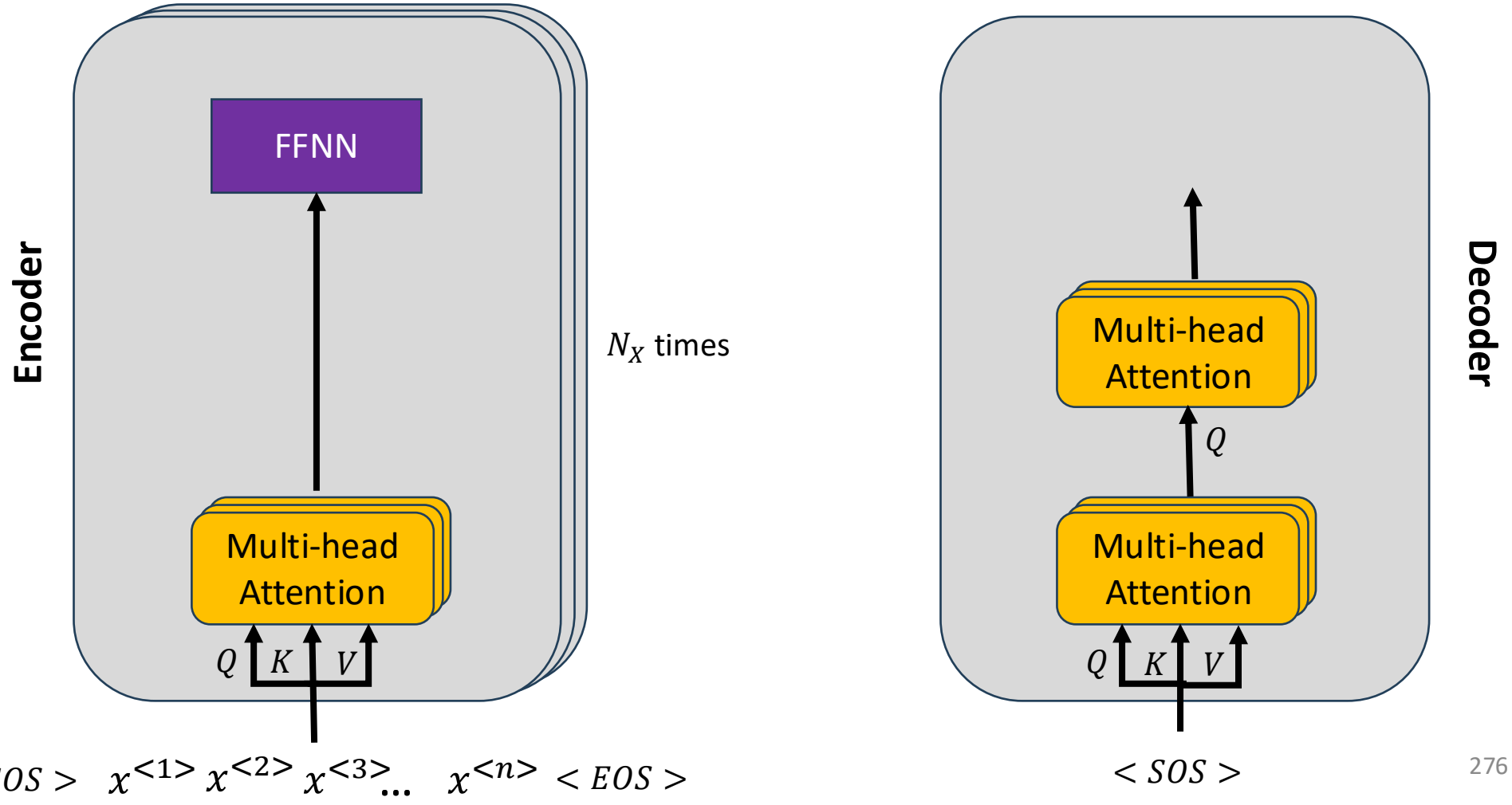
Machine Learning: *Transformers*

Architecture



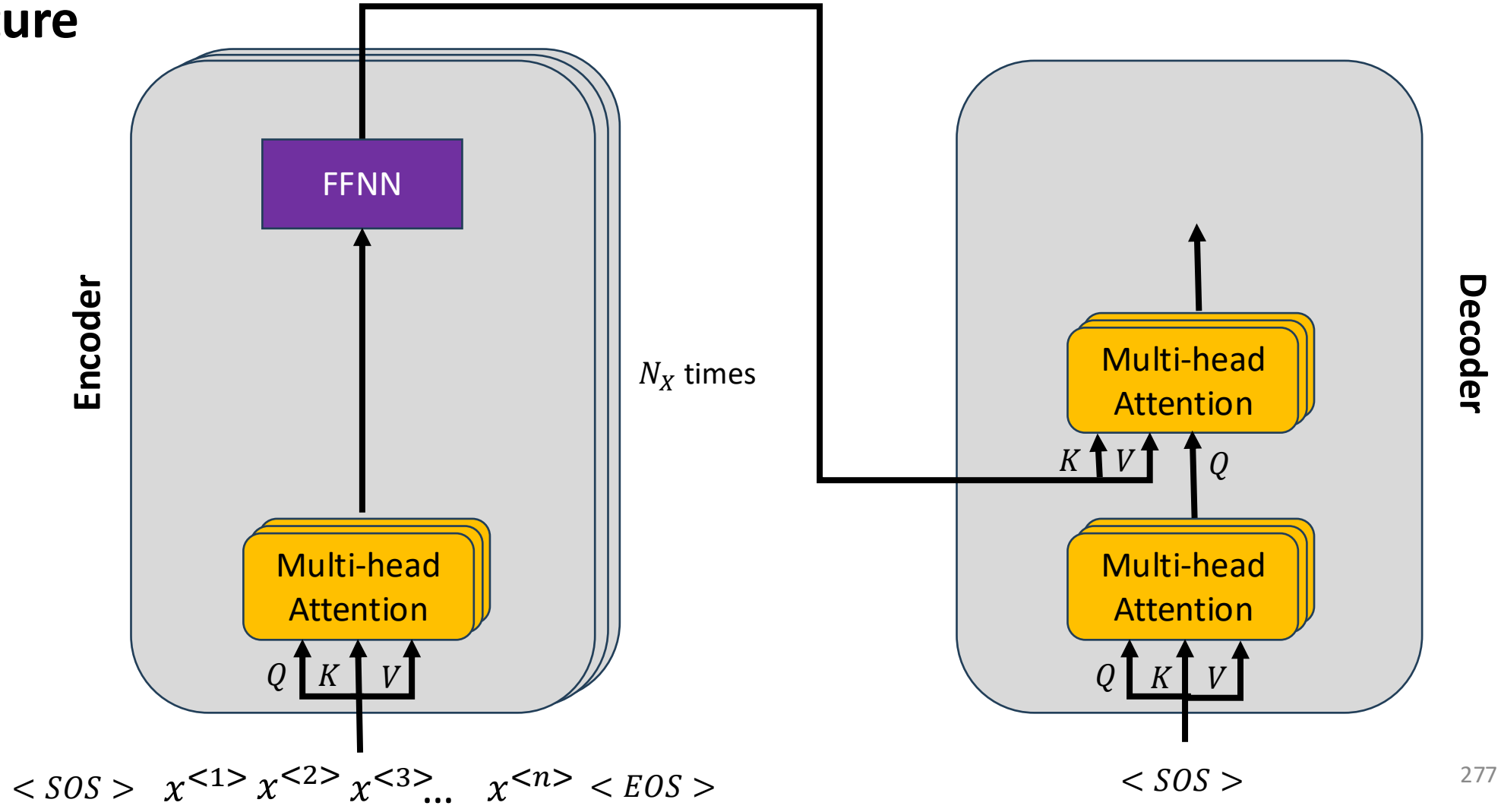
Machine Learning: *Transformers*

Architecture



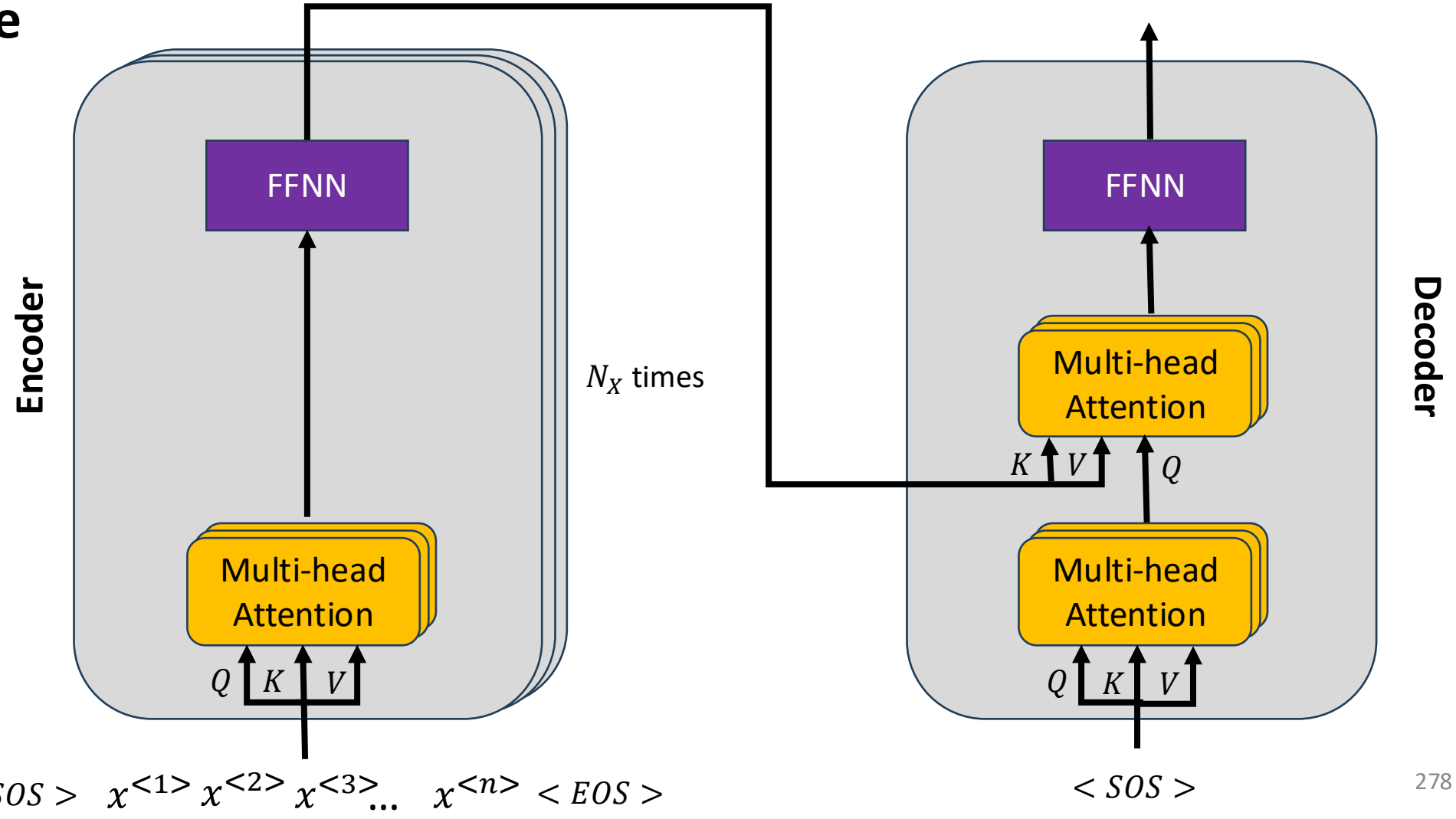
Machine Learning: *Transformers*

Architecture



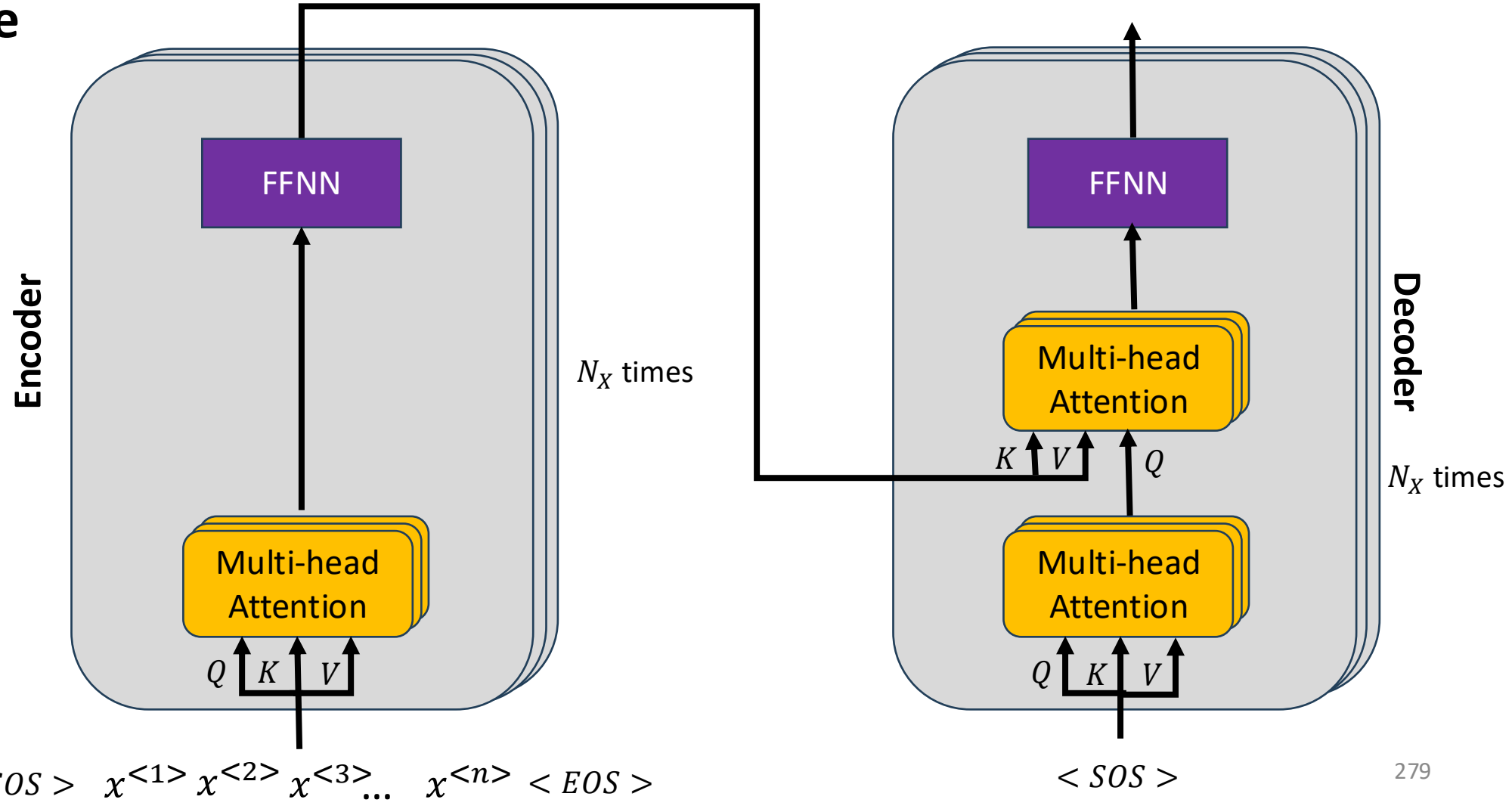
Machine Learning: *Transformers*

Architecture



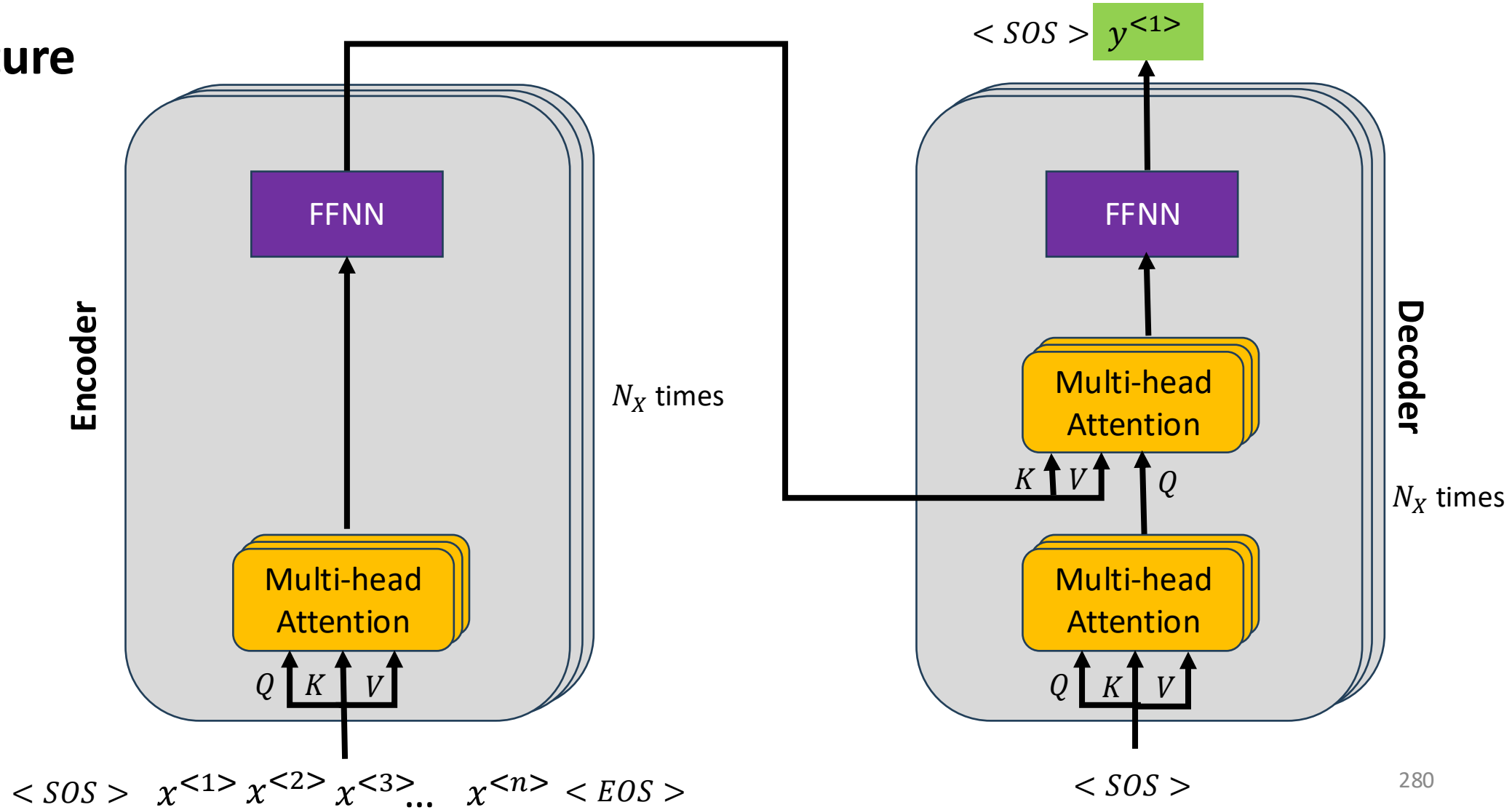
Machine Learning: *Transformers*

Architecture



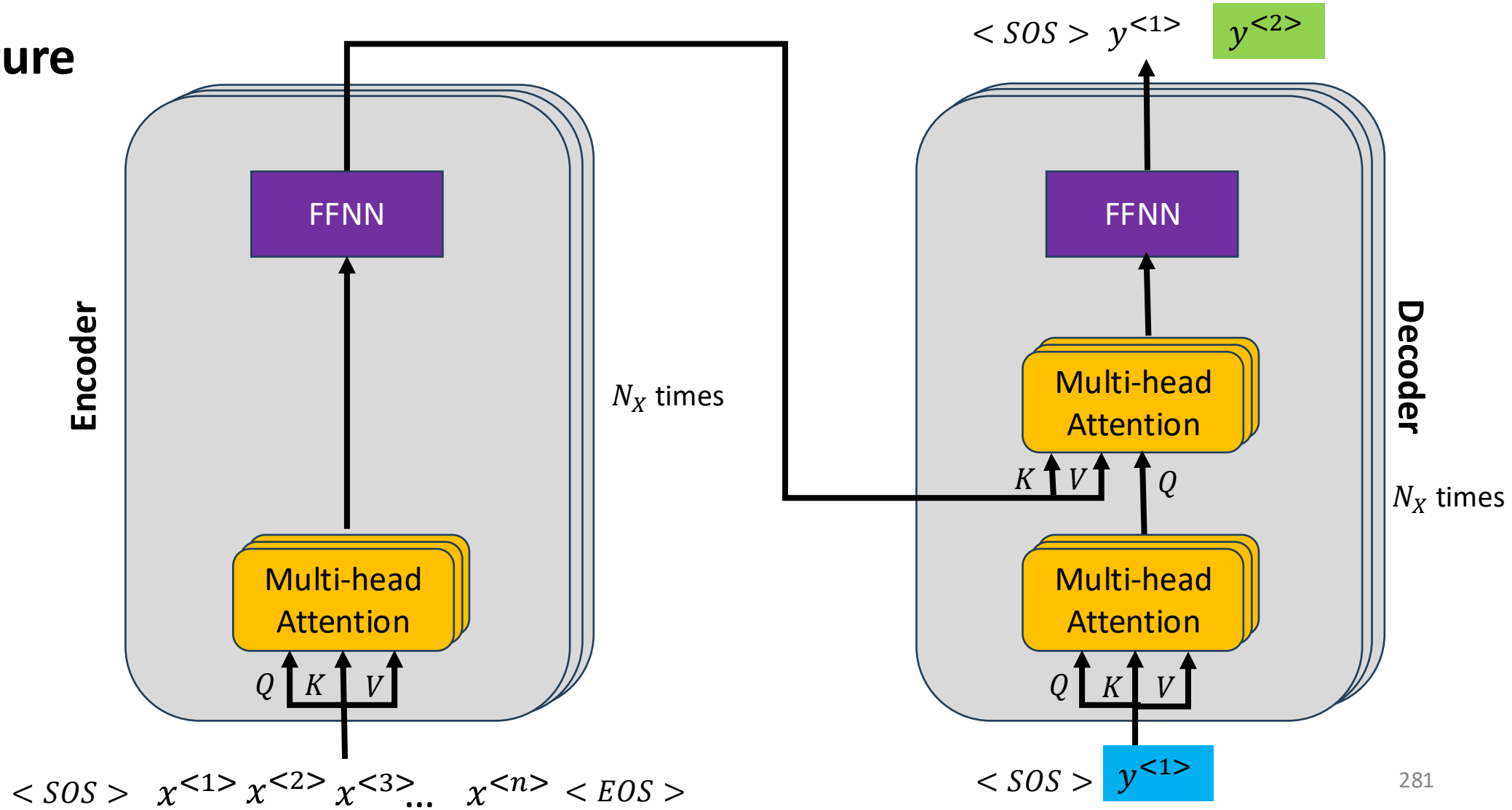
Machine Learning: *Transformers*

Architecture



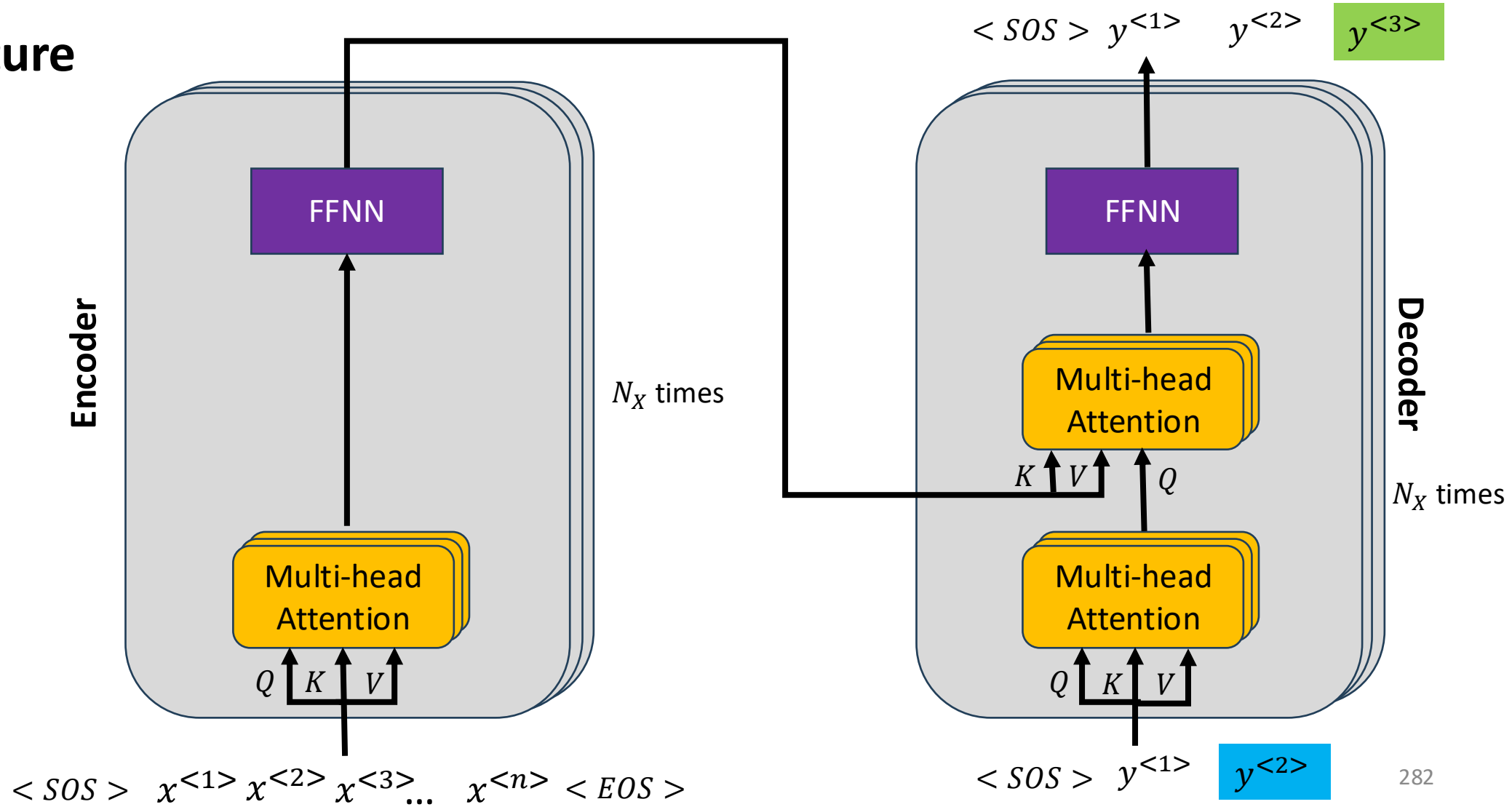
Machine Learning: *Transformers*

Architecture



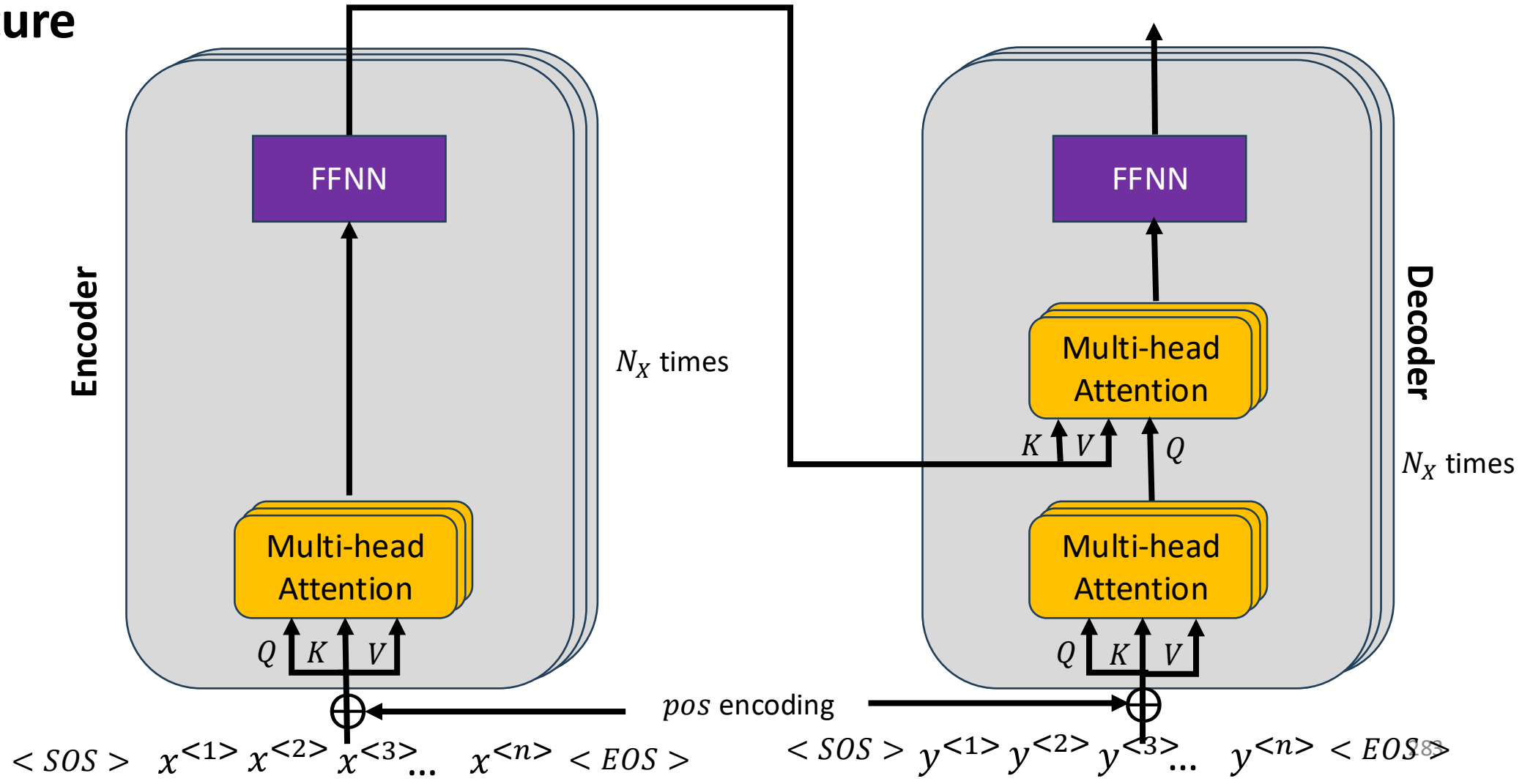
Machine Learning: *Transformers*

Architecture



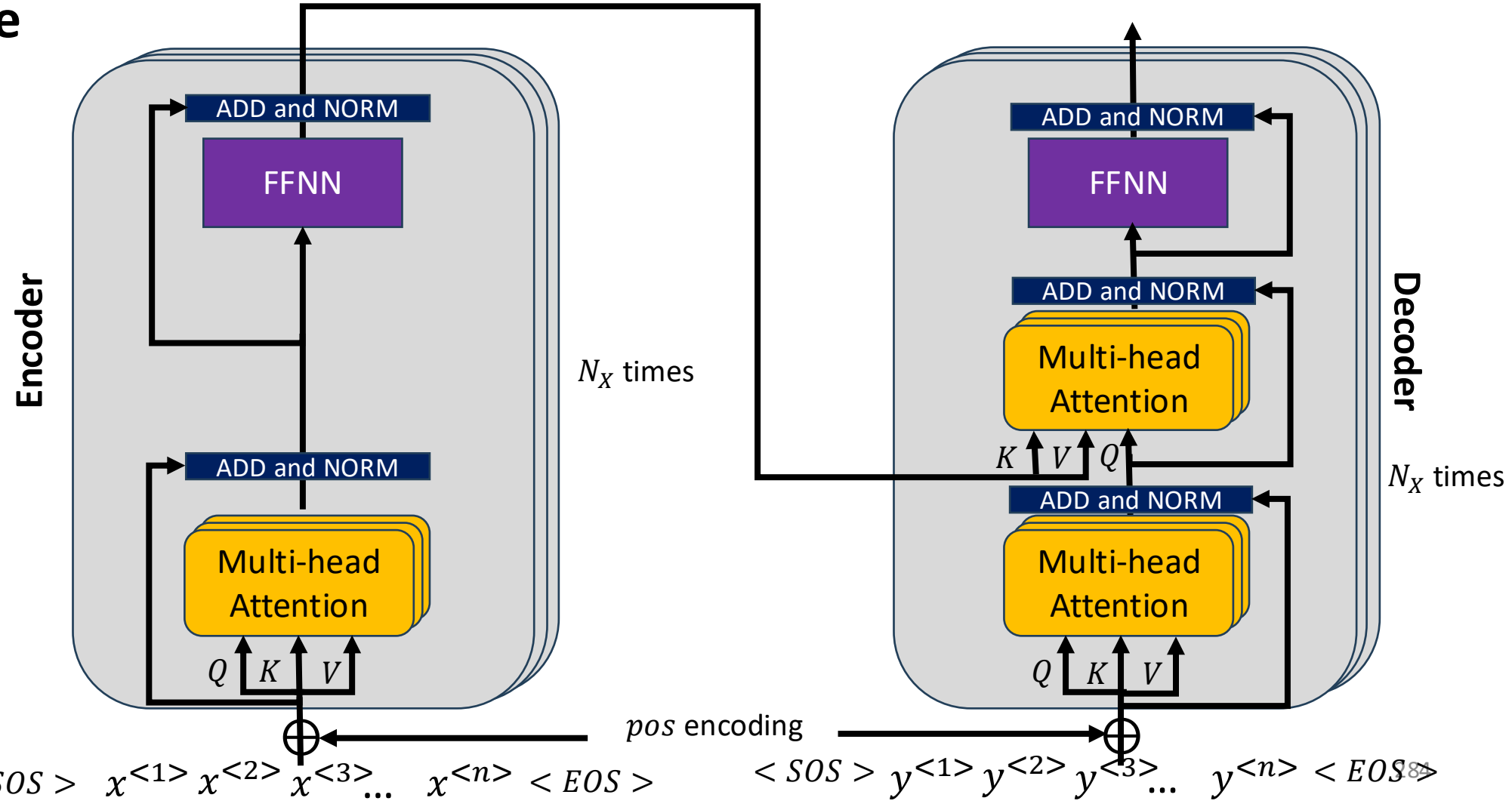
Machine Learning: *Transformers*

Architecture



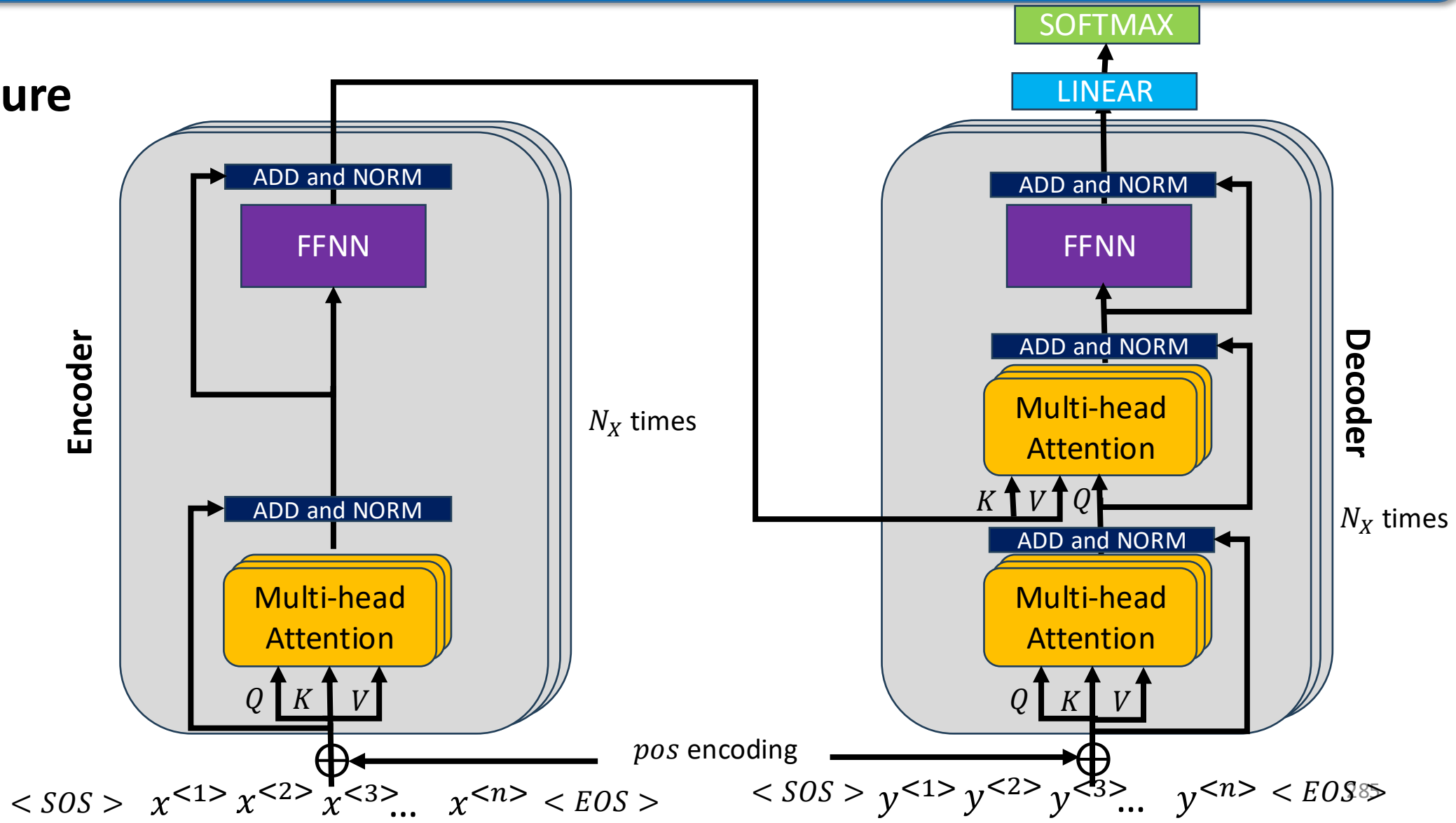
Machine Learning: *Transformers*

Architecture



Machine Learning: *Transformers*

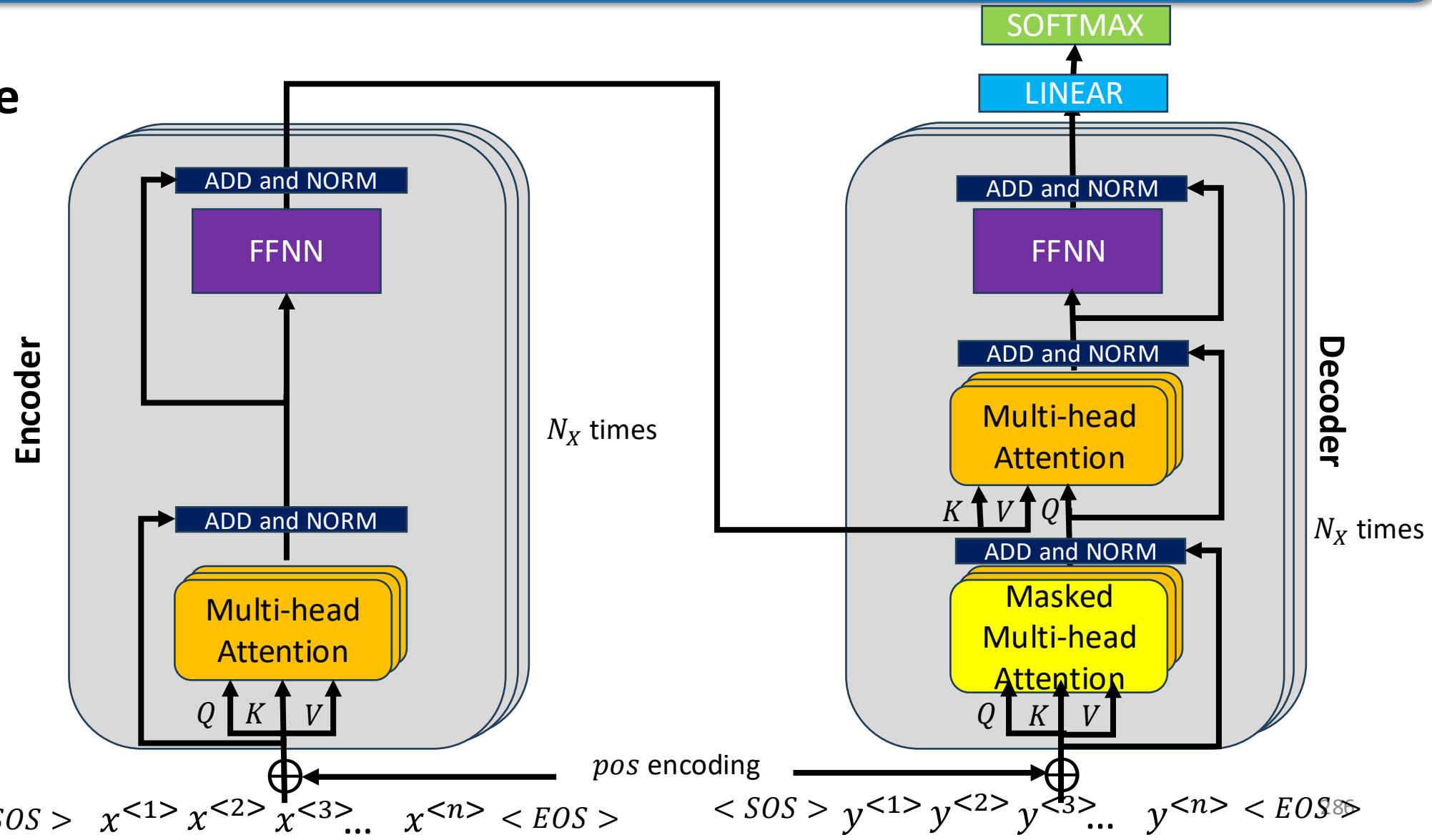
Architecture



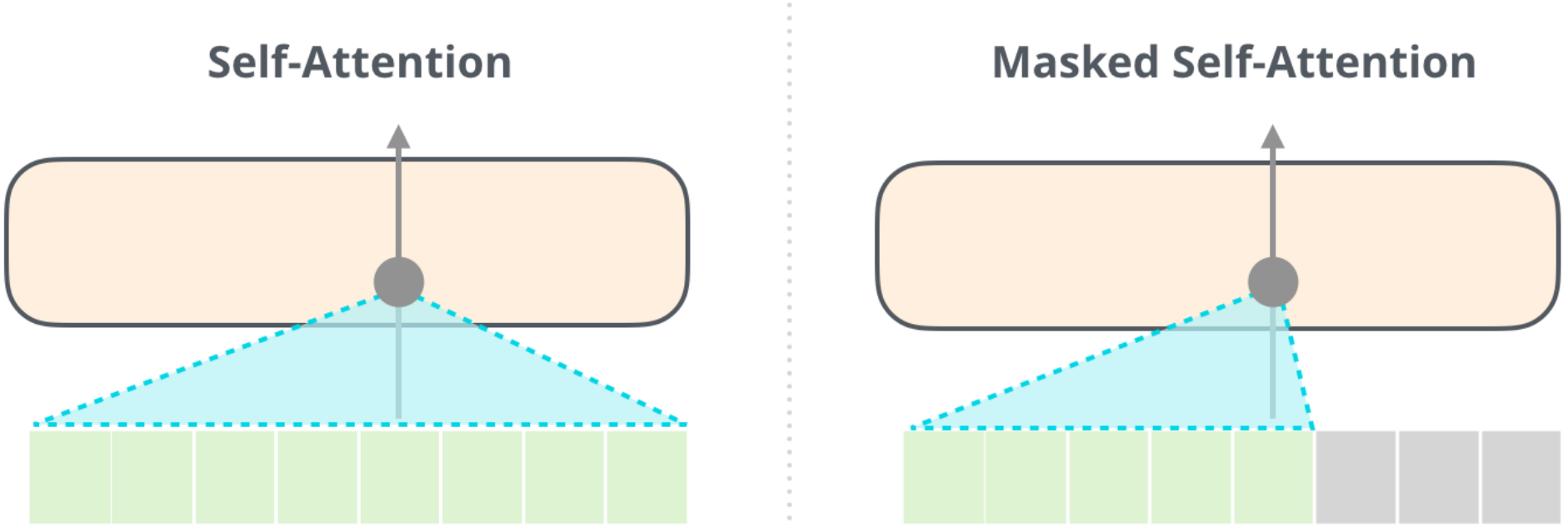
Machine Learning: *Transformers*

Architecture

ONLY during training



Machine Learning: *Transformers*



https://jalammar.github.io/illustrated-gpt2/

Machine Learning: *Neural Networks*

Transformers

Python

https://www.tensorflow.org/text/tutorials/transformer

Machine Learning: *Metrics*

Evaluation Metrics

Regression Metrics

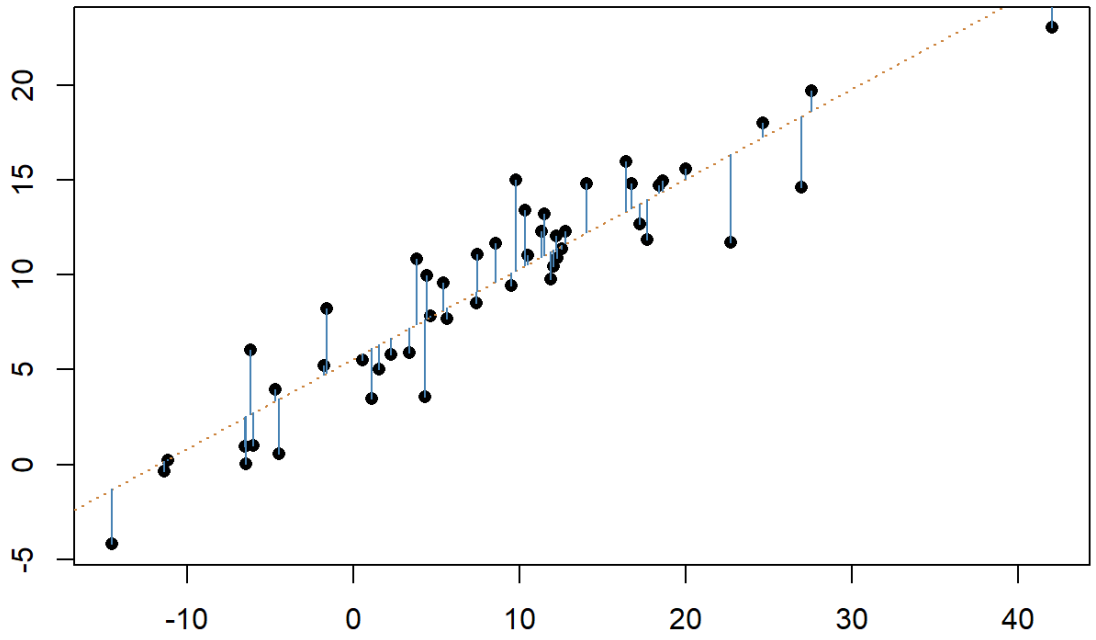
Mean Absolute Error (MAE): The average of the absolute differences between the predictions and actual values.

$$MAE = \frac{1}{N}\sum_{i=1}^{N} |y_i - \hat{y}|$$

Where,

$\hat{y}$ – *predicted value of* y

$\bar{y}$ – *mean value of* y



Machine Learning: *Metrics*

Evaluation Metrics

Regression Metrics

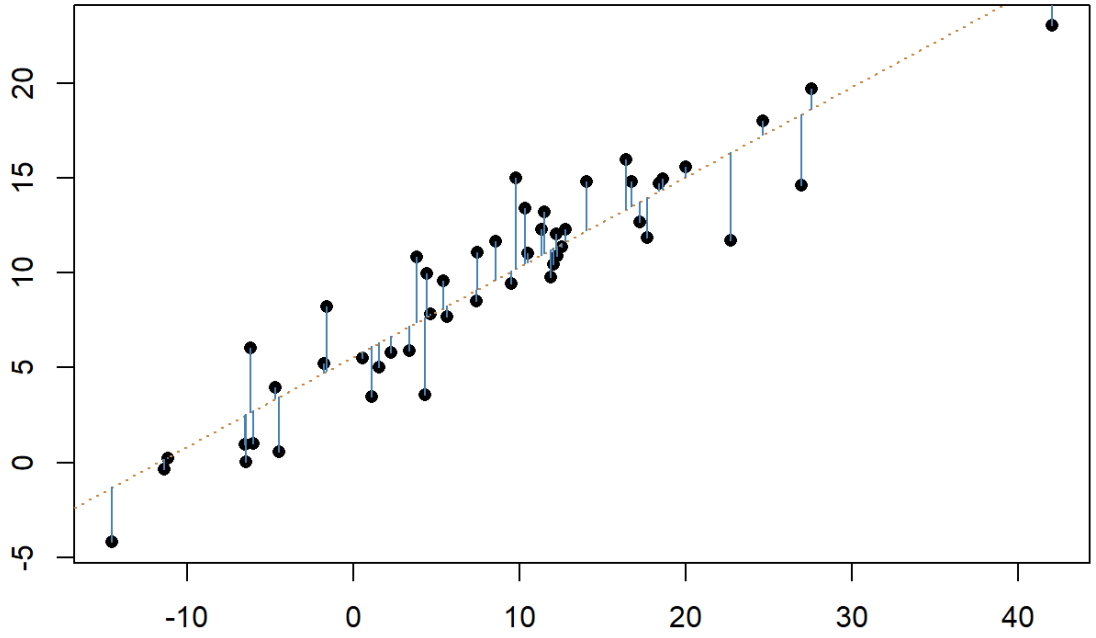
Mean Squared Error (MSE): The average of the squared differences between the predictions and actual values.

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y})^2$$

Where,

$\hat{y}$ – *predicted value of* y

$\bar{y}$ – *mean value of* y



Machine Learning: *Metrics*

Evaluation Metrics

Regression Metrics

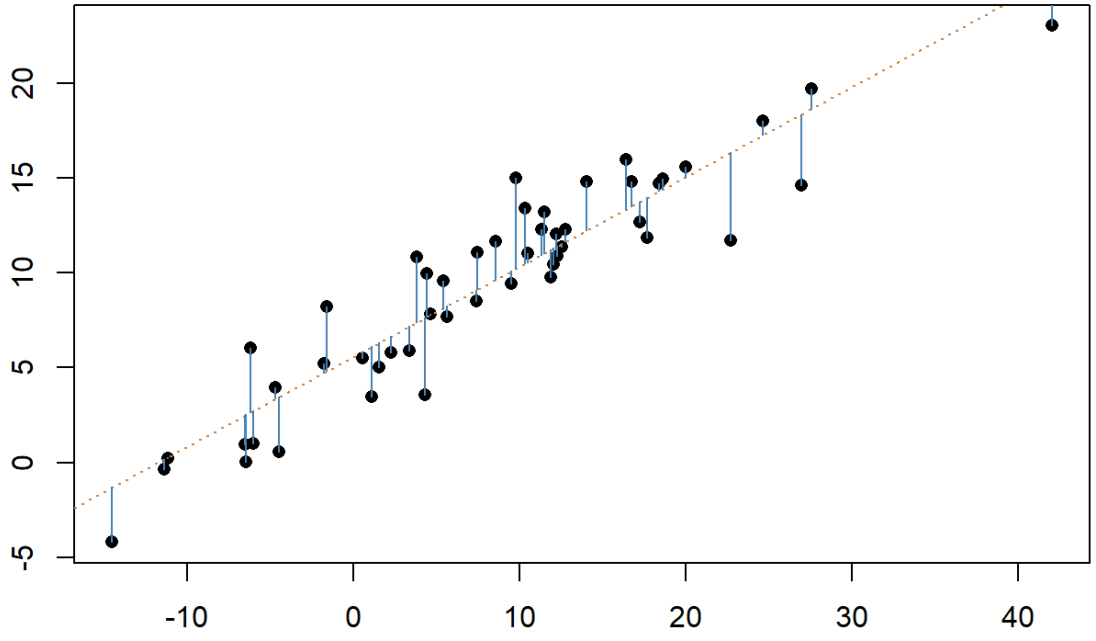
Root Mean Squared Error (RMSE): The square root of MSE.

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y})^2}$$

Where,

$\hat{y}$ – *predicted value of* y

$\bar{y}$ – *mean value of* y



Machine Learning: *Metrics*

Evaluation Metrics

Regression Metrics

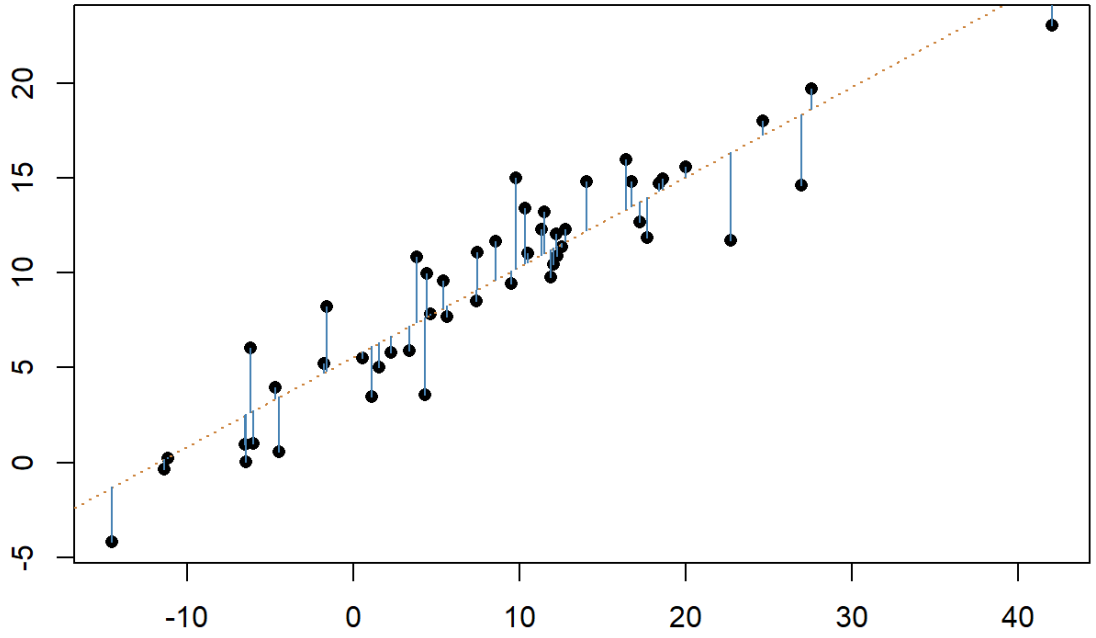
R-squared (Coefficient of Determination): Measures how well the predictions approximate the actual data points. An R-squared of 1 indicates perfect fit.

$$R^2 = 1 - \frac{\sum(y_i - \hat{y})^2}{\sum(y_i - \bar{y})^2}$$

Where,

$\hat{y}$ – *predicted value of* y

$\bar{y}$ – *mean value of* y



Machine Learning: *Metrics*

Evaluation Metrics

Classification Metrics

Accuracy: The proportion of correct predictions over all predictions.

Precision: The proportion of true positive predictions in the positive class

Recall (Sensitivity): The proportion of actual positives correctly identified.

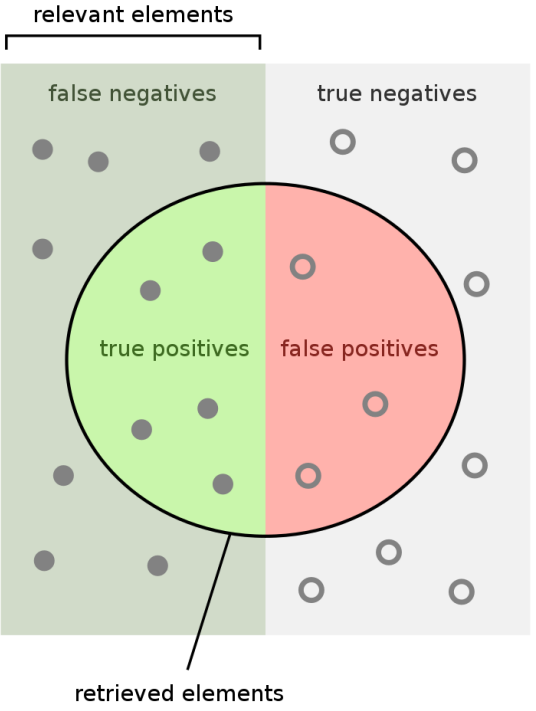
F1 Score: The harmonic mean of precision and recall.

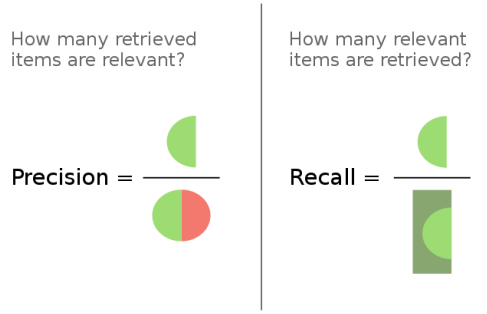
$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$



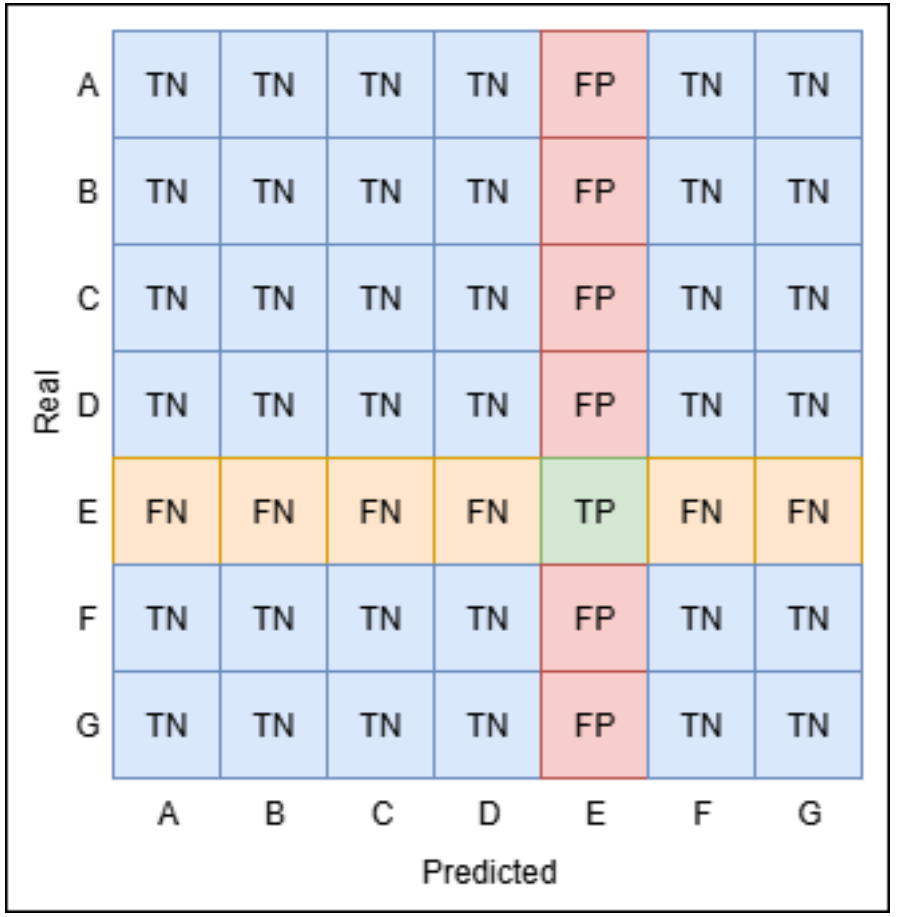


Machine Learning: *Metrics*

Evaluation Metrics

Classification Metrics

Confusion Matrix: A table showing the number of correct and incorrect predictions categorized by type.



wiki

Machine Learning: *Metrics*

Evaluation Metrics

Classification Metrics

Most classifiers (logistic regression, neural networks, etc.) do **not directly output a class**. Instead, they output a **score or probability**.

To convert these probabilities into **class predictions**, we choose a **threshold**.

$$\text{If } P(y = 1 \mid x) \geq \text{ threshold} \Rightarrow \text{predict positive}$$
$$\text{If } P(y = 1 \mid x) < \text{threshold} \Rightarrow \text{predict negative}$$

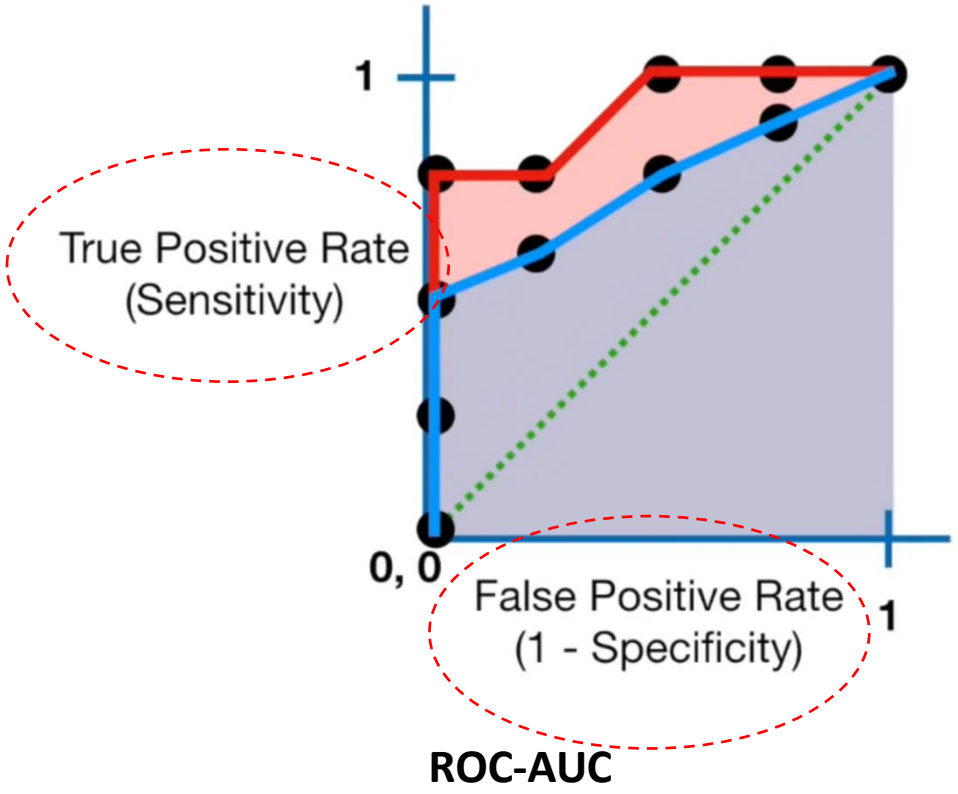
Machine Learning: *Metrics*

Evaluation Metrics

Classification Metrics

ROC-AUC: The area under the Receiver Operating Characteristic curve.

It measures the model's ability to distinguish between classes at different threshold selections.



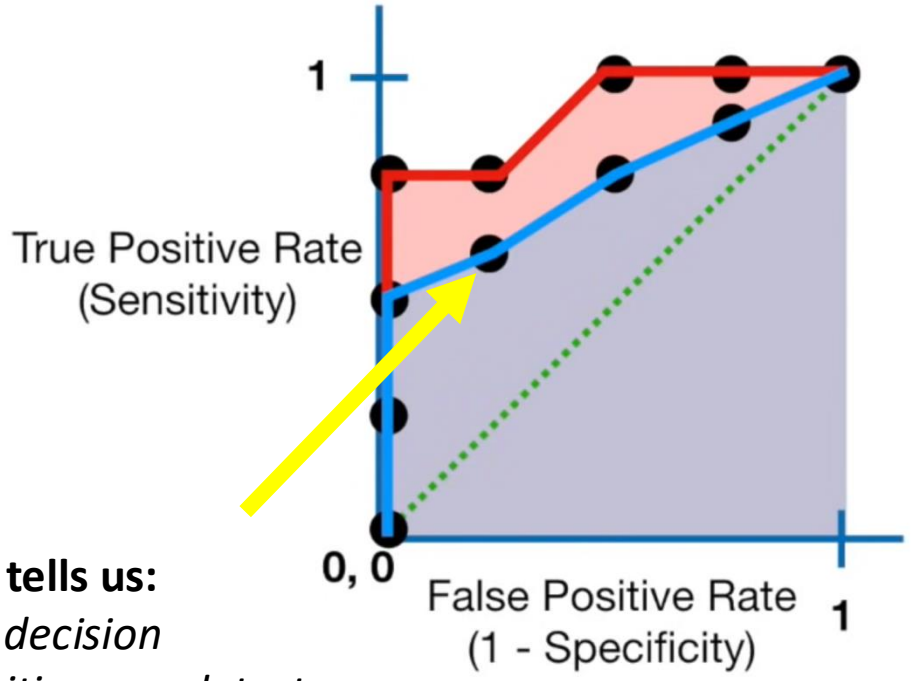
Machine Learning: *Metrics*

Evaluation Metrics

Classification Metrics

ROC-AUC: The area under the Receiver Operating Characteristic curve.

It measures the model's ability to distinguish between classes at different threshold selections.



ROC-AUC

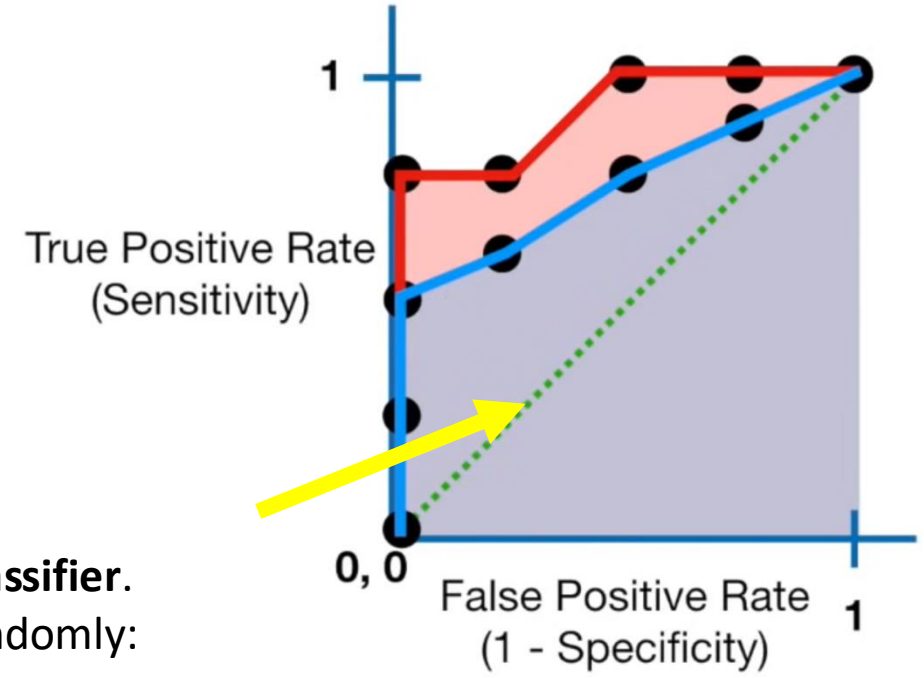
Each point on this graph tells us:
If we choose a particular decision threshold, how many positives we detect vs. how many mistakes we make.

Machine Learning: *Metrics*

Evaluation Metrics

Classification Metrics

ROC-AUC: The area under the Receiver Operating Characteristic curve.



ROC-AUC

represents a **random classifier**.
If the model guesses randomly:

TPR=FPR

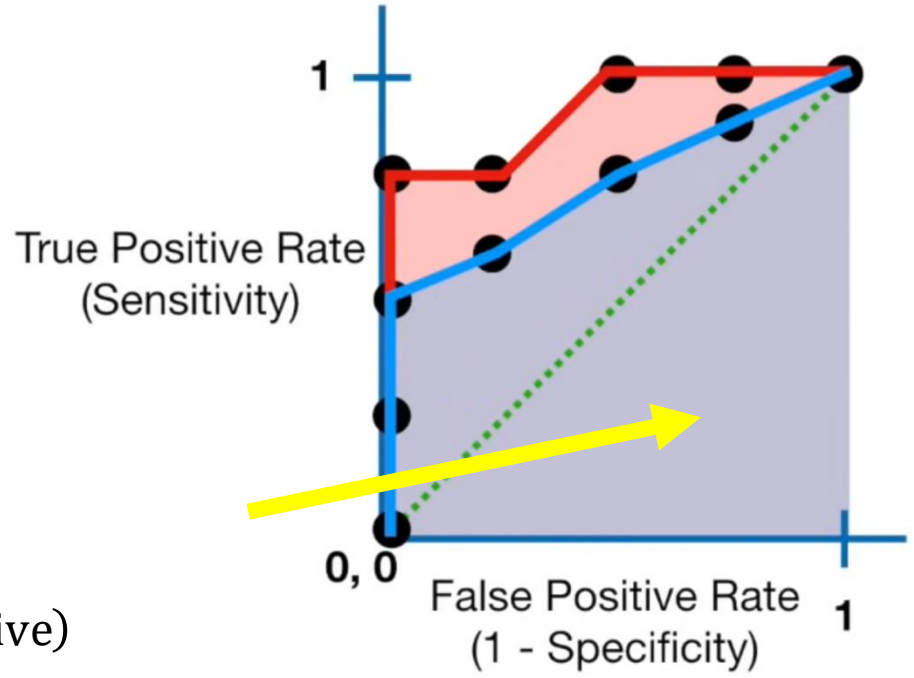
i.e. The model is no better than flipping a coin.

Machine Learning: *Metrics*

Evaluation Metrics

Classification Metrics

ROC-AUC: The area under the Receiver Operating Characteristic curve.



ROC-AUC

The **shaded region** represents the **Area Under the Curve (AUC)**.
Interpretation:

$$AUC = P(\text{model ranks a random positive higher than a random negative})$$

The better the model, the more the curve bends toward the **top-left corner**.